

微网与外部电网电力调控策略

基于因果预测与两日滚动优化的购电及储能调度

摘要

针对微网在负荷、光伏与电价变化下的购电调控问题，建立历史预测、两日计划、日内调整与逐槽执行相结合的优化模型。各层明确区分已知预报和未来实际值，以跨日连续储能协调当前供需与次日需求。

问题一建立包含能量平衡、充放电效率、容量及功率界限和日循环条件的线性规划。典型日最优购电量为 59482.7 kWh，费用 35126.95 元，较无储能基线节省 26.9%。

问题二以历史实现费用标定三源凸组合预测，采用半衰期 5 天的指数加权更新。开发期选定负荷抬升系数 1.02 与光伏折减 25 kW，每天优化两日、执行首日，实际日末电量自由并跨日传递。2 月至 12 月共 334 天总费用为 1395.7 万元，其中紧急购电费 79.6 万元。固定计划的联合残差重采样显示，总成本均值 1464 万元，P95 为 1542 万元。

问题三将官方光伏预报与历史预测按 0.7 和 0.3 组合，在 6、12、18 时调整未执行计划。主方案费用 1347.1 万元，较官方预报无调整基线下降 8.01%。受控消融表明，40 情景对冲扩展反而增加 2.1 万元；冻结验证期也增加 2.30 万元，故正式方案删除对冲。

问题四将价格替换为给定波动电价，两层策略费用分别为 1459.8 和 1410.3 万元。仅用历史价格预测的扩展费用为 1465.9 和 1418.4 万元，对应额外成本 0.42% 和 0.57%。

约束残差达到浮点误差量级，跨日 SOC 连续；334 个日末电量有 330 个不同取值，未被固定。模型同时说明初始状态、单年样本与分层规划近似的适用边界，并以负结果约束复杂机制的使用，为微网购电和储能运行提供可复现的决策依据。

关键词：微网调度；两日滚动；因果预测；储能优化；日内调整；模型消融

一、 问题重述

1.1 研究背景

小区微网通过光伏发电、储能电池和外部购电共同满足负荷。电池能够将低价时段的电能转移到高价时段，也能够吸收光伏富余电量；但预测偏差会使计划与实际需求不匹配，导致额外购电或弃电。因而，调度问题不仅是寻找一条低成本的购电曲线，还要使这条曲线在信息逐步到达的条件下可执行。

1.2 四个问题的递进关系

问题一依据典型日电价、负荷与光伏预测，建立当天购电及储能调度模型，要求日初、日末电量相同，并给出指定时段结果。问题二考虑逐日变化的负荷和光伏，要求每天零点作出购电计划，实际短缺按当期五倍电价紧急购电，常规购电按计划结算。问题三允许利用零点及 6、12、18 时发布的光伏预报制定和调整计划，调整还会产生相对原计划的偏差费用，需要同时评价新增信息和调整时机的作用。问题四进一步考虑波动电价，分别重算无日内调整与有日内调整两种策略。本文在共同的储能约束下逐步改变信息集与结算目标，使四问结果具有清晰的比较基础。

二、 数据理解与总体分析

2.1 数据处理与评价区间

附件 1 给出 144 个 10 分钟时段的典型日电价、负荷和光伏；附件 2 给出 2025 年实际负荷与光伏；附件 3 给出每天四次发布的 24 小时光伏预报；附件 4 给出波动电价。本文以电量为优化变量，功率乘 $\Delta t = 1/6$ 小时后统一为 kWh，电价保持元/kWh。附件与结果模板的时间文字存在一个 10 分钟刻度差异，因此输出沿用模板标签，以行序号一一对应原始槽数据，避免因字符串匹配造成错位。本文指定时段表也沿用结果模板标签。

附件 3 中提前 k 小时的值对应发布时刻加 k 小时，故每次更新都按发布时刻重建时间轴，再插值至 10 分钟粒度。6、12、18 时预报只用于该次发布之后尚未执行的时段，不能直接套用零点预报的映射。负荷与光伏历史预测均仅使用决策日前的实际值，典型日曲线视为题目给定的外生参考资料，不从完整年度实际值重新计算。

根据结果模板，问题二至四统一评价 2025 年 2 月 1 日至 12 月 31 日，共 334 天。1 月数据用于形成滞后特征与预热权重。当前实验在 2 月 1 日以 6000 kWh 初始化储能；这是评价区间的统一初始条件，并非对 1 月运行后的储电量进行推导。题面给出 1 月 1 日初值，因此本文结果应理解为指定报送区间的条件回测，初始状态的局限在模型评价中讨论。

2.2 误差结构与决策目标

数据分析表明，光伏光照样本的预测误差随提前时间增加，近似关系为 $MAE(k) \approx 14.8 + 29.0k$ kW，拟合 $R^2 = 0.982$ 。下午 13 至 18 时的光伏 MAE 随发布时刻由零点推进至 6 时、12 时，从 413 kW 降至 248 kW、76 kW。日内误差的前四阶相关系数为 0.79、0.60、0.40、0.21，说明逐点独立扰动会破坏误差的时间结构。因此，日内更新预报具有调度价值；整日联合残差场景作为候选扩展，通过消融决定是否进入正式方案。

紧急购电与计划购电的价格不对称，最小预测均方误差并不直接等于最小购电成本。本文用历史实现费用标定预测组合，在名义预测上加入小幅保守修正，再利用当前实际状态逐槽执行。候选场景法保留负荷和光伏残差的同日联合结构，避免只考虑光伏误差而遗漏净负荷风险；若留出实验不支持，则按简约原则删除。

2.3 两日规划与滚动执行

每天零点优化当日与次日共 288 个槽，只承诺首日计划；次日计划是储能机会成本的近似参考，到次日零点会重新计算。该思路借鉴滚动时域优化 [1]，但本文采用的有限窗口与实时执行规则并不具有自动成立的无限时域最优性保证。问题三在日内预报发布时重算剩余时段，执行层每 10 分钟只读取当前实际值，日末储电量自然传递至次日。表 1 概括不同问题的信息范围。

表 1 各问题的信息集与调度结构

问题	零点负荷与光伏输入	电价输入	调整与储能结构
一	典型日给定数据	典型日给定	单日规划、日循环
二	因果历史预测	典型日电价	无日内调整、两日滚动
三	历史负荷与组合光伏预报	典型日电价	三次更新、跨日连续
四	分别沿用问题二、三预测	给定波动电价 G	分别重算两层策略
扩展 H	与问题四对应层一致	历史价格预测	评价电价信息价值

三、模型假设

1. 每 10 分钟内功率按常数近似，负荷不可主动削减。该假设使连续能量平衡转化为有限维线性约束。
2. 储能充电和放电各按 90% 效率计损，往返效率为 81%；不另计自放电与寿命衰减。该解释适用于题目给定的固定效率条件。
3. 微网不售电；无法被负荷或储能吸收的供给可弃置。预测规划中的弃光和执行中计划购电富余的弃电应区别理解，后者不获得退款。
4. 常规购电按已承诺电量结算，短缺额外以五倍当期价格采购。问题三各槽按最终有效调整量相对零点原计划结算一次，避免对同一槽重复计收偏差费用。
5. 问题一保留题面要求的日循环；问题二至四不人为固定每日实际末端电量，两日

规划最远端仅受容量边界约束。日内调整规划的末端参考由当日两日计划内生给出，执行允许偏离该参考。

6. 波动电价主分析 G 将附件 4 给定价格作为规划窗口内已知输入；H 作为仅有历史价格的扩展。G 不是对未来实际负荷或光伏已知的假设。

四、符号说明

表 2 主要符号、含义与单位

符号	含义	单位
$d, t; T, H$	日期、日内槽；日槽数 144、规划槽数 288	—
$L_{d,t}, P_{d,t}$	实际负荷、光伏电量	kWh
$\hat{L}, \hat{P}$	决策时刻可得的预测电量	kWh
$p_{d,t}$	交易电价	元/kWh
$x_{d,t}^0, x_{d,t}^a$	零点计划、各槽最终有效调整购电量	kWh
$c_{d,t}, q_{d,t}$	充入、放出的电网侧电量	kWh
$E_{d,t}$	时段末储能电量	kWh
$e_{d,t}, s_{d,t}$	紧急购电量、富余弃电量	kWh
$\eta, \bar{c}$	单向效率 0.9、单槽充放上限 5000/6	—、kWh
w_d, u_d	历史负荷、光伏的三源凸组合权重	—
h, κ, m	EWMA 半衰期、负荷抬升、光伏折减	天、—、kW
λ, S	官方光伏预报占比、情景数	—
$\mathcal{I}_{d,t}$	决策时刻已获得的信息集合	—

带上标 s 的变量表示第 s 个情景， $[z]_+ = \max(z, 0)$ 。下文未列出的局部辅助变量在首次出现时定义。

五、问题一：典型日确定性储能调度

5.1 线性规划模型

对给定的典型日，决策变量为购电 x_t 、充电 c_t 、放电 q_t 、电量 E_t 及弃光 s_t 。在不需要紧急购电的确定性条件下，目标为

$$\min C_1 = \sum_{t=1}^T p_t x_t. \quad (1)$$

每个槽满足供需平衡与储能递推：

$$P_t + x_t + q_t = L_t + c_t + s_t, \quad (2)$$

$$E_t = E_{t-1} + \eta c_t - q_t / \eta. \quad (3)$$

充电量从电网侧计量，故进入电池的电量 ηc_t ；向负荷放出 q_t 需消耗 q_t/η 。容量、功率、非负性与循环条件为

$$1200 \leq E_t \leq 10800, \quad 0 \leq c_t, q_t \leq 5000/6, \quad (4)$$

$$x_t \geq 0, \quad 0 \leq s_t \leq P_t, \quad E_0 = E_T = 6000. \quad (5)$$

所有约束与目标均为线性，共 720 个变量、289 条等式约束，采用稀疏矩阵和 HiGHS 求解。线性规划算法可高效求解该规模问题 [2]。模型未引入充放互斥二元变量，因此解后必须检查同槽同时充放；所得正式解通过该项检查。

5.2 最优策略与题目指定结果

最优全天购电量为 59482.7 kWh，购电费为 35126.95 元，较无储能基线节省 26.9%。电池利用低价时段充电，在高价及光伏不足时段释放电能，全天弃光为 0。题面指定的购电及储能结果列于表 3、表 4。图 1 中的充放电时序同时受峰谷价格和光伏出力影

表 3 问题一指定时段购电量 (kWh)

时间段	购电量
10:00-10:10	0.00
12:00-12:10	486.40
14:00-14:10	0.00
16:00-16:10	394.93
18:00-18:10	636.98
20:00-20:10	0.00
全天购电量	59482.7
全天购电费/元	35126.95

表 4 问题一指定区间储能充放电量 (kWh)

时间段	充电量	放电量
0:00-4:00	4500.00	0.00
4:00-8:00	833.33	6365.84
8:00-12:00	4787.96	1703.00
12:00-16:00	5286.04	91.10
16:00-20:00	0.00	5780.13
20:00-24:00	5333.33	2859.87
0:00 储电量	6000.00	
24:00 储电量	6000.00	

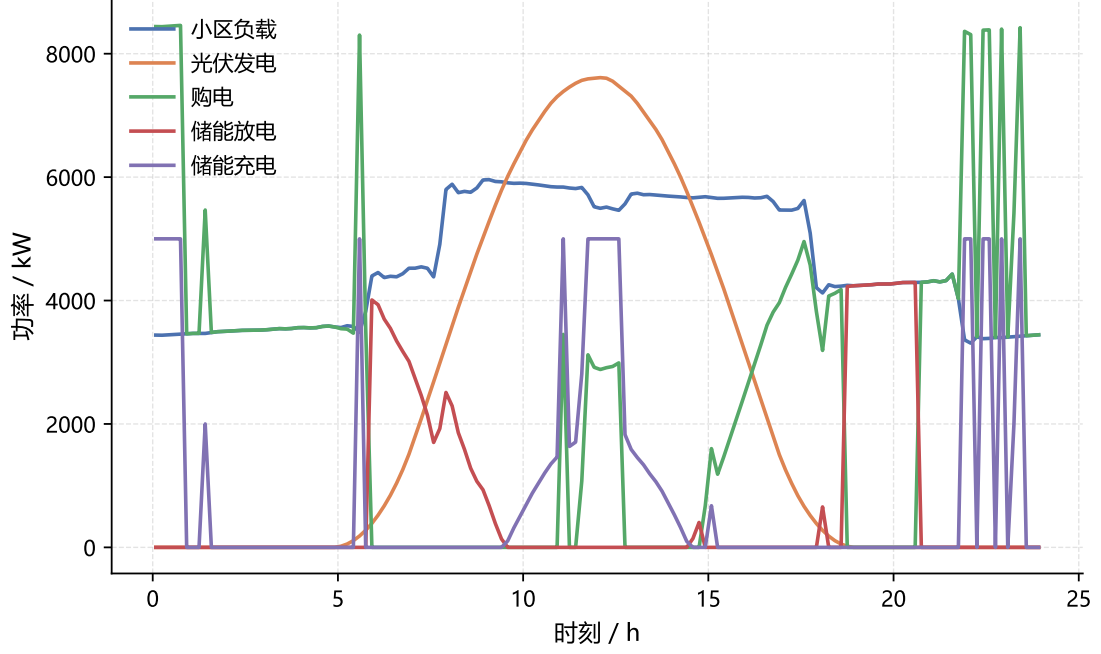


图 1 典型日购电、光伏与储能联合调度

响。由于循环条件约束日末电量，全天节省不是通过消耗初始储能获得的暂时收益。四小时汇总区间内可能同时出现正的累计充电与放电量，这表示不同 10 分钟槽采用不同动作，不意味着同槽同时充放。

本问结论：确定性典型日最优购电费为 35126.95 元，储能节省比例 26.9%；该模型提供后续各问共用的物理约束。

六、 问题二：历史预测与两日滚动购电

6.1 面向费用的历史组合预测

在零点尚不知道当天实际负荷与光伏，因此根据周周期与短期持续性构造

$$\hat{L}_{d,t} = w_{d,1}L_{d-7,t} + w_{d,2}L_{d-14,t} + w_{d,3}\bar{L}_t, \quad (6)$$

$$\hat{P}_{d,t} = u_{d,1}P_{d-1,t} + u_{d,2}P_{d-2,t} + u_{d,3}\bar{P}_t, \quad (7)$$

其中 $\bar{L}_t, \bar{P}_t$ 为题目给定典型日电量，权重非负且各自和为 1。两组三源权重在步长 0.2 的单纯形网格搜索，共 $21 \times 21 = 441$ 个组合。令 $C_j(w, u)$ 为历史日 j 使用该权重作计划并以实际值因果执行后的总费用，递推评分为

$$M_d(w, u) = \rho M_{d-1}(w, u) + (1 - \rho)C_{d-1}(w, u), \quad (8)$$

$$\rho = 2^{-1/h}, \quad (w_d, u_d) = \arg \min_{w, u} M_d(w, u).$$

历史首个可用评分初始化 M ，随后每日只吸收前一日实现费用。半衰期取 $h = 5$ 天。历史成本表采用单日规划、自由末端执行的代理费用，正式运行采用两日滚动，二者不能视为完全相同的优化目标；这种代理降低了逐日权重标定的计算量。

为控制高价短缺，对预测施加

$$\tilde{L}_{d,t} = \kappa \hat{L}_{d,t}, \quad \tilde{P}_{d,t} = [\hat{P}_{d,t} - m\Delta t]_+, \quad \kappa = 1.02, \quad m = 25 \text{ kW}. \quad (9)$$

若忽略储能与跨期耦合，仅考虑计划量 z 和随机净需求 N ，最小化 $pz + 5p\mathbb{E}[(N - z)_+]$ 给出 $F_N(z) = 0.8$ 。这解释了保守修正的方向，但不能据此直接确定含储能模型的最优裕度；具体参数仍由历史开发期费用选择。

6.2 两日计划及次日预测

每日以实际日初电量 $E_{d,0}$ 初始化 288 槽规划，目标为

$$\min \sum_{\tau=1}^{288} \hat{p}_{\tau} x_{\tau} + \varepsilon \sum_{\tau=1}^{288} (c_{\tau} + q_{\tau}), \quad (10)$$

并对预测负荷、光伏施加式 (2)–(4)。微小吞吐量正则项用于减少无意义充放循环，报送购电费不含此数值求解罚项。当日末 E_{144} 和窗口最远端 E_{288} 均不设置等式终值，仅满足容量界限。次日负荷利用 $d+1-7, d+1-14$ 的已发生曲线，当日冻结的光伏权重应用于截至 $d-1$ 的最近两条实际曲线；禁止使用当天尚未发生的光伏来构造次日预测。

6.3 逐槽因果执行

固定当前有效承诺量 x_t ，观察当前实际负荷和光伏，记 $n_t = L_t - P_t - x_t$ 。当 $n_t \geq 0$ 时，

$$q_t = \min\{n_t, \bar{c}, \eta(E_{t-1} - 1200)\}, \quad e_t = n_t - q_t, \quad c_t = s_t = 0. \quad (11)$$

当 $n_t < 0$ 时，

$$c_t = \min\{-n_t, \bar{c}, (10800 - E_{t-1})/\eta\}, \quad s_t = -n_t - c_t, \quad q_t = e_t = 0. \quad (12)$$

然后按式 (3) 更新储电量。这一规则只依赖当前槽及此前信息，不为追赶日末目标而额外紧急购电充能。次日初值为本日实际末值：

$$E_{d+1,0} = E_{d,T}^{\text{exec}}. \quad (13)$$

计划富余不会退费，故实现费用为

$$C_2 = \sum_d \sum_t p_t x_{d,t}^0 + 5 \sum_d \sum_t p_t e_{d,t}. \quad (14)$$

6.4 时间留出选择与总费用

2 月至 6 月为开发期，7 月至 12 月为验证期；后者只报告性能，不用于重新选择全局参数。表 5 显示相邻候选表现接近。所选 $m = 25$ 在开发期最优，在验证期排名第 5； $m = 50$ 的全年描述值略低，也不能据此倒过来改选。因此 334 天汇总包括开发期，不能称为全区间独立样本外测试。正式结果总费用 1395.7 万元，其中计划购电费 1316.0 万元、紧急购电费 79.6 万元；分项独立舍入可能与总额相差 0.1 万元。图 2 给出储能轨迹，日末值随供需变化而变化，验证了跨日状态连续与非固定末端条件。

表 5 问题二部分候选的时间留出结果（万元，均为 $h = 5$ ）

κ	m/kW	开发期	验证期	全区间	验证排名
1.02	25（选定）	574.0	821.6	1395.7	5
1.02	50	574.2	820.6	1394.8	4
1.015	75	574.3	821.7	1396.1	6
1.02	75	575.0	820.3	1395.3	2

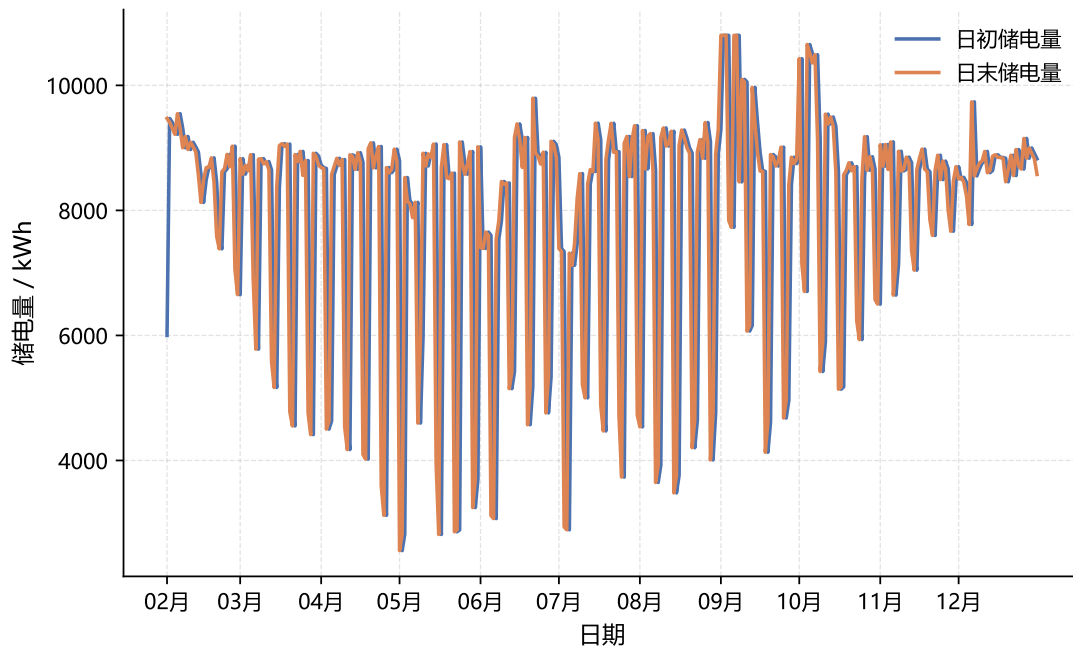


图 2 问题二实际储能电量及日末电量变化

6.5 指定日期结果与年度风险

四个指定日期的计划购电、储能与紧急购电结果见表 6–8。计划购电费与包含紧急费的总费用分别列示，以保持题目结算含义。固定已生成的两日滚动计划，对负荷和光伏联合残差作同月整日块重采样，得到 100 个模拟评价年度。这种模拟是条件于固定计划的离线压力评估，不是重新训练并逐年适应的独立样本外试验。总成本均值 1464 万元，标准差 44.53 万元，P95 为 1542 万元，尾部均值 CVaR95 为 1566 万元。CVaR 刻画超过分位阈值的平均损失 [3]，适合补充均值指标。图 3 表明实际评价区间的 1395.7 万元低于重采样均值，不能把这一实现值直接等同于未来期望成本。

本问结论：基于开发期选择的历史预测与两日滚动策略在 334 天内产生 1395.7 万元费用；实际末端电量自由，风险评价仍显示明显的高费用尾部。

表 6 问题二指定日期计划购电量与购电费

时段或指标	3 月 20 日	6 月 21 日	9 月 23 日	12 月 21 日
10:00-10:10	0.00	0.00	0.00	0.00
12:00-12:10	529.03	0.00	553.22	975.48
14:00-14:10	0.00	0.00	0.00	0.00
16:00-16:10	428.10	169.90	464.46	796.72
18:00-18:10	744.10	534.28	802.45	731.57
20:00-20:10	0.00	0.00	0.00	0.00
全天计划电量/kWh	62281.59	42865.34	69038.87	96267.73
全天计划购电费/元	38733.07	24447.24	42750.14	61810.51
紧急购电费/元	1592.53	0.00	1003.07	83.06
含紧急费总额/元	40325.60	24447.24	43753.21	61893.57

表 7 问题二指定日期储能充放电与首末电量（kWh）

区间	3 月 20 日		6 月 21 日		9 月 23 日		12 月 21 日	
	充电	放电	充电	放电	充电	放电	充电	放电
0:00-4:00	1376.88	122.05	447.73	200.31	2042.44	43.58	2040.21	276.70
4:00-8:00	992.89	4988.27	593.50	1782.09	405.75	6608.26	1608.84	1579.87
8:00-12:00	5356.38	2777.38	7656.06	0.00	5589.93	1896.62	5310.48	7513.53
12:00-16:00	4272.48	534.09	0.00	0.00	5123.65	597.06	7804.74	2276.46
16:00-20:00	313.91	5924.64	242.20	5524.96	213.59	5649.98	0.00	5685.08
20:00-24:00	4089.80	2792.53	8635.18	2570.34	8474.10	2826.51	8405.16	2848.38
0:00 电量	9061.33		5175.11		8709.06		8652.78	
24:00 电量	4780.15		9794.87		8793.57		8883.03	

表 8 问题二指定日期全部紧急购电事件

日期	紧急购电时段	紧急购电量/kWh
03 月 20 日	20:30-20:40	228.2867
06 月 21 日	无	0.0000
09 月 23 日	20:20-20:40	103.1061
09 月 23 日	20:50-21:10	43.2148
09 月 23 日	21:20-21:40	17.9760
12 月 21 日	7:50-8:00	14.2992

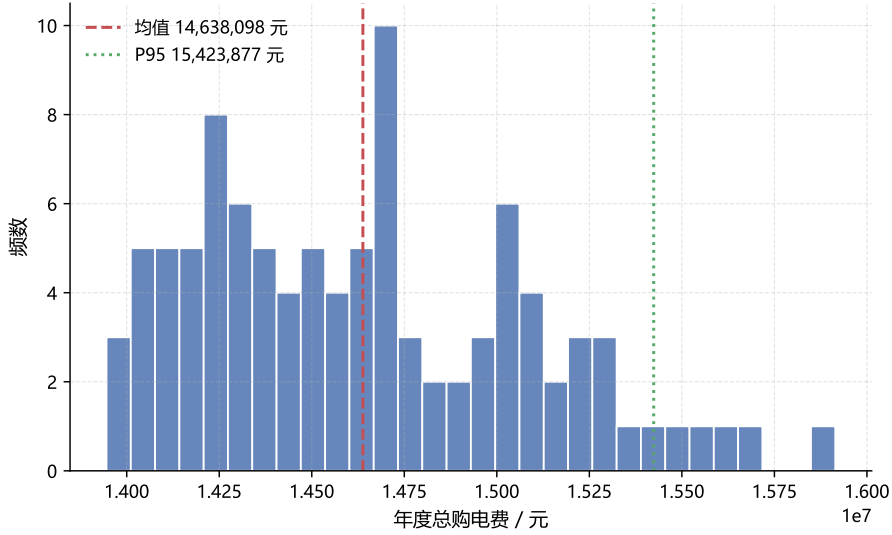


图 3 固定计划下联合残差重采样的总费用分布

七、问题三：组合预报与日内调整

7.1 统一发布相位的组合预报

零点计划层以及各调整时刻 $r \in \{6, 12, 18\}$ 均采用

$$\tilde{P}_{d,t}^{(r)} = \left[\lambda \hat{P}_{d,t}^{\text{off},r} + (1 - \lambda) \hat{P}_{d,t}^{\text{hist}} - m \Delta t \right]_+, \quad \lambda = 0.7. \quad (15)$$

官方预报使用对应发布时刻的曲线，历史项与问题二一致；负荷继续使用因果历史预测和 $\kappa = 1.02$ 修正。第二日尚无对应零点官方预报，使用零点可得的历史外推。采用相同组合形式的目的，是使日内更新仅反映新信息，而不同时改变预测构造。

7.2 调整费用的等价表达

对某个尚未执行的槽，设原计划为 x^0 、最终调整量为 x^a 。当 $x^a \leq x^0$ ，购电加违约费用为 $px^a + 0.5p(x^0 - x^a)$ ；当 $x^a > x^0$ ，费用为 $px^0 + 1.5p(x^a - x^0)$ 。二者统一为

$$g(x^a; x^0) = px^a + 0.5p|x^a - x^0|. \quad (16)$$

因此不能将原计划费用再完整相加，否则会重复计费。引入 $y \leq x^0$, $y \leq x^a$, $y \geq 0$ ，则 $g = 1.5px^a - py + 0.5px^0$ 在最优时成立。最后一项对调整决策是常数，可在求解时省略，但费用统计必须恢复。

7.3 名义调整与日末参考

每次新预报发布后，以当前实际 SOC 初始化剩余时段规划，已执行时段不再修改。零点两日规划产生当日末的参考电量 $E_{d,T}^{\text{ref}} = E_{d,144}^{\text{plan}}$ 。当前日内调整 LP 保留

$$E_{d,T}^{\text{adj}} = E_{d,T}^{\text{ref}} \quad (17)$$

作为规划连接条件；该参考由每日预测、价格与日初状态共同决定，并非恒定 6000 kWh。实际执行仍用式 (11)–(12)，不强制到达此参考。因而必须区分“规划采用动态日末参

考”和“实际日末电量自由”。这种分层连接保留了次日储能需求的部分信息，但日内层尚未直接重算完整两日窗口，属于模型近似。

7.4 联合场景候选扩展

为检验随机规划能否进一步降低短缺风险，先构造一个对冲候选扩展。在 6 时和 12 时，从目标日之前的最近 90 天中选择残差池；其中同月样本达到 14 天时优先使用同月样本。对历史日 $j < d$ ，用其对应发布时刻的名义预测构造联合残差

$$R_{j,t}^L = L_{j,t} - \tilde{L}_{j,t}, \quad R_{j,t}^{P,r} = P_{j,t} - \tilde{P}_{j,t}^{(r)}. \quad (18)$$

同一场景的负荷和光伏取自同一天，并保留整段曲线。每次调整使用 $S = 40$ 个等权情景，包含 1 个名义情景及 39 个有放回重采样情景；两次调整共抽样 78 个残差日索引，并不表示每次有 78 个场景。

不同情景共享调整量 x^a ，各自拥有充放电、弃电、储能与紧急补救变量。对剩余时段集合 $\mathcal{T}_r$ ，求解

$$\min \sum_{t \in \mathcal{T}_r} (1.5p_t x_t^a - p_t y_t) + \frac{1}{S} \sum_{s=1}^S \sum_{t \in \mathcal{T}_r} [5p_t e_t^s + \varepsilon(c_t^s + q_t^s)], \quad (19)$$

约束为各情景下的能量平衡、SOC 动态、容量与功率界限，以及式 (17) 和 y 的线性化约束。目标中的 $1/S$ 不可省略，否则情景数增加等价于人为提高紧急费权重。18 时采用名义剩余时段调整；这是一项计算与信息价值折中，并不意味着 18 时以后完全没有负荷不确定性。

场景内的补救轨迹仅服务于计划评价，其拥有的整段情景信息不等于实际执行可提前获知未来。候选扩展的实现费用也一律用逐槽因果执行回放计算：

$$C_3 = \sum_{d,t} [p_t x_{d,t}^a + 0.5p_t |x_{d,t}^a - x_{d,t}^0| + 5p_t e_{d,t}]. \quad (20)$$

正式方案则不求解式 (19)，在每个发布时刻只按最新组合名义预测调整剩余计划。这样既保留相同的信息结构和动态日末参考，又消除了场景抽样及完整情景补救带来的额外保守性。

7.5 整体表现与指定日期结果

正式主方案 334 天总费用 1347.1 万元，其中计划费 1290.5 万元、调整净额 25.2 万元、紧急费 31.4 万元。此处调整净额定义为 $\sum [g(x^a; x^0) - px^0]$ ，不是绝对偏差费本身。与官方预报无调整基线 1464.5 万元相比，费用降低 8.01%；与官方预报三点调整 1357.5 万元相比降低 0.76%。图 4 说明主要收益来自允许更新和调整，组合预测进一步小幅改善费用。表 9-12 同时列出零点计划与调整后结果，便于区分承诺、实际执行与最终结算。

表 9 问题三指定日期计划购电量与购电费

时段或指标	3 月 20 日	6 月 21 日	9 月 23 日	12 月 21 日
10:00-10:10	0.00	0.00	0.00	0.00
12:00-12:10	567.21	0.00	419.53	966.15
14:00-14:10	0.00	0.00	0.00	0.00
16:00-16:10	618.29	187.98	586.23	811.78
18:00-18:10	706.57	0.00	787.06	735.24
20:00-20:10	0.00	0.00	0.00	0.00
全天计划电量/kWh	64357.79	40036.23	70612.27	96295.77
全天计划购电费/元	39979.30	22282.70	43935.46	61825.34

表 10 问题三指定日期调整购电量与购电费

时段或指标	3 月 20 日	6 月 21 日	9 月 23 日	12 月 21 日
10:00-10:10	0.00	0.00	0.00	0.00
12:00-12:10	567.21	0.00	419.53	999.84
14:00-14:10	0.00	0.00	0.00	0.00
16:00-16:10	520.24	160.44	543.94	822.58
18:00-18:10	706.57	0.00	787.06	735.24
20:00-20:10	0.00	0.00	0.00	0.00
调整后全天电量/kWh	62231.04	40260.15	68561.04	96287.52
买入及偏差费/元	40053.91	22874.05	43321.23	62209.20
紧急购电费/元	20.44	0.00	43.00	74.30
含紧急费总额/元	40074.34	22874.05	43364.23	62283.49

表 11 问题三指定日期储能充放电与首末电量 (kWh)

区间	3 月 20 日		6 月 21 日		9 月 23 日		12 月 21 日	
	充电	放电	充电	放电	充电	放电	充电	放电
0:00-4:00	1292.66	122.05	447.73	200.31	1884.60	43.58	1956.07	276.70
4:00-8:00	774.90	5841.05	525.84	2870.08	424.36	6752.50	1608.84	1579.87
8:00-12:00	4175.67	2777.38	8988.07	0.00	5718.41	1896.62	5457.61	7155.69
12:00-16:00	6164.61	260.50	0.00	0.00	5429.52	500.05	7167.18	2237.06
16:00-20:00	233.47	5454.62	267.54	5481.91	188.48	5720.29	3.13	5687.61
20:00-24:00	4089.80	3017.89	8635.18	2570.34	8474.10	2944.14	8405.16	2848.38
0:00 电量	9137.12		5246.06		8851.11		8728.51	
24:00 电量	4780.15		9865.50		8917.34		8883.03	

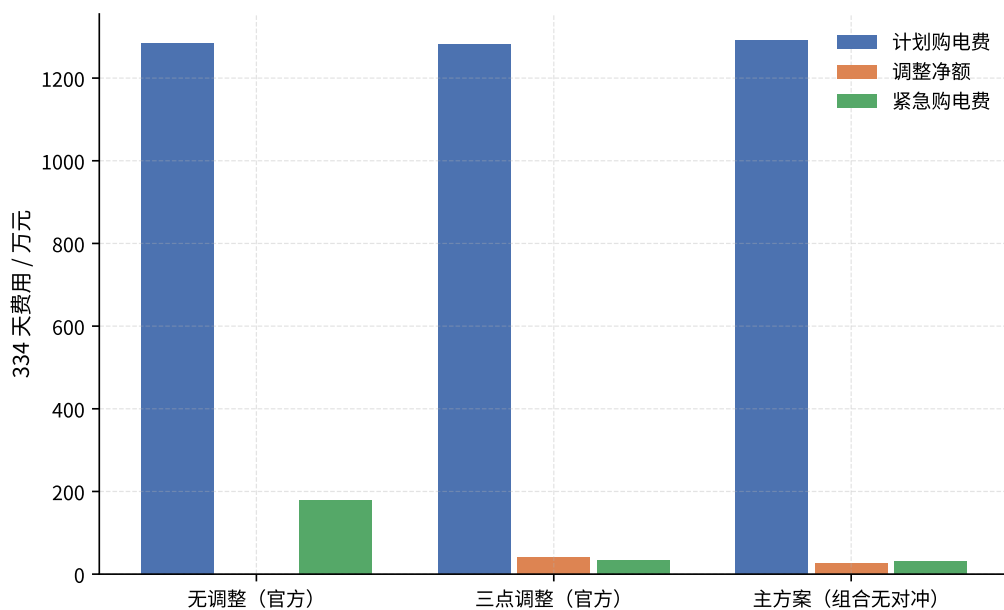


图 4 问题三不同信息与调整策略的费用分解

表 12 问题三指定日期全部紧急购电事件

日期	紧急购电时段	紧急购电量/kWh
3 月 20 日	20:30-20:40	2.9299
6 月 21 日	无	0.0000
9 月 23 日	20:20-20:30	6.3750
12 月 21 日	7:50-8:00	12.7909

7.6 消融检验与其他预报时刻

表 13 表明，组合预报在无对冲结构下比单一官方预报降低 10.4 万元。对冲扩展相对正式方案增加约 2.1 万元，且紧急费与紧急购电量均增加。进一步按预先划分区间核算，开发期对冲仅低 0.20 万元，而冻结验证期无对冲低 2.30 万元；因此这并非只按全年结果事后择优，留出证据同样不支持保留对冲。据此正式方案删除对冲，将其保留为负结果；未来只有在新增独立极端样本并验证 P95 或 CVaR 收益后才考虑恢复。

表 13 组合预报、对冲与信息一致性的消融结果

配置	总费用/万元	紧急费/万元
官方预报、三点调整、无对冲	1357.5	34.1
正式主方案：组合、三点调整、无对冲	1347.1	31.4
候选扩展：组合、三点调整及对冲	1349.2	33.6
计划组合、调整层仅官方且含对冲	1353.0	33.8

对于题目提出的其他发布时刻，现有附件只提供四次预报，无法可靠计算新增时刻的实际收益。可用“新增预报带来的期望紧急费节省，是否超过调整成本与获取成本”作为判据。早晨或中午天气突变时提高预报频率可能有价值，但不能由现有结果宣称某个未观测时刻最优。

本问结论：组合无对冲主方案费用 1347.1 万元，较官方无调整基线下降 8.01%；对冲在冻结验证期与全年均增费，故不进入正式策略。

八、 问题四：波动电价下的两层调度

8.1 给定电价 G 与历史预测 H

主口径 G 直接采用附件 4 的当日及次日电价替换两日规划中的固定价格，负荷和光伏信息范围保持不变。年末没有下一年度价格时，对第二日价格沿用最后可得日的曲线；第二日仅作为窗口延伸，不进入本年度报送费用。该模型评价在题目给定价格条件下能够达到的调度效果，不能将其解释为价格未知时的可实现收益。

为考察电价信息受限情况，扩展 H 采用三源预测

$$\hat{p}_{d,t} = v_{d,1}p_{d-1,t} + v_{d,2}p_{d-7,t} + v_{d,3}\bar{p}_t, \quad v_{d,i} \geq 0, \quad \sum_i v_{d,i} = 1. \quad (21)$$

候选权重按此前 7 天费用选择，再进行 $v_d = 0.1v_d^* + 0.9v_{d-1}$ 平滑。预测次日时，尚未实现的当日价格用最近已观测曲线替代。H 只改变优化时的价格输入，最终计划、偏差和紧急费用仍按实际交易价格结算。

8.2 无调整层与调整层结果

问题四分别沿用问题二与问题三的预测、SOC 与执行结构，仅替换电价。表 14 给出 334 天结果；G 两层费用分别为 1459.8、1410.3 万元，H 分别为 1465.9、1418.4 万元。有调整层的费用由买入、偏差与紧急费用构成，见图 5。日内调整降低了短缺量及

表 14 波动电价下的正式结果与历史价格扩展

指标	无日内调整层	有日内调整层
G 总费用/万元	1459.8	1410.3
G 紧急购电费/万元	92.8	40.5
G 紧急购电量/kWh	155951.8	77378.0
H 总费用/万元	1465.9	1418.4
H 相对 G 费用增幅	0.42%	0.57%

紧急费用；不能将与固定价问题的费用差直接解释成策略改进或退化，因为结算价格环

境已经变化。以 $I_p = (C_H - C_G)/C_G$ 度量条件电价信息价值，得到 0.42% 和 0.57%。该

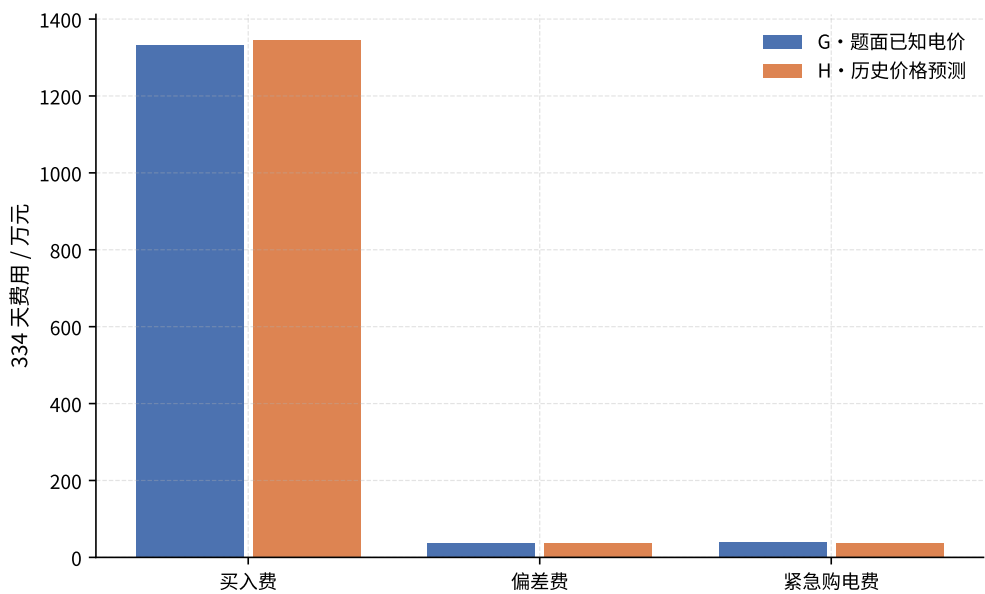


图 5 波动电价调整层在 G、H 两种信息条件下的费用分解

结果说明在这组数据及当前预测策略下，价格预测误差的额外成本较小；它不能推广为所有电价机制下的信息价值都很低。G 仍需面对未知的负荷与光伏，因此也不是所有不确定性消失后的理论最优下界。

8.3 物理可行性与结果校验

G 无调整层和调整层的功率平衡最大残差分别为 4.547×10^{-13} 与 3.411×10^{-13} kWh，SOC 递推最大残差均为 1.137×10^{-13} kWh；储电量落在 1200 至 10800 kWh，同槽同时充放次数为 0。调整层正式方案不使用场景池。完整计划、最终调整、分段充放电与紧急购电事件分别写入题目指定的两份结果表。

本问结论：波动价格条件下，无调整层费用 1459.8 万元，有调整层 1410.3 万元；历史电价预测扩展的额外费用分别为 0.42% 与 0.57%。

九、 灵敏度分析与一致性检验

9.1 已删除对冲扩展的稳定性

为判断否定对冲是否只是某个随机种子的偶然结果，对已退出正式方案的候选扩展改变情景数、种子与历史池。表 15 取自独立灵敏度运行，参考配置为 1349.3 万元；本轮受控消融的对冲扩展为 1349.2 万元。这一 0.1 万元差异不影响其均高于正式无对冲方案 1347.1 万元的判断；由于旧灵敏度缺少未舍入逐日记录，不擅自将差异归因为舍入。40 情景在不同种子下费用变化较小，但各变体并未给出稳定优于 1347.1 万元正式方案的证据。短历史池略增费用，说明季节匹配与样本数量需平衡；该表只解释被删除

表 15 问题三灵敏度运行结果

配置	费用/万元	相对灵敏度参考值
40 情景, 种子 7 (参考)	1349.3	0
5 情景, 种子 7	1350.4	0.08%
10 情景, 种子 7/17/27	1350.6/1350.4/1351.5	0.10/0.08/0.16%
20 情景, 种子 7/17/27	1349.2/1350.1/1349.3	-0.01/0.06/0.00%
40 情景, 种子 17/27	1350.1/1349.0	0.06/-0.02%
历史池回看 30 天	1351.3	0.15%
历史池回看 60 天	1351.0	0.13%
组合权重 $\lambda = 0.5$	1349.1	-0.01%
组合权重 $\lambda = 0.9$	1357.7	0.62%

扩展的行为, 不再用于给正式策略选情景数。 $\lambda = 0.9$ 明显较差, 而 0.5 与 0.7 接近, 支持保留历史预测成分。

9.2 跨问题统一参数的代价

既有问题三参数搜索是在对冲候选结构下完成, 不能直接用于宣称无对冲正式方案的独立最优参数。本轮保持 $\lambda = 0.7, \kappa = 1.02, m = 25$ 不变, 只切换是否对冲, 以保证消融可归因; 其中 κ, m 仍沿用问题二开发期选定值, 便于跨问比较。冻结验证期仅用于判断对冲开关, 没有反过来重选连续参数, 从而避免重复利用验证集。

9.3 信息与约束的分层检验

实际执行按式 (11)–(12) 由当前槽构成, 结构上不读取未来实现值; 残差池检查每个索引严格小于目标日; EWMA 只吸收前一日及更早费用。这些检查验证操作层面的因果性, 但全局超参数使用 2 月至 6 月开发数据, 开发期自身仍是拟合内评价。另需区别场景 LP 的完整情景补救和在线因果控制, 不能把情景目标值直接当作可实现结算值。

五份结果文件均通过工作表名、日期、行数、计划与紧急量汇总检查。问题二和三的跨日 SOC 衔接最大误差为 0, 问题三计划费及计划量回读差为 0; 紧急事件表因小数输出与相邻事件合并产生微小勾稽差, 问题二约 2.300×10^{-4} kWh, 问题三约 1.083×10^{-3} kWh, 远低于实际调度电量尺度。

十、 模型评价与推广

10.1 模型优点

模型把购电结算、储能动态与信息发布时间统一起来。两日滚动使储能能够跨日转移，避免每晚回到固定电量所产生的人为限制；逐槽因果执行使每一步控制可由当时可得信息实现。预测权重按历史费用更新，能响应非对称短缺惩罚；候选联合残差块保留负荷、光伏及日内相关结构，并经消融后未进入正式策略。不同问题共用预测参数、评价区间和初值，有利于比较日内调整与电价信息的影响。

实证结果同时给出主方案、对照、消融与时间留出性能。无对冲更便宜、验证期最优不一定等于开发期最优等负结果被保留，使复杂模块的边际贡献可以检验。

10.2 局限与改进方向

首先，单年数据难以覆盖更广的气象与电价状态，100 次重采样仅是条件压力评估。应增加独立年份或滚动多折验证，不能据本年度费用承诺未来节省。其次，评价区间从 2 月开始，并以 6000 kWh 重新初始化。进一步研究应从题目给定的 1 月 1 日初值连续运行，再提取 2 月至 12 月结果，以排除初始化方式的影响。

第三，历史权重成本表仍以单日规划作代理；日内调整也只优化当日剩余时段，并使用零点规划产生的动态日末参考。将权重标定和日内调整共同迁移到一致的多日滚动目标，可减少层间近似，但需要重新计算正式结果。两日窗口的最远端尚无剩余电量价值函数，也可能出现窗口末端耗尽倾向；延长窗口或使用经验证的终端价值是值得比较的改进方案。

第四，实际执行器只针对当前缺口充放电，未显式考虑未来价格机会成本。被删除的场景扩展采用整段情景补救，与在线执行存在信息差异；可进一步研究多阶段非预见性约束或价格感知的短窗实时控制。最后，当前场景对冲没有表现出冻结验证期或全年降本优势，是否改善新增独立情景下的尾部风险仍需检验；储能衰减和购电容量上限也可在扩展模型中加入。

10.3 结论

典型日优化购电费为 35126.95 元，较无储能节省 26.9%。在统一 334 天评价区间内，历史预测的两日滚动计划费用为 1395.7 万元；加入组合预报与三点调整后，无对冲主方案为 1347.1 万元。波动电价主口径下，两层费用分别为 1459.8 和 1410.3 万元。这些结果支持跨日储能和日内更新的应用价值，同时说明评价复杂调度方法时，应将可执行性、实现费用、尾部风险和假设分别检验。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于建模辅助、程序编写与调试、图表绘制、论文排版与验证，详细使用情况见支撑材料。

参考文献

[1] Mayne D Q, Rawlings J B, Rao C V, Scokaert P O M. Constrained model predictive control: Stability and optimality[J]. Automatica, 2000, 36(6): 789–814. DOI: 10.1016/S0005-1098(99)00214-9.

[2] Huangfu Q, Hall J A J. Parallelizing the dual revised simplex method[J]. Mathematical Programming Computation, 2018, 10(1): 119–142. DOI: 10.1007/s12532-017-0130-5.

[3] Rockafellar R T, Uryasev S. Optimization of conditional value-at-risk[J]. Journal of Risk, 2000, 2(3): 21–41. DOI: 10.21314/JOR.2000.038.

附录 A 复现说明与关键执行步骤

计算使用 Python、NumPy、SciPy 及 HiGHS，结果表通过 OpenPyXL 回读。运行环境由 uv 管理；正式 Q3 无场景随机性，对冲扩展种子为 7、蒙特卡洛种子为 42。复现时先进入随附 Python 工程目录。命令及结果对应关系见表 16，求解中的微小正则项不计入正文购电费用。复现顺序应先形成历史成本表与权重，再执行问题二和问题三，

表 16 核心复现入口

命令	对应内容
uv run python -m solve.q1	典型日调度
uv run python -m solve.q2_tune --result2	问题二正式结果
uv run python -m solve.q3	问题三正式结果
uv run python -m solve.q4_price_fit	历史电价成本标定
uv run python -m solve.q4 --result4-2	波动价格无调整层
uv run python -m solve.q4 --result4-3	波动价格调整层
uv run python -m solve.q3_ablation	正式方案与对冲扩展消融

最后重算电价标定与问题四。同一随机种子有助于比较方案，但线性规划的退化解、求解器版本和浮点阈值仍可能造成小规模数值差异。

```
for day in evaluation_days:
    weights = update_from_past_costs(day)
    forecast = predict_two_days(day, weights)
    plan = optimize_horizon(forecast, actual_soc)
    for slot in current_day:
        if forecast_is_released(slot):
            commitment = adjust_remaining_plan(
                released_forecast, actual_soc, plan)
            load, pv = observe_current_slot(slot)
            charge, discharge, emergency, spill = execute(
                commitment[slot], load, pv, actual_soc)
            actual_soc += eta * charge - discharge / eta
        settle_day_and_export()
```

上述为算法逻辑说明，完整求解、时间映射和回读审计以随附源程序为准。论文指定日期表直接取自正式 Excel，计划量与调整量分别保留；紧急购电表列出全部对应事件，无事件日明确记为零。

附录 B AI 辅助说明

本次论文整理使用 AI 编码助手辅助核对已有结果、组织公式和段落、生成表格与 LaTeX 排版。数值依据为给定附件及随附求解程序输出，文献条目经公开来源核对。此说明记录本次辅助范围，不替代参赛者对题意、模型与最终提交材料的审阅。

附录 C 源程序代码

以下为建模与求解所用的全部源程序（与支撑材料 src/ 目录一致；运行方式见上一节复现说明）。

顶层入口

program/src/solve/__init__.py

```
1 """C 题求解入口（微网与外部电网电力调控策略）。
2
3 在 ``program/`` 目录下运行：
4
5     uv run python -m solve      # 运行问题一（问题二-四后续在此扩展）
6
7 输出约定（工作区根目录）：
8     figures/q1_*.pdf           论文图（矢量 PDF）
9     reports/RESULTS_REPORT.md 结果报告（论文数值唯一来源）
10    code/outputs/*.csv         中间结果与图表数据
11    results/result1.xlsx       官方格式结果文件
12 """
13 from __future__ import annotations
14
15
16 def main() -> None:
17     # 延迟导入：避免 `python -m solve.q1` 先经包初始化再执行子模块时
18     # 触发 runpy 的 "found in sys.modules" 警告。
19     from solve.q1 import run_q1
20
21     run_q1()
22
23
24 if __name__ == "__main__":
25     main()
```

program/src/solve/__main__.py

```
1 """`python -m solve` 入口（转发到 solve.main）。"""
2 from solve import main
3
4 if __name__ == "__main__":
5     main()
```

program/src/solve/common.py

```
1 """C 题求解公共部分：路径解析、储能参数（附录 1）、附件读取。
2
3 运行入口见 ``python -m solve``（在 program/ 目录下执行）。
```

```

4  """
5  from __future__ import annotations
6
7  import os
8  from pathlib import Path
9
10 import pandas as pd
11
12 import program as pm
13
14 # -----
15 # 路径
16 # -----
17 PKG_DIR = Path(__file__).resolve().parents[2] # program/
18 DATA_C = PKG_DIR / "data" / "C" # 附件 1-5 所在目录
19
20
21 def workspace_root() -> Path:
22     """工作区根目录 (figures / reports / code / results 的输出位置)。
23
24     优先级: 环境变量 ``MATHMODEL_ROOT`` > ``program/`` 的上一级 (含 AGENTS.md) > 当前目录。
25     """
26     env = os.environ.get("MATHMODEL_ROOT")
27     if env:
28         return Path(env).resolve()
29     cand = PKG_DIR.parent
30     if (cand / "AGENTS.md").exists():
31         return cand
32     return Path.cwd()
33
34
35 ROOT = workspace_root()
36 RESULTS_DIR = ROOT / "results" # 官方结果文件 (result1.xlsx 等)
37
38 # -----
39 # 附录 1: 储能设备参数 (全文共用; 口径假设见 q1.py 文档)
40 # -----
41 T = 144 # 一天 144 个 10 分钟时段
42 DT_H = 1.0 / 6.0 # 每时段小时数 (10 min)
43 ETA = 0.9 # 充放电效率 (充、放双向计损口径)
44 EO = 6000.0 # 初始储电量 kWh (2025-01-01 0:00)
45 E_MIN, E_MAX = 1200.0, 10800.0
46 P_MAX_KW = 5000.0 # 最大充/放电功率 kW
47 P_MAX_E = P_MAX_KW * DT_H # 每时段最大充/放电量 kWh (= 833.33...)
48
49 # -----
50 # 题目级口径常量 (全文共用)
51 # -----
52 N_DAY = 365 # 全年天数
53 REPORT_START = 31 # 2025-02-01 的日序号 (0 基)
54 EMERG_MULT = 5.0 # 紧急购电价倍数
55 EPS_THROUGHPUT = 1e-3 # LP 唯一化正则项: 执行 LP (元/kWh)
56 EPS_TH = 1e-3 # LP 唯一化正则项: Q3/Q4 计划与调整 LP (元/kWh)
57
58
59 def read_attachment(name: str, sheet: int | str = 0) -> pd.DataFrame:
60     """读取 ``program/data/C`` 下的附件 (如 ``附件1.xlsx``) 。"""
61     return pm.read_table(DATA_C / name, sheet=sheet)

```

program/src/solve/consistency.py

```

1  """项目级模型一致性规范: **唯一参数源**与统一构造 (2026-09-13, v1.4)。
2
3  本模块是整个项目口径参数的唯一定义处; Q1-Q4 的求解脚本必须从这里 import,
4  不得在各自模块中另写常量 (由 ``tests/model_consistency_test.py`` 守卫)。
5
6  统一口径条款 (全文见 ``reports/模型一致性规范.md``):
7
8  1. 预测: 三源自适应凸权重 (负荷: d-7 / d-14 / 典型日; 光伏: d-1 / d-2 / 典型日),

```

```

9     权重按 EWMA 半衰期 ``EWMA_HL`` 由 Q2E 成本表逐日标定（目标日 d 只用 d-1 信息）。
10 2. 风险裕度：负荷抬升 ``KAPPA``、光伏折减 ``MARGIN`` kW（时间留出选定：
11 2-6 月开发期选参，7-12 月冻结验证，不参与选择）。
12 3. 正式 Q3/Q4-3 不使用场景对冲（``Q3_USE_HEDGE=False``）；联合残差情景仅保留为
13 可复现实验扩展，池严格取目标日之前。
14 4. 结构：2 日（288 槽）滚动规划、**规划窗口末端完全自由**；Q2 执行 1 天、
15 Q3/Q4 在 0/6/12/18 决策、执行窗 6 h；储电量跨日连续；逐槽因果执行。
16 5. 起点：2025-02-01（日序号 ``START_DAY``）0:00 以 E0=6000 kWh 起步；1 月仅作预测预热。
17 6. 信息集：Q1 用附件 1；Q2 用附件 1/2（历史自适应预测，不用附件 3 预报）；
18 Q3/Q4 可用附件 3 预报（0:00 计划用组合预测 ·官方 + (1-) ·历史；调整层用最新发布）；
19 Q4 电价：G = 附件 4 已知（题面主口径），H = 历史预测扩展。
20 7. Q3 决策：正式组合权重由 ``Q3_LAM``/``Q3_ADJ_LAM`` 给出，是否对冲只由
21 ``Q3_USE_HEDGE`` 控制；v1.4 固定为关闭。
22 8. Q1 豁免：题面明确单日且要求 0:00 = 24:00，保持日循环；仅共用物理常数
23 （见 ``solve.common``）。
24 """
25 from __future__ import annotations
26
27 import numpy as np
28
29 # -----
30 # 统一参数（唯一来源，调研与选择过程见 reports/ 专题）
31 # -----
32 EWMA_HL = 5.0          # 预测权重标定 EWMA 半衰期（天）
33 KAPPA = 1.02           # 负荷预测抬升系数（全局定稿：Q2 新表留出冠军，Q2+Q3 交叉核对）
34 MARGIN = 25.0          # 光伏预测折减（kW）（同上）
35 N_SCEN = 40            # 对冲/评估等概率情景数
36 SCEN_MIN_SAME_MONTH = 14 # 情景池同月最少天数
37 SCEN_LOOKBACK = 90     # 同月不足时的回看窗口（天）
38 Q3_LAM = 0.7           # Q3 0:00 组合权重（官方预报占比）
39 Q3_ADJ_LAM = 0.7       # Q3 调整层组合权重（保持信息口径一致）
40 Q3_USE_HEDGE = False   # v1.4 正式决策：消融显示对冲增费，故正式方案关闭
41 START_DAY = 31         # 2025-02-01（0 基）；仿真与报送起点
42 PLAN_HORIZON = 288     # 规划窗口槽数（2 日 × 144）
43 EXEC_HOURS_Q2 = 24     # Q2 执行窗口（小时）
44 EXEC_HOURS_Q3 = 6      # Q3/Q4 执行窗口（小时），决策点 0/6/12/18
45 EXEC_POLICY = "free"   # 执行器策略：无段末硬目标（含末段），仅容量/功率约束
46 # 依据（reports/执行器段末目标实验.md，末段修复后重跑）：段末目标会迫使
47 # "紧急购电充能/扣留放电"——中段目标 60 天夏季 +40.3 万（12.2%）、47 天秋季
48 # +8.4 万（4.3%）；末段目标再 +4.6 万（1.6%）/ +0.5 万（0.3%）；
49 # 合计 free 相对 target: -13.6% / -4.5%。
50 HOLDOUT_DEV_START = 31 # 开发期起点（2-1）
51 HOLDOUT_VAL_START = "2025-07-01" # 冻结验证期起点
52 WARM_FROM = 14         # EWMA 预热起始日（d-14 与成本表最早可用日）
53
54 # 历史对照参数（仅用于复现旧口径，禁止进入正式结果）
55 LEGACY = {
56     "EWMA_W7": 7.0,     # 旧官方标定窗口
57     "BETA_SMOOTH": 0.1, # 旧参数平滑（Q3 原型）
58     "N_SCEN_OLD": 10,   # 旧情景数
59     "KAPPA_OLD": 1.02,  # 旧调优参数
60     "MARGIN_OLD": 50.0,
61 }
62
63 # -----
64 # 统一构造工具
65 # -----
66 def ewma_weights_from_table(C, grid, hl: float = EWMA_HL,
67                             first_target: int = START_DAY,
68                             warm_from: int = WARM_FROM,
69                             n_days: int = 365) -> dict:
70     """由 Q2E 成本表 ``C`` 生成逐日 EWMA 标定权重（无前视）。
71
72     目标日 d 的记分矩阵 M = 指数衰减加权 [warm_from, d-1] 的成本表行；
73     返回 ``{d: (w, u)}``（w 为负荷权重、u 为光伏权重，均凸、和=1）。
74     """
75     decay = float(np.exp(-np.log(2.0) / hl))
76     M = None

```

```

78     for k in range(warm_from, first_target - 1):
79         M = C[k].copy() if M is None else decay * M + (1.0 - decay) * C[k]
80     out: dict[int, tuple[np.ndarray, np.ndarray]] = {}
81     for d in range(first_target, n_days):
82         M = C[d - 1].copy() if M is None else decay * M + (1.0 - decay) * C[d - 1]
83         i, j = np.unravel_index(np.argmin(M), M.shape)
84         out[d] = (grid[i].copy(), grid[j].copy())
85     return out
86
87
88 def joint_residuals(load_act, pv_act, load_nom, pv_nom):
89     """(Δ负荷, Δ光伏) 整日联合残差块: 实际 - 名义 (同单位, kWh/槽)。"""
90     RL = np.asarray(load_act, dtype=float) - np.asarray(load_nom, dtype=float)
91     RP = np.asarray(pv_act, dtype=float) - np.asarray(pv_nom, dtype=float)
92     return RL, RP
93
94
95 def scenario_indices(months, target_day: int,
96                     pool_min_same_month: int = SCEN_MIN_SAME_MONTH,
97                     lookback: int = SCEN_LOOKBACK) -> np.ndarray:
98     """无前视情景池 (同月优先; 不足回看 ``lookback`` 天)。所有日序号 < target_day。"""
99     months = np.asarray(months)
100     past = np.arange(max(WARM_FROM, target_day - lookback), target_day, dtype=int)
101     same = past[months[past] == months[target_day]]
102     pool = same if len(same) >= pool_min_same_month else past
103     if not len(pool) or int(pool.max()) >= target_day:
104         raise RuntimeError(f"情景池因果性校验失败: D={target_day}, pool={pool.tolist()}")
105     return pool
106
107
108 def kappa_load(load_kw: np.ndarray, kappa: float = KAPPA) -> np.ndarray:
109     """负荷名义预测:  $\cdot L$  (裁剪非负)。"""
110     return np.clip(np.asarray(load_kw, dtype=float) * kappa, 0.0, None)
111
112
113 def margin_pv(pv_kw: np.ndarray, margin: float = MARGIN) -> np.ndarray:
114     """光伏名义预测:  $\max(P - m, 0)$ 。"""
115     return np.clip(np.asarray(pv_kw, dtype=float) - margin, 0.0, None)

```

program/src/solve/q1.py

```

1  """C 题 问题一: 典型日购电 - 储能连续 LP 调度。
2
3  数学模型 (连续线性规划;  $t = 1..144$  个 10 分钟时段):
4
5      min     $E_t p_t \cdot x_t$                 (全天购电费)
6      s.t.    $P_t/6 + x_t + d_t = L_t/6 + c_t + s_t$     (功率平衡;  $s_t 0$  为弃光)
7             $E_t = E_{t-1} + \cdot c_t - d_t/$           (储能动态;  $=0.9$ )
8             $E_0 = E_{144} = 6000$                     (储能电量循环回归)
9             $0 \leq x_t; 0 \leq c_t, d_t \leq 833.33$     (购电非负; 充放功率上限)
10            $1200 \leq E_t \leq 10800$                 (储电量安全区间)
11
12  变量:  $x$  (购电)、 $c$  (充电)、 $d$  (放电)、 $s$  (弃光)、 $E$  (时段末储电量), 共  $5 \times 144$  个, 全部连续。
13  同槽「充放互斥」无需 0-1 变量: 若  $c_t, d_t > 0$ , 同时减 可使储电量增加  $(1/-) > 0$ ,
14  其余约束不变 —— 连续松弛的最优解可取到无同充同放的解。
15
16  口径说明 (论文假设):
17  1. 附件标签为时段右端点; 附件标签与附件 5 结果模板相差一个 10 分钟刻度 (附件第 60 行为 10:00,
18     模板第 60 行为 10:00-10:10)。数据按行序号对齐 (附件第  $k$  槽 模板第  $k$  行 论文第  $k$  时段),
19     结果文件沿用模板标签, 模板标签仅作行标识;
20  2. 充电乘、放电除 (往返 81%; 另有「仅充电计损」口径, 可切换 ETA_D 做敏感性);
21  3. 允许弃光 ( $s_t = 0$ ), 不允许向外部电网售电 ( $x_t = 0$ )。
22  """
23
24  from __future__ import annotations
25
26  from pathlib import Path
27
28  import numpy as np

```

```

29 import openpyxl
30 import pandas as pd
31
32 import program as pm
33 from solve.common import (
34     DATA_C,
35     EO,
36     E_MAX,
37     E_MIN,
38     ETA,
39     P_MAX_E,
40     RESULTS_DIR,
41     ROOT,
42     T,
43     read_attachment,
44 )
45 from solve.core.lp import plan_lp
46
47 N_VAR = 5 * T # 5 类变量 × 144 时段
48
49
50 # -----
51 # 建模与求解
52 # -----
53 def build_lp(price: np.ndarray, load_kwh: np.ndarray, pv_kwh: np.ndarray):
54     """构建并求解问题一连续 LP, 返回 ``(OptResult, 解字典)``。
55
56     复用统一计划 LP (core.lp.plan_lp, 日循环 E(0)=E(144)=EO, eps=0) ;
57     所有能量量纲为 kWh/时段, 功率量纲为 kW (乘 1/6 转 kWh)。
58     """
59     sol, res = plan_lp(price, load_kwh, pv_kwh, EO, EO, eps=0.0)
60     return res, sol
61
62
63 # -----
64 # 校验
65 # -----
66 def verify(sol: dict, load_kwh: np.ndarray, pv_kwh: np.ndarray) -> dict:
67     """逐项校验功率平衡、储能动态、边界与同充同放 (返回指标 dict)。"""
68     x, c, d, s, E = (sol[k] for k in ("x", "c", "d", "s", "E"))
69     balance_res = float(np.max(np.abs(x + d - c - s - (load_kwh - pv_kwh))))
70     dyn_res = float(
71         np.max(np.abs(np.diff(np.concatenate([[EO], E])) - (ETA * c - d / ETA)))
72     )
73     return {
74         "功率平衡最大残差/kWh": balance_res,
75         "储能动态最大残差/kWh": dyn_res,
76         "储电量下限/kWh": float(max(0.0, E_MIN - E.min())),
77         "储电量上限/kWh": float(max(0.0, E.max() - E_MAX)),
78         "充放电越功率限/kWh": float(max(0.0, c.max() - P_MAX_E, d.max() - P_MAX_E)),
79         "0:00 储电量/kWh": float(EO),
80         "24:00 储电量/kWh": float(E[-1]),
81         "同槽同时充放最大值/kWh": float(np.max(np.minimum(c, d))),
82         "弃光总量/kWh": float(s.sum()),
83         "购电量最小值/kWh": float(x.min()),
84     }
85
86
87 # -----
88 # 图表
89 # -----
90 def make_figures(
91     sol: dict,
92     price: np.ndarray,
93     load_kw: np.ndarray,
94     pv_kw: np.ndarray,
95     baseline_cost: float,
96     cost: float,
97 ) -> None:

```

```

98     """生成问题一论文图 (PDF 存 figures/, 作图数据存 code/outputs/figure_data/)。"""
99     x, c, d, E = (sol[k] for k in ("x", "c", "d", "E"))
100     t_h = (np.arange(T) + 0.5) / 6.0 # 时段中点 (小时)
101
102     pm.line(
103         t_h,
104         [load_kw, pv_kw, x * 6.0, d * 6.0, c * 6.0],
105         labels=["小区负载", "光伏发电", "购电", "储能放电", "储能充电"],
106         xlabel="时刻 / h",
107         ylabel="功率 / kW",
108         save="Q1_调度时序",
109     )
110     pm.dual_axis(
111         t_h,
112         E,
113         price,
114         labels=("储电量", "电价"),
115         xlabel="时刻 / h",
116         ylabel1="储电量 / kWh",
117         ylabel2="电价 / (元 * kWh$^{-1})$",
118         save="Q1_储电量与电价",
119     )
120     pm.bar(
121         ["无储能基线", "储能最优调度"],
122         [baseline_cost, cost],
123         ylabel="全天购电费 / 元",
124         value_labels=True,
125         save="Q1_费用对比",
126     )
127
128
129 # -----
130 # 结果文件
131 # -----
132 def write_result1(sol: dict) -> Path:
133     """按附件 5 模板填写 ``result1.xlsx`` (计划购电量 + 充放电量), 输出到 results/。"""
134     x, c, d, E = (sol[k] for k in ("x", "c", "d", "E"))
135     wb = openpyxl.load_workbook(DATA_C / "附件5" / "result1.xlsx")
136
137     ws = wb["计划购电量"] # 144 行, 与附件同时段序号对齐
138     for t in range(T):
139         ws.cell(row=2 + t, column=2, value=float(x[t]))
140
141     ws = wb["充放电量"] # 6 个 4 小时块 + 两端储电量
142     for b in range(6):
143         ws.cell(row=2 + b, column=2, value=float(c[24 * b : 24 * (b + 1)].sum()))
144         ws.cell(row=2 + b, column=3, value=float(d[24 * b : 24 * (b + 1)].sum()))
145     ws.cell(row=2, column=5, value=float(E[0])) # 0:00 储电量
146     ws.cell(row=3, column=5, value=float(E[-1])) # 24:00 储电量
147
148     RESULTS_DIR.mkdir(parents=True, exist_ok=True)
149     out = RESULTS_DIR / "result1.xlsx"
150     wb.save(out)
151     wb.close()
152     return out
153
154
155 # -----
156 # 主流程
157 # -----
158 def run_q1() -> dict:
159     """问题一: 读数据 → 建模求解 → 校验 → 记录 → 出图 → 写 result1.xlsx。"""
160     pm.init(root=str(ROOT))
161     log = pm.get_logger("q1")
162
163     df = read_attachment("附件1.xlsx")
164     label = df["时间"].astype(str).to_numpy()
165     price = df["电价"].to_numpy(dtype=float)
166     load_kw = df["小区负载"].to_numpy(dtype=float)

```

```

167 pv_kw = df["光伏发电预测功率"].to_numpy(dtype=float)
168 load_kwh, pv_kwh = load_kw / 6.0, pv_kw / 6.0
169
170 with pm.Timer("问题一连续 LP 求解", logger=log):
171     res, sol = build_lp(price, load_kwh, pv_kwh)
172
173 x, c, d, s, E = (sol[k] for k in ("x", "c", "d", "s", "E"))
174 cost = float(price @ x)
175 baseline = float(np.sum(price * np.maximum(load_kwh - pv_kwh, 0.0)))
176
177 # ---- 记录: 数据概览 ----
178 pm.record_result(
179     "问题一 输入数据概览",
180     {
181         "时段数": T,
182         "电价范围/(元/kWh)": f"{price.min():.4f} ~ {price.max():.4f}",
183         "负载范围/kW": f"{load_kw.min():.1f} ~ {load_kw.max():.1f}",
184         "光伏出力范围/kW": f"{pv_kw.min():.1f} ~ {pv_kw.max():.1f}",
185     },
186     note="来源: 附件 1 (典型日: 电价 / 小区负载 / 光伏发电预测功率, 10 分钟粒度)。",
187 )
188
189 # ---- 记录: 结果总览 ----
190 pm.record_result(
191     "问题一 结果总览",
192     {
193         "全天购电量/kWh": float(x.sum()),
194         "全天购电费/元": cost,
195         "无储能基线购电费/元": baseline,
196         "储能节省/元": baseline - cost,
197         "储能节省比例/%": 100.0 * (baseline - cost) / baseline,
198         "总充电量/kWh": float(c.sum()),
199         "总放电量/kWh": float(d.sum()),
200         "弃光总量/kWh": float(s.sum()),
201         "储电量最小值/kWh": float(E.min()),
202         "储电量最大值/kWh": float(E.max()),
203     },
204     note=(
205         "方法: 连续线性规划 (HiGHS, %d 变量、%d 条等式约束), 目标为全天购电费最小;"
206         "储能充放电效率 %.2f (充放双向计损), E(0:00)=E(24:00)=%.0f kWh; 弃光松弛 s 0。"
207         "对照基线: 无储能时逐时段购电 max(负载-光伏, 0)。"
208         "图表: figures/Q1_调度时序.pdf、Q1_储电量与电价.pdf、Q1_费用对比.pdf;"
209         "完整时段表: code/outputs/q1_schedule.csv。" % (N_VAR, 2 * T + 1, ETA, EO)
210     ),
211 )
212
213 # ---- 记录: 表 1 (指定 10 分钟窗口) ----
214 windows = [10, 12, 14, 16, 18, 20] # 表 1 要求的整点窗口
215 w_idx = [6 * h - 1 for h in windows] # 按序号对齐口径的槽序号
216 table1 = pd.DataFrame(
217     {
218         "时间段": [f"{h}:00-{h}:10" for h in windows],
219         "购电量/kWh": [float(x[i]) for i in w_idx],
220     }
221 )
222 pm.record_result(
223     "问题一 表1 指定时段购电量",
224     table1,
225     note=(
226         "口径: 数据按行序号对齐 (附件第 k 槽 附件 5 第 k 行 论文第 k 时段); 附件标签为时段"
227         "右端点, 附件与结果模板标签相差一个 10 分钟刻度 (10:00-10:10 第 60 个时段), 模板标签"
228         "仅作行标识。"
229         "全天购电量 %.1f kWh, 全天购电费 %.2f 元。" % (x.sum(), cost)
230     ),
231 )
232 pm.save_outputs(table1, "q1_table1_buy")
233
234 # ---- 记录: 表 2 (4 小时块充放电) ----
235 table2 = pd.DataFrame(

```

```

236     {
237         "时间段": [f"{4 * b}:00-{4 * (b + 1)}:00" for b in range(6)],
238         "充电量/kWh": [float(c[24 * b : 24 * (b + 1)].sum()) for b in range(6)],
239         "放电量/kWh": [float(d[24 * b : 24 * (b + 1)].sum()) for b in range(6)],
240     }
241 )
242 pm.record_result(
243     "问题一 表2 储能充放电",
244     table2,
245     note=f"0:00 储电量 = {E0:.1f} kWh, 24:00 储电量 = {E[-1]:.1f} kWh (相等, 满足循环约束)。",
246 )
247 pm.save_outputs(table2, "q1_table2_storage")
248
249 # ---- 记录: 约束与一致性校验 ----
250 checks = verify(sol, load_kwh, pv_kwh)
251 pm.record_result(
252     "问题一 约束与一致性校验",
253     checks,
254     note=(
255         "残差均为数值误差量级 ( $10^{-3}$ ) ; 储电量全程在 [1200, 10800] kWh 内; "
256         "无同槽充放、无弃光、购电量非负。"
257     ),
258 )
259
260 # ---- 中间数据: 完整时段调度表 ----
261 schedule = pd.DataFrame(
262     {
263         "时段": np.arange(1, T + 1),
264         "时间标签": label,
265         "电价/(元/kWh)": price,
266         "负载/kW": load_kw,
267         "光伏/kW": pv_kw,
268         "购电量/kWh": x,
269         "充电量/kWh": c,
270         "放电量/kWh": d,
271         "弃光量/kWh": s,
272         "储电量/kWh": E,
273     }
274 )
275 pm.save_outputs(schedule, "q1_schedule")
276
277 # ---- 图 ----
278 make_figures(sol, price, load_kw, pv_kw, baseline, cost)
279
280 # ---- 官方结果文件 ----
281 out = write_result1(sol)
282
283 log.info(
284     "问题一完成: 购电量 {:.1f} kWh, 购电费 {:.2f} 元 (无储能 {:.2f} 元, 节省 {:.1f}%); result1.xlsx + {}",
285     x.sum(),
286     cost,
287     baseline,
288     100.0 * (baseline - cost) / baseline,
289     out,
290 )
291 return {"cost": cost, "baseline": baseline, "sol": sol, "checks": checks}
292
293
294 if __name__ == "__main__":
295     run_q1()

```

program/src/solve/q2.py

```

1 """C 题 问题二: 日前计划 + 紧急购电 (口径 D) + 年度成本概率分布。
2
3 口径 (论文假设, 常量区可调):
4 - 计划: 每天 0:00 做 48 小时滚动计划 (core.lp.plan_horizon, 视野末端完全自由),
5   只执行当日; 计划购电量 take-or-pay, 储能日末 SOC 自由并跨日传递;
6 - 执行: 逐槽因果执行 (core.causal.exec_segment_causal, 统一 free 策略,

```

```

7   不读取未来实际值; core.lp.exec_day 事后 LP 仅作前视下界), 缺口按 5 倍电价紧急购电;
8   - 结果自 2025-02-01 起报送。
9
10  概率扩展: 以附件 3 的"整日原始残差块(预报-实际)"重采样, 蒙特卡洛评估年度
11  总成本分布; 对冲决策的场景池严格限于目标日之前, 离线风险评估则使用同月全年样本。
12
13  本模块保留既有入口与公开 API (兼容门面):
14      uv run python -m solve.q2          # 全年滚动 + 100 年蒙特卡洛 + 对冲对照
15      uv run python -m solve.q2 --write-result2-d  # 口径 D 对照写 result2_D_backup.xlsx
16
17  分层结构: 核心 LP/执行在 solve.core, 数据在 solve.data, 年度流程在 solve.flows,
18  结果表与校验在 solve.io; 本文件只做编排(run_q2)与出图(make_figures)。
19  """
20  from __future__ import annotations
21
22  import time
23
24  import numpy as np
25  import pandas as pd
26
27  import program as pm
28  from solve.common import (
29      DATA_C,
30      EO,
31      E_MAX,
32      E_MIN,
33      EMERG_MULT,
34      EPS_TH,
35      EPS_THROUGHPUT,
36      ETA,
37      N_DAY,
38      P_MAX_E,
39      REPORT_START,
40      RESULTS_DIR,
41      ROOT,
42      T,
43  )
44  from solve.core.causal import (
45      CAUSAL_POLICY_VERSION,
46      exec_day_causal,
47      exec_segment_causal,
48      run_exec,
49  )
50  from solve.core.lp import exec_day, plan_day, plan_horizon
51  from solve.core.slots import _events, _fmt_time, _hour_to_slots
52  from solve.data.attachments import load_all
53  from solve.flows.q2_year import hedge_day, run_deterministic, run_hedge, run_mc
54  from solve.io.checks import verify
55  from solve.io.excel import write_result2
56  from solve.io.figures import month_axis
57  from solve.models.errors import scenario_from_residual as _scenario_from_residual
58
59
60  def make_figures(dates, recs, mc_costs, plan_cost_report):
61      """Q2 论文图: 逐日紧急购电量 + 年度总费用分布。
62
63      上图: 每日紧急购电量(kWh), 按月份刻度; 作图数据同步 figure_data/*.csv。
64      下图: 年度总购电费直方图(计划费 + 蒙特卡洛紧急费), 标均值与 P95。
65      """
66      import matplotlib.pyplot as plt
67
68      rep = list(range(REPORT_START, N_DAY))
69      emerg_daily = np.array([recs[d]["emerg_kwh"] for d in rep])
70      fig, ax = pm.line(np.arange(len(rep)), emerg_daily,
71                      xlabel="日期", ylabel="紧急购电量 / kWh")
72      month_axis(ax, [dates[d] for d in rep])
73      pm.save_fig(fig, "Q2_逐日紧急购电",
74                  data=pd.DataFrame({"日期": [dates[d] for d in rep],
75                                   "紧急购电量_kWh": emerg_daily}))

```

```

76
77 if mc_costs is not None and len(mc_costs):
78     total = plan_cost_report + mc_costs
79     fig2, ax2 = plt.subplots(figsize=(7, 4.3))
80     ax2.hist(total, bins=30, color="#4C72B0", alpha=0.85, edgecolor="white")
81     ax2.axvline(total.mean(), color="#C44E52", ls="--",
82                 label=f"均值 {total.mean():.0f} 元")
83     ax2.axvline(np.percentile(total, 95), color="#55A868", ls=":",
84                 label=f"P95 {np.percentile(total, 95):.0f} 元")
85     ax2.set_xlabel("年度总购电费 / 元")
86     ax2.set_ylabel("频数")
87     ax2.legend()
88     pm.save_fig(fig2, "Q2_总费用分布",
89                 data=pd.DataFrame({"年度总购电费_元": total}))
90
91
92 # -----
93 # 主流程
94 # -----
95 def run_q2(n_mc: int = 100, seed: int = 42, hedge: bool = True,
96           write_result2_file: bool = False) -> dict:
97     """口径 D 主流程: 滚动 → MC → 对冲对照 → 校验 → 记录 → 出图。"""
98     pm.init(root=str(ROOT))
99     log = pm.get_logger("q2")
100    data = load_all()
101
102    t0 = time.time()
103    x_plans, recs = run_deterministic(data)
104    log.info("确定性滚动完成 (365 天, 用时 {:.1f}s)", time.time() - t0)
105
106    rep = list(range(REPORT_START, N_DAY))
107    plan_cost = sum(recs[d]["plan_cost"] for d in rep)
108    emerg_cost = sum(recs[d]["emerg_cost"] for d in rep)
109    emerg_kwh = sum(recs[d]["emerg_kwh"] for d in rep)
110    spill_kwh = sum(recs[d]["spill_kwh"] for d in rep)
111    n_days = sum(1 for d in rep if recs[d]["emerg_kwh"] > 1e-3)
112    max_day = max(rep, key=lambda d: recs[d]["emerg_kwh"])
113
114    pm.record_result(
115        "问题二 (口径D) 确定性结果 (2025-02-01 ~ 12-31)",
116        {
117            "计划购电量/kWh": float(x_plans[rep].sum()),
118            "计划购电费/元": float(plan_cost),
119            "紧急购电量/kWh": float(emerg_kwh),
120            "紧急购电费/元": float(emerg_cost),
121            "发生紧急购电的天数": n_days,
122            "单日最大紧急购电量/kWh": float(recs[max_day]["emerg_kwh"]),
123            "全程弃电量/kWh": float(spill_kwh),
124            "日初=日末储电量/kWh": float(recs[REPORT_START]["E_start"]),
125            "日内储电量最小值/kWh": float(min(recs[d]["E"].min() for d in rep)),
126            "日内储电量最大值/kWh": float(max(recs[d]["E"].max() for d in rep)),
127        },
128        note=(
129            "口径 D: 0:00 用附件 3 的 0:00 光伏预报制定计划 (负荷视为已知), "
130            "48 小时滚动视野、只执行首日, 储能日末 SOC 自由并跨日传递; "
131            "计划购电 take-or-pay, 实际按附件 2 执行, 缺口按 5 倍交易时刻电价紧急购电; "
132            "结果自 2 月 1 日报送; 图中位置: figures/Q2_逐日紧急购电.pdf."
133        ),
134    )
135
136    checks = verify(data, x_plans, recs)
137    pm.record_result("问题二 约束与一致性校验", checks,
138                    note="残差为数值误差量级即可; 储电量须在 [1200, 10800] kWh 内。")
139
140    daily = pd.DataFrame({
141        "日期": [data["dates"][d] for d in rep],
142        "计划购电量/kWh": [float(x_plans[d].sum()) for d in rep],
143        "计划购电费/元": [recs[d]["plan_cost"] for d in rep],
144        "紧急购电量/kWh": [recs[d]["emerg_kwh"] for d in rep],

```

```

145     "紧急购电费/元": [recs[d]["emerg_cost"] for d in rep],
146     "弃电量/kWh": [recs[d]["spill_kwh"] for d in rep],
147     "日初储电量/kWh": [recs[d]["E_start"] for d in rep],
148     "日末储电量/kWh": [recs[d]["E_end"] for d in rep],
149 })
150 pm.save_outputs(daily, "q2_daily_summary")
151
152 if write_result2_file:
153     # 口径 D 仅为使用附件 3 的额外信息对照；正式 result2 由 q2_tune 生成。
154     out = write_result2(
155         data["dates"], data["price"], x_plans, recs,
156         out_name="result2_D_backup.xlsx",
157     )
158     log.info("口径 D 对照结果 -> {}", out)
159
160 mc_costs = None
161 if n_mc > 0:
162     t1 = time.time()
163     e_targets = np.array([r["E_end"] for r in recs], dtype=float)
164     e_starts = np.array([r["E_start"] for r in recs], dtype=float)
165     mc_costs, _mc_kwhs = run_mc(data, x_plans, recs[REPORT_START]["E_start"], e_targets,
166                               n_years=n_mc, seed=seed)
167     log.info("蒙特卡洛完成 ({} 年, 用时 {:.1f}s)", n_mc, time.time() - t1)
168     total = plan_cost + mc_costs
169     p95 = float(np.percentile(total, 95))
170     pm.record_result(
171         "问题二 年度总成本分布 (蒙特卡洛)",
172         {
173             "模拟年数": n_mc,
174             "年度总成本均值/元": float(total.mean()),
175             "年度总成本标准差/元": float(total.std()),
176             "P5/元": float(np.percentile(total, 5)),
177             "P50/元": float(np.percentile(total, 50)),
178             "P95/元": p95,
179             "CVaR95 (尾部均值) /元": float(total[total >= p95].mean()),
180             "年度紧急费用均值/元": float(mc_costs.mean()),
181             "年度紧急费用P95/元": float(np.percentile(mc_costs, 95)),
182         },
183         note=(
184             "固定计划、离线重采样附件 3 同月整日原始残差块 (预报-实际, 保留日内相关); "
185             "计划费固定, 分布差异来自紧急购电; 此分布只作事后风险评价, 不参与在线决策; "
186             "图 figures/Q2_总费用分布.pdf。"
187         ),
188     )
189     pm.save_outputs(
190         pd.DataFrame({"计划费_元": plan_cost, "紧急费_元": mc_costs,
191                     "总费用_元": plan_cost + mc_costs}),
192         "q2_mc_annual_costs",
193     )
194
195 if hedge:
196     import matplotlib.pyplot as plt
197
198     t2 = time.time()
199     x_hedge = run_hedge(data, e_starts, e_targets, n_scen=20, seed=7)
200     log.info("对冲计划求解完成 (用时 {:.1f}s)", time.time() - t2)
201     plan_cost_h = float(sum(float(data["price"] @ x_hedge[d]) for d in rep))
202     mc_h, _ = run_mc(data, x_hedge, recs[REPORT_START]["E_start"], e_targets,
203                     n_years=n_mc, seed=seed)
204     total_h = plan_cost_h + mc_h
205     p95h = float(np.percentile(total_h, 95))
206     pm.record_result(
207         "问题二 对冲计划 vs 朴素计划 (蒙特卡洛对比)",
208         {
209             "朴素: 计划购电费/元": float(plan_cost),
210             "朴素: 紧急费均值/元": float(mc_costs.mean()),
211             "朴素: 总成本均值/元": float(total.mean()),
212             "朴素: 总成本P95/元": p95,
213             "对冲: 计划购电费/元": plan_cost_h,

```

```

214         "对冲: 紧急费均值/元": float(mc_h.mean()),
215         "对冲: 总成本均值/元": float(total_h.mean()),
216         "对冲: 总成本P95/元": p95h,
217         "期望费用下降/元": float(total.mean() - total_h.mean()),
218         "期望费用下降/%": float(100 * (total.mean() - total_h.mean()) / total.mean()),
219         "P95 下降/元": float(p95 - p95h),
220     },
221     note=(
        "对冲计划 = 两阶段场景 LP (每日 20 个等概率原始残差情景) 最小化期望总费用; "
        "目标日场景只取此前 90 日、同月样本不少于 14 日时优先同月; "
        "两套计划用同一种子作离线评估。"),
222 )
223
224 pm.save_outputs(
225     pd.DataFrame({"朴素_总费用_元": total, "对冲_总费用_元": total_h,
226                  "朴素_紧急费_元": mc_costs, "对冲_紧急费_元": mc_h}),
227     "Q2_对冲费用分布",
228 )
229
230 fig3, ax3 = plt.subplots(figsize=(7, 4.3))
231 ax3.hist(total, bins=25, alpha=0.55, color="#4C72B0", label="朴素计划")
232 ax3.hist(total_h, bins=25, alpha=0.55, color="#C44E52", label="对冲计划")
233 ax3.set_xlabel("年度总购电费 / 元")
234 ax3.set_ylabel("频数")
235 ax3.legend()
236 pm.save_fig(fig3, "Q2_对冲费用分布",
237             data=pd.DataFrame({"朴素_总费用_元": total,
238                               "对冲_总费用_元": total_h}))
239
240 make_figures(data["dates"], recs, mc_costs, float(plan_cost))
241 log.info("问题二完成: 计划购电费 {:.0f} 元, 紧急购电费 {:.0f} 元 ({} 天有缺口)",
242         plan_cost, emerg_cost, n_days)
243 return {"plan_cost": plan_cost, "emerg_cost": emerg_cost,
244         "emerg_kwh": emerg_kwh, "x_plans": x_plans, "recs": recs,
245         "mc_costs": mc_costs}
246
247
248 if __name__ == "__main__":
249     import argparse
250
251     ap = argparse.ArgumentParser()
252     ap.add_argument("--mc", type=int, default=100, help="蒙特卡洛年数; 0 表示跳过")
253     ap.add_argument("--no-hedge", action="store_true", help="跳过两阶段场景对冲")
254     ap.add_argument("--write-result2-d", action="store_true",
255                     help="把口径 D 对照写为 result2_D_backup.xlsx (不覆盖正式 result2)")
256     args = ap.parse_args()
257     run_q2(n_mc=args.mc, hedge=not args.no_hedge,
258           write_result2_file=args.write_result2_d)

```

program/src/solve/q2_adaptive.py

```

1  """C 题 问题二 口径 E: 历史自适应加权预测 + 网格定位 + 梯度精化 (兼容门面)。
2
3  实现已迁至 ``solve.models.adaptive`` (类 :class:`AdaptiveWeightModel`);
4  本文件保留原命令行入口与全部公开名 (``softmax`` / ``simplex_grid`` /
5  ``record`` / ``_verify_result2`` / N_DAY / REPORT_START), 供旧脚本与测试导入。
6
7  用法:
8      from solve.q2_adaptive import AdaptiveWeightModel
9
10     model = AdaptiveWeightModel(W=1).load()
11     result = model.build_table().run()          # 主结果 (含对照/图/报告)
12     model.window_sensitivity()                  # 标定窗口敏感性
13
14  命令行: uv run python -m solve.q2_adaptive    (在 program/ 目录; 首次约 8 分钟)
15  """
16  from __future__ import annotations
17
18  from solve.common import N_DAY, REPORT_START
19  from solve.io.excel import verify_result2 as _verify_result2
20  from solve.io.report import record
21  from solve.models.adaptive import (

```

```

22     AdaptiveWeightModel,
23     _init_worker,
24     _worker_day,
25     hist_forecast,
26     total_of,
27 )
28 from solve.models.weights import simplex_grid, softmax
29
30 if __name__ == "__main__":
31     import sys
32
33     _model = AdaptiveWeightModel().load().build_table()
34     if "--result2" in sys.argv:
35         _model.write_result2()
36     else:
37         _model.run()

```

program/src/solve/q2_tune.py

```

1  """Q2 口径 E 参数搜索: 标定窗口 / EWMA / 决策裕度 / 风险目标 / 权重收缩。
2
3  扫描对象 (全部因果执行, 2025-02-01 ~ 12-31, 334 天):
4  A. 标定窗口 W (网格表 argmin) 与 EWMA 半衰期 (对日费用行做指数衰减加权);
5  B. 决策裕度: 全局常数 m、逐时段/逐小时/逐月分位数 (数据分布参考值)、
6     仿射  $m(t)=a \cdot q80(t)+b$ 、比例式  $P(1-)$ 、负荷抬升  $_L$ 、动态 (滚动分位数);
7  C. 风险目标: 窗口选择目标改成  $(1-)$  均值 +  $\cdot CVaR20$  (日费用最差 20% 均值);
8  D. 权重收缩:  $w \leftarrow (1-) \cdot w + \cdot /3$  (抑制极端权重)。
9  E. 精扫与稳健性: 细网格、 $\times m$ 、 $\times$ 小时裕度、动态、分半验证。
10
11  参考值来自 `solve/experiments/q2e_structure.py` (报童临界比 0.8  $\rightarrow$  残差分位数)。
12  运行 (在 program/ 下): uv run python -m solve.q2_tune
13  """
14  from __future__ import annotations
15
16  import os
17  import shutil
18  import time
19
20  import numpy as np
21  import pandas as pd
22
23  import program as pm
24  from solve import consistency as cs
25  from solve import q2
26  from solve.common import EO, RESULTS_DIR, ROOT
27  from solve.q2_adaptive import AdaptiveWeightModel
28  from solve.io.report import record
29
30  REPORT_START = q2.REPORT_START
31  REP = list(range(REPORT_START, q2.N_DAY))
32  EPS_PLAN = AdaptiveWeightModel.EPS_PLAN
33  HOUR_OF_SLOT = np.arange(q2.T) // 6
34
35  _P = None # 子进程全局负载
36
37
38  # -----
39  # 权重序列构造
40  # -----
41  def seqs_grid(model: AdaptiveWeightModel, W: int):
42      """窗口 W 的网格 argmin 权重序列 (官方口径)。"""
43      return model.weights(W=W, use_gd=False)
44
45
46  def seqs_ewma(model: AdaptiveWeightModel, hl: float):
47      """日费用行指数衰减加权 (半衰期 hl 天); hl 1e8 退化为历史均值。
48
49      统一实现见 ``consistency.ewma_weights_from_table`` (唯一参数源);
50      本函数仅做 {day: (w, u)}  $\rightarrow$  报送期列表的接口适配。

```

```

51     """
52     table = cs.ewma_weights_from_table(model.C, model.grid, hl=hl)
53     return [table[d] for d in REP]
54
55
56 def seqs_risk(model: AdaptiveWeightModel, W: int, lam: float, q: float = 0.2):
57     """窗口目标 (1-) * 均值 + * CVaR_q (最差 q 比例的日费用均值)。"""
58     grid = model.grid
59     out = []
60     for d in REP:
61         lo = max(model.START, d - W)
62         block = model.C[lo:d]
63         mean = block.mean(axis=0)
64         k = max(1, int(np.ceil(q * block.shape[0])))
65         cvar = np.sort(block, axis=0)[-k:].mean(axis=0)
66         score = (1 - lam) * mean + lam * cvar
67         i, j = np.unravel_index(np.argmin(score), mean.shape)
68         out.append((grid[i].copy(), grid[j].copy()))
69     return out
70
71
72 def seqs_shrink(model: AdaptiveWeightModel, W: int, alpha: float):
73     """权重向等权收缩: (1-) * w + /3 (网格基座)。"""
74     base = model.weights(W=W, use_gd=False)
75     return [((1 - alpha) * w + alpha / 3.0, (1 - alpha) * u + alpha / 3.0)
76             for (w, u) in base]
77
78
79 def shrink_seq(seq, alpha: float):
80     """对任意权重序列做向等权收缩。"""
81     return [((1 - alpha) * w + alpha / 3.0, (1 - alpha) * u + alpha / 3.0)
82            for (w, u) in seq]
83
84
85 # -----
86 # 裕度参考值 (分布结构)
87 # -----
88
89 def net_residuals(model: AdaptiveWeightModel, ws_list):
90     """净需求残差 e 与负荷预测 L (kW), 形状均为 (n_days, 144)。"""
91     e = np.empty((len(REP), q2.T))
92     lh = np.empty_like(e)
93     for i, d in enumerate(REP):
94         w, u = ws_list[i]
95         l, p = model.forecast_kw(w, u, d)
96         lh[i] = np.clip(l, 0.0, None)
97         e[i] = (model.L[d] - model.P[d]) - (1 - p)
98     return e, lh
99
100
101 def margin_refs(model: AdaptiveWeightModel, seq, months) -> dict:
102     """由残差分布得到参考裕度: 全局/时段/小时/月/比例式/负荷系数。"""
103     e, lh = net_residuals(model, seq)
104     refs = {
105         "global": float(np.quantile(e, 0.80)),
106         "slot": np.quantile(e, 0.80, axis=0),
107         "hour": np.array([np.quantile(e[:, 6 * h:6 * (h + 1)], 0.80)
108                           for h in range(24)]),
109         "month": np.array([
110             np.quantile(e[months[REP] == m], 0.80) if (months[REP] == m).any() else 0.0
111             for m in range(1, 13)]),
112         "kappa": 1.0 + float(np.quantile((e / np.maximum(lh, 1.0)).ravel(), 0.80)),
113     }
114     ratios = []
115     for i, d in enumerate(REP):
116         w, u = seq[i]
117         l, p = model.forecast_kw(w, u, d)
118         mask = p > 500.0
119         if mask.any():
120             ratios.append(e[i][mask] / p[mask])

```

```

120     refs["rho"] = float(np.quantile(np.concatenate(ratios), 0.80))
121     refs["e"] = e
122     return refs
123
124
125 def rolling_kappa(model: AdaptiveWeightModel, seq, window: int = 60, q: float = 0.8):
126     """动态负荷系数：每日用过去 window 天残差比 e/L 的 q 分位数（无前视）。"""
127     e, lh = net_residuals(model, seq)
128     ratio = e / np.maximum(lh, 1.0)
129     kappa = np.ones(len(REP))
130     for i in range(len(REP)):
131         lo = max(0, i - window)
132         if i - lo >= 14:
133             kappa[i] = 1.0 + float(np.quantile(ratio[lo:i].ravel(), q))
134     return kappa
135
136
137 # -----
138 # 并行评估
139 # -----
140 def _init_worker(payload):
141     global _P
142     _P = payload
143
144
145 def _task(t):
146     ci, wsid, msid, d = t
147     p = _P
148     w, u = p["seqs"][wsid][d - REPORT_START]
149     l = w[0] * p["L"][d - 7] + w[1] * p["L"][d - 14] + w[2] * p["L_typ"]
150     pp = u[0] * p["P"][d - 1] + u[1] * p["P"][d - 2] + u[2] * p["P_typ"]
151     spec = p["margins"][msid]
152     if spec is not None:
153         kind = spec["kind"]
154         if kind == "pv_global":
155             pp = pp - spec["m"]
156         elif kind == "pv_slot":
157             pp = pp - spec["m"]
158         elif kind == "pv_hour":
159             pp = pp - spec["m"][HOURLY_SLOT]
160         elif kind == "pv_month":
161             pp = pp - spec["m"][p["months"][d] - 1]
162         elif kind == "pv_affine":
163             pp = pp - (spec["a"] * spec["m"][HOURLY_SLOT] + spec["b"])
164         elif kind == "pv_prop":
165             pp = pp * (1.0 - spec["rho"])
166         elif kind == "load_kappa":
167             l = l * spec["k"]
168         elif kind == "load_kappa_dyn":
169             l = l * spec["kappa"][d - REPORT_START]
170         elif kind == "combo":
171             l = l * spec["k"]
172             pp = pp - spec["m"]
173         elif kind == "combo_hour":
174             l = l * spec["k"]
175             pp = pp - spec["m"][HOURLY_SLOT]
176     l = np.clip(l, 0.0, None) / 6.0
177     pp = np.clip(pp, 0.0, None) / 6.0
178     x, _E, _ = q2.plan_day(p["price"], l, pp, E0, eps=EPS_PLAN)
179     ex = q2.exec_day_causal(p["price"], p["L"][d] / 6.0, p["P"][d] / 6.0, x, E0)
180     return ci, d - REPORT_START, float(p["price"] @ x), float(q2.EMERG_MULT * (p["price"] @ ex["e"]))
181
182
183 def evaluate(model: AdaptiveWeightModel, configs: list[dict],
184             workers: int | None = None):
185     """并行评估配置：返回（汇总表，日计划费矩阵，日紧急费矩阵）。"""
186     seqs, sid_by_obj = [], {}
187     for cfg in configs:
188         key = id(cfg["seq"])

```

```

189         if key not in sid_by_obj:
190             sid_by_obj[key] = len(seqs)
191             seqs.append(cfg["seq"])
192     margins, mid_by_key = [], {}
193     for cfg in configs:
194         key = id(cfg["margin"]) if cfg["margin"] is not None else -1
195         if key not in mid_by_key:
196             mid_by_key[key] = len(margins)
197             margins.append(cfg["margin"])
198     tasks = []
199     for ci, cfg in enumerate(configs):
200         sid = sid_by_obj[id(cfg["seq"])]
201         mid = mid_by_key[id(cfg["margin"])] if cfg["margin"] is not None else -1
202         for d in REP:
203             tasks.append((ci, sid, mid, d))
204
205     payload = {
206         "seqs": seqs, "margins": margins,
207         "L": model.L, "P": model.P, "L_tpy": model.L_tpy, "P_tpy": model.P_tpy,
208         "price": model.price, "months": model.data["months"],
209     }
210     t0 = time.time()
211     n = len(configs)
212     daily_plan = np.zeros((n, len(REP)))
213     daily_emerg = np.zeros((n, len(REP)))
214     nw = workers or min(8, os.cpu_count() or 1)
215     if nw > 1:
216         from multiprocessing import Pool
217
218         with Pool(nw, initializer=_init_worker, initargs=(payload,)) as pool:
219             for ci, di, c_plan, c_em in pool.imap_unordered(_task, tasks, chunksize=64):
220                 daily_plan[ci, di] = c_plan
221                 daily_emerg[ci, di] = c_em
222     else:
223         _init_worker(payload)
224         for t in tasks:
225             ci, di, c_plan, c_em = _task(t)
226             daily_plan[ci, di] = c_plan
227             daily_emerg[ci, di] = c_em
228     df = pd.DataFrame({
229         "配置": [c["名称"] for c in configs],
230         "计划购电费/万元": np.round(daily_plan.sum(axis=1) / 1e4, 1),
231         "紧急购电费/万元": np.round(daily_emerg.sum(axis=1) / 1e4, 1),
232     })
233     df["总费用/万元"] = (df["计划购电费/万元"] + df["紧急购电费/万元"]).round(1)
234     print(f" 批量评估 {len(configs)} 个配置完成 ({time.time() - t0:.0f}s)")
235     return df, daily_plan, daily_emerg
236
237
238 def simulate_two_day(model: AdaptiveWeightModel, seq, kappa: float, margin: float,
239                     detail: bool = False):
240     """2 日滚动正式口径：日末 SOC 自由、跨日连续，规划窗口末端自由（不锚定）。"""
241     E = float(E0)
242     rows = []
243     x_plans = np.zeros((q2.N_DAY, q2.T)) if detail else None
244     recs = [None] * q2.N_DAY if detail else None
245     for i, d in enumerate(REP):
246         w, u = seq[i]
247         l0, p0 = model.forecast_kw(w, u, d)
248         # 次日预测在 d 日 0:00 形成；负荷的 d+1-7/d+1-14 已知；
249         # 光伏不能读取尚未实现的 P[d]，只能沿用 P[d-1]/P[d-2]/典型日组合。
250         l1 = w[0] * model.L[d - 6] + w[1] * model.L[d - 13] + w[2] * model.L_tpy
251         p1 = u[0] * model.P[d - 1] + u[1] * model.P[d - 2] + u[2] * model.P_tpy
252         l0 = cs.kappa_load(l0, kappa)
253         l1 = cs.kappa_load(l1, kappa)
254         p0 = cs.margin_pv(p0, margin)
255         p1 = cs.margin_pv(p1, margin)
256         xh, Eh, _ = q2.plan_horizon(
257             np.tile(model.price, 2), np.concatenate([l0, l1]) / 6.0,

```

```

258         np.concatenate([p0, p1]) / 6.0, E, None, eps=EPS_PLAN,
259     )
260     x = xh[:q2.T]
261     # 统一执行策略 (consistency.EXEC_POLICY="free"): 无段末硬目标
262     ex = q2.exec_segment_causal(
263         model.L[d] / 6.0, model.P[d] / 6.0, x, E, None)
264     if detail:
265         x_plans[d] = x
266         recs[d] = {
267             "c": ex["c"], "d": ex["d"], "s": ex["s"], "e": ex["e"],
268             "E": ex["E"], "E_start": E, "E_end": float(ex["E"][-1]),
269         }
270     rows.append({
271         "日期": model.dates[d],
272         "计划购电量/kWh": float(x.sum()),
273         "计划购电费/元": float(model.price @ x),
274         "紧急购电量/kWh": float(ex["e"].sum()),
275         "紧急购电费/元": float(q2.EMERG_MULT * (model.price @ ex["e"])),
276         "日初SOC/kWh": E,
277         "日末SOC/kWh": float(ex["E"][-1]),
278     })
279     E = float(ex["E"][-1])
280     df = pd.DataFrame(rows)
281     return (df, x_plans, recs) if detail else df
282
283
284 def rolling_holdout_search() -> pd.DataFrame:
285     """在 2 日滚动结构下做小范围开发/冻结验证, 正式参数只由 2-6 月选择。"""
286     pm.init(root=str(ROOT))
287     model = AdaptiveWeightModel().load().build_table()
288     split_at = next(i for i, d in enumerate(REP) if model.dates[d] == "2025-07-01")
289     rows = []
290     t0 = time.time()
291     for h in (3.0, 5.0, 7.0):
292         seq = seqs_ewma(model, h)
293         for kappa in (1.01, 1.015, 1.02):
294             for margin in (25.0, 50.0, 75.0):
295                 df = simulate_two_day(model, seq, kappa, margin)
296                 costs = df["计划购电费/元"].to_numpy() + df["紧急购电费/元"].to_numpy()
297                 rows.append({
298                     "配置": f"EWMA h={h:g} = {kappa:g}+m={margin:g}",
299                     "h": h, "kappa": kappa, "margin": margin,
300                     "开发期(2-6月)/万元": round(float(costs[:split_at].sum()) / 1e4, 1),
301                     "冻结验证期(7-12月)/万元": round(float(costs[split_at:].sum()) / 1e4, 1),
302                     "全年描述值/万元": round(float(costs.sum()) / 1e4, 1),
303                 })
304             print(f" 2日滚动候选 {len(rows)}/27 ({time.time() - t0:.0f}s) ")
305     out = pd.DataFrame(rows).sort_values(
306         ["开发期(2-6月)/万元", "冻结验证期(7-12月)/万元"]
307     ).reset_index(drop=True)
308     out["开发期选中"] = out.index == 0
309     out["验证期排名"] = out["冻结验证期(7-12月)/万元"].rank(method="min").astype(int)
310     out.to_csv(pm.outputs_dir() / "q2e_tune_2day_holdout.csv", index=False, encoding="utf-8-sig")
311     record(
312         "问题二 2日滚动参数时间留出验证",
313         out.head(10),
314         note=(
315             "每个候选均按 48 小时预测窗口逐日滚动; 当前日末 SOC 不固定并跨日传递, "
316             "规划窗口末端完全自由 (不锚定终值)。2-6 月选参, 7-12 月冻结验证; 完整表见 "
317             "code/outputs/q2e_tune_2day_holdout.csv。"),
318     )
319     return out
320
321 # -----
322 # 主流程
323 # -----
324 def main():
325     pm.init(seed=42, root=str(ROOT))
326     model = AdaptiveWeightModel().load().build_table()

```

```

327 months = model.data["months"]
328 rep_str = f"{model.dates[REPORT_START]} ~ {model.dates[-1]}"
329
330 # ----- 阶段 1: 窗口与 EWMA -----
331 W_LIST = [1, 2, 3, 4, 5, 6, 7, 8, 10, 12, 14, 16, 18, 21, 25, 28, 30, 32, 35,
332           45, 60, 90, 120, 180, 240, 300]
333 HL_LIST = [0.5, 1, 2, 3, 4, 5, 6, 7, 10, 14, 21, 30, 60, 120, 1e9]
334 cfigs = [{"名称": f"网格 W={W}", "seq": seqs_grid(model, W), "margin": None}
335           for W in W_LIST]
336 cfigs += [{"名称": ("历史均值" if hl > 1e8 else f"EWMA h={hl}"),
337            "seq": seqs_ewma(model, hl), "margin": None} for hl in HL_LIST]
338 df1, dp1, de1 = evaluate(model, cfigs)
339 df1.to_csv(pm.outputs_dir() / "q2e_tune_window.csv", index=False, encoding="utf-8-sig")
340
341 grid_rows = df1[df1["配置"].str.startswith("网格")]
342 best_grid = grid_rows.loc[grid_rows["总费用/万元"].idxmin()]
343 w_best = int(best_grid["配置"].split("=")[1])
344 ew_rows = df1[df1["配置"].str.startswith("EWMA")]
345 hl_best = float(ew_rows.loc[ew_rows["总费用/万元"].idxmin(), "配置"].split("=")[1])
346 print(f"阶段1 网格最优: W*={w_best} = {best_grid['总费用/万元']} 万元; "
347       f"EWMA 最优: h={hl_best:g} = {ew_rows['总费用/万元'].min()} 万元")
348
349 seq_w7 = seqs_grid(model, 7)
350 seq_best = seq_w7 if w_best == 7 else seqs_grid(model, w_best)
351 seq_ew = seqs_ewma(model, hl_best)
352 bases = {"W7": seq_w7}
353 if w_best != 7:
354     bases[f"W{w_best}"] = seq_best
355 bases[f"EWMA h={hl_best:g}"] = seq_ew
356
357 # ----- 阶段 2: 决策裕度 (分布参考值) -----
358 refs = {name: margin_refs(model, seq, months) for name, seq in bases.items()}
359 for name, r in refs.items():
360     print(f" {name} 参考裕度: 全局 {r['global']:.0f} kW、"
361           f"小时范围 {r['hour'].min():.0f}~{r['hour'].max():.0f}、"
362           f"比例式 = {r['rho']:.3f}、_ref={r['kappa']:.3f}")
363
364 cfigs2 = []
365 for bname, seq in bases.items():
366     r = refs[bname]
367     for m in range(0, 501, 50):
368         cfigs2.append({"名称": f"{bname} 全局裕度 m={m}", "seq": seq,
369                       "margin": None if m == 0 else
370                       {"kind": "pv_global", "m": float(m)}})
371     for g in (0.5, 0.75, 1.0, 1.25, 1.5):
372         cfigs2.append({"名称": f"{bname} 小时裕度 = {g}", "seq": seq,
373                       "margin": {"kind": "pv_hour", "m": r["hour"] * g}})
374     for g in (0.75, 1.0, 1.25):
375         cfigs2.append({"名称": f"{bname} 时段裕度 = {g}", "seq": seq,
376                       "margin": {"kind": "pv_slot", "m": r["slot"] * g}})
377     cfigs2.append({"名称": f"{bname} 月裕度 = 1", "seq": seq,
378                   "margin": {"kind": "pv_month", "m": r["month"]}})
379     for rho in (0.02, 0.04, 0.06, 0.08, 0.12):
380         cfigs2.append({"名称": f"{bname} 比例裕度 = {rho}", "seq": seq,
381                       "margin": {"kind": "pv_prop", "rho": rho}})
382     for k in (1.005, 1.01, 1.015, 1.02):
383         cfigs2.append({"名称": f"{bname} 负荷抬升 = {k}", "seq": seq,
384                       "margin": {"kind": "load_kappa", "k": k}})
385     for a in (0.5, 0.75, 1.0, 1.25):
386         for b in (50.0, 100.0, 150.0):
387             cfigs2.append({"名称": f"W7 仿射裕度 a={a}, b={b}", "seq": seq_w7,
388                           "margin": {"kind": "pv_affine", "a": a, "b": b,
389                                     "m": refs["W7"]["hour"]}})
390 df2, dp2, de2 = evaluate(model, cfigs2)
391 df2.to_csv(pm.outputs_dir() / "q2e_tune_margin.csv", index=False, encoding="utf-8-sig")
392 top2 = df2.loc[df2["总费用/万元"].idxmin()]
393 print(f"阶段2 最优: {top2['配置']} = {top2['总费用/万元']} 万元")
394
395 # ----- 阶段 3: 风险目标与收缩 -----

```

```

396 cfigs3 = []
397 for bname, seq in bases.items():
398     for lam in (0.25, 0.5, 0.75, 1.0):
399         for W in (7, 21, 30):
400             cfigs3.append({"名称": f"{bname} 风险 = {lam}, W={W}",
401                           "seq": seqs_risk(model, W, lam), "margin": None})
402         for alpha in (0.05, 0.1, 0.2, 0.35):
403             cfigs3.append({"名称": f"{bname} 收缩 = {alpha}",
404                           "seq": shrink_seq(seq, alpha), "margin": None})
405 df3, dp3, de3 = evaluate(model, cfigs3)
406 df3.to_csv(pm.outputs_dir() / "q2e_tune_risk.csv", index=False, encoding="utf-8-sig")
407 top3 = df3.loc[df3["总费用/万元"].idxmin()]
408 print(f"阶段3 最优: {top3['配置']} = {top3['总费用/万元']} 万元")
409
410 # ----- 阶段 4: 精扫、组合、动态 -----
411 cfigs4 = []
412 for bname, seq in bases.items():
413     for k in (1.015, 1.02, 1.025, 1.03, 1.035, 1.04):
414         cfigs4.append({"名称": f"{bname} = {k}", "seq": seq,
415                       "margin": {"kind": "load_kappa", "k": k}})
416     for k in (1.015, 1.02, 1.025):
417         for m in (50.0, 100.0, 150.0):
418             cfigs4.append({"名称": f"{bname} = {k}+m={m:.0f}", "seq": seq,
419                           "margin": {"kind": "combo", "k": k, "m": m}})
420     for g in (0.5, 0.75):
421         cfigs4.append({"名称": f"{bname} = 1.02+小时裕度 = {g}", "seq": seq,
422                       "margin": {"kind": "combo_hour", "k": 1.02,
423                                   "m": refs[bname]["hour"] * g}})
424     cfigs4.append({"名称": f"{bname} 动态 (60天q80)", "seq": seq,
425                   "margin": {"kind": "load_kappa_dyn",
426                               "kappa": rolling_kappa(model, seq)}})
427 # EWMA 基座的细网格 (围绕 =1.02, m=50)
428 seq_ew_name = f"EWMA h={hl_best:g}"
429 if seq_ew_name in bases:
430     seq = bases[seq_ew_name]
431     for k in (1.02, 1.0225, 1.025):
432         for m in (25.0, 50.0, 75.0):
433             cfigs4.append({"名称": f"{seq_ew_name} = {k}+m={m:.0f}", "seq": seq,
434                           "margin": {"kind": "combo", "k": k, "m": m}})
435 df4, dp4, de4 = evaluate(model, cfigs4)
436 df4.to_csv(pm.outputs_dir() / "q2e_tune_combo.csv", index=False, encoding="utf-8-sig")
437 top4 = df4.loc[df4["总费用/万元"].idxmin()]
438 print(f"阶段4 最优: {top4['配置']} = {top4['总费用/万元']} 万元")
439
440 # ----- 时间留出验证 (开发期: 2-6 月; 冻结验证期: 7-12 月) -----
441 # 不能按 334 天机械对半: 真正的 7 月 1 日切点是第 150 个报告日。
442 split_at = next(i for i, d in enumerate(REP) if model.dates[d] == "2025-07-01")
443 tagged = []
444 for family, df, dp, de in (
445     ("窗口/EWMA", df1, dp1, de1),
446     ("决策裕度", df2, dp2, de2),
447     ("风险/收缩", df3, dp3, de3),
448     ("组合精扫", df4, dp4, de4),
449 ):
450     for i in range(len(df)):
451         tagged.append((family, df.iloc[i]["配置"], dp[i], de[i]))
452 tagged = pd.DataFrame(tagged, columns=["族", "配置", "dp", "de"])
453
454 split_rows = []
455 for _, row in tagged.iterrows():
456     dev = float(row["dp"][:split_at].sum() + row["de"][:split_at].sum()) / 1e4
457     val = float(row["dp"][split_at:].sum() + row["de"][split_at:].sum()) / 1e4
458     split_rows.append({
459         "族": row["族"], "配置": row["配置"],
460         "开发期(2-6月)/万元": round(dev, 1),
461         "冻结验证期(7-12月)/万元": round(val, 1),
462         "全年描述值/万元": round(dev + val, 1),
463     })
464 holdout = pd.DataFrame(split_rows).sort_values(

```

```

465     ["开发期(2-6月)/万元", "冻结验证期(7-12月)/万元"]
466 ).reset_index(drop=True)
467 holdout["开发期选中"] = holdout.index == 0
468 holdout["验证期排名"] = holdout["冻结验证期(7-12月)/万元"].rank(
469     method="min"
470 ).astype(int)
471 selected = holdout.iloc[0]
472 holdout.to_csv(pm.outputs_dir() / "q2e_tune_holdout.csv", index=False,
473     encoding="utf-8-sig")
474
475 keep = {"网格 W=7", str(selected["配置"]), str(top4["配置"])}
476 split = holdout[holdout["配置"].isin(keep)].copy()
477 split.to_csv(pm.outputs_dir() / "q2e_tune_split.csv", index=False,
478     encoding="utf-8-sig")
479 print("时间留出验证: 开发期选中 {} (开发 {:.1f} 万, 冻结验证 {:.1f} 万, 验证排名 {}/{})".format(
480     selected["配置"], selected["开发期(2-6月)/万元"],
481     selected["冻结验证期(7-12月)/万元"], selected["验证期排名"], len(holdout)))
482
483 # ----- 汇总 -----
484 official = df1[df1["配置"] == "网格 W=7"].iloc[0]
485 all_best = pd.concat([df1, df2, df3, df4]).sort_values("总费用/万元")
486 final = all_best.iloc[0]
487 summary = pd.DataFrame({
488     "方案": ["官方 W=7 (网格)", f"最优配置: {final['配置']}"],
489     "计划购电费/万元": [official["计划购电费/万元"], final["计划购电费/万元"]],
490     "紧急购电费/万元": [official["紧急购电费/万元"], final["紧急购电费/万元"]],
491     "总费用/万元": [official["总费用/万元"], final["总费用/万元"]],
492     "相对官方/%": [0.0, round(100 * (final["总费用/万元"] / official["总费用/万元"] - 1), 2)],
493 })
494 print(summary.to_string(index=False))
495 print(split.to_string(index=False))
496 pm.save_outputs(summary, "q2e_tune_summary")
497 pm.save_outputs(split, "q2e_tune_split")
498
499 # 最优配置逐日表 (供后续 result2 引用)
500 best_row = tagged[tagged["配置"] == final["配置"]]
501 if len(best_row):
502     row = best_row.iloc[0]
503     pd.DataFrame({
504         "日期": [model.dates[d] for d in REP],
505         "计划购电费/元": row["dp"],
506         "紧急购电费/元": row["de"],
507     }).to_csv(pm.outputs_dir() / "q2e_tune_daily_best.csv", index=False,
508         encoding="utf-8-sig")
509
510 # ----- 图: 参数搜索 -----
511 import matplotlib.pyplot as plt
512
513 fig, axes = plt.subplots(2, 2, figsize=(7.2, 5.2))
514 ax = axes[0, 0]
515 g = df1[df1["配置"].str.startswith("网格")]
516 ax.plot([int(s.split("=")[-1]) for s in g["配置"]], g["总费用/万元"],
517     marker="o", ms=3, color="#4C72B0", label="网格 W")
518 ew = df1[df1["配置"].str.startswith("EWMA")]
519 ax.plot([float(s.split("=")[-1]) for s in ew["配置"]], ew["总费用/万元"],
520     marker="s", ms=3, color="#C44E52", label="EWMA h")
521 ax.set_xscale("log")
522 ax.set_xlabel("标定窗口 W / EWMA 半衰期 / 天")
523 ax.set_ylabel("年度总费用 / 万元")
524 ax.set_title("(a) 权重标定时间尺度", fontsize=9)
525 ax.legend(fontsize=8)
526
527 ax = axes[0, 1]
528 for i, bname in enumerate(bases):
529     color = ["#4C72B0", "#C44E52", "#55A868"][i % 3]
530     gg = df2[df2["配置"].str.startswith(f"{bname} 全局裕度")]
531     ax.plot([int(s.split("=")[-1]) for s in gg["配置"]], gg["总费用/万元"],
532         marker="o", ms=3, color=color, label=f"{bname} 全局 m")
533     ax.set_xlabel("全局裕度 m / kW")

```

```

534 ax.set_ylabel("年度总费用 / 万元")
535 ax.set_title("(b) 光伏裕度", fontsize=9)
536 ax.legend(fontsize=7)
537
538 ax = axes[1, 0]
539 for i, bname in enumerate(bases):
540     color = ["#4C72B0", "#C44E52", "#55A868"][i % 3]
541     kk = df4[df4["配置"].str.startswith(f"{bname} =")
542             & ~df4["配置"].str.contains("\\+")]
543     ax.plot([float(s.split("=")[-1]) for s in kk["配置"]], kk["总费用/万元"],
544            marker="o", ms=3, color=color, label=f"{bname} 负荷 ")
545     ax.axvline(refs[bname]["kappa"], color=color, ls=":", lw=1.0, alpha=0.7)
546 ax.set_xlabel("负荷抬升系数 (虚线 = 分布参考)")
547 ax.set_ylabel("年度总费用 / 万元")
548 ax.set_title("(c) 负荷裕度 (虚线 = 分布参考)", fontsize=9)
549 ax.legend(fontsize=7)
550
551 ax = axes[1, 1]
552 labels = [s.replace(":", "\n") for s in summary["方案"]]
553 ax.bar(labels, summary["计划购电费/万元"], label="计划购电费", color="#4C72B0")
554 ax.bar(labels, summary["紧急购电费/万元"], bottom=summary["计划购电费/万元"],
555        label="紧急购电费", color="#C44E52")
556 for i, v in enumerate(summary["总费用/万元"]):
557     ax.text(i, v + 10, f"{v:.1f}", ha="center", fontsize=8)
558 ax.set_ylabel("年度费用 / 万元")
559 ax.set_title("(d) 官方 vs 最优配置", fontsize=9)
560 ax.legend(fontsize=8)
561 fig.tight_layout()
562 fig_data = pd.concat([
563     df1.assign(族="窗口/EWMA")["配置", "总费用/万元", "族"],
564     df2.assign(族="裕度")["配置", "总费用/万元", "族"],
565     df3.assign(族="风险/收缩")["配置", "总费用/万元", "族"],
566     df4.assign(族="精扫")["配置", "总费用/万元", "族"],
567 ], ignore_index=True)
568 pm.save_fig(fig, "Q2E_参数搜索", data=fig_data)
569
570 # ----- 报告 -----
571 from solve.io.report import record
572
573 record(
574     "问题二 口径E 参数搜索: 标定窗口与 EWMA",
575     df1,
576     note=(f"对照期 {rep_str}, 因果执行; W=1 对应日度标定; "
577          "EWMA 对日费用行做指数衰减加权(无前视); 历史均值 = 全历史等权。"),
578 )
579 record(
580     "问题二 口径E 参数搜索: 决策裕度",
581     df2,
582     note=("裕度为加在净需求(光伏)预测上的保守修正, 参考值 = 残差 0.8 分位数"
583          "(报童临界比 1-1/5); 小时/时段/月裕度使用各自通道的分布分位数, "
584          " 为分位数缩放; 比例式 =  $P(1-)$ ; 负荷抬升 =  $\cdot L$ 。"),
585 )
586 record(
587     "问题二 口径E 参数搜索: 风险目标与权重收缩",
588     df3,
589     note="风险目标 =  $(1-)$  * 窗口均值 +  $\cdot CVaR_{20}$  (最差 20% 日费用均值); 收缩 =  $(1-)$  *  $w$  +  $/3$ 。",
590 )
591 record(
592     "问题二 口径E 参数搜索: 精扫与动态参考",
593     df4,
594     note=(" 为负荷预测抬升系数(分布参考 _ref =  $1 + q_{80}(e/L)$  见各基座行); "
595          "动态 = 过去 60 天  $e/L$  的 0.8 分位数逐日重估(无前视); "
596          "组合 = 与光伏裕度  $m$ /小时裕度叠加。"),
597 )
598 record(
599     "问题二 口径E 参数搜索: 时间留出验证",
600     split,
601     note=("开发期 2025-02-01-06-30 只负责选参, 冻结验证期 2025-07-01-12-31 不参与选择; "
602          "全部候选见 code/outputs/q2e_tune_holdout.csv. 全年列仅作描述, 不用于选参。"),

```

```

603     )
604     record(
605         "问题二 口径E 调优配置汇总",
606         summary,
607         note=(f"最优配置 {final['配置']}: 总费用 {final['总费用/万元']} 万元, "
608              f"相对官方 W=7 ({official['总费用/万元']} 万元) "
609              f"{100 * (final['总费用/万元'] / official['总费用/万元'] - 1):+.2f}%."
610              "全部明细见 code/outputs/q2e_tune_*.csv; 图 figures/Q2E_参数搜索.pdf。"),
611     )
612     return summary
613
614
615 def joint_residual_mc(model, seq, kappa, margin, x_plans, e_start,
616                      n_years=100, seed=42):
617     """统一口径年度分布: 固定计划, 重采样 (Δ负荷, Δ光伏) 同月整日联合残差块。
618
619     名义 = *L 与 max(P - m, 0) (EWMA h=5 权重, 构造同 ``simulate_two_day``);
620     残差 = 实际 - 名义; 池同月优先、严格取目标日之前 (``consistency``)。
621     """
622     data = model.data
623     months = np.asarray(data["months"])
624     wu = cs.ewma_weights_from_table(model.C, model.grid, hl=cs.EWMA_HL,
625                                     first_target=15)
626     L_nom = np.zeros((q2.N_DAY, q2.T))
627     P_nom = np.zeros((q2.N_DAY, q2.T))
628     for d in range(15, q2.N_DAY):
629         w, u = wu[d]
630         l = w[0] * model.L[d - 7] + w[1] * model.L[d - 14] + w[2] * model.L_typ
631         p = u[0] * model.P[d - 1] + u[1] * model.P[d - 2] + u[2] * model.P_typ
632         L_nom[d] = cs.kappa_load(np.clip(l, 0.0, None), kappa) / 6.0
633         P_nom[d] = cs.margin_pv(np.clip(p, 0.0, None), margin) / 6.0
634     RL = data["load"] / 6.0 - L_nom
635     RP = data["pv_act"] / 6.0 - P_nom
636     rng = np.random.default_rng(seed)
637     emerg = np.zeros(n_years)
638     kwh = np.zeros(n_years)
639     for y in range(n_years):
640         E = float(e_start)
641         for d in REP:
642             pool = cs.scenario_indices(months, d)
643             j = int(rng.choice(pool))
644             load_s = np.clip(L_nom[d] + RL[j], 0.0, None)
645             pv_s = np.clip(P_nom[d] + RP[j], 0.0, None)
646             # 统一执行策略 (free): 无段末硬目标
647             ex = q2.exec_segment_causal(load_s, pv_s, x_plans[d], E, None)
648             kwh[y] += float(ex["e"].sum())
649             emerg[y] += float(q2.EMERG_MULT * (model.price @ ex["e"]))
650             E = float(ex["E"][-1])
651         if (y + 1) % 20 == 0:
652             print(f" MC(联合残差) {y + 1}/{n_years}")
653     return emerg, kwh
654
655
656 def write_result2_tuned(W: float = 5.0, kappa: float = 1.02,
657                        margin: float = 25.0) -> dict:
658     """把 2 日滚动时间留出选定配置写入 ``results/result2.xlsx``。
659
660     流程: EWMA 权重序列 → 48 小时计划 (当前日末 SOC 自由) → 逐槽因果执行 →
661     备份现官方文件 → 写出 + 回读校验 + 报告。默认参数只由 2-6 月开发期选择,
662     7-12 月作为冻结验证期, 不参与选择。
663     """
664     from solve.q2_adaptive import _verify_result2
665
666     pm.init(root=str(ROOT))
667     log = pm.get_logger("q2-tune")
668     model = AdaptiveWeightModel().load().build_table()
669     seq = seqs_ewma(model, W)
670
671     df, x_plans, recs = simulate_two_day(model, seq, kappa, margin, detail=True)

```

```

672 plan = float(df["计划购电费/元"].sum())
673 emerg = float(df["紧急购电费/元"].sum())
674
675 cur = RESULTS_DIR / "result2.xlsx"
676 if cur.exists():
677     backup = pm.outputs_dir() / "result2_pre_2day_backup.xlsx"
678     if not backup.exists():
679         shutil.copy2(cur, backup)
680         log.info("2 日滚动修正前 result2 备份 -> {}", backup)
681 out = q2.write_result2(model.dates, model.price, x_plans, recs, out_name="result2.xlsx")
682 checks = _verify_result2(out, df)
683 pm.save_outputs(df, "q2e_tuned_daily")
684
685 # ---- 官方口径图与年度成本分布（论文引用；全部来自本官方计划）----
686 import matplotlib.pyplot as plt
687
688 idx = np.arange(len(df))
689 month_starts = [i for i, s in enumerate(df["日期"]) if s.endswith("-01")]
690 month_labels = [df["日期"].iloc[i][5:7] + "月" for i in month_starts]
691
692 fig, ax = pm.line(idx, df["紧急购电量/kWh"].to_numpy(),
693                  xlabel="日期", ylabel="紧急购电量 / kWh")
694 ax.set_xticks(month_starts)
695 ax.set_xticklabels(month_labels)
696 pm.save_fig(fig, "Q2_逐日紧急购电", data=df[["日期", "紧急购电量/kWh"]])
697
698 fig2, ax2 = pm.line(
699     idx, [df["日初SOC/kWh"].to_numpy(), df["日末SOC/kWh"].to_numpy()],
700     labels=["日初储电量", "日末储电量"], xlabel="日期", ylabel="储电量 / kWh")
701 ax2.set_xticks(month_starts)
702 ax2.set_xticklabels(month_labels)
703 pm.save_fig(fig2, "Q2_储能轨迹",
704             data=df[["日期", "日初SOC/kWh", "日末SOC/kWh"]])
705
706 mc_costs, _mc_kwhs = joint_residual_mc(
707     model, seq, kappa, margin, x_plans,
708     float(df["日初SOC/kWh"].iloc[0]), n_years=100, seed=42)
709 total = plan + mc_costs
710 p95 = float(np.percentile(total, 95))
711 fig3, ax3 = plt.subplots(figsize=(7, 4.3))
712 ax3.hist(total, bins=30, color="#4C72B0", alpha=0.85, edgecolor="white")
713 ax3.axvline(total.mean(), color="#C44E52", ls="--",
714             label=f"均值 {total.mean():.0f} 元")
715 ax3.axvline(p95, color="#55A868", ls=":", label=f"P95 {p95:.0f} 元")
716 ax3.set_xlabel("年度总购电费 / 元")
717 ax3.set_ylabel("频数")
718 ax3.legend()
719 pm.save_fig(fig3, "Q2_总费用分布",
720             data=pd.DataFrame({"年度总购电费_元": total}))
721 record(
722     "问题二 年度总成本分布（2 日滚动·蒙特卡洛）",
723     {
724         "模拟年数": 100,
725         "年度总成本均值/元": float(total.mean()),
726         "年度总成本标准差/元": float(total.std()),
727         "P5/元": float(np.percentile(total, 5)),
728         "P50/元": float(np.percentile(total, 50)),
729         "P95/元": p95,
730         "CVaR95（尾部均值）/元": float(total[total >= p95].mean()),
731         "年度紧急费用均值/元": float(mc_costs.mean()),
732         "年度紧急费用P95/元": float(np.percentile(mc_costs, 95)),
733     },
734     note=(
735         "固定 2 日滚动官方计划，重采样（ $\Delta$ 负荷， $\Delta$ 光伏）同月整日联合残差块"
736         "（相对统一名义预测 /m），保留日内与两通道相关；"
737         "逐日执行沿用统一策略 free（无段末硬目标）、SOC 跨日连续；"
738         "仅作事后风险评价，不参与在线决策；图 figures/Q2_总费用分布.pdf。"),
739 )
740 record(
741     f"问题二 口径E（2日滚动时间留出选定：EWMA h={W:g} + ={kappa:g} + m={margin:g}）官方 result2",

```

```

741     {
742         "计划购电费/万元": round(plan / 1e4, 1),
743         "紧急购电费/万元": round(emerg / 1e4, 1),
744         "总费用/万元": round((plan + emerg) / 1e4, 1),
745         "天数": len(df),
746         **checks,
747     },
748     note=(
749         f"官方 results/result2.xlsx 由 2 日滚动时间留出配置生成: 2-6 月开发期选择 EWMA "
750         f"h={W:g}、={kappa:g}、m={margin:g} kW, 7-12 月冻结验证不参与选参; "
751         "每天以 48 小时为规划窗口、只执行次日, 当前日末 SOC 不固定并传递到下一日, "
752         "规划窗口末端完全自由 (不锚定任何终值)。"
753         "旧 W=7 日循环版仍保留在 "
754         "code/outputs/result2_W7_backup.xlsx, 留出修正前版本保留在 "
755         "code/outputs/result2_pre_2day_backup.xlsx; 复现: "
756         "uv run python -m solve.q2_tune --result2。"
757         "图 figures/Q2_逐日紧急购电.pdf、Q2_储能轨迹.pdf、Q2_总费用分布.pdf。"
758     ),
759 )
760 log.info("result2.xlsx (2 日滚动时间留出选定) 写出完成: {}, 总费用 {:.1f} 万元 (计划 {:.1f} + 紧急 {:.1f})",
761         out, (plan + emerg) / 1e4, plan / 1e4, emerg / 1e4)
762 return {"plan": plan, "emerg": emerg, "checks": checks, "out": out}
763
764
765 def make_weight_figure(W: float = 5.0) -> None:
766     """生成权重重演化图 (EWMA h=5 标定口径, 与官方模型一致)。"""
767     import matplotlib.pyplot as plt
768
769     pm.init(root=str(ROOT))
770     model = AdaptiveWeightModel().load().build_table()
771     seq = seqs_ewma(model, W)
772     dates = [model.dates[d] for d in REP]
773     Wm = np.array([w for w, _ in seq])
774     Um = np.array([u for _, u in seq])
775     w_cols = ["w1_L(d-7)", "w2_L(d-14)", "w3_典型日"]
776     u_cols = ["u1_P(d-1)", "u2_P(d-2)", "u3_典型日"]
777     df = pd.DataFrame({"日期": dates,
778                       **{c: Wm[:, i] for i, c in enumerate(w_cols)},
779                       **{c: Um[:, i] for i, c in enumerate(u_cols)}})
780
781     x = np.arange(len(dates))
782     month_starts = [i for i, s in enumerate(dates) if s.endswith("-01")]
783     fig, axes = plt.subplots(2, 1, figsize=(7, 5.6), sharex=True)
784     for ax, cols, ylabel, labels in (
785         (axes[0], w_cols, "负权重重", ["d-7", "d-14", "典型日"]),
786         (axes[1], u_cols, "光伏权重", ["d-1", "d-2", "典型日"]),
787     ):
788         ax.stackplot(x, [df[c].to_numpy() for c in cols], labels=labels, alpha=0.9)
789         ax.set_ylabel(ylabel)
790         ax.set_ylim(0, 1)
791         ax.legend(loc="upper center", ncol=3, fontsize=8, framealpha=0.9)
792     axes[1].set_xticks(month_starts)
793     axes[1].set_xticklabels([dates[i][5:7] + "月" for i in month_starts])
794     axes[1].set_xlabel("日期")
795     pm.save_fig(fig, "Q2E_权重重演化", data=df)
796     print(f"Q2E_权重重演化.pdf 已按 EWMA h={W:g} 重新生成")
797
798 if __name__ == "__main__":
799     if "--result2" in __import__("sys").argv:
800         write_result2_tuned()
801     elif "--rolling-holdout" in __import__("sys").argv:
802         rolling_holdout_search()
803     elif "--fig-weights" in __import__("sys").argv:
804         make_weight_figure()
805     else:
806         main()

```

program/src/solve/q3.py

```

1  """C 题 问题三：预报驱动的日内调整（三层结算）——主方案、result3 与图表。
2
3  正式主方案（v1.4，见 reports/Q3_对冲取舍实验.md）：
4  - 0:00 计划：组合预测 * 官方f0 + (1-) * 历史 EWMA, =0.7
5  - 6/12/18 调整：同组合口径的最新预报重优化未执行时段（口径 A: 6:00 重优化 [6,24)、
6    12:00 重优化 [12,24)、18:00 重优化 [18,24)，只承诺执行未来 6 h)
7  - 不使用场景对冲；联合残差对冲仅作为负结果扩展保留在 q3_ablation.py
8  - 储能：2 日预测窗口，每日只执行首日；日末 SOC 自由并跨日传递，规划窗口末端完全自由
9    （不锚定终值）
10 - 执行：逐槽因果 free (q2.exec_segment_causal, 无段末硬目标)，缺口按 5 倍交易时刻电价紧急购电
11 - 结算：费用 =  $\Sigma [p \cdot x_{adj} + 0.5 \cdot p \cdot |x_{plan} - x_{adj}|] + 5 \cdot \Sigma p \cdot e$ 
12
13 # =====
14 # 三层费用口径（结算规则）
15 # 计划层: C_plan =  $\Sigma_t p_t \cdot x_{plan,t}$ 
16 # 调整层: C_adj =  $\Sigma_t [p_t \cdot x_{adj,t} + 0.5 \cdot p_t \cdot |x_{plan,t} - x_{adj,t}|]$ 
17 # 紧急层: C_em =  $5 \cdot \Sigma_t p_t \cdot e_t$ 
18 # 总费用: C = C_adj + C_em
19 # （调整层系数 0.5 即“调低 50% 违约金”的线性化；调高部分按 1.5p 体现在
20 # 1.5p * adj - p * y 中, y = min(x_plan, adj)）
21 # =====
22
23 输出：
24 - results/result3.xlsx（计划购电量 / 调整购电量 / 充放电量 / 紧急购电量）
25 - figures/Q3_策略费用对比.pdf、Q3_逐日紧急购电.pdf、Q3_调整量分布.pdf
26 - reports/RESULTS_REPORT.md 章节
27
28 运行（program/ 下）：uv run python -m solve.q3
29 """
30 from __future__ import annotations
31
32 import time
33
34 import numpy as np
35 import pandas as pd
36
37 import program as pm
38 from solve import consistency as cs
39 from solve import q2
40 from solve import q3_proto as qp
41 from solve.common import ROOT, T
42 from solve.io.excel import verify_result3, write_result3
43 from solve.io.report import record
44
45 LAM = cs.Q3_LAM
46 ADJ_LAM = cs.Q3_ADJ_LAM
47 USE_HEDGE = cs.Q3_USE_HEDGE
48 DAY_START = q2.REPORT_START # 31 (2025-02-01)
49 N_Q3 = q2.N_DAY - DAY_START # 334
50
51
52 def _day_result(data, D, e_start, **kw):
53     if USE_HEDGE: # 仅用于显式复现实验；正式唯一参数源固定为 False。
54         r = qp.simulate_day_rt_hedge(
55             data, D, LAM, adj_lam=ADJ_LAM, n_scen=cs.N_SCEN, seed=7,
56             e_start=e_start, **kw,
57         )
58     else:
59         r = qp.simulate_day_rt(
60             data, D, LAM, adj_lam=ADJ_LAM, e_start=e_start, **kw,
61         )
62     return r
63
64
65 def run_main(data):
66     """主方案全年模拟，返回明细矩阵与逐日表（日末 SOC 跨日传递）。
67
68     输出矩阵形状 (N_Q3, T): xp 计划购电、xa 调整购电、c/d 充放、E 执行储电量、e 紧急；
69     另含 E_start（每日日初 SOC，写入结果表充放电页）。

```

```

70     """
71     days = list(range(DAY_START, DAY_START + N_Q3))
72     out = {k: np.zeros((N_Q3, T)) for k in ("xp", "xa", "c", "d", "E", "e")}
73     out["E_start"] = np.zeros(N_Q3)
74     rows = []
75     t0 = time.time()
76     E = float(qp.E0)
77     for i, D in enumerate(days):
78         r = _day_result(data, D, E)
79         out["xp"][i] = r["x_plan"]; out["xa"][i] = r["x_final"]
80         out["c"][i] = r["c"]; out["d"][i] = r["d"]
81         out["E"][i] = r["E_exec"]; out["e"][i] = r["e"]
82         out["E_start"][i] = r["E_start"]
83         rows.append({"日期": data["dates"][D], "计划费/元": r["plan_cost"],
84                     "调整净额/元": r["adjust_net"], "紧急费/元": r["emerg"],
85                     "总费用/元": r["total"], "紧急量/kWh": float(r["e"].sum()),
86                     "调整量/kWh": float(np.abs(r["x_final"] - r["x_plan"]).sum()),
87                     "场景最大日序号": int(r.get("scenario_max_day", -1)),
88                     "场景数": int(r.get("scenario_count", 0))})
89     E = float(r["E_end"])
90     if (i + 1) % 50 == 0:
91         print(f" 主方案 {i + 1}/{N_Q3} 天, 用时 {time.time() - t0:.0f}s")
92     return {k: v for k, v in out.items()}, pd.DataFrame(rows)
93
94
95 def run_baseline(data, tag, **kw):
96     """基线模拟 (=1 官方, 无对冲), 用于对照 (日末 SOC 跨日传递)。"""
97     days = list(range(DAY_START, DAY_START + N_Q3))
98     xa = np.zeros((N_Q3, T)); e = np.zeros((N_Q3, T))
99     tot = plan = adj = em = 0.0
100    E = float(qp.E0)
101    for i, D in enumerate(days):
102        r = qp.simulate_day_rt(data, D, 1.0, e_start=E, **kw)
103        xa[i] = r["x_final"]; e[i] = r["e"]
104        tot += r["total"]; plan += r["plan_cost"]; adj += r["adjust_net"]; em += r["emerg"]
105        E = float(r["E_end"])
106    print(f" {tag}: 总 {tot / 1e4:.1f} 万 (计划 {plan / 1e4:.1f} + 调整 {adj / 1e4:.1f} + 紧急 {em / 1e4:.1f}) ")
107    return {"total": tot, "plan": plan, "adj": adj, "emerg": em, "e": e, "xa": xa}
108
109
110 def make_figures(daily, base_noadj, base_off, out, dates):
111     """Q3 论文图: 三策略费用构成、逐日紧急购电、逐日调整量分布。"""
112     import matplotlib.pyplot as plt
113
114     # 1) 策略费用对比 (三策略 × 三构成)
115     names = ["无调整 (官方)", "三点调整 (官方)", "主方案 (组合无对冲)"]
116     plan = [base_noadj["plan"] / 1e4, base_off["plan"] / 1e4,
117             float(sum(daily["计划费/元"])) / 1e4]
118     adj = [base_noadj["adj"] / 1e4, base_off["adj"] / 1e4,
119            float(sum(daily["调整净额/元"])) / 1e4]
120     em = [base_noadj["emerg"] / 1e4, base_off["emerg"] / 1e4,
121           float(sum(daily["紧急费/元"])) / 1e4]
122     pm.bar_group(names, {"计划购电费": plan, "调整净额": adj, "紧急购电费": em},
123                  ylabel="334 天费用 / 万元", save="Q3_策略费用对比")
124
125     # 2) 逐日紧急购电量
126     fig, ax = pm.line(np.arange(N_Q3), [out["e"].sum(axis=1), base_off["e"].sum(axis=1)],
127                       labels=["主方案 (组合无对冲)", "三点调整 (官方)"],
128                       xlabel="日期 (2025-02-01 起)", ylabel="紧急购电量 / kWh")
129     pm.save_fig(fig, "Q3_逐日紧急购电",
130                 data=pd.DataFrame({"日期": daily["日期"], "主方案紧急量/kWh": out["e"].sum(axis=1),
131                                   "官方三点紧急量/kWh": base_off["e"].sum(axis=1)}))
132
133     # 3) 调整量分布 (主方案: |x_adj - x_plan| 的逐日分布)
134     dev = np.abs(out["xa"] - out["xp"]).sum(axis=1)
135     fig2, ax2 = pm.hist(dev, bins=30, xlabel="逐日调整量 |x_adj - x_plan| / kWh",
136                          ylabel="天数")
137     pm.save_fig(fig2, "Q3_调整量分布", data=pd.DataFrame({"逐日调整量_kWh": dev}))
138

```

```

139
140 def main():
141     pm.init(seed=42, root=str(ROOT))
142     log = pm.get_logger("q3")
143     t0 = time.time()
144     data = qp.load_extended() # 含 EWMA h=5 统一权重 (consistency.py)
145     print("基线对照.....")
146     base_noadj = run_baseline(data, "无调整 (官方)", adj_hours=())
147     base_off = run_baseline(data, "三点调整 (官方)")
148     print("主方案.....")
149     out, daily = run_main(data)
150     cost = {"plan": float(sum(daily["计划费/元"])), "adj": float(sum(daily["调整净额/元"])),
151            "emerg": float(sum(daily["紧急费/元"])), "total": float(sum(daily["总费用/元"]))}
152     print(f"主方案: 总 {cost['total'] / 1e4:.1f} 万 (计划 {cost['plan'] / 1e4:.1f} "
153           f"+ 调整 {cost['adj'] / 1e4:.1f} + 紧急 {cost['emerg'] / 1e4:.1f}), "
154           f"用时 {time.time() - t0:.0f}s")
155
156     p = write_result3(data["dates"], data["price"], out)
157     chk = verify_result3(p, data["dates"], data["price"], out)
158     log.info("result3.xlsx -> {}", p)
159
160     make_figures(daily, base_noadj, base_off, out, data["dates"])
161     daily.to_csv(ROOT / "code" / "outputs" / "q3_daily.csv", index=False, encoding="utf-8-sig")
162
163     # 场景池无前视硬校验: 采样日序号必须小于目标日
164     day_idx = np.arange(DAY_START, DAY_START + N_Q3)
165     chk["场景池前视违规天数"] = int(
166         (daily["场景最大日序号"].to_numpy() >= day_idx).sum()
167     )
168     chk["场景对冲是否启用"] = bool(USE_HEDGE)
169
170     record(
171         "问题三 主方案结果 (2025-02-01 ~ 12-31, 334 天)",
172         {
173             "计划购电费/万元": round(cost["plan"] / 1e4, 1),
174             "调整净额/万元": round(cost["adj"] / 1e4, 1),
175             "紧急购电费/万元": round(cost["emerg"] / 1e4, 1),
176             "总费用/万元": round(cost["total"] / 1e4, 1),
177             "对照·无调整 (官方) /万元": round(base_noadj["total"] / 1e4, 1),
178             "对照·三点调整 (官方) /万元": round(base_off["total"] / 1e4, 1),
179             "主方案相对无调整": f"{100 * (cost['total'] / base_noadj['total'] - 1):.2f}%",
180             "主方案相对官方三点": f"{100 * (cost['total'] / base_off['total'] - 1):.2f}%",
181             "发生紧急购电天数": int((daily["紧急量/kWh"] > 1e-3).sum()),
182             "全程紧急购电量/kWh": float(daily["紧急量/kWh"].sum()),
183             "全程调整量/kWh": float(daily["调整量/kWh"].sum()),
184             **chk,
185         },
186         note=(
187             f"统一口径 (consistency.py): 负荷使用目标日前 EWMA h=5 权重预测并抬升 "
188             f"f={cs.KAPPA:g}, "
189             f"0:00 光伏用组合预测 (={LAM:g}·官方f0 + {1 - LAM:g}·历史 EWMA) "
190             f"并折减 m={cs.MARGIN:g} kW; "
191             "6/12/18 点用同组合口径的最新预报调整; 正式方案不使用场景对冲, "
192             "联合残差对冲仅保留为消融扩展; "
193             "储能采用 2 日预测窗口滚动优化, 当前日末 SOC 不固定并传递到下一日, "
194             "规划窗口末端完全自由 (不锚定终值); "
195             "逐槽因果执行 (q2.exec_segment_causal, "
196             "不读取未来实际值), 缺口 5 倍紧急. 结算 =  $\Sigma[p \cdot x_{adj} + 0.5p|x_{plan}-x_{adj}|] + 5\Sigma p \cdot e$ . "
197             "result3.xlsx 已生成并回读校验; 图 figures/Q3_策略费用对比.pdf、Q3_逐日紧急购电.pdf、"
198             "Q3_调整量分布.pdf; 逐日表 code/outputs/q3_daily.csv."
199         ),
200     )
201     print("完成。")
202
203
204 if __name__ == "__main__":
205     main()

```

program/src/solve/q3_proto.py

```

1  """C 题 问题三原型：官方预报 + 历史预测组合、滚动成本标定（兼容门面 + 原型入口）。
2
3  口径（原型，正式口径见 solve/q3.py）：
4  - 0:00 计划：光伏预测 = 官方 f0 + (1-) * 历史预测（口径 E 加权；u 来自 Q2E 缓存）
5  - 6/12/18 调整：用附件 3 最新预报对未执行时段重优化（偏差按 50%/150% 结算）
6  - 执行：购电承诺 take-or-pay，储能再调度（日循环），缺口 5 倍价紧急购电
7  - 结算：p * min(plan,adj) + 0.5p * (plan-adj) + 1.5p * (adj-plan) + 5p * e
8
9  对比策略：
10 - A: =1（纯官方）；B: =0.5（固定组合）；C: 滚动窗口 W 天费用最优（动态组合）
11 - D: =0（纯历史）；P: 完美下界（0:00 用实际光伏做计划，不调整）
12
13 实现分层（本文件只保留原型批处理 main 与兼容导出）：
14 - 调整/对冲 LP solve.core.lp; 槽位/残差池 solve.core.slots / solve.core.residual
15 - 单日模拟 solve.flows.q3_day; 数据扩展 solve.data.attachments
16 - 历史预报/平滑 solve.models.adaptive / solve.models.weights
17
18 运行（program/ 下）：
19     uv run python -m solve.q3_proto --start 52 --days 67
20 """
21 from __future__ import annotations
22
23 import argparse
24 import time
25
26 import numpy as np
27 import pandas as pd
28
29 import program as pm
30 from solve.common import EO, EMERG_MULT, EPS_TH, ROOT, T
31 from solve.core.causal import (
32     exec_day_causal,
33     exec_segment_causal,
34     run_exec,
35 )
36 from solve.core.lp import (
37     adjust_day,
38     adjust_day_hedge,
39     exec_day,
40     exec_segment_hindsight,
41     plan_day,
42     plan_horizon,
43 )
44 from solve.core.residual import (
45     causal_residual_pool,
46     forecast_at_publish,
47     latest_forecast,
48 )
49 from solve.core.slots import fc_slots
50 from solve.data.attachments import load_extended
51 from solve.flows.q3_day import (
52     forecast_residual,
53     joint_residual_blocks,
54     perfect_day,
55     plan_two_day,
56     simulate_day,
57     simulate_day_rt,
58     simulate_day_rt_hedge,
59 )
60 from solve.models.adaptive import (
61     hist_forecast,
62     hist_forecast_asof,
63     hist_load_forecast,
64     hist_load_forecast_asof,
65 )
66 from solve.models.weights import make_smooth_u
67 from solve.models.weights import softmax as _softmax
68
69 LAMBDA_GRID = np.round(np.arange(0.0, 1.0001, 0.1), 2) # 11 个组合权重

```

```

70 W_WINDOW = 7          # 滚动标定窗口（天）
71 DAY_TABLE_START = 52  # 建表起点（给滚动窗口预热）
72 DAY_EVAL_START = 59   # 评估起点 2025-03-01
73 DAY_END = 119         # 评估终点（不含）= 2025-04-30
74
75
76 # -----
77 # 主流程： 网格成本表 + 滚动窗口选择（无前视）
78 # -----
79 def main():
80     ap = argparse.ArgumentParser()
81     ap.add_argument("--start", type=int, default=DAY_TABLE_START)
82     ap.add_argument("--days", type=int, default=DAY_END - DAY_TABLE_START)
83     ap.add_argument("--eval-start", type=int, default=DAY_EVAL_START)
84     args = ap.parse_args()
85
86     pm.init(seed=42, root=str(ROOT))
87     t0 = time.time()
88     data = load_extended()
89     print(f"数据加载完成 {time.time() - t0:.1f}s, Q2E 权重示例 u = {np.round(_softmax(data['TH'][58, 3:6]), 3)}")
90
91     # ---- 成本表：行=日历日，列= 网格；simulate_day 为事后执行口径（原型对照） ----
92     days = list(range(args.start, args.start + args.days))
93     n1 = len(LAMBDA_GRID)
94     table = np.zeros((len(days), n1))
95     detail = []
96     ts = time.time()
97     for i, D in enumerate(days):
98         rd = []
99         for j, lam in enumerate(LAMBDA_GRID):
100             r = simulate_day(data, D, float(lam))
101             table[i, j] = r["total"]
102             rd.append({k: r[k] for k in ("total", "plan_cost", "adjust_net", "emerg", "adj_abs_kwh")})
103         detail.append(rd)
104         if (i + 1) % 10 == 0 or i == len(days) - 1:
105             print(f" {i + 1}/{len(days)} 天完成, 用时 {time.time() - ts:.0f}s")
106
107     # ---- 评估（滚动窗口在 table 行上，无前视） ----
108     i_eval = max(0, args.eval_start - args.start)
109     idx_l1, idx_half, idx_l0 = n1 - 1, int(np.argmax(np.abs(LAMBDA_GRID - 0.5))), 0
110
111     rows = []
112     for i in range(i_eval, len(days)):
113         i0 = max(0, i - W_WINDOW)
114         li = int(np.argmax(table[i0:i].mean(axis=0))) if i > i0 else idx_half
115         rows.append({
116             "day": days[i],
117             "date": data["dates"][days[i]],
118             "A_official": table[i, idx_l1],
119             "B_mix50": table[i, idx_half],
120             "C_adaptive": table[i, li],
121             "D_hist": table[i, idx_l0],
122             "lam_star": LAMBDA_GRID[li],
123             "perfect": perfect_day(data, days[i]),
124         })
125         if (i - i_eval + 1) % 20 == 0:
126             print(f" 评估 {i - i_eval + 1}/{len(days) - i_eval} 天")
127
128     df = pd.DataFrame(rows)
129     n = len(df)
130     print("\n==== Q3 原型结果 (%d 天: %s ~ %s) ===== % (n, df["date"].iloc[0], df["date"].iloc[-1]))
131     print(f"{'策略':<12}{'总费用/万元':>12}{'vs A':>9}{'日均/元':>11}")
132     for name, col in [("A 纯官方", "A_official"), ("B 固定0.5", "B_mix50"),
133                      ("C 动态", "C_adaptive"), ("D 纯历史", "D_hist"),
134                      ("P 完美下界", "perfect")]:
135         tot = df[col].sum()
136         tag = "--" if col == "A_official" else f"{{100 * (tot / df['A_official'].sum() - 1):+.2f}}%"
137         print(f"{{name:<12}}{{tot / 1e4:>12.1f}}{{tag:>9}}{{tot / n:>11.0f}}")
138

```

```

139 print("\n* 取值分布: ", dict(zip(*np.unique(df["lam_star"], return_counts=True))))
140 print(" * 均值: %.2f" % df["lam_star"].mean())
141
142 dec = []
143 for i in range(i_eval, len(days)):
144     i0 = max(0, i - W_WINDOW)
145     li = int(np.argmax(table[i0:i].mean(axis=0))) if i > i0 else idx_half
146     d_c = detail[i][li]
147     d_a = detail[i][idx_half]
148     dec.append({"plan_C": d_c["plan_cost"], "adj_C": d_c["adjust_net"], "em_C": d_c["emerg"],
149               "plan_A": d_a["plan_cost"], "adj_A": d_a["adjust_net"], "em_A": d_a["emerg"]})
150 dec = pd.DataFrame(dec)
151 print("\n费用分解 (元, 60 天合计): ")
152 print(f" A: 计划 {dec['plan_A'].sum() / 1e4:.1f} 万 + 调整净额 {dec['adj_A'].sum() / 1e4:.1f} 万 + 紧急 {dec['em_A'].sum() / 1e4:.1f} 万")
153 print(f" C: 计划 {dec['plan_C'].sum() / 1e4:.1f} 万 + 调整净额 {dec['adj_C'].sum() / 1e4:.1f} 万 + 紧急 {dec['em_C'].sum() / 1e4:.1f} 万")
154
155 out_dir = ROOT / "code" / "outputs"
156 out_dir.mkdir(parents=True, exist_ok=True)
157 pd.DataFrame(table, index=[data["dates"][d] for d in days],
158             columns=[f"lam={v:.1f}" for v in LAMBDA_GRID]).to_csv(
159     out_dir / "q3_proto_lambda_table.csv", encoding="utf-8-sig")
160 df.to_csv(out_dir / "q3_proto_eval.csv", index=False, encoding="utf-8-sig")
161 print(f"\n已保存: {out_dir / 'q3_proto_lambda_table.csv'}")
162 print(f"总用时 {time.time() - t0:.0f}s")
163
164
165 if __name__ == "__main__":
166     main()

```

program/src/solve/q3_ablation.py

```

1 """C 题 问题三 关键消融 (因果执行口径): 对冲 / 组合 / 信息退化的单独贡献。
2
3 正式主方案 = 组合 (=0.7) + 三点调整 + 无对冲 (统一口径 v1.4, free 执行器)。
4 联合残差对冲仅保留为实验扩展; 信息退化配置验证调整层必须同口径。
5
6 运行 (program/ 下): uv run python -m solve.q3_ablation
7 """
8 from __future__ import annotations
9
10 import time
11
12 import numpy as np
13 import pandas as pd
14
15 import program as pm
16 from solve import consistency as cs
17 from solve import q3_proto as qp
18 from solve.common import ROOT
19 from solve.io.report import record
20
21 # 统一口径: EWMA 权重由 load_extended() 提供; 情景数取唯一参数源。
22 N_SCEN = cs.N_SCEN
23 SEED = 7
24 LAM = cs.Q3_LAM
25 DAYS = list(range(31, 365))
26
27
28 def run_cfg(tag, fn, *, lam, adj_lam, hedge, data):
29     t0 = time.time()
30     tot = plan = em = 0.0
31     e_kwh = 0.0
32     E = float(qp.E0) # 跨日 SOC 连续 (与官方主程序一致)
33     for D in DAYS:
34         kw = {"adj_lam": adj_lam, "e_start": E}
35         if hedge:
36             kw.update({"n_scen": N_SCEN, "seed": SEED})

```

```

37     r = fn(data, D, lam, **kw)
38     tot += r["total"]
39     plan += r["plan_cost"]
40     em += r["emerg"]
41     e_kwh += float(r["e"].sum())
42     E = float(r["E_end"])
43     print(f" {tag:<22} 总 {tot/1e4:7.1f} 万 (计划 {plan/1e4:.1f} + 紧急 {em/1e4:.1f}) "
44           f" 紧急量 {e_kwh:,.0f} kWh ({time.time()-t0:.0f}s) ")
45     return {"配置": tag, "总费用/万元": round(tot / 1e4, 1),
46            "计划费/万元": round(plan / 1e4, 1), "紧急费/万元": round(em / 1e4, 1)}
47
48
49 def main():
50     pm.init(seed=42, root=str(ROOT))
51     data = qp.load_extended() # 含统一 EWMA 权重 (EWMA_WU)
52     print("Q3 消融 (因果口径, 334 天): ")
53     rows = [
54         run_cfg("官方 · 三点 · 无对冲", qp.simulate_day_rt,
55                lam=1.0, adj_lam=1.0, hedge=False, data=data),
56         run_cfg("正式主方案 (组合无对冲)", qp.simulate_day_rt,
57                lam=LAM, adj_lam=LAM, hedge=False, data=data),
58         run_cfg("对冲扩展 (组合+对冲)", qp.simulate_day_rt_hedge,
59                lam=LAM, adj_lam=LAM, hedge=True, data=data),
60         run_cfg("信息退化 (调整层仅官方)", qp.simulate_day_rt_hedge,
61                lam=LAM, adj_lam=1.0, hedge=True, data=data),
62     ]
63     df = pd.DataFrame(rows)
64     pm.save_outputs(df, "q3_ablation")
65     record(
66         "问题三 消融实验 (因果口径, 2025-02-01 ~ 12-31)",
67         df,
68         note=(
69             "因果执行口径下的单项消融 (跨日 SOC 连续, 与官方主程序一致): "
70             "官方 · 三点 · 无对冲 (无组合); 正式主方案 (组合 = 0.7 + 三点、无对冲)"
71             "——应与官方 result3 同值; 对冲扩展; 信息退化 (计划组合、调整层仅官方, 含对冲)。"
72             "实验显示对冲增费, 故 v1.4 正式方案关闭对冲。"
73             "对照基线 (无调整 / 三点官方 / 主方案) 见 RESULTS_REPORT「问题三 主方案结果」章节。"
74         ),
75     )
76     print("已保存 code/outputs/q3_ablation.csv")
77
78
79 if __name__ == "__main__":
80     main()

```

program/src/solve/q3_sensitivity.py

```

1  """Q3 对冲扩展灵敏度: 情景数 / 场景池窗口 / 组合权重 (不进入正式结果)。
2
3  对照 GPT 版"场景数 5/10/20 + 窗口"的灵敏度思路; 本工作区 Q3 的 为固定值
4  (无滚动窗口), 因此等价地补测:
5  - 对冲情景数 n_scen {5, 10, 20, 40}, 并对 10/20/40 使用 3 个随机种子
6  - 场景池回看窗口 lookback {30, 60, 90} (扩展基准 90)
7  - 组合权重 {0.5, 0.7, 0.9} (扩展基准 0.7)
8
9  运行 (program/ 下): uv run python -m solve.q3_sensitivity
10 """
11 from __future__ import annotations
12
13 import os
14 import time
15
16 import numpy as np
17 import pandas as pd
18
19 import program as pm
20 from solve import q2
21 from solve import q3_proto as qp
22 from solve.common import ROOT

```

```

23 from solve.io.report import record
24
25 # 统一口径: `qp.load_extended()` 已生成 EWMA h=5 权重 (data["EWMA_WU"]),
26 # 旧版 平滑 (U_SMOOTH) 已退役, 不再设置。
27 DAYS = list(range(q2.REPORT_START, q2.N_DAY))
28
29 _P = None
30
31
32 def _init(payload):
33     global _P
34     _P = payload
35
36
37 def _task(t):
38     """单个变量: 变体内逐日串行、执行 SOC 跨日连续 (与官方主程序一致)。"""
39     vi, name, v = t
40     E = float(qp.E0)
41     plan = adj = em = em_kwh = 0.0
42     viol = 0
43     for D in DAYS:
44         r = qp.simulate_day_rt_hedge(
45             _P, D, v["lam"], adj_lam=v["lam"], n_scen=v["n_scen"], seed=v["seed"],
46             pool_min_month=v["pmm"], pool_lookback=v["plb"], e_start=E,
47         )
48         plan += float(r["plan_cost"])
49         adj += float(r["adjust_net"])
50         em += float(r["emerg"])
51         em_kwh += float(r["e"].sum())
52         viol += int(r.get("scenario_max_day", -1) >= D)
53         E = float(r["E_end"])
54     return (vi, plan, adj, em, em_kwh, viol)
55
56
57 VARIANTS = [
58     ("对冲扩展基准 (情景 40 / seed=7)", dict(n_scen=40, pmm=14, plb=90, lam=0.7, seed=7)),
59     ("情景数 5 / seed=7", dict(n_scen=5, pmm=14, plb=90, lam=0.7, seed=7)),
60     ("情景数 10 / seed=7", dict(n_scen=10, pmm=14, plb=90, lam=0.7, seed=7)),
61     ("情景数 10 / seed=17", dict(n_scen=10, pmm=14, plb=90, lam=0.7, seed=17)),
62     ("情景数 10 / seed=27", dict(n_scen=10, pmm=14, plb=90, lam=0.7, seed=27)),
63     ("情景数 20 / seed=7", dict(n_scen=20, pmm=14, plb=90, lam=0.7, seed=7)),
64     ("情景数 20 / seed=17", dict(n_scen=20, pmm=14, plb=90, lam=0.7, seed=17)),
65     ("情景数 20 / seed=27", dict(n_scen=20, pmm=14, plb=90, lam=0.7, seed=27)),
66     ("情景数 40 / seed=17", dict(n_scen=40, pmm=14, plb=90, lam=0.7, seed=17)),
67     ("情景数 40 / seed=27", dict(n_scen=40, pmm=14, plb=90, lam=0.7, seed=27)),
68     ("场景池回看 30 天", dict(n_scen=10, pmm=14, plb=30, lam=0.7, seed=7)),
69     ("场景池回看 60 天", dict(n_scen=10, pmm=14, plb=60, lam=0.7, seed=7)),
70     ("组合权重 =0.5", dict(n_scen=10, pmm=14, plb=90, lam=0.5, seed=7)),
71     ("组合权重 =0.9", dict(n_scen=10, pmm=14, plb=90, lam=0.9, seed=7)),
72 ]
73
74
75
76
77 def main():
78     pm.init(seed=42, root=str(ROOT))
79     log = pm.get_logger("q3-sens")
80     t0 = time.time()
81     data = qp.load_extended() # 含统一 EWMA 权重 (EWMA_WU)
82
83     # 变体间并行、变体内逐日串行: 执行 SOC 跨日连续, 与官方主程序一致。
84     tasks = [(vi, name, v) for vi, (name, v) in enumerate(VARIANTS)]
85     n = len(VARIANTS)
86     plan = np.zeros(n)
87     adj = np.zeros(n)
88     em = np.zeros(n)
89     em_kwh = np.zeros(n)
90     viol = np.zeros(n, dtype=int)
91     nw = min(8, os.cpu_count() or 1)

```

```

92     from multiprocessing import Pool
93
94     with Pool(nw, initializer=_init, initargs=(data,)) as pool:
95         for vi, c_plan, c_adj, c_em, e_kwh, n_viol in pool.imap_unordered(
96             _task, tasks, chunksize=1):
97             plan[vi] += c_plan
98             adj[vi] += c_adj
99             em[vi] += c_em
100            em_kwh[vi] += e_kwh
101            viol[vi] += n_viol
102            print(f" 变体 {vi + 1}/{n} 完成, 用时 {time.time() - t0:.0f}s")
103
104    out = pd.DataFrame({
105        "配置": [name for name, _ in VARIANTS],
106        "计划费/万元": np.round(plan / 1e4, 1),
107        "调整净额/万元": np.round(adj / 1e4, 1),
108        "紧急费/万元": np.round(em / 1e4, 1),
109    })
110    out["总费用/万元"] = (out["计划费/万元"] + out["调整净额/万元"] + out["紧急费/万元"]).round(1)
111    base = float(out.loc[0, "总费用/万元"])
112    out["相对扩展基准/%"] = (100 * (out["总费用/万元"] / base - 1)).round(2)
113    out["场景前视违规天数"] = viol
114    pm.save_outputs(out, "q3_sensitivity")
115    print(out.to_string(index=False))
116
117    record(
118        "问题三 对冲扩展灵敏度: 情景数 / 场景池窗口 / 组合权重 (因果口径)",
119        out,
120        note=(
121            "对已退出正式方案的场景对冲扩展做单因素灵敏度: "
122            "对冲情景数使用 seed=7/17/27 检查随机稳定性, 并单独改变场景池回看窗口、"
123            "组合权重 ; 场景前视违规天数应全为 0。"
124            "数据 code/outputs/q3_sensitivity.csv"),
125    )
126    log.info("Q3 灵敏度完成 ({:.0f}s)".format(time.time() - t0))
127    return out
128
129 if __name__ == "__main__":
130     main()

```

program/src/solve/q3_tune.py

```

1  """Q3 对冲扩展下的历史风险参数时间留出实验 ( / / m) 。
2
3  在统一结构 (2 日滚动 + 因果负荷 + EWMA h=5 预测 + /m + (Δ负荷, Δ光伏) 联合残差
4  40 情景对冲) 下, 对已退出正式方案的扩展模型做两级扫描:
5
6  A. 粗筛: =0.7 固定, {1.0, 1.01, 1.015, 1.02} × m {0, 25, 50, 75} (16 组);
7  B. 细选: 最优 (, m) 上, {0.3, 0.5, 0.7, 0.9, 1.0} (5 组)。
8
9  协议: 开发期 2025-02-01 ~ 06-30 只负责选参; 冻结验证期 2025-07-01 ~ 12-31
10  不参与选择 (同 q2_tune 的时间留出做法)。参数经 ``consistency.py`` 透传,
11  随机流由 (seed, D, pub) 固定, 各配置共用随机数。
12
13  输出: code/outputs/q3e_tune_unified_{coarse,fine}.csv、
14        figures/Q3E_参数搜索.pdf、RESULTS_REPORT 章节。
15
16  运行 (program/ 下): uv run python -m solve.q3_tune
17  """
18  from __future__ import annotations
19
20  import time
21  from multiprocessing import Pool
22
23  import numpy as np
24  import pandas as pd
25
26  import program as pm
27  from solve import consistency as cs

```

```

28 from solve import q3_proto as qp
29 from solve.common import ROOT
30 from solve.io.report import record
31
32 REP = list(range(cs.START_DAY, 365))
33 LAM_FIX = 0.7
34 KAPPA_GRID = (1.0, 1.01, 1.015, 1.02)
35 MARGIN_GRID = (0.0, 25.0, 50.0, 75.0)
36 LAM_GRID = (0.3, 0.5, 0.7, 0.9, 1.0)
37 SEED = 7
38 WORKERS = 6
39
40 _P = None # 子进程全局负载
41
42
43 def _init_worker(payload) -> None:
44     global _P
45     _P = payload
46
47
48 def _task(cfg) -> dict:
49     lam, kappa, margin = cfg
50     p = _P
51     data = p["data"]
52     t0 = time.time()
53     E = float(qp.E0)
54     costs = np.zeros(len(REP))
55     for i, D in enumerate(REP):
56         r = qp.simulate_day_rt_hedge(
57             data, D, lam, adj_lam=lam, n_scen=cs.N_SCEN, seed=SEED,
58             kappa=kappa, margin=margin, e_start=E)
59         costs[i] = r["total"]
60         E = float(r["E_end"])
61     split = p["split"]
62     dev = float(costs[split:].sum()) / 1e4
63     val = float(costs[split:].sum()) / 1e4
64     print(f"    {lam:g} = {kappa:g} m={margin:g}: 开发 {dev:.1f} 万 / "
65           f"冻结 {val:.1f} 万 ({time.time() - t0:.0f}s) ")
66     return {"配置": f"{lam:g} = {kappa:g} m={margin:g}", "lam": lam,
67            "kappa": kappa, "margin": margin,
68            "开发期(2-6月)/万元": round(dev, 1),
69            "冻结验证期(7-12月)/万元": round(val, 1),
70            "全年描述值/万元": round(dev + val, 1)}
71
72
73 def _run(payload, cfgs, workers=WORKERS) -> pd.DataFrame:
74     with Pool(workers, initializer=_init_worker, initargs=(payload,)) as pool:
75         rows = pool.map(_task, cfgs)
76     return pd.DataFrame(rows)
77
78
79 def main() -> None:
80     pm.init(seed=42, root=str(ROOT))
81     t0 = time.time()
82     data = qp.load_extended()
83     split = next(i for i, d in enumerate(REP)
84                  if data["dates"][d] == cs.HOLDOUT_VAL_START)
85     payload = {"data": data, "split": split}
86     print(f"对冲扩展历史参数搜索: 开发期到 2025-06-30 ({split} 天), "
87           f"冻结验证期起 {cs.HOLDOUT_VAL_START}; 情景数 {cs.N_SCEN}, workers={WORKERS}")
88
89     # ----- A. 粗筛 x m (固定) -----
90     cfgs = [(LAM_FIX, k, m) for k in KAPPA_GRID for m in MARGIN_GRID]
91     coarse = _run(payload, cfgs)
92     coarse = coarse.sort_values(
93         ["开发期(2-6月)/万元", "冻结验证期(7-12月)/万元"]).reset_index(drop=True)
94     coarse["开发期选中"] = coarse.index == 0
95     coarse["验证期排名"] = coarse["冻结验证期(7-12月)/万元"].rank(method="min").astype(int)
96     coarse.to_csv(pm.outputs_dir() / "q3e_tune_unified_coarse.csv",

```

```

97         index=False, encoding="utf-8-sig")
98     best = coarse.iloc[0]
99     print(f"粗筛选中: {best['配置']} (开发 {best['开发期(2-6月)']/万元'}) 万, "
100           f"冻结 {best['冻结验证期(7-12月)']/万元'}) 万, 验证排名 {best['验证期排名']}")
101
102     # ----- B. 细选 -----
103     cfgs2 = [(lam, float(best["kappa"]), float(best["margin"])) for lam in LAM_GRID]
104     fine = _run(payload, cfgs2, workers=min(WORKERS, len(cfgs2)))
105     fine = fine.sort_values(
106         ["开发期(2-6月)'/万元", "冻结验证期(7-12月)'/万元"]).reset_index(drop=True)
107     fine["开发期选中"] = fine.index == 0
108     fine["验证期排名"] = fine["冻结验证期(7-12月)'/万元"].rank(method="min").astype(int)
109     fine.to_csv(pm.outputs_dir() / "q3e_tune_unified_fine.csv",
110               index=False, encoding="utf-8-sig")
111     best2 = fine.iloc[0]
112     print(f"细筛选中: {best2['配置']} (开发 {best2['开发期(2-6月)']/万元'}) 万, "
113           f"冻结 {best2['冻结验证期(7-12月)']/万元'}) 万, 验证排名 {best2['验证期排名']}")
114
115     # ----- 图 -----
116     import matplotlib.pyplot as plt
117
118     fig, axes = plt.subplots(1, 2, figsize=(7.2, 4.0))
119     ax = axes[0]
120     for k in KAPPA_GRID:
121         g = coarse[np.isclose(coarse["kappa"], k)]
122         ax.plot(g["margin"], g["开发期(2-6月)'/万元"], marker="o", ms=3,
123               label=f"={k:g} (开发期)")
124     ax.set_xlabel("光伏折减 m / kW")
125     ax.set_ylabel("开发期费用 / 万元")
126     ax.set_title("(a) 粗筛  $\times m$  (开发期)", fontsize=9)
127     ax.legend(fontsize=7)
128     ax = axes[1]
129     ax.plot(fine["lam"], fine["开发期(2-6月)'/万元"], marker="o", label="开发期 (选参)")
130     ax.plot(fine["lam"], fine["冻结验证期(7-12月)'/万元"], marker="s", label="冻结验证期")
131     ax.set_xlabel("组合权重")
132     ax.set_ylabel("费用 / 万元")
133     ax.set_title("(b) 细选", fontsize=9)
134     ax.legend(fontsize=7)
135     fig.tight_layout()
136     pm.save_fig(fig, "Q3E_参数搜索",
137               data=pd.concat([coarse.assign(阶段="粗筛  $\times m$ "),
138                             fine.assign(阶段="细选")], ignore_index=True))
139
140     # ----- 报告 -----
141     record("问题三 对冲扩展历史参数搜索: 粗筛 ( $\times m$ )", coarse,
142           note=(f"已退出正式方案的对冲扩展 (2 日滚动+因果负荷+EWMA+联合残差 40 情景) 下, "
143                 f"={LAM_FIX:g} 固定, {KAPPA_GRID}、m {MARGIN_GRID}; "
144                 f"开发期 2-6 月选参, 7-12 月冻结验证; 常用随机数 (seed=7)。"))
145     record("问题三 对冲扩展历史参数搜索: 细选", fine,
146           note=(f"在粗筛最优 ( $\times m$ )=({best['kappa']:g},{best['margin']:g}) 上扫描 "
147                 f"={LAM_GRID}; 开发期选择、冻结验证。"))
148     record("问题三 对冲扩展参数搜索: 汇总",
149           pd.DataFrame([{"阶段": "粗筛", **best.to_dict()},
150                         {"阶段": "细选", **best2.to_dict()}]),
151           note=(f"这是对对冲扩展的历史搜索, 不再决定 v1.4 正式策略; 验证期仅报告排名。"
152                 f"完整表见 code/outputs/q3e_tune_unified_*.csv; "
153                 f"图 figures/Q3E_参数搜索.pdf。"))
154     print(f"完成, 用时 {time.time() - t0:.0f}s")
155
156
157 if __name__ == "__main__":
158     main()

```

program/src/solve/q4.py

```

1 """C 题 问题四: 实时波动电价下的重算 (Q2 层 / Q3 层)。
2
3 口径:
4 - 电价 G (题面主口径): 未来电价已由附件 4 给出, 决策直接使用对应日价格;

```

```

5     H (仅历史价格) 是额外的信息受限扩展, 用滚动费用标定  $v(W=7) + \approx 0.1$  动态修正。
6 - 负荷由目标日前历史数据预测 (•EWMA  $h=5$ ); 光伏用附件 3 的 0:00 官方预报,
7   2 日视野中的次日用 asof 历史自适应预报 (无前视)。
8 - 执行: 逐槽因果 (core.causal.exec_segment_causal, free 策略, 不读未来实际值)。
9 - 储能结构: 日循环 vs 2 日滚动 (跨日连续, 末端自由), 按费用择优。
10
11 # =====
12 # 费用口径 (结算一律用附件 4 真实电价 p4)
13 #   Q2 层:  $C = \sum p4 \cdot x + 5 \cdot \sum p4 \cdot e$ 
14 #   Q3 层:  $C = \sum [p4 \cdot x_{adj} + 0.5 \cdot p4 \cdot |x_{plan} - x_{adj}|] + 5 \cdot \sum p4 \cdot e$ 
15 # 决策价 (H):  $P = v1 \cdot P4(D-1) + v2 \cdot P4(D-7) + v3 \cdot P_{typ}$  (见 models/price.py)
16 # 结构: daily = 日循环 plan_day; 2day = plan_two_day (48h 滚动、末端自由)
17 # =====
18
19 实现分层: 年度流程在 solve.flows.q4_year, 结果表在 solve.io.excel,
20 电价模型在 solve.models.price; 本文件只做编排、校验与出图。
21
22 运行 (program/ 下):
23     uv run python -m solve.q4 --probe          # 结构对比探针
24     uv run python -m solve.q4 --result4-2      # 生成 result4-2.xlsx (Q2 层)
25     uv run python -m solve.q4 --result4-3      # 生成 result4-3.xlsx (Q3 层)
26
27 from __future__ import annotations
28
29 import argparse
30 import time
31
32 import numpy as np
33 import pandas as pd
34
35 import program as pm
36 from solve import consistency as cs
37 from solve.common import E_MAX, E_MIN, ETA, N_DAY, REPORT_START, ROOT
38 from solve.core.lp import plan_horizon # noqa: F401 (公开 API: 供探针与测试引用)
39 from solve.data.attachments import load_q4_data
40 from solve.flows.q4_year import (
41     run_year,
42     run_year_q3,
43     simulate_day_q3, # noqa: F401 (公开 API: 守卫测试校验默认对冲开关)
44     total_of,
45     total_of_q3,
46 )
47 from solve.io.checks import verify_records
48 from solve.io.excel import (
49     verify_result4_2,
50     verify_result4_3,
51     write_result4_2,
52     write_result4_3,
53 )
54 from solve.io.report import record
55 from solve.models.price import price_forecast_d, price_forecast_next, rolling_v_seq # noqa: F401
56
57
58 # -----
59 # 图表 / 报告
60 # -----
61 def make_figures(data, recs_h, recs_g, s_h, s_g, daily) -> None:
62     """Q2 层论文图: H/G 费用对比 + 逐日紧急购电。"""
63     pm.bar_group(
64         ["计划购电费", "紧急购电费"],
65         {"H·历史预测": [s_h["plan"] / 1e4, s_h["emerg"] / 1e4],
66          "G·题面已知电价": [s_g["plan"] / 1e4, s_g["emerg"] / 1e4]},
67         ylabel="334 天费用 / 万元", save="Q4_H与G费用对比",
68     )
69     days = list(range(REPORT_START, N_DAY))
70     e_h = np.array([float(recs_h[D]["e"]).sum() for D in days])
71     e_g = np.array([float(recs_g[D]["e"]).sum() for D in days])
72     fig, ax = pm.line(
73         np.arange(len(days)), [e_h, e_g],

```

```

74     labels=["H·历史预测", "G·题面已知电价"],
75     xlabel="日期 (2025-02-01 起)", ylabel="紧急购电量 / kWh",
76 )
77 pm.save_fig(fig, "Q4_逐日紧急购电",
78             data=pd.DataFrame({"日期": [data["dates"][D] for D in days],
79                                "H紧急量/kWh": e_h, "G紧急量/kWh": e_g}))
80
81
82 # -----
83 # Q2 层主流程
84 # -----
85 def run_result42() -> None:
86     """生成 result4-2 (H·2 日滚动为主口径; G 同结构对照)。"""
87     pm.init(seed=42, root=str(ROOT))
88     log = pm.get_logger("q4")
89     t0 = time.time()
90     data, p4, p_typ = load_q4_data()
91     vseq = rolling_v_seq()
92     recs_h = run_year(data, p4, p_typ, vseq, price_mode="H", storage="2day")
93     recs_g = run_year(data, p4, p_typ, vseq, price_mode="G", storage="2day")
94     s_h, s_g = total_of(recs_h), total_of(recs_g)
95     chk = verify_records(recs_g, data, x_key="x")
96     print(f"问题四 Q2 层: H {s_h['total']/1e4:.1f} 万 (计划 {s_h['plan']/1e4:.1f} + "
97           f"紧急 {s_h['emerg']/1e4:.1f}); G {s_g['total']/1e4:.1f} 万; "
98           f"电价信息价值 {100*(s_h['total']-s_g['total'])/s_h['total']:.2f}% ({time.time()-t0:.0f}s) ")
99
100     p = write_result4_2(data, p4, recs_g)
101     excel = verify_result4_2(p, data, p4, recs_g)
102     log.info("result4-2.xlsx -> {}", p)
103
104     days = list(range(REPORT_START, N_DAY))
105     daily = pd.DataFrame({
106         "日期": [data["dates"][D] for D in days],
107         "计划购电费/元": [recs_g[D]["plan_cost"] for D in days],
108         "紧急购电量/kWh": [float(recs_g[D]["e"].sum()) for D in days],
109         "紧急购电费/元": [recs_g[D]["emerg"] for D in days],
110         "总费用/元": [recs_g[D]["total"] for D in days],
111     })
112     pm.save_outputs(daily, "q4_result42_daily")
113     make_figures(data, recs_h, recs_g, s_h, s_g, daily)
114
115     record(
116         "问题四 结果: Q2 层 (result4-2, 2025-02-01 ~ 12-31)",
117         {
118             "题面口径 G·总费用/万元": round(s_g["total"] / 1e4, 1),
119             "G·计划购电费/万元": round(s_g["plan"] / 1e4, 1),
120             "G·紧急购电费/万元": round(s_g["emerg"] / 1e4, 1),
121             "G·紧急购电量/kWh": float(s_g["emerg_kwh"]),
122             "扩展口径 H·历史价格预测/万元": round(s_h["total"] / 1e4, 1),
123             "电价信息价值 (H 比 G 高)": f"{100 * (s_h['total'] - s_g['total']) / s_h['total']:.2f}%",
124             "储能结构": "2 日滚动 (跨日连续)",
125             **excel,
126         },
127         note=(
128             "题面口径 G: 附件 4 已给未来电价, 0:00 用历史负荷预测 + 附件 3 的 0:00 光伏预报; "
129             "采用 2 日滚动计划、逐槽因果执行并按附件 4 结算。H 口径不读取目标日价格, "
130             "仅作为信息受限扩展, 不写入正式结果表。"
131             "图 figures/Q4_H与G费用对比.pdf、Q4_逐日紧急购电.pdf; "
132             "逐日表 code/outputs/q4_result42_daily.csv; 结构探针 code/outputs/q4_structure_probe.csv。"
133         ),
134     )
135     record("问题四 约束与一致性校验", chk,
136           note="功率平衡、SOC 递推、区间与充放互斥全部回代; 残差应为浮点误差量级。")
137
138
139 # -----
140 # Q3 层: 0:00 计划 + 6/12/18 调整 (+6/12 对冲), 结算用附件 4 真实价
141 # -----
142 def probe_q3() -> None:

```

```

143     """Q4-3 结构探针: 价格口径 × 储能结构 × 调整/对冲。"""
144     pm.init(seed=42, root=str(ROOT))
145     data, p4, p_typ = load_q4_data()
146     vseq = rolling_v_seq()
147     cfigs = [
148         ("H", "daily", (), False, 1.0, 1.0),
149         ("H", "daily", (6, 12, 18), False, 1.0, 1.0),
150         ("H", "daily", (6, 12, 18), True, 1.0, 1.0),
151         ("H", "daily", (6, 12, 18), True, 0.7, 0.7),
152         ("H", "2day", (6, 12, 18), True, 0.7, 0.7),
153         ("G", "2day", (6, 12, 18), True, 0.7, 0.7),
154     ]
155     rows = []
156     for mode, storage, adj, hedge, lam, adj_lam in cfigs:
157         t0 = time.time()
158         recs = run_year_q3(data, p4, p_typ, vseq, price_mode=mode,
159                           storage=storage, adj_hours=adj, hedge=hedge,
160                           lam=lam, adj_lam=adj_lam)
161         s = total_of_q3(recs)
162         tag = (f"{mode} • {storage} • {'三点' if adj else '无调整'}"
163               f"{'+对冲' if hedge else ''}{'组合' if lam < 0.9 else '官方'}")
164         print(f" [{tag:<16}] 总 {s['total']/1e4:7.1f} 万 (买入 {s['buy']/1e4:.1f} + "
165               f"偏差 {s['dev']/1e4:.1f} + 紧急 {s['emerg']/1e4:.1f}) "
166               f"调整量 {s['adj_kwh']:,.0f} kWh ({time.time()-t0:.0f}s) ")
167         rows.append({"配置": tag, "计划费": s["plan"], "买入": s["buy"],
168                    "偏差费": s["dev"], "紧急": s["emerg"], "总费用": s["total"]})
169     pd.DataFrame(rows).to_csv(ROOT / "code" / "outputs" / "q4_q3_probe.csv",
170                    index=False, encoding="utf-8-sig")
171
172
173 def make_figures_q3(data, recs_g, recs_h, s_g, s_h, daily_df) -> None:
174     """Q3 层论文图: G/H 费用构成对比 + 题面口径 G 的逐日紧急购电。"""
175     # 1) 同一正式策略下的电价信息口径对比
176     pm.bar_group(
177         ["买入费", "偏差费", "紧急购电费"],
178         {"G • 题面已知电价": [s_g["buy"] / 1e4, s_g["dev"] / 1e4, s_g["emerg"] / 1e4],
179          "H • 历史价格预测": [s_h["buy"] / 1e4, s_h["dev"] / 1e4, s_h["emerg"] / 1e4]},
180         ylabel="334 天费用 / 万元", save="Q4_调整层策略对比",
181     )
182     # 2) 题面口径 G 的逐日紧急购电
183     fig, ax = pm.line(
184         np.arange(len(daily_df)), daily_df["紧急购电量/kWh"],
185         xlabel="日期 (2025-02-01 起)", ylabel="紧急购电量 / kWh",
186     )
187     pm.save_fig(fig, "Q4_调整层逐日紧急购电",
188                 data=daily_df[["日期", "紧急购电量/kWh"]])
189
190
191 def run_result43() -> None:
192     """生成 result4-3: G • 2 日 • 三点调整 • 组合预测 • 无对冲; H 作扩展对照。"""
193     pm.init(seed=42, root=str(ROOT))
194     log = pm.get_logger("q4")
195     t0 = time.time()
196     data, p4, p_typ = load_q4_data()
197     vseq = rolling_v_seq()
198     kw = dict(storage="2day", adj_hours=(6, 12, 18), hedge=cs.Q3_USE_HEDGE,
199               n_scen=cs.N_SCEN, seed=7, lam=cs.Q3_LAM, adj_lam=cs.Q3_ADJ_LAM)
200     recs_h = run_year_q3(data, p4, p_typ, vseq, price_mode="H", **kw)
201     recs_g = run_year_q3(data, p4, p_typ, vseq, price_mode="G", **kw)
202     s_h, s_g = total_of_q3(recs_h), total_of_q3(recs_g)
203     chk = verify_records(recs_g, data, x_key="x_final", scenario=True)
204     print(f"问题四 Q3 层: H {s_h['total']/1e4:.1f} 万 (买入 {s_h['buy']/1e4:.1f} + "
205           f"偏差 {s_h['dev']/1e4:.1f} + 紧急 {s_h['emerg']/1e4:.1f}); G {s_g['total']/1e4:.1f} 万"
206           f" ({time.time()-t0:.0f}s) ")
207
208     p = write_result4_3(data, p4, recs_g)
209     excel = verify_result4_3(p, data, p4, recs_g)
210     log.info("result4-3.xlsx -> {}", p)
211

```

```

212 days = list(range(REPORT_START, N_DAY))
213 daily = pd.DataFrame({
214     "日期": [data["dates"][D] for D in days],
215     "计划购电费/元": [recs_g[D]["plan_cost"] for D in days],
216     "买入/元": [recs_g[D]["buy"] for D in days],
217     "偏差费/元": [recs_g[D]["dev"] for D in days],
218     "紧急购电费/元": [recs_g[D]["emerg"] for D in days],
219     "总费用/元": [recs_g[D]["total"] for D in days],
220     "紧急购电量/kWh": [float(recs_g[D]["e"].sum()) for D in days],
221     "调整量/kWh": [float(np.abs(recs_g[D]["x_final"] - recs_g[D]["x_plan"]).sum())
222                     for D in days],
223 })
224 pm.save_outputs(daily, "q4_result43_daily")
225 make_figures_q3(data, recs_g, recs_h, s_g, s_h, daily)
226
227 record(
228     "问题四 结果: Q3 层 (result4-3, 2025-02-01 ~ 12-31) ",
229     {
230         "题面口径 G·总费用/万元": round(s_g["total"] / 1e4, 1),
231         "G·买入/万元": round(s_g["buy"] / 1e4, 1),
232         "G·偏差费/万元": round(s_g["dev"] / 1e4, 1),
233         "G·紧急购电费/万元": round(s_g["emerg"] / 1e4, 1),
234         "G·紧急购电量/kWh": float(s_g["emerg_kwh"]),
235         "G·调整量/kWh": float(s_g["adj_kwh"]),
236         "扩展口径 H·历史价格预测/万元": round(s_h["total"] / 1e4, 1),
237         "电价信息价值 (H 比 G 高)": f"{100 * (s_h['total'] - s_g['total']) / s_h['total']:.2f}%",
238         "储能结构": "2 日滚动 (跨日连续)",
239         **excel,
240     },
241     note=(
242         "题面口径 G: 0:00 计划使用附件 4 已知电价、历史负荷预测及组合光伏预测; "
243         "6/12/18 更新光伏并调整: 正式方案关闭场景对冲, 逐槽因果执行; "
244         "采用 2 日滚动跨日储能并按附件 4 结算。H 不读取目标日价格, 仅作扩展对照。"
245         "图 figures/Q4_调整层策略对比.pdf、Q4_调整层逐日紧急购电.pdf; "
246         "逐日表 code/outputs/q4_result43_daily.csv; 探针表 code/outputs/q4_q3_probe.csv。"
247     ),
248 )
249 record("问题四 Q3 层约束与校验", chk,
250     note="功率平衡、SOC 递推、区间与充放互斥全部回代通过; 正式方案未启用场景池。")
251
252
253 def fig_weights() -> None:
254     """电价预测权重演化图: 滚动费用标定 v* + 动态修正 =0.1 (2025-02-01 起)。"""
255     pm.init(seed=42, root=str(ROOT))
256     data, p4, p_ttyp = load_q4_data()
257     vseq = rolling_v_seq()
258     days = list(range(REPORT_START, N_DAY))
259     df = pd.DataFrame({
260         "日期": [data["dates"][D] for D in days],
261         "v1(D-1)": vseq[days, 0], "v2(D-7)": vseq[days, 1], "v3(典型日)": vseq[days, 2],
262     })
263     fig, ax = pm.line(
264         np.arange(len(days)), [vseq[days, 0], vseq[days, 1], vseq[days, 2]],
265         labels=["$v_1$·P(D-1)", "$v_2$·P(D-7)", "$v_3$·典型日"],
266         xlabel="日期 (2025-02-01 起)", ylabel="电价预测权重",
267     )
268     pm.save_fig(fig, "Q4_电价权重演化", data=df)
269     print("已生成 figures/Q4_电价权重演化.pdf")
270
271
272 # -----
273 # 结构探针
274 # -----
275 def probe() -> None:
276     """Q4-2 结构探针: 价格口径 × 储能结构 (daily vs 2day)。"""
277     pm.init(seed=42, root=str(ROOT))
278     t0 = time.time()
279     data, p4, p_ttyp = load_q4_data()
280     vseq = rolling_v_seq()

```

```

281 print(f"数据就绪 ({time.time()-t0:.0f}s), v 序列 D=31..364: ")
282     f"v1 {vseq[31:,0].mean():.2f} / v2 {vseq[31:,1].mean():.2f} / v3 {vseq[31:,2].mean():.2f}")
283 rows = []
284 for price_mode in ("H", "G"):
285     for storage in ("daily", "2day"):
286         t1 = time.time()
287         recs = run_year(data, p4, p_ttyp, vseq, price_mode=price_mode, storage=storage)
288         s = total_of(recs)
289         print(f" [{price_mode} * {storage:5}] 总 {s['total']/1e4:7.1f} 万"
290               f" (计划 {s['plan']/1e4:.1f} + 紧急 {s['emerg']/1e4:.1f}) "
291               f" 紧急量 {s['emerg_kwh']:, .0f} kWh ({time.time()-t1:.0f}s) ")
292         rows.append({"价格口径": price_mode, "储能结构": storage, **s})
293 pd.DataFrame(rows).to_csv(ROOT / "code" / "outputs" / "q4_structure_probe.csv",
294                       index=False, encoding="utf-8-sig")
295
296
297 if __name__ == "__main__":
298     ap = argparse.ArgumentParser()
299     ap.add_argument("--probe", action="store_true", help="储能/价格口径结构对比探针")
300     ap.add_argument("--probe-q3", action="store_true", help="Q3 层结构探针")
301     ap.add_argument("--result4-2", action="store_true", help="生成 result4-2.xlsx (Q2 层) ")
302     ap.add_argument("--result4-3", action="store_true", help="生成 result4-3.xlsx (Q3 层) ")
303     ap.add_argument("--fig-weights", action="store_true", help="生成电价权重演化图")
304     args = ap.parse_args()
305     if args.probe:
306         probe()
307     elif args.probe_q3:
308         probe_q3()
309     elif args.result4_2:
310         run_result42()
311     elif args.result4_3:
312         run_result43()
313     elif args.fig_weights:
314         fig_weights()
315     else:
316         ap.print_help()

```

program/src/solve/q4_price_fit.py

```

1  """Q4 电价预测参数结构原型（仿口径 E）：三源自适应加权的误差与费用验证。
2
3  结构：
4       $\hat{p}(D,t) = v_1 \cdot P(D-1,t) + v_2 \cdot P(D-7,t) + v_3 \cdot P(t)$ ,  $v = \text{softmax}$  凸权重（网格 0.1）
5  - 误差口径：全期 MAE 网格扫描（找最优权重结构）
6  - 费用口径：预测价  $\rightarrow$  计划 LP (take-or-pay)  $\rightarrow$  执行按真实价（缺口 5 倍真实价）
7      • 事后最优权重（参考上界）
8      • 滚动 W=7 标定（可部署，无前视）
9
10 运行（program/ 下）：uv run python -m solve.q4_price_fit
11
12 口径（2026-09-13 统一 v1.2）：计划输入 =  $\cdot L$  与  $\max(P-m,0)$  (consistency.py),
13 执行 = 逐槽因果 free（无段末硬目标）；末端自由。表随执行/风险口径变化时须重跑。
14 """
15 from __future__ import annotations
16
17 import time
18
19 import numpy as np
20 import pandas as pd
21
22 import program as pm
23 from solve import consistency as cs
24 from solve import q2
25 from solve.common import DATA_C, EO, ROOT, T
26 from solve.data.attachments import load_extended
27 from solve.models.adaptive import hist_forecast_asof, hist_load_forecast_asof
28 from solve.models.price import ( # 唯一实现 (models/price.py), 此处再导出
29     forecast_price,
30     forecast_price_next_asof,

```

```

31 )
32 from solve.models.weights import simplex_grid # 唯一实现 (models/weights.py)
33
34 DAY_START = q2.REPORT_START # 31
35 N_DAY = q2.N_DAY # 365
36
37
38 def mae_of(p4, p_typ, v, days):
39     errs = [np.abs(forecast_price(D, v, p4, p_typ) - p4[D]).mean() for D in days]
40     return float(np.mean(errs))
41
42
43 def main():
44     pm.init(seed=42, root=str(ROOT))
45     data = load_extended()
46     p_typ = data["price"] # 附件 1 = 逐槽均值
47     df4 = pm.read_table(DATA_C / "附件4.xlsx")
48     p4 = df4.iloc[:, 1:145].to_numpy(float)
49     days = list(range(DAY_START, N_DAY))
50     grid = simplex_grid(0.1)
51     t0 = time.time()
52
53     # ---- 1) 误差口径 (逐日矩阵, 供滚动误差标定复用) ----
54     rows = []
55     daily_mae = np.zeros((N_DAY, len(grid)))
56     for j, v in enumerate(grid):
57         for D in range(7, N_DAY):
58             daily_mae[D, j] = np.abs(forecast_price(D, v, p4, p_typ) - p4[D]).mean()
59             rows.append({"v1_D-1": v[0], "v2_D-7": v[1], "v3_典型日": v[2],
60                         "MAE/元": float(daily_mae[days, j].mean())})
61     df = pd.DataFrame(rows).sort_values("MAE/元")
62     best_err = df.iloc[0]
63     print("=== 误差口径网格 (前 8) ===")
64     print(df.head(8).to_string(index=False))
65     for name, vec in ((("D-1 单源", (1, 0, 0)), ("D-7 单源", (0, 1, 0))),
66                     (("典型日单源", (0, 0, 1)), ("等权", (1 / 3, 1 / 3, 1 / 3)))):
67         print(f" {name:<10} MAE = {mae_of(p4, p_typ, vec, days):.4f} 元")
68
69     # ---- 2) 费用: 一次遍历 (事后最优 + 滚动标定共用) ----
70     print("\n费用网格 (一次遍历, 66 组) .....")
71     daily_costs = np.zeros((N_DAY, len(grid)))
72     E_state = np.full(len(grid), float(E0))
73     for D in range(N_DAY):
74         for j, v in enumerate(grid):
75             p_hat = forecast_price(D, v, p4, p_typ) if D >= 7 else p4[D]
76             p_hat1 = forecast_price_next_asof(D, v, p4, p_typ)
77             load0 = cs.kappa_load(hist_load_forecast_asof(data, D, D)) / 6.0
78             load1 = cs.kappa_load(hist_load_forecast_asof(data, D + 1, D)) / 6.0
79             pv0 = cs.margin_pv(q2._hour_to_slots(data["fc0"][D])) / 6.0
80             pv1 = cs.margin_pv(hist_forecast_asof(data, D + 1, D)) / 6.0
81             xh, _Eh, _ = q2.plan_horizon(
82                 np.concatenate([p_hat, p_hat1]), np.concatenate([load0, load1]),
83                 np.concatenate([pv0, pv1]), E_state[j], None, eps=1e-3,
84             )
85             x = xh[:T]
86             ex = q2.exec_segment_causal(
87                 data["load"][D] / 6.0, data["pv_act"][D] / 6.0, x, E_state[j], None,
88             )
89             E_state[j] = float(ex["E"][-1])
90             daily_costs[D, j] = float(p4[D] @ x + q2.EMERG_MULT * (p4[D] @ ex["e"]))
91         if (D + 1) % 60 == 0:
92             print(f" {D + 1}/{N_DAY} 天 ({time.time() - t0:.0f}s) ")
93
94     idx = {(round(v[0], 4), round(v[1], 4), round(v[2], 4)): j for j, v in enumerate(grid)}
95     after_tot = daily_costs[days].sum(axis=0)
96     rows_c = [{"v1_D-1": v[0], "v2_D-7": v[1], "v3_典型日": v[2],
97               "总费用/万元": round(after_tot[j] / 1e4, 1)} for j, v in enumerate(grid)]
98     dfc = pd.DataFrame(rows_c).sort_values("总费用/万元")
99     print("=== 费用口径 · 事后最优 (前 8) ===")

```

```

100     print(dfc.head(8).to_string(index=False))
101
102     # ---- 3) 滚动 W=7 标定 (无前视, 复用 daily_costs) ----
103     roll = 0.0
104     picks = []
105     for D in days:
106         i0 = max(DAY_START, D - 7)
107         j = (idx[(0.0, 0.0, 1.0)] if i0 == D
108              else int(np.argmin(daily_costs[i0:D].mean(axis=0))))
109         roll += daily_costs[D, j]
110         picks.append(grid[j])
111     print(f"滚动 W=7 总费用 {roll / 1e4:.1f} 万元, 平均权重 "
112           f"v={({np.mean([p[0] for p in picks]):.2f}, {np.mean([p[1] for p in picks]):.2f}, "
113             f"{np.mean([p[2] for p in picks]):.2f})}")
114
115     # ---- 滚动 (误差标定, 对照) ----
116     roll_e = 0.0
117     pe = []
118     for D in days:
119         i0 = max(DAY_START, D - 7)
120         j = (idx[(0.0, 0.0, 1.0)] if i0 == D
121              else int(np.argmin(daily_mae[i0:D].mean(axis=0))))
122         roll_e += daily_costs[D, j]
123         pe.append(grid[j])
124     print(f"滚动 W=7 (误差标定) 总费用 {roll_e / 1e4:.1f} 万元, 平均权重 "
125           f"v={({np.mean([p[0] for p in pe]):.2f}, {np.mean([p[1] for p in pe]):.2f}, "
126             f"{np.mean([p[2] for p in pe]):.2f})}")
127
128     # ---- 对照 (全部从 daily_costs 取) ----
129     print("\n=== 对照 (334 天费用) ===")
130     for name, v in (("D-1 单源", (1, 0, 0)), ("D-7 单源", (0, 1, 0)),
131                    ("典型日单源", (0, 0, 1)), ("近似等权(0.3/0.3/0.4)", (0.3, 0.3, 0.4))):
132         j = idx[(round(v[0], 4), round(v[1], 4), round(v[2], 4))]
133         print(f" {name:<18}: {daily_costs[days, j].sum() / 1e4:7.1f} 万元")
134     ve = (round(best_err['v1_D-1'], 4), round(best_err['v2_D-7'], 4), round(best_err['v3_典型日'], 4))
135     if ve in idx:
136         print(f" 误差最优权重 {ve}: {daily_costs[days, idx[ve]].sum() / 1e4:7.1f} 万元")
137     vc = (round(dfc.iloc[0]['v1_D-1'], 4), round(dfc.iloc[0]['v2_D-7'], 4), round(dfc.iloc[0]['v3_典型日'], 4))
138     print(f" 费用最优权重 {vc}: {daily_costs[days, idx[vc]].sum() / 1e4:7.1f} 万元")
139     print(f" 滚动 W=7 (费用标定) : {roll / 1e4:7.1f} 万元")
140     print(f" 滚动 W=7 (误差标定) : {roll_e / 1e4:7.1f} 万元")
141
142     df.to_csv(ROOT / "code" / "outputs" / "q4_price_fit_mae.csv", index=False, encoding="utf-8-sig")
143     dfc.to_csv(ROOT / "code" / "outputs" / "q4_price_fit_cost.csv", index=False, encoding="utf-8-sig")
144     np.savez_compressed(ROOT / "code" / "outputs" / "q4_price_fit_daily.npz",
145                        daily_costs=daily_costs, daily_mae=daily_mae, grid=np.array(grid))
146     print(f"\n已保存 code/outputs/q4_price_fit*.csv (总用时 {time.time() - t0:.0f}s)")
147
148
149 if __name__ == "__main__":
150     main()

```

core

program/src/solve/core/__init__.py

```

1  """求解内核: LP 模型、因果执行器、槽位映射与无前视残差池 (不含文件 I/O) 。
2
3  分层约定:
4      core ← models ← flows ← 顶层入口 (q1-q4、q2_tune 等)
5  core 只依赖 ``solve.common`` (物理常量与路径) 。
6  """
7  from solve.core.causal import (
8      CAUSAL_POLICY_VERSION,
9      exec_day_causal,
10     exec_segment_causal,
11     run_exec,

```

```

12 )
13 from solve.core.lp import (
14     adjust_day,
15     adjust_day_hedge,
16     exec_day,
17     exec_segment_hindsight,
18     plan_day,
19     plan_horizon,
20 )
21 from solve.core.residual import causal_residual_pool, forecast_at_publish, latest_forecast
22 from solve.core.slots import (
23     _events,
24     _fmt_time,
25     _hour_to_slots,
26     emergency_events,
27     fc_slots,
28     fmt_time,
29     hour_to_slots,
30 )
31
32 __all__ = [
33     "CAUSAL_POLICY_VERSION", "exec_day_causal", "exec_segment_causal", "run_exec",
34     "plan_day", "plan_horizon", "exec_day", "exec_segment_hindsight",
35     "adjust_day", "adjust_day_hedge",
36     "causal_residual_pool", "latest_forecast", "forecast_at_publish",
37     "hour_to_slots", "fc_slots", "fmt_time", "emergency_events",
38     "_hour_to_slots", "_fmt_time", "_events",
39 ]

```

program/src/solve/core/causal.py

```

1  """逐槽因果执行器：每槽只读取当前已实现的负荷与光伏，不使用未来实际值。
2
3  正式口径（2026-09-12 修订）为 ``exec_segment_causal`` / ``exec_day_causal``；
4  事后 LP（``solve.core.lp.exec_day`` 等）只能作为理论下界对照。
5  修改执行策略时必须同步更新 ``CAUSAL_POLICY_VERSION``（参与 Q2E 缓存键）。
6  """
7  from __future__ import annotations
8
9  import numpy as np
10
11  from solve.common import E_MAX, E_MIN, ETA, P_MAX_E
12
13  CAUSAL_POLICY_VERSION = "free-no-terminal-v1"  # 因果执行策略版本（参与缓存键）
14
15  # =====
16  # 逐槽因果执行算法（贪心 + 终端可达性投影）
17  #
18  # 记槽 i 的净需求 net_i = L_i - P_i - x_i          (>0 缺口, <0 富余)
19  # 单槽功率极限（折算为 kWh/槽）：
20  #   最大充电量 up   = • P
21  #   最大放电量 down = P/
22  # 自然平衡所需的 SOC 变化：
23  #   Δ_balance = -net_i/          （缺口 → 放电, E 下降）
24  #   Δ_balance = - • net_i       （富余 → 充电, E 上升）
25  #
26  # 末段目标与执行策略（EXEC_POLICY）：
27  #   - e_end 给定 ("target")：把下一时刻 SOC 投影到仍能到达 e_end 的区间
28  #       lo_i = max(E_min, E_now - down, e_end - remain • up)
29  #       hi_i = min(E_max, E_now + up, e_end + remain • down)
30  #   - e_end=None ("free", 统一口径) 或 no_purchase_charge=True:
31  #       仅受容量/功率约束 lo_i = max(E_min, E_now-down), hi_i = min(E_max, E_now+up)
32  #   free 策略不迫使"紧急购电充电/扣留放电", 与 dayend 同值且更简洁
33  #   （见 reports/执行器段末目标实验.md）。
34  #   E_{i+1} = clip(E_now + Δ_balance, lo_i, hi_i)      remain = m-i-1
35  #
36  # 由 E_{i+1} 反推充放量：
37  #   Δ   0:  c_i = Δ/          （充电）

```

```

39 #  $\Delta < 0$ :  $d_i = -\Delta$  (放电)
40 # 功率平衡残差转为紧急购电或弃光:
41 #  $gap_i = L_i + c_i - P_i - x_i - d_i$ 
42 #  $gap_i \geq 0 \rightarrow e_i = gap_i$  (紧急购电);  $gap_i < 0 \rightarrow s_i = -gap_i$  (弃光)
43 # =====
44 def exec_segment_causal(load_kwh, pv_kwh, x_commit, e_start, e_end,
45                         no_purchase_charge: bool = False):
46     """逐槽因果执行: 每槽只使用当前已实现的负荷和光伏, 不读取未来实际值。
47
48     策略: 先用储能吸收当前富余或填补当前缺口, 再把下一时刻 SOC 投影到
49     "仍能在剩余时段到达 e_end"的可达区间; 无法被储能覆盖的缺口记为紧急购电,
50     富余且无法消纳的部分记为弃电。
51
52     可选策略开关 (2026-09-13 增设, 供"段末目标机制"对照实验):
53     - ``e_end=None``: 完全自由末端 (仅受容量/功率约束, 不做可达性投影);
54     - ``no_purchase_charge=True``: 禁止"为充能而紧急购电"——SOC 不得被抬升到
55       自然平衡点之上, 段末目标允许偏离 (偏差在返回值中报告)。
56     参数均为长度 m 的 kWh/槽序列; 返回 c/d/s/E/e (另附诊断量)。
57     """
58     load_kwh = np.asarray(load_kwh, dtype=float)
59     pv_kwh = np.asarray(pv_kwh, dtype=float)
60     x_commit = np.asarray(x_commit, dtype=float)
61     if not (load_kwh.ndim == pv_kwh.ndim == x_commit.ndim == 1):
62         raise ValueError("因果执行输入必须是一维序列")
63     if not (len(load_kwh) == len(pv_kwh) == len(x_commit)):
64         raise ValueError("因果执行的负荷、光伏与购电序列必须等长")
65     if not (E_MIN <= e_start <= E_MAX):
66         raise ValueError("因果执行的初始 SOC 超出安全区间")
67     if e_end is not None and not (E_MIN <= e_end <= E_MAX):
68         raise ValueError("因果执行的末端 SOC 超出安全区间")
69
70     m = len(load_kwh)
71     c = np.zeros(m)
72     d = np.zeros(m)
73     spill = np.zeros(m)
74     E = np.zeros(m)
75     emerg = np.zeros(m)
76     e_now = float(e_start)
77     max_up = ETA * P_MAX_E # 单槽最大充电量 (kWh)
78     max_down = P_MAX_E / ETA # 单槽最大放电量 (kWh)
79     tol = 1e-8
80
81     for i in range(m):
82         remain = m - i - 1
83         if e_end is None or no_purchase_charge:
84             # 自由末端 (或禁止购电充电): 仅受容量与功率约束
85             lo = max(E_MIN, e_now - max_down)
86             hi = min(E_MAX, e_now + max_up)
87         else:
88             # E_next 既要能由当前 SOC 一步到达, 也要能在剩余槽回到 e_end
89             lo = max(E_MIN, e_now - max_down, float(e_end) - remain * max_up)
90             hi = min(E_MAX, e_now + max_up, float(e_end) + remain * max_down)
91         if lo > hi + tol:
92             raise RuntimeError(
93                 f"终端 SOC 不可达: i={i}, interval=[{lo:.6f}, {hi:.6f}]"
94             )
95
96         net = load_kwh[i] - pv_kwh[i] - x_commit[i]
97         # 使当前槽供需平衡的自然 SOC 变化; 若会破坏终端可达性则投影
98         delta_balance = -net / ETA if net >= 0.0 else -ETA * net
99         e_next = float(np.clip(e_now + delta_balance, lo, hi))
100         delta = e_next - e_now
101         if delta >= 0.0:
102             c[i] = delta / ETA
103         else:
104             d[i] = -delta * ETA
105
106         gap = load_kwh[i] + c[i] - pv_kwh[i] - x_commit[i] - d[i]
107         if gap >= 0.0:

```

```

108         emerg[i] = gap
109     else:
110         spill[i] = -gap
111     E[i] = e_next
112     e_now = e_next
113
114     term_err = abs(E[-1] - float(e_end)) if e_end is not None else 0.0
115     if e_end is not None and m and not no_purchase_charge and term_err > 1e-6:
116         raise RuntimeError(f"因果执行末端 SOC 偏差 {E[-1] - e_end:.3e} kWh")
117     return {"c": c, "d": d, "s": spill, "E": E, "e": emerg,
118            "terminal_error_kwh": float(term_err)}
119
120
121 def exec_day_causal(price, load_kwh, pv_kwh, x_plan, e_start, e_end=None):
122     """旧版（日循环）因果执行包装: e_end=None 时终端取 e_start。
123
124     正式口径（`consistency.EXEC_POLICY="free"`）已改为无段末硬目标：
125     直接调用 `exec_segment_causal(..., e_end=None)`；本函数仅供历史对照。
126     """
127     del price
128     return exec_segment_causal(
129         load_kwh, pv_kwh, x_plan, e_start, e_start if e_end is None else e_end
130     )
131
132
133 def run_exec(policy: str, load, pv, x, e0, e1, last: bool = False):
134     """按执行策略调用逐槽因果执行器（段末目标机制对照，consistency 规范扩展）。
135
136     policy:
137         "target" 段末目标硬跟踪（原口径，历史对照）；
138         "free" 无段末目标（仅容量/功率约束；统一口径 EXEC_POLICY）；
139         "dayend" 仅最后一段跟踪段末目标，中间段自由（与 free 同值，见探针）。
140     """
141     if policy == "free" or (policy == "dayend" and not last):
142         return exec_segment_causal(load, pv, x, e0, None)
143     return exec_segment_causal(load, pv, x, e0, e1)

```

program/src/solve/core/lp.py

```

1  """LP 求解器：日前/多日计划、事后执行下界、Q3 调整与场景对冲。
2
3  变量块口径（每个时段 t 一类变量，按块顺序拼接）：
4  - 计划/多日计划 ``plan_day``/``plan_horizon``:
5      [ x 购电 | c 充电 | d 放电 | s 弃光 | E 储电量 ]
6  - 事后执行 ``exec_day``/``exec_segment_hindsight``:
7      [ c 充电 | d 放电 | s 弃光 | E 储电量 | e 紧急购电 ]
8  - 调整 ``adjust_day``/``adjust_day_hedge``:
9      [ adj 购电 | c | d | s | E | y 结算基准 ]
10
11  统一实现：
12  - ``plan_lp`` 是计划/多日计划 LP 的唯一构造（由 ``plan_day``（H=T 日循环）
13    与 ``plan_horizon``（H 可变、跨日）两个薄包装复用）；
14  - ``exec_lp`` 是事后执行 LP 的唯一构造（``exec_day`` 段长 T、日循环，
15    ``exec_segment_hindsight`` 任意段与终端）。
16
17  HiGHS 对矩阵顺序敏感：本模块的矩阵构造在重构期间由金标测试锁定数值
18  (plan_day plan_horizon(H=T) 旧 q1.build_lp(eps=0);
19  exec_day exec_segment_hindsight(t0=0, e_end=e_start))。
20  """
21  from __future__ import annotations
22
23  import numpy as np
24  from scipy.sparse import csr_matrix, lil_matrix
25
26  import program as pm
27  from solve.common import (
28      EO,
29      E_MAX,
30      E_MIN,

```

```

31     EMERG_MULT,
32     EPS_TH,
33     EPS_THROUGHPUT,
34     ETA,
35     P_MAX_E,
36     T,
37 )
38
39
40 # =====
41 # 一、计划 LP (日前 / 多日滚动)
42 #
43 #     min  E_t p_t * x_t +  E_t (c_t + d_t)          (购电费 + 充放唯一化罚项)
44 #     s.t. P_t + x_t + d_t = L_t + c_t + s_t          (功率平衡, t)
45 #           E_t = E_{t-1} +  c_t - d_t/              (储能动态, t)
46 #           0 ≤ x_t, 0 ≤ c_t, d_t ≤ P, 0 ≤ s_t ≤ P_t  (边界; s 为弃光松弛)
47 #           E_min ≤ E_t ≤ E_max                    (储电量安全区间)
48 #           E_0 = e_start, E_{H-1} = e_terminal      (跨日/日循环终端条件)
49 #
50 #     =0 时与"纯购电费最小"完全一致 (计划 LP); >0 在等费用平面上
51 #     唯一化充放方案, 供口径 E 标定与 Q3/Q4 使用。
52 # =====
53 def plan_lp(price_h, load_h, pv_h, e_start, e_terminal=None, eps=0.0):
54     """统一计划 LP: 返回完整解字典与 OptResult。
55
56     ``e_terminal=None`` (统一口径默认) 时规划窗口末端完全自由 (不锚定终值);
57     传入数值时只约束视野最远端 E(H)=e_terminal (供结构对照)。
58     中间每个自然日的末端 SOC 都是优化变量; 每日只执行前 144 槽,
59     次日以真实末端 SOC 重新求解。
60     解字典键: x 购电、c 充、d 放、s 弃光、E 储电量 (均为长度 H 数组)。
61     """
62     price_h = np.asarray(price_h, dtype=float)
63     load_h = np.asarray(load_h, dtype=float)
64     pv_h = np.asarray(pv_h, dtype=float)
65     H = len(price_h)
66     if len(load_h) != H or len(pv_h) != H:
67         raise ValueError("price/load/pv 的滚动视野长度必须一致")
68     n = 5 * H
69     c_obj = np.zeros(n)
70     # 目标函数: 仅购电量 x 按电价计费; c/d 加 罚项抑制无意义充放
71     c_obj[0:H] = price_h
72     c_obj[H:3 * H] = eps
73
74     # ---- 等式约束行布局: [0, H) 平衡; [H, 2H) 动态; 可选第 2H 行终端条件 ----
75     n_rows = 2 * H + (1 if e_terminal is not None else 0)
76     A_eq = lil_matrix((n_rows, n))
77     b_eq = np.zeros(n_rows)
78     for t in range(H):
79         # 功率平衡: +x_t + d_t - c_t - s_t = L_t - P_t
80         A_eq[t, 0 * H + t] = 1.0
81         A_eq[t, 1 * H + t] = -1.0
82         A_eq[t, 2 * H + t] = 1.0
83         A_eq[t, 3 * H + t] = -1.0
84         b_eq[t] = load_h[t] - pv_h[t]
85     for t in range(H):
86         # 储能动态: E_t - E_{t-1} -  c_t + d_t/ = 0 (t=0 时右端为 e_start)
87         r = H + t
88         A_eq[r, 4 * H + t] = 1.0
89         if t:
90             A_eq[r, 4 * H + t - 1] = -1.0
91             A_eq[r, 1 * H + t] = -ETA
92             A_eq[r, 2 * H + t] = 1.0 / ETA
93             b_eq[r] = e_start if t == 0 else 0.0
94     # 终端条件 (可选): E_{H-1} = e_terminal (日循环/跨日循环锚定)
95     if e_terminal is not None:
96         A_eq[2 * H, 4 * H + H - 1] = 1.0
97         b_eq[2 * H] = float(e_terminal)
98
99     # 变量边界: x [0,∞), c,d [0,P], s [0,该槽可用光伏], E [E_min,E_max]

```

```

100     bounds = (
101         [(0.0, None)] * H
102         + [(0.0, P_MAX_E)] * H
103         + [(0.0, P_MAX_E)] * H
104         + [(0.0, float(v)) for v in pv_h]
105         + [(E_MIN, E_MAX)] * H
106     )
107     res = pm.optimize.solve_lp(c_obj, A_eq=csr_matrix(A_eq), b_eq=b_eq, bounds=bounds)
108     if not res.success:
109         raise RuntimeError(f"计划 LP 失败: {res.message}")
110     sol = {
111         "x": res.x[0:H],
112         "c": res.x[H:2 * H],
113         "d": res.x[2 * H:3 * H],
114         "s": res.x[3 * H:4 * H],
115         "E": res.x[4 * H:5 * H],
116     }
117     return sol, res
118
119
120 def plan_day(price, load_kwh, pv_kwh, e_start, eps=0.0):
121     """当日计划 LP:  $\min \Sigma p \cdot x + \text{eps} \cdot \Sigma(c+d)$ ; 储能循环  $E(0)=E(24)=e\_start$ 。
122
123     eps > 0 为极小正则项, 用于在等价位内唯一化购电/充放方案 (口径 E 标定);
124     默认 0 与原有口径完全一致。返回 (x, E, res)。
125     """
126     sol, res = plan_lp(price, load_kwh, pv_kwh, e_start, e_start, eps)
127     return sol["x"], sol["E"], res
128
129
130 def plan_horizon(price_h, load_h, pv_h, e_start, e_terminal=None, eps=0.0):
131     """有限视野计划 LP; 正式滚动策略使用 2 日 (288 槽) 视野。
132
133     默认末端完全自由 (``e_terminal=None``, 统一口径); 传入数值时锚定
134      $E(H)=e\_terminal$  (供结构对照)。返回 ``(x, E, res)``。
135     每日只执行前 144 槽, 次日以真实末端 SOC 重新求解。
136     """
137     sol, res = plan_lp(price_h, load_h, pv_h, e_start, e_terminal, eps)
138     return sol["x"], sol["E"], res
139
140
141 # =====
142 # 二、事后执行下界 LP (不得用于正式结果)
143 #
144 #     min     $\Sigma(c_t + d_t) + \Sigma_t 5 \cdot p_t \cdot e_t$ 
145 #     s.t.  $P_t + d_t + e_t = L_t + c_t + s_t + x_t$     (x 已承诺 take-or-pay)
146 #            $E_t = E_{t-1} + c_t - d_t$ 
147 #            $E_{end} = e_{end}$     (日循环或段末端)
148 #
149 #     仅作"全天/整段信息已知"的理论下界; 正式口径为逐槽因果执行 (core.causal)。
150 # =====
151 def _exec_lp(price, load_kwh, pv_kwh, x_seg, e_start, e_end, t0, eps):
152     """统一事后执行 LP: 对段 [t0, t0+m) 重优化储能; 返回 c/d/s/E/e 字典。"""
153     m = len(x_seg)
154     n = 5 * m
155
156     def I(b, i):
157         return b * m + i
158
159     c_obj = np.zeros(n)
160     # 目标: c/d 唯一化罚项; 紧急购电按 5 倍电价惩罚
161     c_obj[0:m] = eps
162     c_obj[m:2 * m] = eps
163     c_obj[4 * m:5 * m] = EMERG_MULT * price[t0:t0 + m]
164
165     rows = 2 * m + 1
166     A_eq = lil_matrix((rows, n))
167     b_eq = np.zeros(rows)
168     for i in range(m):

```

```

169     # 供需平衡:  $-c_i + d_i - s_i + e_i = L - P - x_{seg}$  (缺口由  $e$  或放电补齐)
170     A_eq[i, I(0, i)] = -1.0
171     A_eq[i, I(1, i)] = 1.0
172     A_eq[i, I(2, i)] = -1.0
173     A_eq[i, I(4, i)] = 1.0
174     b_eq[i] = load_kwh[t0 + i] - pv_kwh[t0 + i] - x_seg[i]
175
176     # 储能动态:  $E_i - E_{i-1} - \cdot c_i + d_i / = 0$ 
177     r = m + i
178     A_eq[r, I(3, i)] = 1.0
179     if i:
180         A_eq[r, I(3, i - 1)] = -1.0
181     A_eq[r, I(0, i)] = -ETA
182     A_eq[r, I(1, i)] = 1.0 / ETA
183     b_eq[r] = e_start if i == 0 else 0.0
184     # 段末终端:  $E_{m-1} = e_{end}$ 
185     A_eq[2 * m, I(3, m - 1)] = 1.0
186     b_eq[2 * m] = e_end
187
188     bounds = (
189         [(0.0, P_MAX_E)] * m
190         + [(0.0, P_MAX_E)] * m
191         + [(0.0, None)] * m
192         + [(E_MIN, E_MAX)] * m
193         + [(0.0, None)] * m
194     )
195     res = pm.optimize.solve_lp(c_obj, A_eq=csr_matrix(A_eq), b_eq=b_eq, bounds=bounds)
196     if not res.success:
197         raise RuntimeError(f"执行 LP 失败 (t0={t0}): {res.message}")
198     return {
199         "c": res.x[0:m], "d": res.x[m:2 * m], "s": res.x[2 * m:3 * m],
200         "E": res.x[3 * m:4 * m], "e": res.x[4 * m:5 * m],
201     }
202
203
204 def exec_day(price, load_kwh, pv_kwh, x_plan, e_start):
205     """事后执行下界:  $x$  已承诺, 使用全天实际曲线重优化储能。
206
207     本函数保留给历史对照和理论下界, **不得**标记为"实时执行";
208     正式因果口径使用 :func:`solve.core.causal.exec_day_causal` /
209     :func:`solve.core.causal.exec_segment_causal`。
210     """
211     return _exec_lp(price, load_kwh, pv_kwh, x_plan, e_start, e_start, 0,
212                     EPS_THROUGHPUT)
213
214
215 def exec_segment_hindsight(price, load_kwh, pv_kwh, x_seg, e_start, e_end, t0, eps=EPS_TH):
216     """事后段执行下界: 使用整段实际功率重优化储能。
217
218     仅用于理论下界对照, **不得**进入实时口径; 正式因果执行使用
219     :func:`solve.core.causal.exec_segment_causal` (逐槽只读当前已实现值)。
220     """
221     return _exec_lp(price, load_kwh, pv_kwh, x_seg, e_start, e_end, t0, eps)
222
223
224 # =====
225 # 三、Q3 调整 LP
226 #
227 # 分段结算规则 (题目): 调低部分按 50%、调高部分按 150% 结算,
228 # 即 费用 =  $p \cdot adj + 0.5 \cdot p \cdot |x_{plan} - adj|$ 
229 # LP 线性化 (用  $y$  作分段下包络, 目标含  $-p \cdot y$ ):
230 #      $y_t$    $x_{plan,t}$    $y_t$    $adj_t$    $y_t$   0
231 #      $\Sigma [1.5 \cdot p_t \cdot adj_t - p_t \cdot y_t] + \cdot \Sigma (c_t + d_t)$ 
232 # (最优时  $y_t = \min(x_{plan,t}, adj_t)$ , 与原分段费用等价)
233 #
234 # 变量块: [  $adj$  |  $c$  |  $d$  |  $s$  |  $E$  |  $y$  ]
235 # 约束:  $P_t + adj_t + d_t = L_t + c_t + s_t$ 
236 #      $E_t = E_{t-1} + \cdot c_t - d_t /$ 
237 #      $s_t$  预测光伏 (弃光不能超过可用光伏)

```

```

238 # E_t0-1 = e_init, E_{T-1} = e_terminal
239 # =====
240 def adjust_day(price, load_kwh, pv_fc_kwh, x_plan, e_init, t0, eps=EPS_TH, e_terminal=E0):
241     """Q3 调整 LP: 对 [t0, T) 重优化购电与储能, 偏差费线性化 (新预报视为已知)。
242
243     e_terminal: 段末 (当天 24:00) 目标储电量, 默认 E0 (日循环); 跨日模式传 2 日计划的当日末值。
244
245     变量块 (各 m = T-t0 个): 0=adj 购电, 1=c 充, 2=d 放, 3=s 弃光, 4=E 储电量, 5=y 结算基准
246     目标:  $\min \sum_{t=t0} [1.5 \cdot p \cdot adj - p \cdot y] + \cdot \Sigma(c+d)$ 
247     其中  $y = \min(x\_plan, adj)$  (系数为负  $\rightarrow$  约束  $y \leq x\_plan, y \leq adj$  下自动取等)
248     返回 [t0, T) 的 adj 序列。
249     """
250     m = T - t0
251     n = 6 * m
252
253     def I(b, i):
254         return b * m + i
255
256     c_obj = np.zeros(n)
257     # 目标系数: adj 记 1.5p, y 记 -p (等价于  $p \cdot adj + 0.5p|x\_plan - adj|$ ), c/d 罚项
258     c_obj[0 * m : 1 * m] = 1.5 * price[t0:]
259     c_obj[1 * m : 2 * m] = eps
260     c_obj[2 * m : 3 * m] = eps
261     c_obj[5 * m : 6 * m] = -price[t0:]
262
263     # ---- 等式约束: [0,m) 平衡; [m,2m) 储能动态; 第 2m 行终端 ----
264     rows = 2 * m + 1
265     A_eq = lil_matrix((rows, n))
266     b_eq = np.zeros(rows)
267     for i in range(m):
268         t = t0 + i
269         A_eq[i, I(0, i)] = 1.0
270         A_eq[i, I(1, i)] = -1.0
271         A_eq[i, I(2, i)] = 1.0
272         A_eq[i, I(3, i)] = -1.0
273         b_eq[i] = load_kwh[t] - pv_fc_kwh[t]
274
275         r = m + i
276         A_eq[r, I(4, i)] = 1.0
277         if i:
278             A_eq[r, I(4, i - 1)] = -1.0
279             A_eq[r, I(1, i)] = -ETA
280             A_eq[r, I(2, i)] = 1.0 / ETA
281             b_eq[r] = e_init if i == 0 else 0.0
282     A_eq[2 * m, I(4, m - 1)] = 1.0
283     b_eq[2 * m] = e_terminal
284
285     # ---- 不等式:  $y_i \leq x\_plan_i$  (上次承诺);  $y_i - adj_i \leq 0$  ----
286     A_ub = lil_matrix((2 * m, n))
287     b_ub = np.zeros(2 * m)
288     for i in range(m):
289         t = t0 + i
290         A_ub[i, I(5, i)] = 1.0
291         b_ub[i] = x_plan[t]
292         A_ub[m + i, I(5, i)] = 1.0
293         A_ub[m + i, I(0, i)] = -1.0
294     bounds = (
295         [(0.0, None)] * m
296         + [(0.0, P_MAX_E)] * m
297         + [(0.0, P_MAX_E)] * m
298         + [(0.0, float(pv_fc_kwh[t0 + i])) for i in range(m)] # 弃光 预测光伏
299         + [(E_MIN, E_MAX)] * m
300         + [(0.0, None)] * m
301     )
302     res = pm.optimize.solve_lp(
303         c_obj, A_ub=csr_matrix(A_ub), b_ub=b_ub,
304         A_eq=csr_matrix(A_eq), b_eq=b_eq, bounds=bounds,
305     )
306     if not res.success:

```

```

307         raise RuntimeError(f"调整 LP 失败 (t0={t0}): {res.message}")
308     return res.x[0:m], res.x[4 * m:5 * m]
309
310
311 # =====
312 # 四、Q3 调整 LP (场景对冲版, 两阶段随机规划)
313 #
314 #     第一阶段 (共享): adj 为所有情景共用的购电决策;
315 #     第二阶段 (每情景独立): c_s/d_s/s_s/E_s/e_s。
316 #
317 #     min  $\Sigma [1.5p \cdot adj - p \cdot y] + (1/S) \cdot \Sigma_s [(c_s + d_s) + 5p \cdot e_s]$ 
318 #     s.t. 每情景 s: 功率平衡、储能动态、终端  $E_{T-1} = e_{terminal}$ 
319 #           y x_plan, y adj (第一阶段结算基准)
320 #
321 #     变量块顺序: [ adj | y | (c_s, d_s, s_s, E_s, e_s) for s=1..S ]
322 # =====
323 def adjust_day_hedge(price, load_kwh, pv_scens, x_plan, e_init, t0, eps=EPS_TH,
324                     e_terminal=E0, load_scens=None):
325     """调整 LP (场景对冲): adj 为第一阶段共享决策; 每场景储能/紧急独立。
326
327     e_terminal: 段末 (当天 24:00) 目标储电量 (各场景同值), 默认 E0。
328     load_scens: 每场景负荷 (list, 长度 m; None 时所有场景共用 ``load_kwh``);
329                 统一口径下与 ``pv_scens`` 一起由 (Δ负荷, Δ光伏) 联合残差块构造。
330
331     pv_scens: list, 第 0 个为名义预测 (用于返回基准储能轨迹), 其余为扰动场景;
332               每个元素是长度 m = T-t0 的 kWh/槽数组。
333     目标: min  $\Sigma [1.5p \cdot adj - p \cdot y] + (1/S) \cdot \Sigma_s [(c_s + d_s) + 5p \cdot e_s]$ 
334     返回 (adj, 名义储能轨迹 E_nom)。
335     """
336     m = T - t0
337     S = len(pv_scens)
338     n = 2 * m + S * 5 * m
339
340     def A(i):
341         return i
342
343     def Y(i):
344         return m + i
345
346     def base(s):
347         return 2 * m + s * 5 * m
348
349     # 目标: 第一阶段 adj/y; 第二阶段各情景的罚项与期望紧急费 (1/S 缩放)
350     c_obj = np.zeros(n)
351     c_obj[0:m] = 1.5 * price[t0:]
352     c_obj[m:2 * m] = -price[t0:]
353     for s in range(S):
354         b0 = base(s)
355         c_obj[b0:b0 + m] = eps / S
356         c_obj[b0 + m:b0 + 2 * m] = eps / S
357         c_obj[b0 + 4 * m:b0 + 5 * m] = EMERG_MULT * price[t0:] / S
358
359     # ---- 等式约束: 每情景 (2m+1) 行 (平衡 m + 动态 m + 终端 1) ----
360     rows = 2 * m * S + S
361     A_eq = lil_matrix((rows, n))
362     b_eq = np.zeros(rows)
363     r = 0
364     for s in range(S):
365         b0 = base(s)
366         for i in range(m):
367             # 情景 s 的功率平衡: adj + d_s - c_s - s_s + e_s = L - P_s
368             A_eq[r, A(i)] = 1.0
369             A_eq[r, b0 + i] = -1.0
370             A_eq[r, b0 + m + i] = 1.0
371             A_eq[r, b0 + 2 * m + i] = -1.0
372             A_eq[r, b0 + 4 * m + i] = 1.0
373             b_eq[r] = ((load_kwh[t0 + i] if load_scens is None
374                        else load_scens[s][i]) - pv_scens[s][i])
375             r += 1

```

```

376     for i in range(m):
377         # 情景 s 的储能动态
378         A_eq[r, b0 + 3 * m + i] = 1.0
379         if i:
380             A_eq[r, b0 + 3 * m + i - 1] = -1.0
381             A_eq[r, b0 + i] = -ETA
382             A_eq[r, b0 + m + i] = 1.0 / ETA
383             b_eq[r] = e_init if i == 0 else 0.0
384             r += 1
385         # 情景 s 的段末端
386         A_eq[r, b0 + 3 * m + m - 1] = 1.0
387         b_eq[r] = e_terminal
388         r += 1
389
390     # ---- 不等式: y ≤ x_plan; y - adj ≤ 0 (第一阶段结算) ----
391     A_ub = lil_matrix((2 * m, n))
392     b_ub = np.zeros(2 * m)
393     for i in range(m):
394         A_ub[i, Y(i)] = 1.0
395         b_ub[i] = x_plan[t0 + i]
396         A_ub[m + i, Y(i)] = 1.0
397         A_ub[m + i, A(i)] = -1.0
398
399     bounds = [(0.0, None)] * m + [(0.0, None)] * m
400     for _ in range(S):
401         bounds += [(0.0, P_MAX_E)] * m + [(0.0, P_MAX_E)] * m
402             + [(0.0, None)] * m + [(E_MIN, E_MAX)] * m + [(0.0, None)] * m
403     res = pm.optimize.solve_lp(
404         c_obj, A_ub=csr_matrix(A_ub), b_ub=b_ub,
405         A_eq=csr_matrix(A_eq), b_eq=b_eq, bounds=bounds,
406     )
407     if not res.success:
408         raise RuntimeError(f"对冲调整 LP 失败 (t0={t0}): {res.message}")
409     E_nom = res.x[base(0) + 3 * m: base(0) + 4 * m]
410     return res.x[0:m], E_nom

```

program/src/solve/core/residual.py

```

1  """无前视残差池与"最新可用预报" (Q3/Q4 对冲与场景采样共用)。"""
2  from __future__ import annotations
3
4  import numpy as np
5
6  from solve import consistency as cs
7  from solve.common import T
8  from solve.core.slots import fc_slots, hour_to_slots
9
10
11 def latest_forecast(data: dict, D: int) -> np.ndarray:
12     """当天"最新可用预报" (144 槽 kW)：按段取 0:00/6:00/12:00/18:00 发布值。"""
13     v = np.empty(T)
14     v[:36] = hour_to_slots(data["fc0"][D])[:36]
15     v[36:72] = fc_slots(data["fc6"][D], 6)[36:72]
16     v[72:108] = fc_slots(data["fc12"][D], 12)[72:108]
17     v[108:] = fc_slots(data["fc18"][D], 18)[108:]
18     return v
19
20
21 def forecast_at_publish(data: dict, D: int, publish: int) -> np.ndarray:
22     """返回历史日 ``D`` 在指定发布时刻可得到的整条当日预报 (144 槽, kW)。
23
24     场景残差必须与当前决策的信息集一致。例如 6:00 调整只能使用历史 6:00
25     发布预报的误差，不能把历史 12:00/18:00 的更新拼接进剩余时段。
26     """
27     if publish == 0:
28         return hour_to_slots(data["fc0"][D])
29     if publish not in (6, 12, 18):
30         raise ValueError(f"不支持的预报发布时刻: {publish}")
31     return fc_slots(data[f"fc{publish}"][D], publish)

```

```

32
33
34 def causal_residual_pool(data: dict, D: int, min_same_month: int = 14,
35                           lookback: int = 90) -> np.ndarray:
36     """构造目标日 D 的历史残差池，严格保证所有日序号小于 D（无前视）。
37
38     统一实现见 ``consistency.scenario_indices``（同月优先、不足回看）。
39     """
40     if D <= 14:
41         raise ValueError("残差池至少需要 14 天预热数据")
42     return cs.scenario_indices(np.asarray(data["months"]), D,
43                                 pool_min_same_month=min_same_month, lookback=lookback)

```

program/src/solve/core/slots.py

```

1  """时间-槽位映射与紧急购电事件（相位敏感的公共实现）。
2
3  关键口径：``hour_to_slots`` 只适用于 0:00 发布的预报（24 整点 → 144 槽）；
4  6:00/12:00/18:00 发布的预报必须用 ``fc_slots`` 做相位对齐，
5  否则整体错位 6/12/18 小时（费用翻倍级错误）。
6  """
7  from __future__ import annotations
8
9  import numpy as np
10
11  from solve.common import T
12
13
14  def hour_to_slots(y24: np.ndarray) -> np.ndarray:
15     """24 个整点值 → 144 个 10 分钟槽（线性插值到槽中心）。
16
17     插值形式：以 (0, 0), (1, y1), ..., (24, y24) 为节点做分段线性插值，
18     在槽中心  $_k = (k + 0.5)/6$  处取值， $k = 0..143$ 。
19     """
20     fp = np.concatenate([[0.0], np.asarray(y24, dtype=float)])
21     centers = (np.arange(T) + 0.5) / 6.0
22     return np.interp(centers, np.arange(0, 25, dtype=float), fp)
23
24
25  def fc_slots(fc24: np.ndarray, publish: int) -> np.ndarray:
26     """附件 3 预报行 → 144 槽 (kW)，相位对齐（预报 k 小时 = 发布时刻 + k 时）。
27
28     构造 0..24 整点的值序列 hv（发布时刻及以前用第一个预报值填充），
29     再线性插值到 144 个槽中心。调整 LP 只使用 [publish*6, 144) 段。
30
31     相位规则（关键）：
32         hv[0..publish] = y[0] (发布时刻已过/当前，用首值)
33         hv[publish + k] = y[k-1], k = 1..24-publish
34     即预报第 k 个值对应“发布时刻 + k 小时”，与 hour_to_slots 的 0:00 口径区分。
35     """
36     y = np.asarray(fc24, dtype=float)
37     hv = np.empty(25)
38     hv[: publish + 1] = y[0]
39     hv[publish + 1: 25] = y[: 24 - publish]
40     centers = (np.arange(T) + 0.5) / 6.0
41     return np.interp(centers, np.arange(25, dtype=float), hv)
42
43
44  def fmt_time(minute: int) -> str:
45     """分钟数 → 时间标签 (1440 → 0:00+1)。"""
46     if minute >= 1440:
47         return "0:00+1"
48     return f"{minute // 60}:{minute % 60:02d}"
49
50
51  def emergency_events(e_slots: np.ndarray, tol: float = 1e-3):
52     """逐槽紧急购电合并为事件区间：[(起槽, 止槽, kWh), ...]。
53
54     合并规则：相邻槽  $e_t > tol$  时归为同一事件，区间电量 =  $\Sigma e_t$ 。

```

```

55     时间标签转换见 :func:`fmt_time` (起槽 ×10 分钟 → "H:MM")。
56     """
57     out = []
58     t = 0
59     while t < T:
60         if e_slots[t] > tol:
61             j = t
62             while j + 1 < T and e_slots[j + 1] > tol:
63                 j += 1
64             out.append((t, j, float(e_slots[t:j + 1].sum())))
65             t = j + 1
66         else:
67             t += 1
68     return out
69
70
71 # 兼容别名 (旧脚本按私有调用)
72 _hour_to_slots = hour_to_slots
73 _fmt_time = fmt_time
74 _events = emergency_events

```

models

program/src/solve/models/__init__.py

```

1 """预测模型层: 凸权重工具、预报误差模型、历史自适应加权模型与电价模型。"""

```

program/src/solve/models/adaptive.py

```

1 """C 题 问题二 口径 E: 历史自适应加权预测 + 网格定位 + 梯度精化。
2
3 预测 (每天 0:00, 只用历史实际与典型日, 无前视):
4     负荷  $L(d,t) = w_1 \cdot L(d-7,t) + w_2 \cdot L(d-14,t) + w_3 \cdot L(t)$ 
5     光伏  $P(d,t) = u_1 \cdot P(d-1,t) + u_2 \cdot P(d-2,t) + u_3 \cdot P(t)$ 
6  $w$ 、 $u$  为凸权重 (softmax 参数化), 每天一组、144 个时段共用; 典型日取附件 1。
7
8 标定 (目标: 窗口内实际总费用 = 计划费 + 紧急购电费):
9     1) 每个日历日  $k$  在权重网格 (步长 grid_step) 上算成本表  $C[k, i, j]$ ;
10    2) 目标日  $d$  的权重 = 窗口  $[d-W, d-1]$  平均成本最小的格子 (查表);
11    3)  $W=1$  主结果再用 torch.optim.Adam + 中心差分数值梯度在该窗口上精化,
12       保留更优者。成本表并行预计算并缓存。
13
14 执行沿用问题二口径 (见 solve/q2.py): 计划购电 take-or-pay、储能日循环  $E=6000$ 、
15 缺口按 5 倍交易时刻电价紧急购电。result2.xlsx 由 io.excel 负责写出。
16
17 用法:
18     from solve.q2_adaptive import AdaptiveWeightModel
19
20     model = AdaptiveWeightModel(W=1).load()
21     result = model.build_table().run()          # 主结果 (含对照/图/报告)
22     model.window_sensitivity()                 # 标定窗口敏感性
23
24 命令行: uv run python -m solve.q2_adaptive (在 program/ 目录; 首次约 8 分钟)
25 """
26
27 from __future__ import annotations
28
29 import hashlib
30 import os
31 import time
32
33 import numpy as np
34 import pandas as pd
35 import torch
36
37 import program as pm

```

```

38 from solve.common import (
39     DATA_C,
40     EO,
41     EMERG_MULT,
42     N_DAY,
43     REPORT_START,
44     RESULTS_DIR,
45     ROOT,
46     T,
47 )
48 from solve.core.causal import (
49     CAUSAL_POLICY_VERSION,
50     exec_day_causal,
51     exec_segment_causal,
52 )
53 from solve.core.lp import plan_day
54 from solve.data.attachments import load_all
55 from solve.flows.q2_year import run_deterministic
56 from solve.io.excel import verify_result2 as _verify_result2
57 from solve.io.excel import write_result2
58 from solve.io.figures import month_axis
59 from solve.io.report import record
60 from solve.models.weights import simplex_grid, softmax
61
62 # =====
63 # 口径 E 模型总览
64 #
65 # 预测（每天 0:00 给出，只用历史与典型日）：
66 #  $L(d,t) = w_1 \cdot L(d-7,t) + w_2 \cdot L(d-14,t) + w_3 \cdot L(t)$  ( $\Sigma w = 1, w_0$ )
67 #  $P(d,t) = u_1 \cdot P(d-1,t) + u_2 \cdot P(d-2,t) + u_3 \cdot P(t)$  ( $\Sigma u = 1, u_0$ )
68 # 单日实际费用（计划 take-or-pay + 因果执行 + 5 倍紧急）：
69 #  $J_d(w,u) = E_t p_t \cdot x_t + 5 \cdot E_t p_t \cdot e_t$ 
70 # 其中  $x = \text{plan\_day}(\text{price}, L/6, P/6, EO; \approx 1e-3)$ ,  $e$  由 exec_day_causal 给出。
71 # 标定（窗口平均费用最小）：
72 #  $(w^*, u^*) = \text{argmin}_{\{w,u \text{ 网格}\}} (1/|D_d|) \cdot \Sigma_{\{k D_d\}} J_k(w,u)$ ,
73 #  $D_d = [d-W, d-1] [14, d]$ ;  $W=1$  主结果再用 Adam + 中心差分精化。
74 # =====
75 _WORKER: "AdaptiveWeightModel | None" = None # 子进程中的模型副本
76
77
78 # -----
79 # 工具函数
80 # -----
81 def total_of(df: pd.DataFrame) -> float:
82     """逐日表的年度总费用（计划 + 紧急）。"""
83     return float(df["计划购电费/元"].sum() + df["紧急购电费/元"].sum())
84
85
86 def _init_worker(model: "AdaptiveWeightModel") -> None:
87     global _WORKER
88     _WORKER = model
89
90
91 def _worker_day(d: int):
92     return _WORKER._one_day(d)
93
94
95 def hist_forecast(data: dict, D: int) -> np.ndarray:
96     """统一口径光伏预测（144 槽, kW）：三源凸加权, EWMA h=5 权重（无前视）。
97
98     与 ``consistency.py`` 一致:  $P = u_1 \cdot P(D-1) + u_2 \cdot P(D-2) + u_3 \cdot \text{典型日}$ , 裁剪非负;
99     权重取 ``data["EWMA_WU"][D]``（旧数据若无该字段回退 U_SMOOTH/TH）。
100     """
101     if "PV_OVERRIDE" in data and D in data["PV_OVERRIDE"]:
102         return np.clip(np.asarray(data["PV_OVERRIDE"][D]), dtype=float, 0.0, None)
103     if D < 2:
104         return np.clip(np.asarray(data["pv_typ"], dtype=float), 0.0, None)
105     if "EWMA_WU" in data and D in data["EWMA_WU"]:
106         u = data["EWMA_WU"][D][1]

```

```

107     elif "U_SMOOTH" in data:
108         u = data["U_SMOOTH"][D]
109     else:
110         u = softmax(data["TH"][D - 1, 3:6])
111     pv, typ = data["pv_act"], data["pv_typ"]
112     return np.clip(u[0] * pv[D - 1] + u[1] * pv[D - 2] + u[2] * typ, 0.0, None)
113
114
115 def hist_load_forecast(data: dict, D: int) -> np.ndarray:
116     """统一口径负荷预测（144 槽，kW）：同星期几/两周前/典型日三源凸加权。
117
118     权重取 ``data["EWMA_WU"][D]``（EWMA h=5，只用 D-1 及更早信息）；
119     目标日实际负荷只用于执行回放与残差统计。
120     """
121     if D < 14:
122         return np.clip(np.asarray(data["load_typ"], dtype=float), 0.0, None)
123     if "EWMA_WU" in data and D in data["EWMA_WU"]:
124         w = data["EWMA_WU"][D][0]
125     else:
126         w = softmax(data["TH"][D - 1, :3])
127     load, typ = data["load"], data["load_typ"]
128     d7, d14 = max(0, D - 7), max(0, D - 14)
129     return np.clip(w[0] * load[d7] + w[1] * load[d14] + w[2] * typ, 0.0, None)
130
131
132 def hist_forecast_asof(data: dict, target_D: int, asof_D: int) -> np.ndarray:
133     """在 ``asof_D`` 日 0:00 预测目标日光伏，禁止读取当日及以后实际值。
134
135     两日滚动时 ``target_D=asof_D+1``，附件 3 尚无次日 0:00 预报，因此沿用
136     当日已冻结的 EWMA 权重与截至 ``asof_D-1`` 的最近两条实际日曲线。
137     """
138     if target_D == asof_D:
139         return hist_forecast(data, target_D)
140     if target_D < asof_D or target_D > asof_D + 1:
141         raise ValueError("历史光伏预测仅支持当前日或次日两日窗口")
142     if asof_D < 2:
143         return np.clip(np.asarray(data["pv_typ"], dtype=float), 0.0, None)
144     if "EWMA_WU" in data and asof_D in data["EWMA_WU"]:
145         u = data["EWMA_WU"][asof_D][1]
146     elif "U_SMOOTH" in data:
147         u = data["U_SMOOTH"][asof_D]
148     else:
149         u = softmax(data["TH"][max(asof_D - 1, 13), 3:6])
150     pv, typ = data["pv_act"], data["pv_typ"]
151     d1, d2 = max(0, asof_D - 1), max(0, asof_D - 2)
152     return np.clip(u[0] * pv[d1] + u[1] * pv[d2] + u[2] * typ, 0.0, None)
153
154
155 def hist_load_forecast_asof(data: dict, target_D: int, asof_D: int) -> np.ndarray:
156     """在 ``asof_D`` 日 0:00 预测当前/次日负荷（两日窗口，无前视）。"""
157     if target_D < asof_D or target_D > asof_D + 1:
158         raise ValueError("历史负荷预测仅支持当前日或次日两日窗口")
159     if asof_D < 14:
160         return np.clip(np.asarray(data["load_typ"], dtype=float), 0.0, None)
161     if "EWMA_WU" in data and asof_D in data["EWMA_WU"]:
162         w = data["EWMA_WU"][asof_D][0]
163     else:
164         w = softmax(data["TH"][asof_D - 1, :3])
165     load, typ = data["load"], data["load_typ"]
166     d7, d14 = max(0, target_D - 7), max(0, target_D - 14)
167     if d7 >= asof_D or d14 >= asof_D:
168         raise RuntimeError("两日负荷预测读取了尚未实现的实际日")
169     return np.clip(w[0] * load[d7] + w[1] * load[d14] + w[2] * typ, 0.0, None)
170
171
172 # -----
173 # 口径 E 模型
174 # -----
175 class AdaptiveWeightModel:

```

```

176 """问题二口径 E: 三源自适应加权预测 + 网格/梯度标定 + 全年滚动模拟。
177
178 Parameters
179 -----
180 W : int
181     标定窗口 (天): 目标日 d 用 [d-W, d-1] 的实际总费用标定。主结果 1。
182 grid_step : float
183     权重网格步长 (0.2 → 21×21 = 441 组/天)。
184 steps, lr : int, float
185     Adam 精化步数与学习率。
186 eps_grad : float
187     中心差分步长 ( 空间)。
188 workers : int | None
189     并行进程数, 默认 min(8, CPU 核数)。
190 """
191
192 START = 14 # 最早可用于标定的日历日: 需要 d-14 的历史负荷
193 LAG_L = (7, 14) # 负荷来源滞后 (同星期几, 周周期强)
194 LAG_P = (1, 2) # 光伏来源滞后 (短期持续性)
195 TYP_L = "小区负载" # 附件 1 典型日负荷列名
196 TYP_P = "光伏发电预测功率" # 附件 1 典型日光伏列名
197 EPS_PLAN = 1e-3 # 计划 LP 正则项 (唯一化最优方案)
198
199 def __init__(
200     self,
201     W: int = 1,
202     grid_step: float = 0.2,
203     steps: int = 10,
204     lr: float = 0.05,
205     eps_grad: float = 0.1,
206     workers: int | None = None,
207 ):
208     self.W = W
209     self.grid_step = grid_step
210     self.steps = steps
211     self.lr = lr
212     self.eps_grad = eps_grad
213     self.workers = workers or min(8, os.cpu_count() or 1)
214     self.grid = simplex_grid(grid_step)
215
216     self.dates = self.L = self.P = self.price = None
217     self.L_typ = self.P_typ = None
218     self.data = None
219     self.C = self.TH = self.imp = None
220
221 # -----
222 # 数据
223 # -----
224 def load(self, data: dict | None = None) -> "AdaptiveWeightModel":
225     """读取附件 2 实际序列 + 附件 1 典型日 (负荷/光伏, kW)。"""
226     if data is None:
227         data = load_all()
228         a1 = pm.read_table(DATA_C / "附件1.xlsx")
229         data["load_typ"] = a1[self.TYP_L].to_numpy(float)
230         data["pv_typ"] = a1[self.TYP_P].to_numpy(float)
231     self.dates = data["dates"]
232     self.L = data["load"]
233     self.P = data["pv_act"]
234     self.price = data["price"]
235     self.L_typ = data["load_typ"]
236     self.P_typ = data["pv_typ"]
237     self.data = data
238     return self
239
240 # -----
241 # 预测与单日成本
242 # -----
243 # 预测式
244 # 
$$L = w_1 \cdot L(d-7) + w_2 \cdot L(d-14) + w_3 \cdot L$$
 [kW], 除以 6 → kWh/槽

```

```

245 # P = u1 * P(d-1) + u2 * P(d-2) + u3 * P [kW], 除以 6 → kWh/槽
246 # 凸权重保证非负; 预测值再裁剪 0。
247 def forecast_kw(self, w, u, d: int):
248     """第 d 天三源预测功率 (kW, 未裁剪): forecast 的共享实现。"""
249     l_kw = (
250         w[0] * self.L[d - self.LAG_L[0]]
251         + w[1] * self.L[d - self.LAG_L[1]]
252         + w[2] * self.L_typ
253     )
254     p_kw = (
255         u[0] * self.P[d - self.LAG_P[0]]
256         + u[1] * self.P[d - self.LAG_P[1]]
257         + u[2] * self.P_typ
258     )
259     return l_kw, p_kw
260
261 def forecast(self, w, u, d: int):
262     """第 d 天预测 (kWh/时段): 三源凸组合, 裁剪非负。"""
263     l_kw, p_kw = self.forecast_kw(w, u, d)
264     return np.clip(l_kw, 0.0, None) / 6.0, np.clip(p_kw, 0.0, None) / 6.0
265
266 # 单日费用 (标定目标)
267 # x = plan_day(price, L/6, P/6, E0; =1e-3) (take-or-pay 计划)
268 # e = exec_day_causal(price, L/6, P/6, x, E0) (逐槽因果执行)
269 # J = Σ p * x + 5 * Σ p * e (计划费 + 紧急费)
270 def day_cost(self, d: int, w, u) -> float:
271     """第 d 天实际总费用: 按预测制定计划 (take-or-pay) + 按实际执行 + 紧急购电。
272
273     执行口径与正式统一口径一致 (``consistency.EXEC_POLICY="free"``):
274     无段末硬目标, 仅容量/功率约束。
275     """
276     l_kwh, p_kwh = self.forecast(w, u, d)
277     x, _E, _ = plan_day(self.price, l_kwh, p_kwh, E0, eps=self.EPS_PLAN)
278     ex = exec_segment_causal(
279         self.L[d] / 6.0, self.P[d] / 6.0, x, E0, None
280     )
281     return float(self.price @ x + EMERG_MULT * (self.price @ ex["e"]))
282
283 def window_days(self, d: int, W: int | None = None):
284     """目标日 d 的标定窗口 [d-W, d-1] (截止到最早可用日 START)。"""
285     W = self.W if W is None else W
286     return list(range(max(self.START, d - W), d))
287
288 # -----
289 # 梯度精化 (softmax + 数值梯度 + torch.optim.Adam)
290 # -----
291 # 数值梯度精化 (softmax + 中心差分 + Adam)
292 # 窗口目标 J(.) = (1/|D|) * Σ_{d ∈ D} J_d(softmax([:3]), softmax([3:]))
293 # 中心差分 J/_{e_i} [J(+e_i) - J(-e_i)] / (2), e_i 为第 i 个单位方向
294 # 用 torch.optim.Adam 迭代 steps 步 (只保留优于网格解时才采用)。
295 def window_cost(self, theta, days) -> float:
296     """窗口平均实际总费用 (标定目标; 为 6 维: 负荷 3 + 光伏 3)。"""
297     w, u = softmax(theta[:3]), softmax(theta[3:])
298     return float(np.mean([self.day_cost(d, w, u) for d in days]))
299
300 def calibrate(self, d: int, theta, W: int | None = None) -> np.ndarray:
301     """在窗口 [d-W, d-1] 上做 steps 步 Adam (中心差分梯度), 返回 6 维。"""
302     days = self.window_days(d, W)
303     th = torch.tensor(theta, dtype=torch.float64)
304     opt = torch.optim.Adam([th], lr=self.lr)
305     for _ in range(self.steps):
306         base = th.detach().numpy()
307         grad = np.empty(6)
308         for i in range(6):
309             plus, minus = base.copy(), base.copy()
310             plus[i] += self.eps_grad
311             minus[i] -= self.eps_grad
312             grad[i] = (
313                 self.window_cost(plus, days) - self.window_cost(minus, days)

```

```

314         ) / (2 * self.eps_grad)
315         th.grad = torch.from_numpy(grad)
316         opt.step()
317         opt.zero_grad()
318         return th.detach().numpy()
319
320 # -----
321 # 成本表（并行预计算 + 缓存）
322 # -----
323 def _one_day(self, d: int):
324     """日历日 d: 网格成本行 + 从网格最优出发的 Adam 精化权重（供次日使用）。"""
325     n = len(self.grid)
326     row = np.empty((n, n))
327     best = (np.inf, 0, 0)
328     for i, w in enumerate(self.grid):
329         for j, u in enumerate(self.grid):
330             c = self.day_cost(d, w, u)
331             row[i, j] = c
332             if c < best[0]:
333                 best = (c, i, j)
334     theta0 = np.concatenate(
335         [np.log(self.grid[best[1]] + 1e-9), np.log(self.grid[best[2]] + 1e-9)]
336     )
337     if d + 1 < N_DAY: # 精化“用于次日”的权重: 目标日 d+1, 标定在日 d
338         theta = self.calibrate(d + 1, theta0, W=1)
339         w, u = softmax(theta[:3]), softmax(theta[3:])
340         polished = self.day_cost(d, w, u)
341         if polished < best[0]:
342             return d, row, theta, float(best[0]), float(polished)
343     return d, row, theta0, float(best[0]), float(best[0])
344
345 # 缓存键
346 # key = MD5( L || P || price || L || P || (grid_step, steps, lr, eps_grad)
347 #         || CAUSAL_POLICY_VERSION )[:12]
348 # 执行口径变更必须升 CAUSAL_POLICY_VERSION, 否则旧缓存会污染新结果。
349 def _cache_key(self) -> str:
350     h = hashlib.md5()
351     for arr in (self.L, self.P, self.price, self.L_typ, self.P_typ):
352         h.update(np.ascontiguousarray(arr, dtype=float).tobytes())
353     h.update(
354         np.asarray(
355             [self.grid_step, self.steps, self.lr, self.eps_grad], float
356         ).tobytes()
357     )
358     h.update(CAUSAL_POLICY_VERSION.encode("utf-8"))
359     return h.hexdigest()[:12]
360
361 # 成本表并行预计算 + 缓存
362 # 表结构: C[k,i,j] 为日历日 k 在权重网格格 (i,j) 的实际总费用;
363 #         TH[k] 为从网格最优出发的 Adam 精化参数 (6 维 );
364 #         imp[k] = max(0, 网格最优 - 精化后) 为精化收益。
365 # 缓存: code/outputs/cache/q2e/table_{key}.npz (子类可用 CACHE_TAG 隔离)。
366 def build_table(self, force: bool = False) -> "AdaptiveWeightModel":
367     """计算/加载成本表: C[k,i,j] 日 k 网格费用, TH[k] 日 k 精化, imp 精化收益。"""
368     pm.init(root=str(ROOT))
369     tag = getattr(self, "CACHE_TAG", "q2e") # 子类变体隔离缓存 (如回归源模型)
370     cache = pm.outputs_dir() / "cache" / tag
371     cache.mkdir(parents=True, exist_ok=True)
372     path = cache / f"table_{self._cache_key()}.npz"
373     if path.exists() and not force:
374         z = np.load(path)
375         self.C, self.TH, self.imp = z["C"], z["TH"], z["imp"]
376         return self
377
378     days = list(range(self.START, N_DAY))
379     t0 = time.time()
380     if self.workers > 1:
381         from multiprocessing import Pool
382

```

```

383         with Pool(self.workers, initializer=_init_worker, initargs=(self,)) as pool:
384             results = pool.map(_worker_day, days)
385         else:
386             results = [self._one_day(d) for d in days]
387
388         n = len(self.grid)
389         self.C = np.zeros((N_DAY, n, n))
390         self.TH = np.zeros((N_DAY, 6))
391         self.imp = np.zeros(N_DAY)
392         for d, row, theta, c_grid, c_pol in results:
393             self.C[d], self.TH[d], self.imp[d] = row, theta, max(0.0, c_grid - c_pol)
394         np.savez_compressed(path, C=self.C, TH=self.TH, imp=self.imp, tag=tag)
395         print(
396             f"[q2e] 成本表完成: {len(days)} 天 * {n * n} 组, 用时 {time.time() - t0:.0f}s"
397         )
398         return self
399
400 # -----
401 # 权重选择
402 # -----
403 def weights(self, W: int | None = None, use_gd: bool = True):
404     """目标日 d 的权重: W=1 用精化, 其余用窗口成本表最小格。"""
405     W = self.W if W is None else W
406     out = []
407     for d in range(REPORT_START, N_DAY):
408         if use_gd and W == 1:
409             theta = self.TH[d - 1]
410             out.append((softmax(theta[:3]), softmax(theta[3:])))
411         else:
412             lo = max(self.START, d - W)
413             i, j = np.unravel_index(
414                 np.argmin(self.C[lo:d].mean(axis=0)),
415                 (len(self.grid), len(self.grid)),
416             )
417             out.append((self.grid[i].copy(), self.grid[j].copy()))
418     return out
419
420 def oracle_weights(self):
421     """事后最优权重: 每个目标日直接取该日成本最小的网格格 (仅作下界参考)。"""
422     out = []
423     for d in range(REPORT_START, N_DAY):
424         i, j = np.unravel_index(
425             np.argmin(self.C[d]), (len(self.grid), len(self.grid))
426         )
427         out.append((self.grid[i].copy(), self.grid[j].copy()))
428     return out
429
430 def const_weights(self, w, u):
431     """固定权重序列 (对照基线)。"""
432     return [(np.asarray(w, float), np.asarray(u, float))] * (N_DAY - REPORT_START)
433
434 # -----
435 # 全年模拟
436 # -----
437 def simulate(self, weights, detail: bool = False):
438     """全年滚动: 计划 (按预测) → 执行 (按实际) → 紧急购电。
439
440     detail=False 返回逐日表; detail=True 额外返回 (x_plans, recs),
441     供官方 result2 填表 (recs[d] 含 c/d/e/E/E_start/E_end)。
442     """
443     rows = []
444     x_plans = np.zeros((N_DAY, T)) if detail else None
445     recs = [None] * N_DAY if detail else None
446     for i, d in enumerate(range(REPORT_START, N_DAY)):
447         w, u = weights[i]
448         l_kwh, p_kwh = self.forecast(w, u, d)
449         x, _E, _ = plan_day(self.price, l_kwh, p_kwh, E0, eps=self.EPS_PLAN)
450         ex = exec_day_causal(
451             self.price, self.L[d] / 6.0, self.P[d] / 6.0, x, E0

```

```

452     )
453     if detail:
454         x_plans[d] = x
455         recs[d] = {"c": ex["c"], "d": ex["d"], "s": ex["s"], "e": ex["e"],
456                  "E": ex["E"], "E_start": E0, "E_end": float(ex["E"][-1])}
457     residual = (
458         self.P[d] / 6.0
459         + x
460         + ex["d"]
461         + ex["e"]
462         - self.L[d] / 6.0
463         - ex["c"]
464         - ex["s"]
465     )
466     rows.append(
467         {
468             "日期": self.dates[d],
469             "w1_L(d-7)": w[0],
470             "w2_L(d-14)": w[1],
471             "w3_典型日": w[2],
472             "u1_P(d-1)": u[0],
473             "u2_P(d-2)": u[1],
474             "u3_典型日": u[2],
475             "计划购电量/kWh": float(x.sum()),
476             "计划购电费/元": float(self.price @ x),
477             "紧急购电量/kWh": float(ex["e"].sum()),
478             "紧急购电费/元": float(EMERG_MULT * (self.price @ ex["e"])),
479             "平衡残差/kWh": float(np.abs(residual).max()),
480             "储电量最小/kWh": float(ex["E"].min()),
481             "储电量最大/kWh": float(ex["E"].max()),
482         }
483     )
484     df = pd.DataFrame(rows)
485     return (df, x_plans, recs) if detail else df
486
487 def write_result2(self, W: int = 7, out_name: str = "result2.xlsx"):
488     """用口径 E (默认 W=7) 生成官方 ``results/result2.xlsx`` (三张表)。
489
490     写出前把现有 result2 备份到 ``code/outputs/result2_D_backup.xlsx``,
491     写出后回读校验并记录到 RESULTS_REPORT.md。
492     """
493     import shutil
494
495     pm.init(root=str(ROOT))
496     log = pm.get_logger("q2e")
497     if self.C is None:
498         self.build_table()
499     df, x_plans, recs = self.simulate(self.weights(W=W, use_gd=False), detail=True)
500
501     plan = float(df["计划购电费/元"].sum())
502     emerg = float(df["紧急购电费/元"].sum())
503     cur = RESULTS_DIR / out_name
504     if cur.exists():
505         backup = pm.outputs_dir() / "result2_D_backup.xlsx"
506         shutil.copy2(cur, backup)
507         log.info("旧 result2 已备份 -> {}", backup)
508     out = write_result2(self.dates, self.price, x_plans, recs, out_name=out_name)
509     checks = _verify_result2(out, df)
510     pm.save_outputs(df, f"q2e_w{W}_daily")
511     record(
512         f"问题二 口径E (W={W}) 官方 result2 结果与校验",
513         {
514             "计划购电费/万元": round(plan / 1e4, 1),
515             "紧急购电费/万元": round(emerg / 1e4, 1),
516             "总费用/万元": round((plan + emerg) / 1e4, 1),
517             "天数": len(df),
518             **checks,
519         },
520         note=(

```

```

521         f"官方 results/{out_name} 由口径 E (W={W}, 7 天窗口成本表 argmin) 生成; "
522         "旧口径 D 版备份在 code/outputs/result2_D_backup.xlsx。"
523         "复现: uv run python -m solve.q2_adaptive --result2。"
524     ),
525 )
526 log.info("result2.xlsx (口径 E, W={}) 写出完成: {}, 总费用 {:.1f} 万元",
527         W, out, (plan + emerg) / 1e4)
528 return out
529
530 # -----
531 # 主流程
532 # -----
533 def run(self, force_table: bool = False) -> dict:
534     """口径 E 主流程: 成本表 → 权重 → 模拟 → 对照 → 校验 → 出图 → 记录。"""
535     import matplotlib.pyplot as plt
536
537     pm.init(root=str(ROOT))
538     log = pm.get_logger("q2e")
539     if self.C is None:
540         self.build_table(force=force_table)
541     W = self.W
542
543     df_e = self.simulate(self.weights(W=W, use_gd=True)) # 主结果
544     df_g = self.simulate(self.weights(W=1, use_gd=False)) # 仅网格
545     df_b = self.simulate(self.const_weights([0, 0, 1], [0, 0, 1])) # 典型日
546     df_q = self.simulate(self.const_weights([1 / 3] * 3, [1 / 3] * 3)) # 等权
547     _x_d, recs_d = run_deterministic(self._as_data()) # 口径 D
548     rep = list(range(REPORT_START, N_DAY))
549     df_d = pd.DataFrame(
550         {
551             "日期": [self.dates[d] for d in rep],
552             "计划购电费/元": [recs_d[d]["plan_cost"] for d in rep],
553             "紧急购电量/kWh": [recs_d[d]["emerg_kwh"] for d in rep],
554             "紧急购电费/元": [recs_d[d]["emerg_cost"] for d in rep],
555         }
556     )
557     df_o = self.simulate(self.oracle_weights()) # 事后最优下界
558
559     # ---- 对照表 ----
560     names = [
561         "典型日 (B)",
562         "等权组合",
563         "网格选择 (W=1)",
564         f"网格+梯度 (E, W={W})",
565         "0:00 预报 (D)",
566         "事后最优 (下界)",
567     ]
568     dfs = [df_b, df_q, df_g, df_e, df_d, df_o]
569     table = pd.DataFrame(
570         {
571             "口径": names,
572             "计划购电费/万元": [
573                 round(d["计划购电费/元"].sum() / 1e4, 1) for d in dfs
574             ],
575             "紧急购电费/万元": [
576                 round(d["紧急购电费/元"].sum() / 1e4, 1) for d in dfs
577             ],
578         }
579     )
580     table["总费用/万元"] = (
581         table["计划购电费/万元"] + table["紧急购电费/万元"]
582     ).round(1)
583     record(
584         "问题二 口径E 结果对照 (2025-02-01 ~ 12-31)",
585         table,
586         note=(
587             "预测式: 负荷 w1 * L(d-7)+w2 * L(d-14)+w3 * 典型日; 光伏 u1 * P(d-1)+u2 * P(d-2)+u3 * 典型日。"
588             f"权重每天在窗口 W={W} 上标定后用于次日: 先步长 {self.grid_step} 网格全局定位, "
589             f"再做 {self.steps} 步 Adam (中心差分) 精化并保留更优者。"

```

```

590         "对照：典型日、等权、纯网格、附件 3 的 0:00 预报（口径 D）、事后最优（网格下界）。"
591         "执行口径与问题二一致。"
592     ),
593 )
594 pm.save_outputs(table, "q2e_compare")
595 pm.save_outputs(df_e, "q2e_daily")
596
597 # ---- 一致性校验 ----
598 w_cols = ["w1_L(d-7)", "w2_L(d-14)", "w3_典型日"]
599 u_cols = ["u1_P(d-1)", "u2_P(d-2)", "u3_典型日"]
600 n_imp = int((self.imp > 1e-6).sum())
601 record(
602     "问题二 口径E 一致性与梯度精化校验",
603     {
604         "平衡最大残差/kWh": float(df_e["平衡残差/kWh"].max()),
605         "负荷权重和最小值": float(df_e[w_cols].sum(axis=1).min()),
606         "负荷权重和最大值": float(df_e[w_cols].sum(axis=1).max()),
607         "光伏权重和最小值": float(df_e[u_cols].sum(axis=1).min()),
608         "光伏权重和最大值": float(df_e[u_cols].sum(axis=1).max()),
609         "储电量最小值/kWh": float(df_e["储电量最小/kWh"].min()),
610         "储电量最大值/kWh": float(df_e["储电量最大/kWh"].max()),
611         "发生紧急购电天数": int((df_e["紧急购电量/kWh"] > 1e-3).sum()),
612         "梯度精化改善天数": n_imp,
613         "梯度精化日均改善/元": float(self.imp[self.imp > 0].mean()),
614         if n_imp
615         else 0.0,
616         "梯度精化年改善/元": float(self.imp[REPORT_START:].sum()),
617     },
618     note=(
619         "权重由 softmax 生成、和恒为 1；平衡残差为数值误差量级即通过。"
620         f"梯度精化相对网格最优的改善：{n_imp} 天，"
621         f"年累计 {self.imp[REPORT_START:].sum():.0f} 元。"
622     ),
623 )
624
625 # ---- 图：权重演化 ----
626 dates = df_e["日期"].tolist()
627 x = np.arange(len(dates))
628
629 fig, axes = plt.subplots(2, 1, figsize=(7, 5.6), sharex=True)
630 groups = [
631     (axes[0], w_cols, "负荷权重", ["d-7", "d-14", "典型日"]),
632     (axes[1], u_cols, "光伏权重", ["d-1", "d-2", "典型日"]),
633 ]
634 for ax, cols, ylabel, labels in groups:
635     ax.stackplot(
636         x, [df_e[c].to_numpy() for c in cols], labels=labels, alpha=0.9
637     )
638     ax.set_ylabel(ylabel)
639     ax.set_ylim(0, 1)
640     ax.legend(loc="upper center", ncol=3, fontsize=8, framealpha=0.9)
641 axes[1].set_xlabel("日期")
642 month_axis(axes[1], dates)
643 pm.save_fig(fig, "Q2E_权重演化", data=df_e[["日期"] + w_cols + u_cols])
644
645 # ---- 图：费用对照 ----
646 pm.bar_group(
647     names[:5],
648     {
649         "计划购电费": table["计划购电费/万元"].tolist()[:5],
650         "紧急购电费": table["紧急购电费/万元"].tolist()[:5],
651     },
652     ylabel="年度费用 / 万元",
653     rot=10,
654     save="Q2E_费用对比",
655 )
656
657 # ---- 图：逐日紧急购电量 ----
658 fig3, ax3 = pm.line(

```

```

659         x,
660         [df_e["紧急购电量/kWh"].to_numpy(), df_d["紧急购电量/kWh"].to_numpy()],
661         labels=["口径E（网格+梯度）", "口径D（0:00预报）"],
662         xlabel="日期",
663         ylabel="紧急购电量 / kWh",
664     )
665     ax3.legend()
666     month_axis(ax3, dates)
667     pm.save_fig(
668         fig3,
669         "Q2E_紧急购电对比",
670         data=pd.DataFrame(
671             {
672                 "日期": dates,
673                 "口径E_紧急购电量_kWh": df_e["紧急购电量/kWh"],
674                 "口径D_紧急购电量_kWh": df_d["紧急购电量/kWh"],
675             }
676         ),
677     )
678
679     log.info(
680         "口径 E (W={}) 总费用 {:.1f} 万元 (计划 {:.1f} + 紧急 {:.1f})",
681         W,
682         total_of(df_e) / 1e4,
683         df_e["计划购电费/元"].sum() / 1e4,
684         df_e["紧急购电费/元"].sum() / 1e4,
685     )
686     return {
687         "daily": df_e,
688         "compare": table,
689         "C": self.C,
690         "TH": self.TH,
691         "imp": self.imp,
692     }
693
694     def _as_data(self) -> dict:
695         """还原 q2 所需的完整 data 字典（供口径 D 对照）。"""
696         return self.data
697
698     # -----
699     # 窗口敏感性
700     # -----
701     def window_sensitivity(
702         self, W_list=(1, 3, 7, 14, 30), force_table=False
703     ) -> pd.DataFrame:
704         """标定窗口 W 敏感性：查成本表选择权重后全年模拟。"""
705         pm.init(root=str(ROOT))
706         log = pm.get_logger("q2e")
707         if self.C is None:
708             self.build_table(force=force_table)
709         rows = []
710         for W in W_list:
711             t0 = time.time()
712             df = self.simulate(self.weights(W=W, use_gd=False))
713             rows.append(
714                 {
715                     "窗口W/天": W,
716                     "计划购电费/万元": round(float(df["计划购电费/元"].sum()) / 1e4, 1),
717                     "紧急购电费/万元": round(float(df["紧急购电费/元"].sum()) / 1e4, 1),
718                     "总费用/万元": round(total_of(df) / 1e4, 1),
719                 }
720             )
721             log.info(
722                 "W={} 总费用 {:.1f} 万元 ( {:.0f}s )",
723                 W,
724                 rows[-1]["总费用/万元"],
725                 time.time() - t0,
726             )
727         out = pd.DataFrame(rows)

```

```

728     pm.save_outputs(out, "Q2E_窗口敏感性")
729     record(
730         "问题二 口径E 标定窗口敏感性",
731         out,
732         note="主结果 W=1 (含梯度精化)；本表为纯网格选择, W 越大权重越平滑。",
733     )
734     pm.line(
735         out["窗口W/天"],
736         out["总费用/万元"],
737         xlabel="标定窗口 W / 天",
738         ylabel="年度总费用 / 万元",
739         marker="o",
740         save="Q2E_窗口敏感性",
741     )
742     return out

```

program/src/solve/models/errors.py

```

1  """附件 3 预报误差模型（相对（时刻,月）期望出力；见 C题_预报误差分析.md）。
2
3  系统偏差 (k) = 0.0050 - 0.00128k; 误差尺度 (k) = 0.0006 + 0.0084k.
4  """
5  from __future__ import annotations
6
7  import numpy as np
8
9
10 # =====
11 # 误差模型（相对（时刻,月）期望出力的乘性误差）
12 #
13 # (k) = 0.0050 - 0.00128 * k (系统偏差, k = 1..24)
14 # (k) = 0.0006 + 0.0084 * k (误差尺度)
15 #
16 # 标准化误差日块:
17 # z_dk = [ (F_dk - A_dk)/T_mk - (k) ] / (k) (仅 T_mk > 100 kW 的槽)
18 # 情景还原（蒙特卡洛/对冲采样）:
19 # P_dk = F_dk + T_mk * ( (k) + (k) * z_dk ), 再裁剪 0
20 #
21 # 其中 F 为附件 3 的 0:00 预报、A 为附件 2 实际、T_mk 为典型日（时刻,月）均值。
22 # 保留整日误差块（含日内相关与左尾）是 Q2 概率扩展的关键口径。
23 # =====
24 def mu(k):
25     """系统偏差 (k) = 0.0050 - 0.00128k. """
26     return 0.0050 - 0.00128 * np.asarray(k, dtype=float)
27
28
29 def sigma(k):
30     """误差尺度 (k) = 0.0006 + 0.0084k. """
31     return 0.0006 + 0.0084 * np.asarray(k, dtype=float)
32
33
34 def scenario_hourly(fc0_row, month, z_row, typ_hm) -> np.ndarray:
35     """由标准化误差块生成情景小时序列（裁剪到 0）。
36
37     P_k = F_k - T_k * ( _k + _k * z_k ), 其中 k = 1..24（整点口径）。
38     注意 err = 预报 - 实际, 因此实际情景 = 预报 - err.
39     """
40     ks = np.arange(1, 25)
41     typ = typ_hm[1:25, month]
42     err = typ * (mu(ks) + sigma(ks) * z_row)
43     return np.clip(fc0_row - err, 0.0, None)
44
45
46 def scenario_from_residual(fc0_row, residual_row) -> np.ndarray:
47     """由原始残差块生成小时实际情景: 残差定义为"预报 - 实际"。"""
48     return np.clip(np.asarray(fc0_row, float) - np.asarray(residual_row, float),
49                    0.0, None)
50
51

```

```

52 # 兼容别名（旧调用点）
53 _mu = mu
54 _sigma = sigma
55 _scenario_hourly = scenario_hourly

```

program/src/solve/models/price.py

```

1  """Q4 电价预测模型：三源凸组合 + 滚动费用标定 + 动态修正。
2
3  # =====
4  # 三源预测（对附件 4 的实时波动电价）：
5  #   P(D,t) = v1 * P4(D-1,t) + v2 * P4(D-7,t) + v3 * P_typ(t)
6  #   其中 P4 为附件 4 逐槽电价，P_typ 为附件 1 典型日电价（逐槽均值），
7  #   v = (v1,v2,v3) 为凸权重（softmax 参数化或网格）。
8  #
9  # 滚动标定（无前视）：v*_D = argmin_{v 网格} (1/|H_D|) * Σ_{k H_D} C_k(v)
10 #   H_D = [max(31, D-7), D), C_k(v) 为日 k 按预测价计划、按真实价结算的费用；
11 # 动态修正（参数平滑）：v_D = β * v*_D + (1-β) * v_{D-1}, β = 0.1。
12 # =====
13 """
14 from __future__ import annotations
15
16 import numpy as np
17
18 from solve.common import N_DAY, REPORT_START, ROOT
19
20 BETA = 0.1          # 电价权重的动态修正（新旧混合）
21 W_V = 7            # 滚动标定窗口（天）
22
23
24 def forecast_price(D, v, p4, p_typ):
25     """三源预测（kW 同量纲，元/kWh）；D 需要 7。
26
27     P = v1 * P4(D-1) + v2 * P4(D-7) + v3 * P_typ。
28     """
29     return (v[0] * p4[D - 1] + v[1] * p4[D - 7] + v[2] * p_typ)
30
31
32 def price_forecast_d(D: int, v: np.ndarray, p4: np.ndarray, p_typ: np.ndarray) -> np.ndarray:
33     """目标日 D 的电价预测（D 7）；D<7 退化为典型日。"""
34     return forecast_price(D, v, p4, p_typ) if D >= 7 else p_typ
35
36
37 def price_forecast_next(D: int, v: np.ndarray, p4: np.ndarray, p_typ: np.ndarray) -> np.ndarray:
38     """day D+1 的预测（仅用 day D 可得信息）：P(D) 未知，用 P(D-1) 替代。
39
40     q4 年度流程专用（末端夹取到 N_DAY-1）。
41     """
42     d1 = min(D + 1, N_DAY - 1)
43     return v[0] * p4[d1 - 2] + v[1] * p4[d1 - 7] + v[2] * p_typ
44
45
46 def forecast_price_next_asof(D, v, p4, p_typ):
47     """在 D 日 0:00 预测 D+1 电价，不读取 P(D)（电价参数拟合/动态修正用，末端不夹取）。
48
49     D<7 退化为典型日；P(D+1) = v1 * P4(D-1) + v2 * P4(D-6) + v3 * P_typ。
50     """
51     if D < 7:
52         return p_typ
53     return v[0] * p4[D - 1] + v[1] * p4[D - 6] + v[2] * p_typ
54
55
56 def rolling_v_star() -> np.ndarray:
57     """逐日电价三源权重 v*：窗口 W=7 费用标定的 argmin（不平滑，无前视）。
58
59     v*_D = argmin_{v 网格} (1/|H_D|) * Σ_{k H_D} C_k(v), H_D = [max(31, D-7), D)。
60     返回 (N_DAY, 3)；D < REPORT_START 的位置为占位零向量（调用方不取）。
61     依赖 code/outputs/q4_price_fit_daily.npz（由 solve.q4_price_fit 生成）。
62     """

```

```

63     z = np.load(ROOT / "code" / "outputs" / "q4_price_fit_daily.npz")
64     daily_costs = z["daily_costs"]
65     grid = [tuple(v) for v in z["grid"]]
66     seq = np.zeros((N_DAY, 3))
67     for D in range(REPORT_START, N_DAY):
68         hist = daily_costs[max(REPORT_START, D - W_V):D]
69         # 预热首日窗口为空: 返回纯典型日 (0,0,1) (与旧实现一致)
70         seq[D] = (np.array(grid[int(np.argmin(hist.mean(axis=0)))]))
71         if len(hist) else np.array([0.0, 0.0, 1.0]))
72     return seq
73
74
75 def rolling_v_seq() -> np.ndarray:
76     """逐日电价三源权重: 滚动费用标定 (W=7) + 动态修正 =0.1 (无前视)。
77
78     v_D =  $\beta \cdot v_D + (1-\beta) \cdot v_{D-1}$ ; D<31 预热用纯典型日 (0,0,1)。
79     """
80     v_star = rolling_v_star()
81     seq = np.tile(np.array([0.0, 0.0, 1.0]), (N_DAY, 1))
82     prev = None
83     for D in range(REPORT_START, N_DAY):
84         v_new = v_star[D]
85         prev = v_new if prev is None else BETA * v_new + (1.0 - BETA) * prev
86         seq[D] = prev
87     return seq

```

program/src/solve/models/weights.py

```

1  """凸权重工具: softmax 参数化、单纯形网格与权重序列平滑。"""
2  from __future__ import annotations
3
4  import numpy as np
5
6  from solve.common import N_DAY
7
8
9  # =====
10 # 一、凸权重参数化
11 #     softmax:  $w_i = \exp(-i - \max) / \sum_j \exp(-j - \max)$ 
12 #             (减最大值仅用于数值稳定, 不改变结果; 输出非负且  $\sum w = 1$ )
13 #     单纯形网格 (步长  $h = 1/n$ ):
14 #             { (i/n, j/n, (n-i-j)/n) : i,j = 0, ..., n }
15 #             组数 = (n+1)(n+2)/2
16 # =====
17 def softmax(theta) -> np.ndarray:
18     """logit 向量  $\rightarrow$  凸权重 (非负、和=1)。"""
19     e = np.exp(theta - theta.max())
20     return e / e.sum()
21
22
23 def simplex_grid(step: float) -> np.ndarray:
24     """3 维凸权重网格 (非负、和=1), 组数 (n+1)(n+2)/2。"""
25     n = int(round(1.0 / step))
26     return np.array(
27         [
28             (i / n, j / n, (n - i - j) / n)
29             for i in range(n + 1)
30             for j in range(n + 1 - i)
31         ]
32     )
33
34
35 # =====
36 # 二、权重序列平滑 (参数混合)
37 #     u_D =  $\beta \cdot u_{\text{new}}(D) + (1-\beta) \cdot u_{D-1}$ 
38 #     u_new(D) = softmax(TH[D-1][3:6]) (Q2E 在标定日 D-1 为次日给出的权重)
39 #     凸组合保持非负、和=1;  $\beta=1$  退化为不平滑。
40 # =====
41 def make_smooth_u(data: dict, beta: float) -> np.ndarray:

```

```

42     """参数平滑（新旧混合）：u_D =  $\beta$  * u_new + (1- $\beta$ ) * u_{D-1}。
43
44     u_new[D] = softmax(TH[D-1][3:6]) (Q2E 在标定日 D-1 为次日给出的权重)；
45     凸组合自动保持权重非负且和=1； $\beta=1$  退化为不平滑。
46     """
47     TH = data["TH"]
48     U = np.zeros((N_DAY, 3))
49     prev = None
50     for D in range(N_DAY):
51         j = max(D - 1, 13)
52         unew = softmax(TH[j, 3:6])
53         prev = unew if prev is None else  $\beta$  * unew + (1.0 -  $\beta$ ) * prev
54         U[D] = prev
55     return U
56
57
58 # 兼容别名（旧代码以 _softmax 调用）
59 _softmax = softmax

```

flows

program/src/solve/flows/__init__.py

```

1     """问题级年度/单日滚动流程（依赖 core/models/data，不含顶层 CLI）。"""

```

program/src/solve/flows/q2_year.py

```

1     """Q2 年度流程：确定性滚动、蒙特卡洛、两阶段对冲计划（统一口径 v1.1）。
2
3     # =====
4     # 1) 确定性滚动
5     #     每日 d:  $\beta$  * 历史负荷预测 + 0:00 官方光伏预报，做 48 小时滚动计划
6     #     (plan_horizon 视野末端完全自由)，只执行首日；逐槽因果执行 (free)。
7     #     费用：计划费 =  $\sum p \cdot x$ ，紧急费 =  $5 \cdot \sum p \cdot e$ ；日末 SOC 跨日传递。
8     # 2) 蒙特卡洛（固定计划，离线风险评价，不参与在线决策）
9     #     对 d [31, 365] 从同月整日原始残差块 RO（预报-实际）重采样；
10    #     计划费固定，分布差异完全来自紧急购电。
11    # 3) 两阶段对冲计划（期望费用最小）
12    #      $\min \sum p \cdot x + (1/S) \cdot \sum_s [5p \cdot e_s + (c_s + d_s)]$ 
13    #     s.t. x 为所有情景共用第一阶段决策；每情景 s 的储能/紧急独立、
14    #         段末回到 e_terminal；场景池严格取目标日之前（同月 14 天优先，
15    #         否则回看 90 天；Q2 离线评估用同月全年样本，在线对冲无前视）。
16    # =====
17    """
18    from __future__ import annotations
19
20    import time
21
22    import numpy as np
23    from scipy.sparse import csr_matrix
24
25    import program as pm
26    from solve.common import (
27        EO,
28        E_MAX,
29        E_MIN,
30        EMERG_MULT,
31        EPS_THROUGHPUT,
32        ETA,
33        N_DAY,
34        P_MAX_E,
35        REPORT_START,
36        T,
37    )
38    from solve.core.causal import exec_segment_causal
39    from solve.core.lp import plan_horizon

```

```

40 from solve.core.slots import hour_to_slots
41 from solve.models.errors import scenario_from_residual
42
43
44 def run_deterministic(data: dict):
45     """2025 全年确定性滚动 (1 月预热)：按统一预测计划、按实际因果执行。"""
46     price = data["price"]
47     load, pv_act, fc0 = data["load"], data["pv_act"], data["fc0"]
48     E = E0
49     x_plans = np.zeros((N_DAY, T))
50     recs = []
51     for d in range(N_DAY):
52         pv_fc = hour_to_slots(fc0[d]) / 6.0          # kW → kWh/时段
53         load_kwh = load[d] / 6.0
54         # 48 小时滚动：次日无对应 0:00 官方预报，使用典型日作保守占位；
55         # 当前日末 SOC 由两日优化决定并跨日传递，规划窗口末端完全自由。
56         xh, _res = plan_horizon(
57             np.tile(price, 2),
58             np.concatenate([load_kwh, data["load_tpy"] / 6.0]),
59             np.concatenate([pv_fc, data["pv_tpy"] / 6.0]),
60             E, None,
61         )
62         x = xh[:T]
63         # 统一执行策略 (consistency.EXEC_POLICY="free")：无段末硬目标
64         ex = exec_segment_causal(load_kwh, pv_act[d] / 6.0, x, E, None)
65         recs.append({
66             "x": x, "plan_cost": float(price @ x),
67             "c": ex["c"], "d": ex["d"], "s": ex["s"], "e": ex["e"], "E": ex["E"],
68             "emerg_kwh": float(ex["e"].sum()),
69             "emerg_cost": float(EMERG_MULT * (price @ ex["e"])),
70             "E_start": float(E), "E_end": float(ex["E"][-1]),
71             "spill_kwh": float(ex["s"].sum()),
72         })
73         x_plans[d] = x
74         E = float(ex["E"][-1])
75     return x_plans, recs
76
77
78 def run_mc(data, x_plans, e_start_feb, e_targets, n_years=100, seed=42):
79     """固定计划，对 2.1-12.31 重采样同月整日原始残差块，评估年度紧急费用分布。"""
80     price = data["price"]
81     load, fc0, residuals, months = (
82         data["load"], data["fc0"], data["R0"], data["months"],
83     )
84     rng = np.random.default_rng(seed)
85     emerg_costs = np.zeros(n_years)
86     emerg_kwhs = np.zeros(n_years)
87     t0 = time.time()
88     for rep in range(n_years):
89         E = float(e_start_feb)
90         for d in range(REPORT_START, N_DAY):
91             pool = np.flatnonzero(months == months[d])
92             zi = int(rng.choice(pool))
93             pv_s = hour_to_slots(scenario_from_residual(fc0[d], residuals[zi])) / 6.0
94             # 统一执行策略 (free)：无段末硬目标
95             ex = exec_segment_causal(load[d] / 6.0, pv_s, x_plans[d], E, None)
96             emerg_kwhs[rep] += float(ex["e"].sum())
97             emerg_costs[rep] += float(EMERG_MULT * (price @ ex["e"]))
98             E = float(ex["E"][-1])
99         if (rep + 1) % 10 == 0:
100             print(f" MC {rep + 1}/{n_years} 完成, 用时 {time.time() - t0:.1f}s")
101     return emerg_costs, emerg_kwhs
102
103
104 def hedge_day(price, load_kwh, fc0_row, residual_pool, e_start, e_terminal,
105             n_scen=20, rng=None):
106     """两阶段场景 LP：计划 x 所有场景共用；每场景储能再调度 + 紧急购电。
107
108     等概率情景下目标为期望费用：

```

```

109         min     $\Sigma p \cdot x + (1/S) \cdot \Sigma_s [5p \cdot e_s + \cdot (c_s + d_s)]$ 
110
111     计划成本只发生一次，场景补救成本必须按概率 ``1/S`` 加权；否则情景数会
112     人为改变风险项相对计划成本的权重。
113     """
114     if rng is None:
115         rng = np.random.default_rng(0)
116     picks = rng.integers(0, len(residual_pool), size=n_scen)
117     pv_scen = [
118         hour_to_slots(scenario_from_residual(fc0_row, residual_pool[i])) / 6.0
119         for i in picks
120     ]
121
122     # 变量块: 0=x (第一阶段, T), 其后每情景 5T (c/d/s/E/e)
123     n = T + n_scen * 5 * T
124     c_obj = np.zeros(n)
125     c_obj[0:T] = price
126     for s in range(n_scen):
127         base = T + s * 5 * T
128         c_obj[base:base + T] = EPS_THROUGHPUT / n_scen
129         c_obj[base + T:base + 2 * T] = EPS_THROUGHPUT / n_scen
130         c_obj[base + 4 * T:base + 5 * T] = EMERG_MULT * price / n_scen
131
132     # 等式约束按 COO 三元组拼接: 每情景 (2T+1) 行
133     rows = n_scen * (2 * T + 1)
134     rr, cc, vv = [], [], []
135     for s in range(n_scen):
136         r0 = s * (2 * T + 1)
137         base = T + s * 5 * T
138         t = np.arange(T)
139         # 功率平衡:  $x + d_s - c_s - s_s + e_s = L - P_s$ 
140         rr += [r0 + t, r0 + t, r0 + t, r0 + t, r0 + t]
141         cc += [t, base + t, base + T + t, base + 2 * T + t, base + 4 * T + t]
142         vv += [np.ones(T), -np.ones(T), np.ones(T), -np.ones(T), np.ones(T)]
143         # 储能动态:  $E_{s,t} - E_{s,t-1} - \cdot c + d/ = 0$ 
144         rr += [r0 + T + t, r0 + T + t[1:], r0 + T + t, r0 + T + t]
145         cc += [base + 3 * T + t, base + 3 * T + t[:-1], base + t, base + T + t]
146         vv += [np.ones(T), -np.ones(T - 1), -ETA * np.ones(T), (1.0 / ETA) * np.ones(T)]
147         # 情景末 SOC 回到段末目标 (各场景同值)
148         rr.append(np.array([r0 + 2 * T]))
149         cc.append(np.array([base + 3 * T + T - 1]))
150         vv.append(np.array([1.0]))
151     A_eq = csr_matrix(
152         (np.concatenate(vv), (np.concatenate(rr), np.concatenate(cc))),
153         shape=(rows, n),
154     )
155     b_eq = np.zeros(rows)
156     for s in range(n_scen):
157         r0 = s * (2 * T + 1)
158         b_eq[r0:r0 + T] = load_kwh - pv_scen[s]
159         b_eq[r0 + T] = e_start
160         b_eq[r0 + 2 * T] = e_terminal
161
162     bounds = [(0.0, None)] * T
163     for _ in range(n_scen):
164         bounds += [(0.0, P_MAX_E)] * T + [(0.0, P_MAX_E)] * T
165         bounds += [(0.0, None)] * T + [(E_MIN, E_MAX)] * T + [(0.0, None)] * T
166     res = pm.optimize.solve_lp(c_obj, A_eq=A_eq, b_eq=b_eq, bounds=bounds)
167     if not res.success:
168         raise RuntimeError(f"对冲 LP 失败: {res.message}")
169     return res.x[0:T], res
170
171
172 def run_hedge(data: dict, e_starts, e_targets, n_scen: int = 20, seed: int = 7):
173     """对 2.1-12.31 逐日求解无前视对冲计划 (期望费用最小)。
174
175     目标日 d 的场景池只取其前 90 天 (同月样本 14 天时优先同月) 的
176     整日残差块采样, 严格无前视。
177     """

```

```

178 price, load, fc0 = data["price"], data["load"], data["fc0"]
179 months, residuals = data["months"], data["R0"]
180 rng = np.random.default_rng(seed)
181 xh = np.zeros((N_DAY, T))
182 t0 = time.time()
183 for d in range(REPORT_START, N_DAY):
184     past = np.arange(max(0, d - 90), d, dtype=int)
185     same = past[months[past] == months[d]]
186     pool_days = same if len(same) >= 14 else past
187     residual_hist = residuals[pool_days]
188     if not len(residual_hist) or int(pool_days.max()) >= d:
189         raise RuntimeError(f"对冲场景池为空: d={d}")
190     xh[d, _] = hedge_day(
191         price, load[d] / 6.0, fc0[d], residual_hist,
192         float(e_starts[d]), float(e_targets[d]),
193         n_scen=n_scen, rng=rng,
194     )
195     if (d - REPORT_START + 1) % 50 == 0:
196         print(f" hedge {d - REPORT_START + 1}/334, 用时 {time.time() - t0:.0f}s")
197 return xh

```

program/src/solve/flows/q3_day.py

```

1  """Q3 单日流程: 0:00 计划 + 6/12/18 调整 (含场景对冲) + 因果执行/结算。
2
3  # =====
4  # 主方案 (正式口径 = simulate_day_rt_hedge, 统一 v1.1)
5  # 预测: 负荷 = ·三源 EWMA h=5 预测; 光伏 = 0:00 组合预测 (·官方 + (1-)·历史)
6  # 再折减 m; 权重由 Q2E 成本表统一标定 (consistency.py)。
7  # 计划: plan_two_day —— 48 小时滚动视野 (末端完全自由), 只执行首日;
8  # 日末 SOC 自由并跨日传递。
9  # 6/12/18 调整: 用最新发布预报重优化未执行时段 (6/12 叠加无前视场景对冲);
10 # 场景 = (Δ负荷, Δ光伏) 同月整日联合残差块, N_SCEN=40, 严格取目标日之前。
11 # 执行: 逐槽因果 (exec_segment_causal, EXEC_POLICY="free" 无段末硬目标)
12 # 结算: 费用 = Σ [ p · x_adj + 0.5 · p · |x_plan - x_adj| ] + 5 · Σ p · e
13 #
14 # simulate_day (事后口径) 与 exec_day (事后 LP) 仅作下界对照, 禁入正式结果。
15 # =====
16 """
17 from __future__ import annotations
18
19 import numpy as np
20
21 from solve import consistency as cs
22 from solve.common import EO, E_MAX, E_MIN, EMERG_MULT, EPS_TH, T
23 from solve.core.causal import exec_segment_causal, run_exec
24 from solve.core.lp import adjust_day, adjust_day_hedge, exec_day, plan_day, plan_horizon
25 from solve.core.residual import causal_residual_pool, forecast_at_publish, latest_forecast
26 from solve.core.slots import fc_slots, hour_to_slots
27 from solve.models.adaptive import (
28     hist_forecast,
29     hist_forecast_asof,
30     hist_load_forecast,
31     hist_load_forecast_asof,
32 )
33
34
35 def plan_two_day(data: dict, D: int, lam: float, e_start: float,
36                 kappa: float = cs.KAPPA, margin: float = cs.MARGIN,
37                 price2=None):
38     """在 D 日 0:00 做 48 小时滚动计划, 只执行首日 (规划窗口末端完全自由)。
39
40     统一口径 (``consistency.py``): 负荷 = ·L, 光伏 = max(P - m, 0);
41     ``price2`` 可传入 288 槽决策价格 (Q4 用), 默认附件 1 日价格重复两日。
42     返回 (x_day, E_day, res)。
43     """
44     price2 = np.tile(data["price"], 2) if price2 is None else np.asarray(price2, float)
45     f0 = hour_to_slots(data["fc0"][D])
46     ph = hist_forecast_asof(data, D, D)

```

```

47     pv0 = cs.margin_pv(lam * f0 + (1.0 - lam) * ph, margin)
48     pv1 = cs.margin_pv(hist_forecast_asof(data, D + 1, D), margin)
49     load0 = cs.kappa_load(hist_load_forecast_asof(data, D, D), kappa)
50     load1 = cs.kappa_load(hist_load_forecast_asof(data, D + 1, D), kappa)
51     xh, Eh, res = plan_horizon(
52         price2, np.concatenate([load0, load1]) / 6.0,
53         np.concatenate([pv0, pv1]) / 6.0, e_start, None, eps=EPS_TH,
54     )
55     return xh[:T], Eh[:T], res
56
57
58 def joint_residual_blocks(data: dict, pool: int, pub: int, adj_lam,
59                           kappa: float = cs.KAPPA,
60                           margin: float = cs.MARGIN):
61     """历史池日的 (Δ负荷, Δ光伏) 联合整日残差块 (kWh/槽, 144 槽)。
62
63     名义 = 负荷历史预测 与 max(组合光伏预测 - m, 0) (与当前决策同一构造,
64     历史日使用其自身发布时刻的附件 3 预报); 残差 = 实际 - 名义。
65     """
66     RL = np.zeros((len(pool), T))
67     RP = np.zeros((len(pool), T))
68     for k, j in enumerate(pool):
69         j = int(j)
70         l_nom = cs.kappa_load(hist_load_forecast(data, j), kappa) / 6.0
71         off_j = fc_slots(data[f"fc{pub}"])[j], pub
72         pj = hist_forecast(data, j)
73         src_j = off_j if adj_lam is None else adj_lam * off_j + (1.0 - adj_lam) * pj
74         p_nom = cs.margin_pv(np.clip(src_j, 0.0, None), margin) / 6.0
75         RL[k] = data["load"][j] / 6.0 - l_nom
76         RP[k] = data["pv_act"][j] / 6.0 - p_nom
77     return RL, RP
78
79
80 def simulate_day(data: dict, D: int, lam: float, adj_hours=(6, 12, 18),
81                 settle: str = "final", use_new_fc: bool = True,
82                 adj_lam: float | None = None) -> dict:
83     """单日 Q3 模拟 (事后执行口径, 仅对照)。
84
85     adj_hours : 启用哪些调整时刻 (6/12/18 的子集); 空元组 = 无调整 (Q2 口径)。
86     settle : "final": 计划与最终调整值结算一次;
87             "sequential": 每次调整分别与上一次结算 (逐次计费)。
88     use_new_fc: 调整时是否使用新预报; False 时沿用 0:00 预报 (用于价值分解)。
89     adj_lam : 调整层的组合权重 (None = 纯官方新预报; 0-1 = 与历史预测组合)。
90     """
91     price = data["price"]
92     load_fc_kwh = hist_load_forecast(data, D) / 6.0
93     pv_act_kwh = data["pv_act"][D] / 6.0
94
95     f0 = hour_to_slots(data["fc0"])[D]
96     ph = hist_forecast(data, D)
97     pv_plan = (lam * f0 + (1.0 - lam) * ph) / 6.0
98
99     x_plan, E_plan, _ = plan_day(price, load_fc_kwh, pv_plan, E0, eps=EPS_TH)
100
101     x_final = x_plan.copy()
102     x_hist = {0: x_plan.copy()}
103     for t0, fc, pub in ((36, data["fc6"])[D], 6), (72, data["fc12"])[D], 12), (108, data["fc18"])[D], 18)):
104         if pub not in adj_hours:
105             continue
106         if use_new_fc:
107             src = fc_slots(fc, pub)
108             if adj_lam is not None:
109                 src = adj_lam * src + (1.0 - adj_lam) * ph
110         else:
111             src = f0
112         pv_new = np.clip(src, 0.0, None) / 6.0
113         x_adj, _E_adj = adjust_day(
114             price, load_fc_kwh, pv_new, x_plan, float(E_plan[t0 - 1]), t0
115         )

```

```

116     x_final[t0:] = x_adj
117     x_hist[pub] = x_final.copy()
118
119     ex = exec_day(price, load_fc_kwh, pv_act_kwh, x_final, E0)
120     e = ex["e"]
121
122     # 统一结算: 调低部分 50% 违约、调高部分 150% 费用 = p * adj + 0.5p * |plan-adj|
123     buy = float(price @ x_final)
124     if settle == "final":
125         dev = float((0.5 * price * np.abs(x_final - x_plan)).sum())
126     elif settle == "sequential":
127         dev = 0.0
128         base_seq = x_plan.copy()
129         for pub in sorted(k for k in x_hist if k > 0):
130             cur = x_hist[pub]
131             seg = slice(pub * 6, T)
132             dev += float((0.5 * price[seg] * np.abs(cur[seg] - base_seq[seg])).sum())
133             base_seq[seg] = cur[seg]
134     else:
135         raise ValueError(f"未知结算口径: {settle}")
136     emerg = float((EMERG_MULT * price * e).sum())
137     plan_cost = float(price @ x_plan)
138     total = buy + dev + emerg
139     return {
140         "total": total,
141         "plan_cost": plan_cost,
142         "adjust_net": total - plan_cost - emerg,
143         "emerg": emerg,
144         "adj_abs_kwh": float(np.abs(x_final - x_plan).sum()),
145         "x_plan": x_plan, "x_final": x_final, "e": e, "E": ex["E"],
146     }
147
148
149 def simulate_day_rt(data: dict, D: int, lam: float, adj_hours=(6, 12, 18),
150                    settle: str = "final", use_new_fc: bool = True,
151                    adj_lam: float | None = None, kappa: float = cs.KAPPA,
152                    margin: float = cs.MARGIN,
153                    e_start: float = E0, exec_policy: str = cs.EXEC_POLICY) -> dict:
154     """因果实时执行: 调整使用已实现 SOC, 每槽只读取当前实际值。
155
156     与 ``simulate_day`` 的区别 (后者为"事后执行": 全天一次执行 LP):
157     - 每段 (0-6/6-12/12-18/18-24) 用逐槽因果执行器 ``exec_segment_causal``;
158     - 调整 LP 以该实际储电量为初值, 利用已实现信息;
159     - 统一口径: 负荷 =  $\cdot L$ , 光伏 =  $\max(P - m, 0)$ ; 48 小时滚动计划。
160     """
161     price = data["price"]
162     load_kwh = data["load"][D] / 6.0
163     load_fc_kwh = cs.kappa_load(hist_load_forecast(data, D), kappa) / 6.0
164     pv_act_kwh = data["pv_act"][D] / 6.0
165
166     f0 = hour_to_slots(data["fc0"][D])
167     ph = hist_forecast(data, D)
168     pv_plan = cs.margin_pv(np.clip(lam * f0 + (1.0 - lam) * ph, 0.0, None),
169                           margin) / 6.0
170
171     x_plan, E_plan, _ = plan_two_day(data, D, lam, e_start,
172                                     kappa=kappa, margin=margin)
173     e_day_end = float(E_plan[-1])
174
175     x_seq = x_plan.copy()
176     E_target = E_plan.copy()
177     e_total = np.zeros(T)
178     c_all = np.zeros(T)
179     d_all = np.zeros(T)
180     E_exec = np.zeros(T)
181     x_hist = {0: x_plan.copy()}
182     E_now = e_start
183     t = 0
184     for pub in (6, 12, 18):

```

```

185     t1 = pub * 6
186     seg = run_exec(exec_policy, load_kwh[t:t1], pv_act_kwh[t:t1],
187                   x_seq[t:t1], E_now,
188                   float(np.clip(E_target[t1 - 1], E_MIN, E_MAX)))
189     e_total[t:t1] = seg["e"]
190     c_all[t:t1] = seg["c"]
191     d_all[t:t1] = seg["d"]
192     E_exec[t:t1] = seg["E"]
193     E_now = float(seg["E"][-1])
194     if pub in adj_hours:
195         fc = data[f"fc{pub}"][D]
196         if use_new_fc:
197             src = fc_slots(fc, pub)
198             if adj_lam is not None:
199                 src = adj_lam * src + (1.0 - adj_lam) * ph
200         else:
201             src = f0
202         pv_new = cs.margin_pv(np.clip(src, 0.0, None), margin) / 6.0
203         x_adj, E_adj = adjust_day(
204             price, load_fc_kwh, pv_new, x_plan, E_now, t1,
205             e_terminal=e_day_end,
206         )
207         x_seq[t1:] = x_adj
208         E_target[t1:] = E_adj
209         x_hist[pub] = x_seq.copy()
210     t = t1
211     seg = run_exec(exec_policy, load_kwh[108:144], pv_act_kwh[108:144],
212                   x_seq[108:144], E_now,
213                   float(np.clip(E_target[143], E_MIN, E_MAX)), last=True)
214     e_total[108:] = seg["e"]
215     c_all[108:] = seg["c"]
216     d_all[108:] = seg["d"]
217     E_exec[108:] = seg["E"]
218
219     x_final = x_seq.copy()
220     buy = float(price @ x_final)
221     if settle == "final":
222         dev = float((0.5 * price * np.abs(x_final - x_plan)).sum())
223     elif settle == "sequential":
224         dev = 0.0
225         base_seq = x_plan.copy()
226         for pub in sorted(k for k in x_hist if k > 0):
227             cur = x_hist[pub]
228             seg = slice(pub * 6, T)
229             dev += float((0.5 * price[seg] * np.abs(cur[seg] - base_seq[seg])).sum())
230             base_seq[seg] = cur[seg]
231     else:
232         raise ValueError(f"未知结算口径: {settle}")
233     emerg = float((EMERG_MULT * price * e_total).sum())
234     plan_cost = float(price @ x_plan)
235     total = buy + dev + emerg
236     return {
237         "total": total,
238         "plan_cost": plan_cost,
239         "adjust_net": total - plan_cost - emerg,
240         "emerg": emerg,
241         "adj_abs_kwh": float(np.abs(x_final - x_plan).sum()),
242         "x_plan": x_plan, "x_final": x_final, "e": e_total, "E": E_target,
243         "c": c_all, "d": d_all, "E_exec": E_exec,
244         "E_start": float(e_start), "E_end": float(E_exec[-1]),
245     }
246
247
248 def forecast_residual(data: dict, D: int, publish: int,
249                     adj_lam: float | None) -> np.ndarray:
250     """指定发布时刻的"预报 - 实际"残差(kWh/槽; 正 = 高估)。
251
252     场景残差必须与当前决策的信息集一致: 6:00 调整只能使用历史 6:00 发布预报
253     的误差, 不能把历史 12:00/18:00 的更新拼接进剩余时段。

```

```

254     """
255     official = forecast_at_publish(data, D, publish)
256     if adj_lam is None:
257         forecast = official
258     else:
259         forecast = adj_lam * official + (1.0 - adj_lam) * hist_forecast(data, D)
260     return (forecast - data["pv_act"][D]) / 6.0
261
262
263 def simulate_day_rt_hedge(data: dict, D: int, lam: float, adj_hours=(6, 12, 18),
264                          n_scen: int = cs.N_SCEN, seed: int = 0,
265                          adj_lam: float | None = None, kappa: float = cs.KAPPA,
266                          margin: float = cs.MARGIN,
267                          settle: str = "final",
268                          pool_min_month: int = cs.SCEN_MIN_SAME_MONTH,
269                          pool_lookback: int = cs.SCEN_LOOKBACK,
270                          e_start: float = E0,
271                          exec_policy: str = cs.EXEC_POLICY) -> dict:
272     """因果执行 + 无前视场景对冲（6:00/12:00 用对冲 LP, 18:00 段光伏 0 不对冲）。
273
274     统一口径（`consistency.py`）：名义输入 =  $\cdot L$  与  $\max(P - m, 0)$ ；
275     场景 = 名义 + ( $\Delta$ 负荷,  $\Delta$ 光伏) 同月整日联合残差块（池严格取目标日之前，
276     随机流由 `(seed, D, pub)` 唯一确定）；执行层逐槽因果（free 策略）。
277     """
278     price = data["price"]
279     load_kwh = data["load"][D] / 6.0
280     load_nom = cs.kappa_load(hist_load_forecast(data, D), kappa) / 6.0
281     pv_act_kwh = data["pv_act"][D] / 6.0
282
283     f0 = hour_to_slots(data["fc0"][D])
284     ph = hist_forecast(data, D)
285     pv_plan = cs.margin_pv(np.clip(lam * f0 + (1.0 - lam) * ph, 0.0, None),
286                           margin) / 6.0
287     x_plan, E_plan, _ = plan_two_day(data, D, lam, e_start,
288                                     kappa=kappa, margin=margin)
289     e_day_end = float(E_plan[-1])
290
291     x_seq = x_plan.copy()
292     E_target = E_plan.copy()
293     e_total = np.zeros(T)
294     c_all = np.zeros(T)
295     d_all = np.zeros(T)
296     s_all = np.zeros(T)
297     E_exec = np.zeros(T)
298     scenario_days: list[int] = []
299     x_hist = {0: x_plan.copy()}
300     E_now = e_start
301     t = 0
302     for pub in (6, 12, 18):
303         t1 = pub * 6
304         seg = run_exec(exec_policy, load_kwh[t:t1], pv_act_kwh[t:t1],
305                      x_seq[t:t1], E_now,
306                      float(np.clip(E_target[t1 - 1], E_MIN, E_MAX)))
307         e_total[t:t1] = seg["e"]
308         c_all[t:t1] = seg["c"]
309         d_all[t:t1] = seg["d"]
310         s_all[t:t1] = seg["s"]
311         E_exec[t:t1] = seg["E"]
312         E_now = float(seg["E"][-1])
313         if pub in adj_hours:
314             fc = data[f"fc{pub}"][D]
315             off = fc_slots(fc, pub)
316             src = off if adj_lam is None else adj_lam * off + (1.0 - adj_lam) * ph
317             pv_nom_full = cs.margin_pv(np.clip(src, 0.0, None), margin) / 6.0
318             if pub in (6, 12):
319                 # 无前视残差池 + 由 (seed, D, pub) 决定的随机流
320                 pool = causal_residual_pool(data, D, min_same_month=pool_min_month,
321                                             lookback=pool_lookback)
322                 rng = np.random.default_rng(np.random.SeedSequence([seed, D, pub]))

```

```

323     sel = rng.choice(len(pool), size=max(0, n_scen - 1), replace=True)
324     scenario_days.extend(int(pool[k]) for k in sel)
325     RL, RP = joint_residual_blocks(data, pool, pub, adj_lam, kappa, margin)
326     load_scens = [load_nom[t1:]] + [
327         np.clip(load_nom[t1:] + RL[k][t1:], 0.0, None) for k in sel]
328     pv_scens = [pv_nom_full[t1:]] + [
329         np.clip(pv_nom_full[t1:] + RP[k][t1:], 0.0, None) for k in sel]
330     x_adj, E_adj = adjust_day_hedge(
331         price, load_nom, pv_scens, x_plan, E_now, t1,
332         e_terminal=e_day_end, load_scens=load_scens,
333     )
334     else:
335         x_adj, E_adj = adjust_day(
336             price, load_nom, pv_nom_full, x_plan, E_now, t1,
337             e_terminal=e_day_end,
338         )
339         x_seq[t1:] = x_adj
340         E_target[t1:] = E_adj
341         x_hist[pub] = x_seq.copy()
342     t = t1
343     # v1.2 末段修复: 末段也必须走 run_exec (free 时完全无段末硬目标;
344     # v1.1 曾直接传 E_target[143], 使 free 实际退化为 dayend, 见 reports/执行器段末目标实验.md)
345     seg = run_exec(exec_policy, load_kwh[108:144], pv_act_kwh[108:144],
346                   x_seq[108:144], E_now,
347                   float(np.clip(E_target[143], E_MIN, E_MAX)), last=True)
348     e_total[108:] = seg["e"]
349     c_all[108:] = seg["c"]
350     d_all[108:] = seg["d"]
351     s_all[108:] = seg["s"]
352     E_exec[108:] = seg["E"]
353
354     x_final = x_seq.copy()
355     buy = float(price @ x_final)
356     if settle == "final":
357         dev = float((0.5 * price * np.abs(x_final - x_plan)).sum())
358     else:
359         dev = 0.0
360         base_seq = x_plan.copy()
361         for pub in sorted(k for k in x_hist if k > 0):
362             cur = x_hist[pub]
363             seg = slice(pub * 6, T)
364             dev += float((0.5 * price[seg] * np.abs(cur[seg] - base_seq[seg])).sum())
365             base_seq[seg] = cur[seg]
366     emerg = float((EMERG_MULT * price * e_total).sum())
367     plan_cost = float(price @ x_plan)
368     total = buy + dev + emerg
369     return {
370         "total": total, "plan_cost": plan_cost,
371         "buy_cost": buy, "deviation_cost": dev,
372         "adjust_net": total - plan_cost - emerg, "emerg": emerg,
373         "adj_abs_kwh": float(np.abs(x_final - x_plan).sum()),
374         "x_plan": x_plan, "x_final": x_final, "e": e_total, "E": E_target,
375         "c": c_all, "d": d_all, "s": s_all, "E_exec": E_exec,
376         "scenario_max_day": max(scenario_days, default=-1),
377         "scenario_count": len(scenario_days),
378         "E_start": float(e_start), "E_end": float(E_exec[-1]),
379     }
380
381
382 def perfect_day(data: dict, D: int) -> float:
383     """完美下界: 0:00 用实际光伏做计划且不调整 (Q3 规则下不可实现, 仅参考)。"""
384     price = data["price"]
385     load_kwh = data["load"][D] / 6.0
386     pv_act_kwh = data["pv_act"][D] / 6.0
387     x, _E, _ = plan_day(price, load_kwh, pv_act_kwh, E0, eps=EPS_TH)
388     ex = exec_day(price, load_kwh, pv_act_kwh, x, E0)
389     return float(price @ x + EMERG_MULT * price @ ex["e"])

```

program/src/solve/flows/q4_year.py

```

1  """Q4 年度流程：统一口径下的 2 日滚动（Q2 层）与调整层（Q3 层）。
2
3  # =====
4  # Q2 层（run_year）
5  # 决策价：G（题面主口径）= 附件 4 已知电价；H（扩展）= 三源预测（滚动 v + ）
6  # 预测：负荷 = EWMA h=5 历史预测；光伏 = 附件 3 的 0:00 预报（次日用 asof 历史）
7  # 储能：daily = 日循环（plan_day）；2day = 48 小时滚动（plan_horizon，末端自由）
8  # 执行：逐槽因果（free）；结算用附件 4 真实电价： $\Sigma p_4 \cdot x + 5 \cdot p_4 \cdot e$ 
9  #
10 # Q3 层（simulate_day_q3，G 主口径）
11 # 0:00 计划 + 6/12/18 调整（正式无对冲）；
12 # 结算 =  $\Sigma [p_4 \cdot x_{adj} + 0.5 \cdot p_4 \cdot |x_{plan} - x_{adj}|] + 5 \cdot \Sigma p_4 \cdot e$ 
13 # =====
14 """
15 from __future__ import annotations
16
17 import numpy as np
18
19 from solve import consistency as cs
20 from solve.common import (
21     EO,
22     E_MAX,
23     E_MIN,
24     EMERG_MULT,
25     EPS_TH,
26     N_DAY,
27     REPORT_START,
28     T,
29 )
30 from solve.core.causal import exec_segment_causal
31 from solve.core.lp import adjust_day, adjust_day_hedge, plan_day, plan_horizon
32 from solve.core.residual import causal_residual_pool
33 from solve.core.slots import fc_slots, hour_to_slots
34 from solve.flows.q3_day import forecast_residual, joint_residual_blocks, plan_two_day
35 from solve.models.adaptive import hist_forecast, hist_forecast_asof, hist_load_forecast_asof
36 from solve.models.price import price_forecast_d, price_forecast_next
37
38
39 # =====
40 # Q4 Q2 层：全年滚动（计划/执行/结算）
41 # =====
42 def run_year(data, p4, p_typ, vseq, price_mode="H", storage="2day",
43             start=REPORT_START, end=N_DAY) -> list[dict]:
44     """全年滚动模拟（统一口径：EWMA 预测 + /m; 2-1 起 EO 起步）。
45
46     price_mode = "G"（题面已知电价）/ "H"（历史价格预测）；
47     storage = "daily"（日循环）或 "2day"（48 小时滚动、跨日连续、末端自由）。
48     返回逐日记录（含 x/c/d/s/e/E 与费用，REPORT_START 起计费）。
49     """
50     load, pv_act = data["load"], data["pv_act"]
51     E = EO
52     # 按日序号对齐（D < start 的月份仅作参考预热，占位记录不计费）
53     recs: list[dict] = [{"D": D} for D in range(N_DAY)]
54     for D in range(start, end):
55         # 决策价格：G = 当天真实价；H = 三源预测（次日另用 D+1 预测）
56         if price_mode == "G":
57             p_d = p4[D]
58             p_d1 = p4[min(D + 1, N_DAY - 1)]
59         else:
60             v = vseq[D]
61             p_d = price_forecast_d(D, v, p4, p_typ)
62             p_d1 = price_forecast_next(D, v, p4, p_typ) if D + 1 < N_DAY else p_d
63             load_act = load[D] / 6.0
64             load_fc_d = cs.kappa_load(hist_load_forecast_asof(data, D, D)) / 6.0
65             pv_fc_d = cs.margin_pv(hist_forecast_asof(data, D, D)) / 6.0
66
67         if storage == "daily":
68             # 日循环：E(0)=E(24)=当前 SOC
69             x_day, _E_plan, _ = plan_day(p_d, load_fc_d, pv_fc_d, E, eps=EPS_TH)

```

```

70     else:
71         # 48 小时滚动: 次日负荷/光伏用 asof 无前视预测, 窗口末端完全自由
72         load_fc_d1 = cs.kappa_load(hist_load_forecast_asof(data, D + 1, D)) / 6.0
73         pv_fc_d1 = cs.margin_pv(hist_forecast_asof(data, D + 1, D)) / 6.0
74         xh, _Eh, _ = plan_horizon(
75             np.concatenate([p_d, p_d1]),
76             np.concatenate([load_fc_d, load_fc_d1]),
77             np.concatenate([pv_fc_d, pv_fc_d1]),
78             E, None, eps=EPS_TH,
79         )
80         x_day = xh[:T]
81
82         # 统一执行策略 (free): 无段末硬目标
83         ex = exec_segment_causal(load_act, pv_act[D] / 6.0, x_day, E, None)
84         rec = {"D": D, "x": x_day, "c": ex["c"], "d": ex["d"], "s": ex["s"],
85              "e": ex["e"], "E": ex["E"], "E_start": float(E)}
86         if D >= REPORT_START:
87             # 结算用附件 4 真实电价 (与决策价无关)
88             rec["plan_cost"] = float(p4[D] @ x_day)
89             rec["emerg"] = float(EMERG_MULT * (p4[D] @ ex["e"]))
90             rec["total"] = rec["plan_cost"] + rec["emerg"]
91         recs[D] = rec
92         E = float(ex["E"][-1])
93     return recs
94
95
96 def total_of(recs) -> dict:
97     """Q2 层费用汇总 (仅计费日): 计划/紧急/合计与紧急电量。"""
98     days = [r for r in recs if "total" in r]
99     return {
100         "plan": sum(r["plan_cost"] for r in days),
101         "emerg": sum(r["emerg"] for r in days),
102         "total": sum(r["total"] for r in days),
103         "emerg_kwh": sum(r["e"].sum() for r in days),
104         "days": len(days),
105     }
106
107
108 # =====
109 # Q4 Q3 层: 0:00 计划 + 6/12/18 调整; 正式无对冲, 结算用附件 4 真实价
110 # =====
111 def simulate_day_q3(data, p4, p_typ, vseq, D, price_mode="H", storage="daily",
112                    adj_hours=(6, 12, 18), hedge=cs.Q3_USE_HEDGE,
113                    n_scen=cs.N_SCEN, seed=7,
114                    lam=cs.Q3_LAM, adj_lam=cs.Q3_ADJ_LAM,
115                    kappa=cs.KAPPA, margin=cs.MARGIN,
116                    e_start=E0, eps=EPS_TH) -> dict:
117     """Q4 Q3 层单日模拟 (正式: EWMA 预测 + /m + 三点调整、无对冲)。
118
119     H 口径: 决策价格 = 三源预测 (滚动 v); G: 决策价格 = 当天真实价。
120     storage='daily': 日循环 (计划/调整末端回 E0); '2day': 48 小时滚动、跨日连续。
121     结算一律用附件 4 真实价: 买入 + 偏差费 (0.5p|x-x|) + 5 倍紧急。
122     """
123     p_dec = p4[D] if price_mode == "G" else price_forecast_d(D, vseq[D], p4, p_typ)
124     load_act_kwh = data["load"][D] / 6.0
125     load_nom = cs.kappa_load(hist_load_forecast_asof(data, D, D), kappa) / 6.0
126     pv_act_kwh = data["pv_act"][D] / 6.0
127     f0 = hour_to_slots(data["fc0"][D])
128     ph = hist_forecast(data, D)
129     pv_plan = cs.margin_pv(np.clip(lam * f0 + (1.0 - lam) * ph, 0.0, None),
130                             margin) / 6.0
131
132     if storage == "daily":
133         x_plan, E_plan, _ = plan_day(p_dec, load_nom, pv_plan, e_start, eps=eps)
134         e_day_end = E0
135     else:
136         # 48 小时滚动: 次日价格预测 + asof 历史预测, 窗口末端自由
137         D1 = min(D + 1, N_DAY - 1)
138         p_dec1 = (p4[D1] if price_mode == "G"

```

```

139         else price_forecast_next(D, vseq[D], p4, p_typ))
140     x_plan, E_plan, _ = plan_two_day(
141         data, D, lam, e_start, kappa=kappa, margin=margin,
142         price2=np.concatenate([p_dec, p_dec1]))
143     e_day_end = float(E_plan[-1])
144
145     x_seq = x_plan.copy()
146     E_target = E_plan.copy()
147     e_total = np.zeros(T)
148     c_all = np.zeros(T)
149     d_all = np.zeros(T)
150     s_all = np.zeros(T)
151     E_exec = np.zeros(T)
152     scenario_days: list[int] = []
153     E_now = e_start
154     t = 0
155     for pub in (6, 12, 18):
156         t1 = pub * 6
157         # 统一执行策略 (free): 无段末硬目标
158         seg = exec_segment_causal(
159             load_act_kwh[t:t1], pv_act_kwh[t:t1], x_seq[t:t1], E_now, None,
160         )
161         e_total[t:t1] = seg["e"]
162         c_all[t:t1] = seg["c"]
163         d_all[t:t1] = seg["d"]
164         s_all[t:t1] = seg["s"]
165         E_exec[t:t1] = seg["E"]
166         E_now = float(seg["E"][-1])
167         if pub in adj_hours:
168             off = fc_slots(data[f"fc{pub}"])[D], pub)
169             src = off if adj_lam is None else adj_lam * off + (1.0 - adj_lam) * ph
170             pv_new = cs.margin_pv(np.clip(src, 0.0, None), margin) / 6.0
171             if pub in (6, 12) and hedge and D > 14:
172                 # 无前视联合残差场景对冲 (残差块仅取自目标日之前)
173                 pool = causal_residual_pool(data, D)
174                 rng = np.random.default_rng(np.random.SeedSequence([seed, D, pub]))
175                 sel = rng.choice(len(pool), size=max(0, n_scen - 1), replace=True)
176                 scenario_days.extend(int(pool[k]) for k in sel)
177                 RL, RP = joint_residual_blocks(data, pool, pub, adj_lam, kappa, margin)
178                 load_scens = [load_nom[t1:]] + [
179                     np.clip(load_nom[t1:] + RL[k][t1:], 0.0, None) for k in sel]
180                 pv_scens = [pv_new[t1:]] + [
181                     np.clip(pv_new[t1:] + RP[k][t1:], 0.0, None) for k in sel]
182                 x_adj, E_adj = adjust_day_hedge(
183                     p_dec, load_nom, pv_scens, x_plan, E_now, t1,
184                     e_terminal=e_day_end, load_scens=load_scens)
185             else:
186                 x_adj, E_adj = adjust_day(
187                     p_dec, load_nom, pv_new, x_plan, E_now, t1,
188                     e_terminal=e_day_end)
189             x_seq[t1:] = x_adj
190             E_target[t1:] = E_adj
191             t = t1
192         seg = exec_segment_causal(
193             load_act_kwh[108:144], pv_act_kwh[108:144], x_seq[108:144], E_now, None,
194         )
195         e_total[108:] = seg["e"]
196         c_all[108:] = seg["c"]
197         d_all[108:] = seg["d"]
198         s_all[108:] = seg["s"]
199         E_exec[108:] = seg["E"]
200
201     # 结算 (附件 4 真实价): 买入 + 0.5p|x_adj-x_plan| + 5p * 紧急
202     x_final = x_seq.copy()
203     buy = float(p4[D] @ x_final)
204     dev = float((0.5 * p4[D] * np.abs(x_final - x_plan)).sum())
205     emerg = float(EMERG_MULT * (p4[D] @ e_total))
206     return {
207         "D": D, "x_plan": x_plan, "x_final": x_final,

```

```

208         "c": c_all, "d": d_all, "s": s_all, "E": E_exec, "E_start": float(e_start),
209         "e": e_total, "plan_cost": float(p4[D] @ x_plan),
210         "buy": buy, "dev": dev, "emerg": emerg, "total": buy + dev + emerg,
211         "E_end": float(E_exec[-1]),
212         "scenario_max_day": max(scenario_days, default=-1),
213     }
214
215
216 def run_year_q3(data, p4, p_typ, vseq, price_mode="H", storage="daily",
217               adj_hours=(6, 12, 18), hedge=cs.Q3_USE_HEDGE,
218               n_scen=cs.N_SCEN, seed=7,
219               lam=cs.Q3_LAM, adj_lam=cs.Q3_ADJ_LAM) -> list[dict]:
220     """Q4 Q3 层全年滚动（自 REPORT_START 起）：daily 回 E0; 2day 跨日传递 SOC."""
221     E = E0
222     # 按日序号对齐（占位记录仅用于索引对齐，不计费）
223     recs: list[dict] = [{"D": D} for D in range(N_DAY)]
224     for D in range(REPORT_START, N_DAY):
225         r = simulate_day_q3(
226             data, p4, p_typ, vseq, D, price_mode=price_mode, storage=storage,
227             adj_hours=adj_hours, hedge=hedge, n_scen=n_scen, seed=seed,
228             lam=lam, adj_lam=adj_lam,
229             e_start=(E0 if storage == "daily" else E),
230         )
231         recs[D] = r
232         E = r["E_end"] if storage == "2day" else E0
233     return recs
234
235
236 def total_of_q3(recs) -> dict:
237     """Q3 层费用汇总（仅计费日）：计划/买入/偏差/紧急/合计、紧急与调整电量."""
238     days = [r for r in recs if r["D"] >= REPORT_START]
239     return {
240         "plan": sum(r["plan_cost"] for r in days),
241         "buy": sum(r["buy"] for r in days),
242         "dev": sum(r["dev"] for r in days),
243         "emerg": sum(r["emerg"] for r in days),
244         "total": sum(r["total"] for r in days),
245         "emerg_kwh": sum(float(r["e"]).sum() for r in days),
246         "adj_kwh": sum(float(np.abs(r["x_final"] - r["x_plan"])).sum() for r in days),
247         "days": len(days),
248     }

```

data

program/src/solve/data/__init__.py

```

1 """数据层：附件读取与派生数组（全年实际/预报/典型日/误差块/口径 E 权重）。"""

```

program/src/solve/data/attachments.py

```

1 """附件数据装载：附件 1（典型日）、附件 2（实际曲线）、附件 3（预报）、附件 4（电价）。
2
3 # =====
4 # 关键派生量
5 # 1) 时段对齐：一天 144 个 10 分钟槽；附件中的"时间标签"为时段右端点，
6 #    第 k 个整点 (k 时) 对应槽序号 6k-1 (即 6k-1 号槽末)。
7 # 2) 典型日矩阵 T[k, m]: 第 m 月、第 k 个整点在实际曲线上的均值，
8 #    T[k, m] = mean_{d 月 m} A[d, 6k-1], k = 1..24
9 # 3) 误差块 Z[d, k]: 0:00 预报相对期望出力的标准化误差
10 #    z_dk = [ (F_dk - A_dk)/T[k, m_d] - (k) ] / (k) (T > 100 kW 的槽)
11 # 4) 口径 E 历史权重 TH: 由 code/outputs/cache/q2e/table_*.npz 提供，
12 #    hist_forecast 使用 TH[D-1] 生成第 D 天光伏预测（无前瞻）。
13 # =====
14 """
15 from __future__ import annotations
16

```

```

17 import numpy as np
18 import pandas as pd
19
20 import program as pm
21 from solve import consistency as cs
22 from solve.common import DATA_C, N_DAY, ROOT
23 from solve.models.errors import mu, sigma
24 from solve.models.weights import simplex_grid
25
26
27 def load_all() -> dict:
28     """读取附件 1/2/3, 返回数组与辅助量。"""
29     # ---- 附件 2: 实际负荷与光伏 (365 天 × 144 槽) ----
30     sheets = pm.read_sheets(DATA_C / "附件2.xlsx")
31     ldf = sheets["小区负载"]
32     pdf = sheets["光伏发电实际功率"]
33     dates = pd.to_datetime(ldf.iloc[:, 0]).dt.strftime("%Y-%m-%d").tolist()
34     load = ldf.iloc[:, 1:145].to_numpy(float)
35     pv_act = pdf.iloc[:, 1:145].to_numpy(float)
36
37     # ---- 附件 1: 典型日电价 (逐槽) + 典型日负荷/光伏 (统一口径用) ----
38     a1 = pm.read_table(DATA_C / "附件1.xlsx")
39     price = a1["电价"].to_numpy(float)
40     load_typ = a1["小区负载"].to_numpy(float)
41     pv_typ = a1["光伏发电预测功率"].to_numpy(float)
42
43     # ---- 附件 3: 0:00 发布的 24 点光伏预报, 按日期块归并到天 ----
44     fc3 = pm.read_table(DATA_C / "附件3.xlsx")
45     fc0 = np.zeros((N_DAY, 24))
46     day = -1
47     for _, r in fc3.iterrows():
48         cell = r["日期"]
49         if isinstance(cell, str) and cell.strip():
50             day += 1
51             if int(str(r["预报时刻"]).split(":")[0]) == 0:
52                 fc0[day] = r.iloc[2:26].astype(float).to_numpy()
53
54     # ---- 月份标签与 (整点, 月) 典型出力矩阵 T[k, m] ----
55     months = np.array([int(x[5:7]) for x in dates])
56     typ_hm = np.zeros((25, 13))
57     for k in range(1, 25):
58         col = pv_act[:, 6 * k - 1]
59         for m in range(1, 13):
60             typ_hm[k, m] = col[months == m].mean()
61
62     # ---- 标准化误差日块 Z[d, k] (0:00 预报, k=1..24) ----
63     Z = np.zeros((N_DAY, 24))
64     R0 = np.zeros((N_DAY, 24)) # 原始残差: 预报 - 实际 (用于严格因果块重采样)
65     ks = np.arange(1, 25)
66     for i in range(N_DAY):
67         m = months[i]
68         typ = typ_hm[1:25, m]
69         err = fc0[i] - pv_act[i, 6 * ks - 1]
70         R0[i] = err
71         mask = typ > 100
72         zz = np.zeros(24)
73         zz[mask] = (err[mask] / typ[mask] - mu(ks[mask])) / sigma(ks[mask])
74         Z[i] = zz
75
76     return {
77         "dates": dates, "load": load, "pv_act": pv_act, "price": price,
78         "fc0": fc0, "months": months, "typ_hm": typ_hm, "Z": Z, "R0": R0,
79         "load_typ": load_typ, "pv_typ": pv_typ,
80     }
81
82
83 def load_extended() -> dict:
84     """q2 基础数据 + 6/12/18 预报 + 典型日光伏 + Q2E 历史权重。
85

```

```

86 Q2E 权重表选最新且 tag == "q2e" 的缓存（变体表在其它 tag 目录，避免误用）。
87 """
88 data = load_all()
89
90 # ---- 附件 3: 6/12/18 发布的预报（与 0:00 相同解析口径）----
91 fc3 = pm.read_table(DATA_C / "附件3.xlsx")
92 fc = {h: np.zeros((N_DAY, 24)) for h in (6, 12, 18)}
93 day = -1
94 for _, r in fc3.iterrows():
95     cell = r["日期"]
96     if isinstance(cell, str) and cell.strip():
97         day += 1
98         h = int(str(r["预报时刻"]).split(":")[0])
99         if h in fc:
100             fc[h][day] = r.iloc[2:26].astype(float).to_numpy()
101 data["fc6"], data["fc12"], data["fc18"] = fc[6], fc[12], fc[18]
102
103 # ---- 附件 1: 典型日光伏/负荷（列名兼容两种模板）----
104 a1 = pm.read_table(DATA_C / "附件1.xlsx")
105 pv_col = "光伏发电预测功率" if "光伏发电预测功率" in a1.columns else a1.columns[3]
106 load_col = "小区负载" if "小区负载" in a1.columns else a1.columns[2]
107 data["pv_tpy"] = a1[pv_col].to_numpy(float)
108 data["load_tpy"] = a1[load_col].to_numpy(float)
109
110 # ---- 口径 E 历史权重 TH（取最新 q2e 表；无 tag 兼容旧表）----
111 cache_dir = ROOT / "code" / "outputs" / "cache" / "q2e"
112 files = sorted(cache_dir.glob("table_*.npz"),
113                 key=lambda p: p.stat().st_mtime, reverse=True)
114 npz_path = None
115 for p in files:
116     try:
117         with np.load(p) as z:
118             tag = str(z["tag"]) if "tag" in z.files else "q2e"
119     except Exception:
120         continue
121     if tag == "q2e":
122         npz_path = p
123         break
124 if npz_path is None:
125     npz_path = files[0]
126 _z = np.load(npz_path)
127 data["TH"] = _z["TH"]
128 # 统一口径（consistency.py）：EWMA h=5 逐日权重（负荷/光伏），全项目共用；
129 # 覆盖 [15, 365]——日序 <31 的权重仅用于场景池残差的名义预测（无前视）。
130 data["EWMA_WU"] = cs.ewma_weights_from_table(
131     _z["C"], simplex_grid(0.2), first_target=cs.WARM_FROM + 1)
132 return data
133
134
135 def load_q4_data() -> tuple[dict, np.ndarray, np.ndarray]:
136     """附件 1/2/3 + 附件 4 电价 + 统一 EWMA h=5 权重（consistency.py）。
137
138     返回 (data, p4, p_tpy): p4 为附件 4 逐槽实时电价, p_tpy 为典型日电价。
139     """
140     data = load_extended()
141     df4 = pm.read_table(DATA_C / "附件4.xlsx")
142     p4 = df4.iloc[:, 1:145].to_numpy(float)
143     return data, p4, data["price"]

```

io

program/src/solve/io/__init__.py

```
1 """I/O 层：结果报告、官方结果表与约束校验。"""
```

program/src/solve/io/checks.py

```

1  """约束回代校验：功率平衡、储能动态、安全区间与变量非负。
2
3  # =====
4  # 回代公式（逐日、逐槽最大残差）
5  # 功率平衡：  $P_t + x_t + d_t + e_t - L_t - c_t - s_t = 0$ 
6  # 储能动态：  $E_t - E_{t-1} - \cdot c_t + d_t / \text{ETA} = 0$ 
7  # 安全区间：  $E_{\min} \leq E_t \leq E_{\max}$  (1200 ≤ E ≤ 10800 kWh)
8  # 充放互斥：同槽 ( $c_t > 0$  &  $d_t > 0$ ) 次数应为 0
9  # 跨日衔接：  $E_{\text{end}}(d) = E_{\text{start}}(d+1)$  (Q2 日循环口径)
10 # 场景无前视：  $\text{scenario\_max\_day} < D$  (Q3/Q4 对冲场景池)
11 # 残差应为浮点误差量级 ( $\sim 1e-12$ )；负值/越界计数必须为 0。
12 # =====
13 """
14 from __future__ import annotations
15
16 import numpy as np
17
18 from solve.common import E_MAX, E_MIN, ETA, N_DAY, REPORT_START
19
20
21 def _scan_records(recs, load, pv_act, *, x_key="x", start=0, end=None,
22                  cross_day=False) -> dict:
23     """逐日扫描公共指标（供 Q2/Q4 两种输出格式复用）。
24
25     recs : 序列；元素含 D（可选，缺省用下标）、x_key、c/d/s/e/E、E_start；
26            跨日检查需要 E_end。
27     start: 起算日：q2 口径 D 传 0（1 月预热也回代），Q4 传 REPORT_START。
28     返回 dict: bal/dyn/emin/emax/overlap/cross/neg_e/neg_s/future_viol。
29     """
30     end = len(recs) if end is None else end
31     bal = dyn = 0.0
32     emin, emax = np.inf, -np.inf
33     overlap = 0
34     neg_e = neg_s = 0
35     future_viol = 0
36     cross = 0.0
37     prev = None
38     for i in range(end):
39         r = recs[i]
40         if r is None:
41             prev = None
42             continue
43         D = int(r["D"]) if isinstance(r, dict) and "D" in r else i
44         if isinstance(r, dict) and r.get("scenario_max_day", -1) >= D and D >= start:
45             future_viol += 1
46         if D < start:
47             prev = None
48             continue
49         x = r[x_key]
50         # 功率平衡残差
51         res = pv_act[D] / 6.0 + x + r["d"] + r["e"] - load[D] / 6.0 - r["c"] - r["s"]
52         bal = max(bal, float(np.abs(res).max()))
53         # 储能动态残差
54         E = np.concatenate([[r["E_start"]], r["E"]])
55         dyn = max(dyn, float(np.abs(np.diff(E) - (ETA * r["c"] - r["d"] / ETA)).max()))
56         emin = min(emin, float(r["E"].min()))
57         emax = max(emax, float(r["E"].max()))
58         # 同槽充放计数与负值计数
59         overlap += int(np.sum((r["c"] > 1e-7) & (r["d"] > 1e-7)))
60         neg_e += int(np.sum(r["e"] < -1e-6))
61         neg_s += int(np.sum(r["s"] < -1e-6))
62         # 跨日衔接（仅当记录含 E_end）
63         if cross_day and prev is not None and "E_end" in prev:
64             cross = max(cross, abs(float(prev["E_end"]) - float(r["E_start"])))
65         prev = r
66     return {
67         "bal": bal, "dyn": dyn, "emin": emin, "emax": emax,
68         "overlap": overlap, "cross": float(cross),
69         "neg_e": neg_e, "neg_s": neg_s, "future_viol": future_viol,

```

```

70     }
71
72
73 def verify(data: dict, x_plans, recs) -> dict:
74     """Q2 全年逐日约束回代（口径 D/E 共用）。
75
76     返回：平衡残差、储能动态残差、储电量最值与跨日衔接误差、负值计数。
77     """
78     s = _scan_records(recs, data["load"], data["pv_act"], x_key="x",
79                       start=0, end=N_DAY, cross_day=True)
80     return {
81         "功率平衡最大残差/kWh": s["bal"],
82         "储能动态最大残差/kWh": s["dyn"],
83         "储电量最小值/kWh": s["emin"],
84         "储电量最大值/kWh": s["emax"],
85         "跨日储电量衔接最大误差/kWh": s["cross"],
86         "紧急购电量负值数": s["neg_e"],
87         "弃电负值数": s["neg_s"],
88     }
89
90
91 def verify_records(recs, data: dict, *, x_key="x", scenario=False) -> dict:
92     """Q4 通用约束回代（Q2 层 x/x 或 Q3 层 x_final/x_plan）。
93
94     返回：平衡/动态残差、储电量最值、区间检查、同槽充放次数；
95     scenario=True 时附加"场景池前视违规天数"。
96     """
97     s = _scan_records(recs, data["load"], data["pv_act"], x_key=x_key,
98                       start=REPORT_START, end=len(recs))
99     out = {
100         "功率平衡最大残差/kWh": s["bal"],
101         "储能动态最大残差/kWh": s["dyn"],
102         "储电量最小值/kWh": s["emin"],
103         "储电量最大值/kWh": s["emax"],
104         "约束区间检查": bool(s["emin"] >= E_MIN - 1e-6 and s["emax"] <= E_MAX + 1e-6),
105         "同槽同时充放次数": s["overlap"],
106     }
107     if scenario:
108         out["场景池前视违规天数"] = s["future_viol"]
109     return out

```

program/src/solve/io/excel.py

```

1  """官方结果表（result1-4）的写入与回读勾稽。
2
3  # =====
4  # 附件 5 模板结构（4 张表，按问题选用）
5  #   计划购电量：行为日期（2 起始；result2/3 模板无日期列，result4-* 有），
6  #       第 2..145 列为 144 槽购电量，
7  #       第 146 列 =  $\sum_t x_t$ （日合计），第 147 列 =  $\sum_t p_t \cdot x_t$ （日购电费）
8  #   调整购电量：同布局；第 147 列 =  $\sum_t [p_t \cdot x_{adj,t} + 0.5 \cdot p_t \cdot |x_{plan,t} - x_{adj,t}|]$ 
9  #       （调整结算：调低 50% 违约金、调高 150% 购电价）
10 #   充放电量：每日 6 行（4 小时块），列 3/4 为块充/放电量；
11 #       块 0/1 的行在列 5/6 记 "00:00"/"24:00" 与两端储电量
12 #   紧急购电量：每个事件一行（日期、起止时间、电量 kWh），
13 #       事件由 core.slots.emergency_events 合并相邻紧急槽得到
14 #
15 # 统一实现：四类表由 _fill_plan_sheet / _fill_adjust_sheet /
16 # _fill_charge_sheet / _fill_emergency_sheet 参数化填充；
17 # 回读勾稽对 145/146 列合计、储能端点、充放电与紧急电量逐项与内存结果比对，
18 # 残差应为数值误差量级（写入采用 float，无舍入损失）。
19 # =====
20 """
21 from __future__ import annotations
22
23 import numpy as np
24 import openpyxl
25 import pandas as pd
26

```

```

27 from solve.common import DATA_C, N_DAY, REPORT_START, RESULTS_DIR, T
28 from solve.core.slots import emergency_events as _events
29 from solve.core.slots import fmt_time as _fmt_time
30
31
32 # -----
33 # 共享表填充器
34 # -----
35 def _fill_plan_sheet(ws, x_list, prices, dates=None) -> None:
36     """计划购电量表: [(x, p)] 逐日写入; dates=None 时不写日期列 (result2/3 模板)。
37
38     每行: 可选日期、144 槽、日合计 (第 146 列)、日购电费 (第 147 列)。
39     """
40     for i, (x, p) in enumerate(zip(x_list, prices)):
41         row = 2 + i
42         if dates is not None:
43             ws.cell(row=row, column=1, value=dates[i])
44         for t in range(T):
45             ws.cell(row=row, column=2 + t, value=float(x[t]))
46             ws.cell(row=row, column=146, value=float(x.sum()))
47         if p is not None:
48             ws.cell(row=row, column=147, value=float(p @ x))
49
50
51 def _fill_adjust_sheet(ws, xa_list, xp_list, prices, dates=None) -> None:
52     """调整购电量表: 结算费 =  $\sum [p \cdot x_{adj} + 0.5 \cdot p \cdot |x_{plan} - x_{adj}|]$  (调低 50% 违约金)。"""
53     for i, (xa, xp, p) in enumerate(zip(xa_list, xp_list, prices)):
54         row = 2 + i
55         if dates is not None:
56             ws.cell(row=row, column=1, value=dates[i])
57         for t in range(T):
58             ws.cell(row=row, column=2 + t, value=float(xa[t]))
59             ws.cell(row=row, column=146, value=float(xa.sum()))
60         settle = float((p * xa + 0.5 * p * np.abs(xa - xp)).sum())
61         ws.cell(row=row, column=147, value=settle)
62
63
64 def _fill_charge_sheet(ws, entries) -> None:
65     """充放电量表: entries = [(日期对象, c, d, E_start, E_end)], 每日 6 个 4 小时块。"""
66     while ws.max_row > 1:
67         ws.delete_rows(2)
68     row = 2
69     for date_obj, c, d, e_start, e_end in entries:
70         for b in range(6):
71             rr = row + b
72             if b == 0:
73                 ws.cell(row=rr, column=1, value=date_obj)
74                 ws.cell(row=rr, column=2, value=f"{4 * b}:00-{4 * (b + 1)}:00")
75                 ws.cell(row=rr, column=3, value=float(c[24 * b:24 * (b + 1)].sum()))
76                 ws.cell(row=rr, column=4, value=float(d[24 * b:24 * (b + 1)].sum()))
77             if b == 0:
78                 ws.cell(row=rr, column=5, value="00:00")
79                 ws.cell(row=rr, column=6, value=float(e_start))
80             if b == 1:
81                 ws.cell(row=rr, column=5, value="24:00")
82                 ws.cell(row=rr, column=6, value=float(e_end))
83             row += 6
84
85
86 def _fill_emergency_sheet(ws, entries) -> None:
87     """紧急购电量表: entries = [(日期对象, e 槽序列)], 相邻紧急槽合并为一个事件。"""
88     while ws.max_row > 1:
89         ws.delete_rows(2)
90     row = 2
91     for date_obj, e in entries:
92         for j, (a, b, kwh) in enumerate(_events(e)):
93             if j == 0:
94                 ws.cell(row=row, column=1, value=date_obj)
95                 ws.cell(row=row, column=2,

```

```

96         value=f"{_fmt_time(a * 10)}-({_fmt_time((b + 1) * 10)})"
97         ws.cell(row=row, column=3, value=round(kwh, 4))
98         row += 1
99
100
101 def _to_dt(ts) -> object:
102     """ISO 日期字符串/pandas 时间戳 -> datetime (openpyxl 可写)。"""
103     return pd.Timestamp(ts).to_pydatetime()
104
105
106 # -----
107 # result2: 计划 / 充放电 / 紧急 (口径 D/E)
108 # -----
109 def write_result2(dates, price, x_plans, recs, out_name="result2.xlsx"):
110     """按附件 5 模板生成 result2.xlsx (计划购电量 / 充放电量 / 紧急购电量)。"""
111     wb = openpyxl.load_workbook(DATA_C / "附件5" / "result2.xlsx")
112     rep = list(range(REPORT_START, N_DAY))
113
114     _fill_plan_sheet(wb["计划购电量"], [x_plans[d] for d in rep], [price] * len(rep))
115     _fill_charge_sheet(wb["充放电量"], [
116         (_to_dt(dates[d]), recs[d]["c"], recs[d]["d"], recs[d]["E_start"], recs[d]["E_end"])
117         for d in rep
118     ])
119     _fill_emergency_sheet(wb["紧急购电量"],
120                          [(_to_dt(dates[d]), recs[d]["e"]) for d in rep])
121
122     RESULTS_DIR.mkdir(parents=True, exist_ok=True)
123     out = RESULTS_DIR / out_name
124     wb.save(out)
125     wb.close()
126     return out
127
128
129 def verify_result2(path, df: pd.DataFrame) -> dict:
130     """回读 result2.xlsx 并与逐日表勾稽 (返回校验指标 dict)。"""
131     wb = openpyxl.load_workbook(path, read_only=True, data_only=True)
132     checks = {"sheet 名": "/" .join(wb.sheetnames)}
133
134     ws = wb["计划购电量"]
135     rows = [r for r in ws.iter_rows(min_row=2, values_only=True) if r[0] is not None]
136     checks["计划表行数"] = len(rows)
137     checks["全天购电量勾稽差/kWh"] = float(abs(
138         sum(float(r[145]) for r in rows) - df["计划购电量/kWh"].sum()))
139     checks["全天购电费勾稽差/元"] = float(abs(
140         sum(float(r[146]) for r in rows) - df["计划购电费/元"].sum()))
141
142     ws = wb["充放电量"]
143     rows = [r for r in ws.iter_rows(min_row=2, values_only=True) if r[1] is not None]
144     checks["充放表行数"] = len(rows)
145     e0 = [float(rows[6 * i][5]) for i in range(len(rows) // 6)]
146     e24 = [float(rows[6 * i + 1][5]) for i in range(len(rows) // 6)]
147     checks["跨日SOC衔接最大误差/kWh"] = (
148         float(max(abs(e24[i] - e0[i + 1]) for i in range(len(e0) - 1)))
149         if len(e0) > 1 else 0.0)
150     checks["日末SOC非固定取值数"] = len(set(round(v, 3) for v in e24))
151
152     ws = wb["紧急购电量"]
153     kw = sum(float(r[2]) for r in ws.iter_rows(min_row=2, values_only=True)
154              if r[2] is not None)
155     checks["紧急表电量勾稽差/kWh"] = float(abs(kw - df["紧急购电量/kWh"].sum()))
156     wb.close()
157     return checks
158
159
160 # -----
161 # result3: 计划 / 调整 / 充放电 / 紧急 (Q3 主方案)
162 # -----
163 def write_result3(dates, price, out):
164     """按附件 5 模板生成 results/result3.xlsx (计划/调整/充放电/紧急, 334 天)。"""

```

```

165     wb = openpyxl.load_workbook(DATA_C / "附件5" / "result3.xlsx")
166     i0 = REPORT_START
167     n_q3 = N_DAY - REPORT_START
168     rep = range(n_q3)
169
170     _fill_plan_sheet(wb["计划购电量"], [out["xp"][i] for i in rep], [price] * n_q3)
171     _fill_adjust_sheet(wb["调整购电量"], [out["xa"][i] for i in rep],
172                        [out["xp"][i] for i in rep], [price] * n_q3)
173     _fill_charge_sheet(wb["充放电量"], [
174         (_to_dt(dates[i0 + i]), out["c"][i], out["d"][i],
175          out["E_start"][i], out["E"][i, -1])
176         for i in rep
177     ])
178     _fill_emergency_sheet(wb["紧急购电量"],
179                          [(_to_dt(dates[i0 + i]), out["e"][i]) for i in rep])
180
181     RESULTS_DIR.mkdir(parents=True, exist_ok=True)
182     p = RESULTS_DIR / "result3.xlsx"
183     wb.save(p)
184     wb.close()
185     return p
186
187
188 def verify_result3(path, dates, price, out):
189     """回读校验：行数、端点、勾稽、紧急事件。"""
190     wb = openpyxl.load_workbook(path, read_only=True, data_only=True)
191     chk = {"sheets": "/".join(wb.sheetnames)}
192     n_q3 = N_DAY - REPORT_START
193
194     ws = wb["计划购电量"]
195     rows = [r for r in ws.iter_rows(min_row=2, values_only=True) if r[0] is not None]
196     chk["计划表行数"] = len(rows)
197     chk["计划购电量勾稽差/kWh"] = float(abs(sum(float(r[145]) for r in rows) - out["xp"].sum()))
198     chk["计划购电费勾稽差/元"] = float(abs(sum(float(r[146]) for r in rows)
199                                             - sum(float(price @ out["xp"][i]) for i in range(n_q3))))
200
201     ws = wb["调整购电量"]
202     rows = [r for r in ws.iter_rows(min_row=2, values_only=True) if r[0] is not None]
203     chk["调整表行数"] = len(rows)
204     chk["调整购电量勾稽差/kWh"] = float(abs(sum(float(r[145]) for r in rows) - out["xa"].sum()))
205
206     ws = wb["充放电量"]
207     rows = [r for r in ws.iter_rows(min_row=2, values_only=True) if r[1] is not None]
208     chk["充放表行数"] = len(rows)
209     e0 = [float(rows[6 * i][5]) for i in range(n_q3)]
210     e24 = [float(rows[6 * i + 1][5]) for i in range(n_q3)]
211     chk["跨日SOC衔接最大误差/kWh"] = float(
212         max(abs(e24[i] - e0[i + 1]) for i in range(n_q3 - 1))
213     )
214     chk["日末SOC非固定取值数"] = int(len(set(round(v, 3) for v in e24)))
215
216     ws = wb["紧急购电量"]
217     kw = sum(float(r[2]) for r in ws.iter_rows(min_row=2, values_only=True) if r[2] is not None)
218     chk["紧急表电量勾稽差/kWh"] = float(abs(kw - out["e"].sum()))
219     wb.close()
220     return chk
221
222
223 # =====
224 # Q4: result4-2 (Q2 层) 与 result4-3 (Q3 层),
225 #     结算一律用附件 4 真实电价: 买入 + 0.5p|x_adj-x_plan| + 5p * 紧急。
226 # =====
227 def write_result4_2(data, p4, recs, out_name="result4-2.xlsx"):
228     """按附件 5 模板生成 results/result4-2.xlsx (计划 / 充放电 / 紧急, 334 天)。"""
229     wb = openpyxl.load_workbook(DATA_C / "附件5" / "result4-2.xlsx")
230     days = list(range(REPORT_START, N_DAY))
231
232     _fill_plan_sheet(wb["计划购电量"], [recs[D]["x"] for D in days],
233                    [p4[D] for D in days], dates=[_to_dt(data["dates"][D]) for D in days])

```

```

234     _fill_charge_sheet(wb["充放电电量"], [
235         (_to_dt(data["dates"][D])), recs[D]["c"], recs[D]["d"],
236         recs[D]["E_start"], recs[D]["E"][-1])
237         for D in days
238     ])
239     _fill_emergency_sheet(wb["紧急购电量"],
240         [(_to_dt(data["dates"][D])), recs[D]["e"]] for D in days])
241
242     RESULTS_DIR.mkdir(parents=True, exist_ok=True)
243     p = RESULTS_DIR / out_name
244     wb.save(p)
245     wb.close()
246     return p
247
248
249 def verify_result4_2(path, data, p4, recs) -> dict:
250     """回读 result4-2 并与逐日记录勾稽。"""
251     wb = openpyxl.load_workbook(path, read_only=True, data_only=True)
252     chk = {"sheets": "/" .join(wb.sheetnames)}
253     days = list(range(REPORT_START, N_DAY))
254
255     ws = wb["计划购电量"]
256     rows = [r for r in ws.iter_rows(min_row=2, values_only=True) if r[0] is not None]
257     chk["计划表行数"] = len(rows)
258     chk["计划购电量勾稽差/kWh"] = float(abs(
259         sum(float(r[145]) for r in rows) - sum(float(recs[D]["x"].sum()) for D in days)))
260     chk["计划购电费勾稽差/元"] = float(abs(
261         sum(float(r[146]) for r in rows) - sum(float(p4[D] @ recs[D]["x"]) for D in days)))
262
263     ws = wb["充放电电量"]
264     crows = [r for r in ws.iter_rows(min_row=2, values_only=True) if r[1] is not None]
265     chk["充放电表行数"] = len(crows)
266     chk["充放电电量勾稽差/kWh"] = float(abs(
267         sum(float(r[2]) for r in crows if r[2] is not None)
268         - sum(float(recs[D]["c"].sum()) for D in days)))
269
270     ws = wb["紧急购电量"]
271     kw = sum(float(r[2]) for r in ws.iter_rows(min_row=2, values_only=True) if r[2] is not None)
272     chk["紧急表电量勾稽差/kWh"] = float(abs(
273         kw - sum(float(recs[D]["e"].sum()) for D in days)))
274     wb.close()
275     return chk
276
277
278 def write_result4_3(data, p4, recs, out_name="result4-3.xlsx"):
279     """按附件 5 模板生成 results/result4-3.xlsx (计划/调整/充放电/紧急, 334 天)。"""
280     wb = openpyxl.load_workbook(DATA_C / "附件5" / "result4-3.xlsx")
281     days = list(range(REPORT_START, N_DAY))
282     dts = [_to_dt(data["dates"][D]) for D in days]
283
284     _fill_plan_sheet(wb["计划购电量"], [recs[D]["x_plan"] for D in days],
285         [p4[D] for D in days], dates=dts)
286     _fill_adjust_sheet(wb["调整购电量"], [recs[D]["x_final"] for D in days],
287         [recs[D]["x_plan"] for D in days], [p4[D] for D in days],
288         dates=dts)
289     _fill_charge_sheet(wb["充放电电量"], [
290         (_to_dt(data["dates"][D])), recs[D]["c"], recs[D]["d"],
291         recs[D]["E_start"], recs[D]["E"][-1])
292         for D in days
293     ])
294     _fill_emergency_sheet(wb["紧急购电量"],
295         [(_to_dt(data["dates"][D])), recs[D]["e"]] for D in days])
296
297     RESULTS_DIR.mkdir(parents=True, exist_ok=True)
298     p = RESULTS_DIR / out_name
299     wb.save(p)
300     wb.close()
301     return p
302

```

```

303
304 def verify_result4_3(path, data, p4, recs) -> dict:
305     """回读 result4-3 并与逐日记录勾稽。"""
306     wb = openpyxl.load_workbook(path, read_only=True, data_only=True)
307     chk = {"sheets": "/" + ".join(wb.sheetnames)}
308     days = list(range(REPORT_START, N_DAY))
309     for sheet, key, tag in (("计划购电量", "x_plan", "计划"), ("调整购电量", "x_final", "调整")):
310         rows = [r for r in wb[sheet].iter_rows(min_row=2, values_only=True) if r[0] is not None]
311         chk[f"{tag}表行数"] = len(rows)
312         chk[f"{tag}购电量勾稽差/kWh"] = float(abs(
313             sum(float(r[145]) for r in rows) - sum(float(recs[D][key].sum()) for D in days)))
314     rows = [r for r in wb["充放电量"].iter_rows(min_row=2, values_only=True) if r[1] is not None]
315     chk["充放表行数"] = len(rows)
316     chk["充放电量勾稽差/kWh"] = float(abs(
317         sum(float(r[2]) for r in rows if r[2] is not None)
318         - sum(float(recs[D]["c"].sum()) for D in days)))
319     kw = sum(float(r[2]) for r in wb["紧急购电量"].iter_rows(min_row=2, values_only=True)
320              if r[2] is not None)
321     chk["紧急表电量勾稽差/kWh"] = float(abs(
322         kw - sum(float(recs[D]["e"].sum()) for D in days)))
323     wb.close()
324     return chk

```

program/src/solve/io/figures.py

```

1  """共用绘图辅助：月份刻度。
2
3  # =====
4  # month_axis(ax, dates)
5  #     输入：与横轴一一对应的日期字符串列表（ISO，如 "2025-03-01"）
6  #     作用：以每月 1 日的位置设刻度，标签为 "N月"
7  #     等价于旧代码中手写的
8  #         pos = [i for i, s in enumerate(dates) if s.endswith("-01")]
9  #         ax.set_xticks(pos); ax.set_xticklabels([dates[i][5:7] + "月" for i in pos])
10 # =====
11 """
12 from __future__ import annotations
13
14
15 def month_axis(ax, dates) -> None:
16     """在 ax 上设置月份刻度（dates 与横轴索引一一对应）。"""
17     pos = [i for i, s in enumerate(dates) if s.endswith("-01")]
18     ax.set_xticks(pos)
19     ax.set_xticklabels([dates[i][5:7] + "月" for i in pos])

```

program/src/solve/io/report.py

```

1  """结果报告写入：RESULTS_REPORT.md 章节级去重追加。
2
3  报告是论文数值的唯一来源；同名章节重跑时必须先删后写，避免重复堆积。
4  """
5  from __future__ import annotations
6
7  import program as pm
8
9
10 def record(section: str, data, note: str = "") -> None:
11     """写结果报告：先删除同名旧章节再追加（pm.record_result 为追加模式）。
12
13     删除规则：定位 "### {section}" 起始，截到下一个 "\n### " 之前；
14     若为最后一节则截到文件末尾。
15     """
16     path = pm.reports_dir() / "RESULTS_REPORT.md"
17     if path.exists():
18         text = path.read_text(encoding="utf-8")
19         marker = f"### {section}"
20         pos = text.find(marker)
21         if pos != -1:

```

```

22         end = text.find("\n### ", pos + len(marker))
23         text = text[:pos] if end == -1 else text[:pos] + text[end + 1 :]
24         path.write_text(text, encoding="utf-8")
25         pm.record_result(section, data, note=note)

```

experiments

program/src/solve/experiments/__init__.py

```

1  """专题/探索脚本（保留勿删；不属于正式结果生成链）。
2
3  运行（program/ 下）：
4      uv run python -m solve.experiments.q2_arima          # ARIMA 预测对照
5      uv run python -m solve.experiments.q2_bias          # 有偏预测 vs 场景对冲
6      uv run python -m solve.experiments.q2_bias_roll     # 滚动标定
7      uv run python -m solve.experiments.q2_regrsrc       # 回归参考量探索（已搁置）
8      uv run python -m solve.experiments.q2e_smooth      # Q2E 平滑回溯（负结果）
9      uv run python -m solve.experiments.q2e_structure    # 误差分布结构分析
10     uv run python -m solve.experiments.q3_proto_seg      # Q3 分段/规则原型
11     uv run python -m solve.experiments.q3_proto_abl      # Q3 消融+分解+MC
12     uv run python -m solve.experiments.q3_proto_rt       # Q3 实时对比
13     uv run python -m solve.experiments.q4_price_eda      # 附件 4 电价 EDA
14     uv run python -m solve.experiments.q4_price_dyn      # 动态修正验证
15     uv run python -m solve.experiments.exec_policy_probe # 执行器段末目标对照探针
16 说明：正式脚本仍在 solve/ 顶层（q1-q4、q2_tune、q2_adaptive、q3_sensitivity、
17 q3_ablation、q4_price_fit、q3_proto 门面）；本包脚本复用顶层门面与子包实现。
18 """

```

program/src/solve/experiments/eda_cycles.py

```

1  """C 题附件数据周期结构 EDA：日周期与周周期可视化。
2
3  数据：附件2（小区负载、光伏发电实际功率，365×144）、附件4（实时电价，365×144）、
4  附件1（典型日 144 槽；实测等于附件2/4 的全年逐槽均值，作为日周期基准线）。
5
6  输出：
7  - figures/数据_日周期.pdf、figures/数据_周周期.pdf
8    作图数据：code/outputs/figure_data/数据_日周期.csv、数据_周周期.csv
9  - code/outputs/eda_cycles_stats.json（日内/周内解释方差、ACF、星期效应检验）
10 - reports/RESULTS_REPORT.md 章节「附件数据周期结构（EDA）」
11
12 运行（program/ 下）：uv run python -m solve.experiments.eda_cycles
13 """
14 from __future__ import annotations
15
16 import time
17
18 import numpy as np
19 import pandas as pd
20 from scipy import stats
21
22 import program as pm
23 from solve.common import DATA_C, ROOT, T
24
25 HOUR = T // 24          # 每小时 6 槽
26 WEEKDAYS = ["周一", "周二", "周三", "周四", "周五", "周六", "周日"]
27 BLUE_TEAL_YELLOW_RED = ["#2166ac", "#1fa8c9", "#77d7c8", "#fff3a5", "#fdae61", "#d73027"]
28 SERIES = {
29     # 显示名 -> (附件数组键, 单位, 颜色)
30     "小区负载": ("load", "kW", "#2166ac"),
31     "光伏出力": ("pv", "kW", "#1fa8c9"),
32     "实时电价": ("price", "元/kWh", "#d73027"),
33 }
34
35 def _acf(x: np.ndarray, lag: int) -> float:
36     """去均值序列滞后 lag 槽的自相关。"""

```

```

37     y = x - x.mean()
38     return float((y[:-lag] * y[lag:]).sum() / (y * y).sum())
39
40
41 def _acf_curve(x: np.ndarray, max_lag: int) -> tuple[np.ndarray, np.ndarray]:
42     lags = np.arange(1, max_lag + 1)
43     return lags, np.array([_acf(x, int(k)) for k in lags])
44
45
46 def _weekday_r2(day_mean: np.ndarray, weekday: np.ndarray) -> float:
47     """星期效应（各星期几均值）对日均值的解释方差比例。"""
48     eff = np.array([day_mean[weekday == w].mean() for w in weekday])
49     return float(1 - np.var(day_mean - eff) / np.var(day_mean))
50
51
52 def record(section: str, data, note: str = "") -> None:
53     """写结果报告：先删同名旧章节再追加（pm.record_result 为追加模式）。"""
54     path = pm.reports_dir() / "RESULTS_REPORT.md"
55     if path.exists():
56         text = path.read_text(encoding="utf-8")
57         marker = f"### {section}"
58         pos = text.find(marker)
59         if pos != -1:
60             end = text.find("\n### ", pos + len(marker))
61             text = text[:pos] if end == -1 else text[:pos] + text[end + 1:]
62             path.write_text(text, encoding="utf-8")
63     pm.record_result(section, data, note=note)
64
65
66 def main() -> None:
67     t0 = time.perf_counter()
68     pm.init(seed=42, root=STR(ROOT))
69
70     load = pm.read_table(DATA_C / "附件2.xlsx", sheet="小区负载")
71     pv = pm.read_table(DATA_C / "附件2.xlsx", sheet="光伏发电实际功率")
72     price = pm.read_table(DATA_C / "附件4.xlsx")
73     a1 = pm.read_table(DATA_C / "附件1.xlsx")
74
75     dates = pd.to_datetime(load.iloc[:, 0])
76     weekday = dates.dt.weekday.to_numpy()
77     data = {
78         "load": load.iloc[:, 1:145].to_numpy(float),
79         "pv": pv.iloc[:, 1:145].to_numpy(float),
80         "price": price.iloc[:, 1:145].to_numpy(float),
81     }
82     a1_ref = {
83         "load": a1["小区负载"].to_numpy(float),
84         "pv": a1["光伏发电预测功率"].to_numpy(float),
85         "price": a1["电价"].to_numpy(float),
86     }
87
88     t_h = (np.arange(T) + 0.5) * 10 / 60          # 槽中心时刻 / h
89     hour_mean: dict[str, np.ndarray] = {}          # 日内均值曲线（144 槽）
90     wd_slot: dict[str, np.ndarray] = {}            # 星期几×槽均值（7×144）
91     slot_stats: dict[str, dict] = {}
92
93     for name, (key, unit, _c) in SERIES.items():
94         arr = data[key]
95         m = arr.mean(axis=0)
96         hour_mean[key] = m
97         wd_slot[key] = np.stack([arr[weekday == w].mean(axis=0) for w in range(7)])
98         day_mean = arr.mean(axis=1)
99         r2_intra = 1 - np.var(arr - m) / np.var(arr)
100        r2_wd = _weekday_r2(day_mean, weekday)
101        kr = stats.kruskal(*[day_mean[weekday == w] for w in range(7)])
102        wknd = np.isin(weekday, (4, 5))             # 数据中周五+周六为低谷
103        diff_a1 = float(np.abs(a1_ref[key] - m).max())
104        slot_stats[name] = {
105            "单位": unit,

```

```

106     "日内形状R2": round(float(r2_intra), 3),
107     "周内R2_天均值": round(r2_wd, 3),
108     "ACF_lag1天": round(_acf(arr.ravel(), 144), 3),
109     "ACF_lag7天": round(_acf(arr.ravel(), 1008), 3),
110     "周五六天均值": round(float(day_mean[wknd].mean()), 2),
111     "其余天均值": round(float(day_mean[-wknd].mean()), 2),
112     "周五六相对偏离_pct": round(float(100 * (day_mean[wknd].mean() / day_mean.mean() - 1)), 1),
113     "Kruskal_p": float(kr.pvalue),
114     "附件1与年均最大差": diff_a1,
115     "峰值时刻_h": float(t_h[int(np.argmax(m))]),
116 }
117 print(f"{name}: 日内R2={slot_stats[name]['日内形状R2']:.3f} "
118       f"周内R2={slot_stats[name]['周内R2_天均值']:.3f} "
119       f"ACF(1d/7d)={slot_stats[name]['ACF_lag1天']:.3f}/{slot_stats[name]['ACF_lag7天']:.3f} "
120       f"周五六偏离={slot_stats[name]['周五六相对偏离_pct']:+.1f}% "
121       f"Kruskal p={kr.pvalue:.2e}")
122
123 # ---- 图 1: 日周期 (均值 + P25-P75 带 + 附件1 典型日) ----
124 import matplotlib.pyplot as plt
125
126 fig, axes = plt.subplots(1, 3, figsize=(10, 3.2))
127 for ax, (name, (key, unit, color)) in zip(axes, SERIES.items()):
128     arr = data[key]
129     p25, p75 = np.percentile(arr, 25, axis=0), np.percentile(arr, 75, axis=0)
130     ax.fill_between(t_h, p25, p75, color=color, alpha=0.22, lw=0,
131                    label="P25-P75 带")
132     ax.plot(t_h, hour_mean[key], color=color, lw=1.5, label="全年均值")
133     ax.plot(t_h, a1_ref[key], color="k", lw=0.8, ls="--", label="附件1 典型日")
134     ax.axvline(slot_stats[name]["峰值时刻_h"], color="gray", ls=":", lw=0.7)
135     ax.set_xlabel("时刻 / h")
136     ax.set_ylabel(f"{name} / {unit}")
137     ax.set_xlim(0, 24)
138     ax.set_xticks(np.arange(0, 25, 6))
139     ax.set_title(name, fontsize=9)
140     ax.legend(fontsize=6.5, loc="upper left", framealpha=0.9)
141 fig.tight_layout()
142 day_df = pd.DataFrame({
143     "时刻_h": np.round(t_h, 3),
144     **{f"{name}_{stat}": vals for name, (key, _u, _c) in SERIES.items()
145        for stat, vals in (("均值", hour_mean[key]),
146                          ("P25", np.percentile(data[key], 25, axis=0)),
147                          ("P75", np.percentile(data[key], 75, axis=0)),
148                          ("附件1", a1_ref[key]))},
149 })
150 pm.save_fig(fig, "数据_日周期", data=day_df)
151
152 # ---- 图 2: 周周期 (星期几×时刻热力图, 槽级数据 + 双线性平滑; 色带=蓝青→黄红) ----
153 from matplotlib.colors import LinearSegmentedColormap
154
155 cmap = LinearSegmentedColormap.from_list("blue_teal_yellow_red",
156                                         BLUE_TEAL_YELLOW_RED)
157 fig2, axes2 = plt.subplots(1, 3, figsize=(10, 3.0))
158 for ax, (name, (key, unit, _c)) in zip(axes2, SERIES.items()):
159     grid = wd_slot[key]
160     im = ax.imshow(grid, aspect="auto", origin="upper", cmap=cmap,
161                   interpolation="bilinear")
162     ax.set_yticks(np.arange(7))
163     ax.set_yticklabels(WEEKDAYS, fontsize=7)
164     for lbl in ax.get_yticklabels():
165         if lbl.get_text() in ("周五", "周六"):
166             lbl.set_color("#C44E52")
167     ax.set_xticks(np.arange(0, T + 1, 6 * HOUR))
168     ax.set_xticklabels(np.arange(0, 25, 6), fontsize=7)
169     ax.set_xlabel("时刻 / h")
170     ax.set_title(f"{name} / {unit}", fontsize=9)
171     fig2.colorbar(im, ax=ax, fraction=0.046, pad=0.03)
172 fig2.tight_layout()
173 week_df = pd.DataFrame({
174     "星期": np.repeat(WEEKDAYS, T),

```

```

175     "时刻_h": np.round(np.tile(t_h, 7), 3),
176 })
177 for name, (key, _u, _c) in SERIES.items():
178     week_df[name] = wd_slot[key].ravel().round(4)
179 pm.save_fig(fig2, "数据_周周期", data=week_df)
180
181 # ---- 统计落盘 + 报告章节 ----
182 stats_out = {name: slot_stats[name] for name in SERIES}
183 pm.save_outputs(stats_out, "eda_cycles_stats")
184
185 flat = {
186     f"{name} * {k}": v for name, s in stats_out.items() for k, v in s.items()
187 }
188 record(
189     "附件数据周期结构 (EDA) ",
190     flat,
191     note=(
192         "数据: 附件2 (负荷/光伏实际, 365×144)、附件4 (电价, 365×144); "
193         "统计含义: 日内形状R2=144 槽均值对总方差的解释比例, 周内R2=星期几均值对天均值的解释比例, "
194         "ACF 为槽级去均值自相关。结论: 负荷与电价具『日内+周内』双重周期, "
195         "周内低谷为周五+周六 (非通常意义的周六周日), 光伏仅日内周期、无周内效应; "
196         "附件1 三序列=附件2/4 的 全年逐槽均值 (最大差见上表), 可作典型日基准。"
197         "图 figures/数据_日周期.pdf、figures/数据_周周期.pdf; "
198         "报告章节由 solve.experiments.eda_cycles 重跑时自动覆盖。"),
199 )
200 dt = time.perf_counter() - t0
201 print(f"\n完成: figures/数据_日周期.pdf、figures/数据_周周期.pdf、"
202       f"code/outputs/eda_cycles_stats.json ({dt:.1f} s) ")
203
204
205 if __name__ == "__main__":
206     main()

```

program/src/solve/experiments/exec_policy_probe.py

```

1  """执行器"段末硬目标"机制对照探针 (小样本, 供论文与验收引用)。
2
3  背景: 原逐槽因果执行器要求 SOC 在 6:00/12:00/18:00/24:00 跟踪规划轨迹
4  (可达性投影), 实测会迫使"紧急购电充能 / 扣留放电", 在段末小时形成紧急购电尖峰。
5  本探针在统一结构 (2 日滚动 + EWMA + /m + 40 情景联合对冲) 下, 用小样本对比:
6
7  - ``target``: 段末目标硬跟踪 (旧口径);
8  - ``dayend``: 仅当日最后一段跟踪日末目标;
9  - ``free``: 无任何段末目标, 仅容量/功率约束 (现行统一口径 EXEC_POLICY)。
10
11  样本 (小数据, 避免大仿真):
12  - 夏季应力样本: 2025-06-01 ~ 07-30 (60 天);
13  - 秋季应力样本: 2025-10-16 ~ 12-01 (47 天)。
14
15  结论 (2026-09-13, 末段修复后重跑): 段末目标分两段计损——中段目标
16  (target+dayend) 夏季 +40.3 万/12.2%、秋季 +8.4 万/4.3%; 末段目标
17  (dayend+free) 再 +4.6 万/1.6%、+0.5 万/0.3%; 合计 free 相对 target
18  -44.9 万/-13.6%、-8.9 万/-4.5%; SOC 仍全幅利用 ([1200, 10800])。
19
20  运行 (program/ 下): uv run python -m solve.experiments.exec_policy_probe
21  """
22  from __future__ import annotations
23
24  import time
25
26  import numpy as np
27
28  import program as pm
29  from solve import consistency as cs
30  from solve import q3_proto as qp
31  from solve.common import ROOT
32  from solve.io.report import record
33
34  SAMPLES = {

```

```

35     "夏季 (2025-06-01-07-30)": list(range(151, 211)),
36     "秋季 (2025-10-16-12-01)": list(range(288, 335)),
37 }
38 POLICIES = ["target", "dayend", "free"]
39
40
41 def run_sample(data, days, policy, lam=0.7) -> dict:
42     t0 = time.time()
43     E = float(qp.E0)
44     tot = plan = dev = em = spill = 0.0
45     win_em = np.zeros(4)
46     emin, emax = 1e9, -1e9
47     for D in days:
48         r = qp.simulate_day_rt_hedge(
49             data, D, lam, adj_lam=lam, e_start=E, exec_policy=policy)
50         tot += r["total"]
51         plan += r["plan_cost"]
52         dev += r["deviation_cost"]
53         em += r["emerg"]
54         spill += float(r["s"].sum())
55         e = r["e"]
56         win_em += np.array([e[0:36].sum(), e[36:72].sum(),
57                             e[72:108].sum(), e[108:144].sum()])
58         emin = min(emin, float(r["E"].min()))
59         emax = max(emax, float(r["E"].max()))
60         E = float(r["E_end"])
61     return {"样本": days and data["dates"][days[0]], "策略": policy,
62           "天数": len(days),
63           "总费用/万元": round(tot / 1e4, 1),
64           "计划购电费/万元": round(plan / 1e4, 1),
65           "偏差费/万元": round(dev / 1e4, 1),
66           "紧急购电费/万元": round(em / 1e4, 1),
67           "弃电量/kWh": round(spill, 1),
68           "紧急0-6h/万元": round(win_em[0] / 1e4, 1),
69           "紧急6-12h/万元": round(win_em[1] / 1e4, 1),
70           "紧急12-18h/万元": round(win_em[2] / 1e4, 1),
71           "紧急18-24h/万元": round(win_em[3] / 1e4, 1),
72           "SOC最小/kWh": round(emin, 0),
73           "SOC最大/kWh": round(emax, 0),
74           "用时/s": round(time.time() - t0, 0)}
75
76
77 def main() -> None:
78     import pandas as pd
79
80     pm.init(seed=42, root=str(ROOT))
81     data = qp.load_extended()
82     rows = []
83     for name, days in SAMPLES.items():
84         for policy in POLICIES:
85             r = run_sample(data, days, policy)
86             r["样本"] = name
87             rows.append(r)
88     print(f" [{name}][{policy}>6]} 总 {r['总费用/万元']:.61f} 万 "
89           f"(计划 {r['计划购电费/万元']:.1f} + 偏差 {r['偏差费/万元']:.1f} "
90           f"+ 紧急 {r['紧急购电费/万元']:.1f}) 弃电 {r['弃电量/kWh']:.0f} kWh")
91     df = pd.DataFrame(rows)
92     pm.save_outputs(df, "exec_policy_probe")
93     record(
94         "执行器段末目标机制对照 (小样本)",
95         df,
96         note=(
97             "统一结构下对比 target (段末硬目标) / dayend (仅末段跟踪日末目标) / "
98             "free (完全无目标, 现行统一口径 consistency.EXEC_POLICY); "
99             "末段修复后重跑: 中段目标夏季 +40.3 万 (12.2%)、秋季 +8.4 万 (4.3%), "
100             "末段目标再 +4.6 万 (1.6%)、+0.5 万 (0.3%); "
101             "完整探针见 code/outputs/exec_policy_probe.csv 与 "
102             "reports/执行器段末目标实验.md。"),
103     )
104     print("完成。")

```

```

104
105
106 if __name__ == "__main__":
107     main()

```

program/src/solve/experiments/q2_arima.py

```

1  """C 题 问题二 对照：逐时段 ARIMA 预测 vs 自适应加权预测。
2
3  对比两个"预测器"在完全相同的信息结构下（每天 0:00 只看历史）的表现：
4
5  - 自适应加权（口径 E，`q2_adaptive.py`）：负荷 d-7/d-14/典型日，光伏 d-1/d-2/典型日，
6    权重按历史实际费用标定；
7  - ARIMA(2,0,1)：每个 10 分钟槽（144 槽）× 负荷/光伏 各建一条日序列，滚动一步预测；
8  - 参考基线：典型日、持续预测（D-1）、口径 D。
9
10  滚动方式：每 REFIT_EVERY 天用扩展窗口重估参数，其余日期用 Kalman 滤波
11  append 新观测（refit=False）后再做 1 步预测。预测结果缓存到
12  ``code/outputs/cache/q2arima/``。
13
14  评价两条线：
15  1. 预测精度：MAE / RMSE (kW, 2025-02-01 ~ 12-31)；
16  2. 全年费用：把预测代入同一套计划/执行/紧急购电模型（与问题二口径一致）。
17
18  运行：uv run python -m solve.experiments.q2_arima （在 program/ 目录；首次约 5 分钟，之后走缓存）
19  """
20  from __future__ import annotations
21
22  import hashlib
23  import os
24  import time
25  import warnings
26
27  import numpy as np
28  import pandas as pd
29  from statsmodels.tsa.arima.model import ARIMA
30
31  import program as pm
32  from solve import q2
33  from solve.common import DATA_C, EO, ROOT
34  from solve.io.report import record
35  from solve.models.adaptive import AdaptiveWeightModel
36
37  N_DAY = q2.N_DAY # 365
38  REPORT_START = q2.REPORT_START # 31 (2025-02-01)
39  EPS_PLAN = 1e-3
40  ORDER = (2, 0, 1) # 抽样槽位 AIC 对比后固定（负荷 8/12 最优，光伏并列最优）
41  REFIT_EVERY = 7 # 参数重估周期（天）
42  WORKERS = min(8, os.cpu_count() or 1)
43
44  _DATA = None # 子进程数据（initializer 注入）
45
46
47  # -----
48  # ARIMA 预测器
49  # -----
50  def _init_worker(data) -> None:
51      global _DATA
52      _DATA = data
53
54
55  def _fit_slot(task):
56      """单条序列（变量，槽位）的滚动一步预测，返回（which, slot, pred[365]）。"""
57      which, slot, order, refit_every = task
58      warnings.filterwarnings("ignore")
59      y = (_DATA["load"] if which == 0 else _DATA["pv_act"])[ :, slot]
60      pred = np.full(N_DAY, np.nan)
61      res = None
62      for d in range(REPORT_START, N_DAY):

```

```

63     train = y[:d]
64     if np.ptp(train) < 1e-9:                # 常数序列（夜间光伏槽等）
65         pred[d] = float(train[-1])
66         continue
67     try:
68         if res is None or (d - REPORT_START) % refit_every == 0:
69             res = ARIMA(train, order=order).fit(
70                 method_kwargs={"warn_convergence": False})
71         else:
72             res = res.append([y[d - 1]], refit=False)
73             pred[d] = float(res.forecast(1)[0])
74     except Exception:                        # 拟合失败退化为持续预测
75         pred[d] = float(y[d - 1])
76     return which, slot, pred
77
78
79 class ArimaForecaster:
80     """逐时段 ARIMA 滚动一步预测器（144 槽 × 负荷 × 光伏 2 变量）。
81
82     Parameters
83     -----
84     order : tuple
85         ARIMA(p, d, q) 阶数，默认 (2, 0, 1)。
86     refit_every : int
87         参数重估周期（天）；其间用 append(refit=False) 更新状态。
88     workers : int
89         并行进程数（按序列并行），默认 min(8, CPU)。
90     """
91
92     def __init__(self, order=ORDER, refit_every=REFIT_EVERY, workers=WORKERS):
93         self.order = tuple(order)
94         self.refit_every = refit_every
95         self.workers = workers
96         self.data = None
97         self.load_fc = self.pv_fc = None
98         self.seconds = 0.0
99
100     def load(self, data: dict | None = None) -> "ArimaForecaster":
101         self.data = q2.load_all() if data is None else data
102         return self
103
104     def _cache_path(self):
105         h = hashlib.md5()
106         for arr in (self.data["load"], self.data["pv_act"]):
107             h.update(np.ascontiguousarray(arr, dtype=float).tobytes())
108         h.update(np.asarray([self.order, self.refit_every], float).tobytes())
109         cache = pm.outputs_dir() / "cache" / "q2arima"
110         cache.mkdir(parents=True, exist_ok=True)
111         return cache / f"fc_{h.hexdigest()[:12]}.npz"
112
113     def forecast_all(self) -> "ArimaForecaster":
114         """并行预测，结果（kW, 365×144）存 load_fc / pv_fc（31 日之前为 0）。"""
115         pm.init(root=str(ROOT))
116         path = self._cache_path()
117         if path.exists():
118             z = np.load(path)
119             self.load_fc, self.pv_fc = z["load_fc"], z["pv_fc"]
120             return self
121         tasks = [(which, slot, self.order, self.refit_every)
122                 for which in (0, 1) for slot in range(144)]
123         t0 = time.time()
124         if self.workers > 1:
125             from multiprocessing import Pool
126
127             with Pool(self.workers, initializer=_init_worker,
128                       initargs=(self.data,)) as pool:
129                 results = pool.map(_fit_slot, tasks)
130         else:
131             _init_worker(self.data)

```

```

132         results = [_fit_slot(tk) for tk in tasks]
133
134         fl = np.zeros((N_DAY, 144))
135         fp = np.zeros((N_DAY, 144))
136         for which, slot, pred in results:
137             (fl if which == 0 else fp)[:, slot] = np.nan_to_num(pred, nan=0.0)
138         self.load_fc, self.pv_fc = np.clip(fl, 0.0, None), np.clip(fp, 0.0, None)
139         self.seconds = time.time() - t0
140         np.savez_compressed(path, load_fc=self.load_fc, pv_fc=self.pv_fc)
141         return self
142
143
144 # -----
145 # 指标与费用
146 # -----
147 def mae_rmse(pred_kw: np.ndarray, true_kw: np.ndarray):
148     """报告期内的 MAE / RMSE (kW, 全部槽位与日期)。"""
149     err = pred_kw[REPORT_START:] - true_kw[REPORT_START:]
150     return float(np.abs(err).mean()), float(np.sqrt((err ** 2).mean()))
151
152
153 def annual_cost(load_fc_kwh: np.ndarray, pv_fc_kwh: np.ndarray, data: dict):
154     """用给定预测跑全年计划 + 执行 + 紧急购电, 返回 (计划费, 紧急费, 总费用)。"""
155     price, load, pv = data["price"], data["load"], data["pv_act"]
156     plan = emerg = 0.0
157     for d in range(REPORT_START, N_DAY):
158         x, _E, _ = q2.plan_day(price, load_fc_kwh[d], pv_fc_kwh[d], E0, eps=EPS_PLAN)
159         ex = q2.exec_day_causal(price, load[d] / 6.0, pv[d] / 6.0, x, E0)
160         plan += float(price @ x)
161         emerg += float(q2.EMERG_MULT * (price @ ex["e"]))
162     return plan, emerg, plan + emerg
163
164
165 def typical_forecast(data: dict):
166     """附件 1 典型日 (负荷 kW, 光伏 kW) 平铺到全年。"""
167     a1 = pm.read_table(DATA_C / "附件1.xlsx")
168     lt = a1["小区负载"].to_numpy(float)
169     pt = a1["光伏发电预测功率"].to_numpy(float)
170     return np.tile(lt, (N_DAY, 1)), np.tile(pt, (N_DAY, 1))
171
172
173 def adaptive_forecast(data: dict, model: AdaptiveWeightModel):
174     """口径 E 的预测 (kW, 365×144): 目标日 d 用 TH[d-1] 的权重。"""
175     fl = np.zeros((N_DAY, 144))
176     fp = np.zeros((N_DAY, 144))
177     weights = model.weights(W=1, use_gd=True)
178     for i, d in enumerate(range(REPORT_START, N_DAY)):
179         w, u = weights[i]
180         l_kwh, p_kwh = model.forecast(w, u, d)
181         fl[d], fp[d] = l_kwh * 6.0, p_kwh * 6.0
182     return fl, fp
183
184
185 def persistence_forecast(data: dict):
186     """持续预测基线: 用 D-1 的实际值。"""
187     fl = np.zeros((N_DAY, 144))
188     fp = np.zeros((N_DAY, 144))
189     fl[REPORT_START:] = data["load"][REPORT_START - 1:-1]
190     fp[REPORT_START:] = data["pv_act"][REPORT_START - 1:-1]
191     return fl, fp
192
193
194 # -----
195 # 主流程
196 # -----
197 def run_comparison(workers=WORKERS) -> dict:
198     """跑 ARIMA, 与自适应加权/典型日/持续预测比精度与费用, 并写报告与图。"""
199     import matplotlib.pyplot as plt
200

```

```

201 pm.init(root=str(ROOT))
202 log = pm.get_logger("q2arima")
203 data = q2.load_all()
204
205 log.info("ARIMA(2,0,1) 滚动预测中 ({} 进程) ...", workers)
206 ar = ArimaForecaster(workers=workers).load(data).forecast_all()
207 log.info("ARIMA 完成, 用时 {:.0f}s", ar.seconds)
208
209 model = AdaptiveWeightModel().load().build_table() # load() 会补上典型日列
210 fl_ad, fp_ad = adaptive_forecast(data, model)
211 fl_typ, fp_typ = typical_forecast(data)
212 fl_pst, fp_pst = persistence_forecast(data)
213
214 rows = []
215 for name, fl, fp in (
216     ("自适应加权 (E)", fl_ad, fp_ad),
217     ("ARIMA(2,0,1)", ar.load_fc, ar.pv_fc),
218     ("典型日 (B)", fl_typ, fp_typ),
219     ("持续预测 (D-1)", fl_pst, fp_pst),
220 ):
221     mae_l, rmse_l = mae_rmse(fl, data["load"])
222     mae_p, rmse_p = mae_rmse(fp, data["pv_act"])
223     rows.append({
224         "预测模型": name,
225         "负荷MAE/kW": round(mae_l, 1), "负荷RMSE/kW": round(rmse_l, 1),
226         "光伏MAE/kW": round(mae_p, 1), "光伏RMSE/kW": round(rmse_p, 1),
227     })
228 metrics = pd.DataFrame(rows)
229 record(
230     "问题二 对照: ARIMA 与自适应加权的预测精度 (2025-02-01 ~ 12-31)",
231     metrics,
232     note=(
233         "逐时段滚动一步预测 (144 槽 × 334 天), 全部只用 D-1 及以前的数据; "
234         f"ARIMA(2,0,1) 每 {REFIT_EVERY} 天扩展窗口重估参数、其余日期 Kalman append。"
235         "指标单位 kW: 持续预测 = 用前一天同时刻实际值。"
236     ),
237 )
238 pm.save_outputs(metrics, "q2_arima_metrics")
239
240 cost_rows = []
241 for name, fl, fp in (
242     ("自适应加权 (E)", fl_ad, fp_ad),
243     ("ARIMA(2,0,1)", ar.load_fc, ar.pv_fc),
244     ("典型日 (B)", fl_typ, fp_typ),
245 ):
246     plan, emerg, tot = annual_cost(fl / 6.0, fp / 6.0, data)
247     cost_rows.append({
248         "预测模型": name,
249         "计划购电费/万元": round(plan / 1e4, 1),
250         "紧急购电费/万元": round(emerg / 1e4, 1),
251         "总费用/万元": round(tot / 1e4, 1),
252     })
253 _x_d, recs_d = q2.run_deterministic(data) # 口径 D 参考
254 plan_d = sum(recs_d[d]["plan_cost"] for d in range(REPORT_START, N_DAY))
255 emerg_d = sum(recs_d[d]["emerg_cost"] for d in range(REPORT_START, N_DAY))
256 cost_rows.append({
257     "预测模型": "0:00 预报 (D, 参考)",
258     "计划购电费/万元": round(plan_d / 1e4, 1),
259     "紧急购电费/万元": round(emerg_d / 1e4, 1),
260     "总费用/万元": round((plan_d + emerg_d) / 1e4, 1),
261 })
262 costs = pd.DataFrame(cost_rows)
263 record(
264     "问题二 对照: ARIMA 与自适应加权的全年费用",
265     costs,
266     note=(
267         "同一套计划/执行/紧急购电模型 (take-or-pay、储能日循环、5 倍价紧急购电); "
268         "ARIMA 预测为统计拟合 (未按费用标定), 自适应加权权重按费用标定。"
269     ),

```

```

270 )
271 pm.save_outputs(costs, "q2_arima_compare")
272
273 # ---- 图：精度对比 + 费用对比 ----
274 labels = metrics["预测模型"].tolist()
275 pm.bar_group(
276     labels,
277     {"负荷MAE": metrics["负荷MAE/kW"].tolist(),
278      "光伏MAE": metrics["光伏MAE/kW"].tolist()},
279     ylabel="MAE / kW", rot=12, save="Q2ARIMA_误差对比",
280 )
281 pm.bar_group(
282     costs["预测模型"].tolist(),
283     {"计划购电费": costs["计划购电费/万元"].tolist(),
284      "紧急购电费": costs["紧急购电费/万元"].tolist()},
285     ylabel="年度费用 / 万元", rot=12, save="Q2ARIMA_费用对比",
286 )
287
288 # ---- 图：示例日曲线（2025-06-21） ----
289 d = 171
290 t_h = (np.arange(144) + 0.5) / 6.0
291 fig, axes = plt.subplots(2, 1, figsize=(7, 5.6), sharex=True)
292 for ax, actual, ad, arm, ylabel in (
293     (axes[0], data["load"][d], fl_ad[d], ar.load_fc[d], "负荷 / kW"),
294     (axes[1], data["pv_act"][d], fp_ad[d], ar.pv_fc[d], "光伏 / kW"),
295 ):
296     ax.plot(t_h, actual, color="#333333", lw=1.6, label="实际")
297     ax.plot(t_h, ad, color="#4C72B0", lw=1.2, label="自适应加权")
298     ax.plot(t_h, arm, color="#C44E52", lw=1.2, ls="--", label="ARIMA")
299     ax.set_ylabel(ylabel)
300     ax.legend(ncol=3, fontsize=8)
301 axes[1].set_xlabel("时刻 / h")
302 pm.save_fig(fig, "Q2ARIMA_预测示例",
303             data=pd.DataFrame({"时刻/h": t_h,
304                               "负荷实际": data["load"][d], "负荷_E": fl_ad[d],
305                               "负荷_ARIMA": ar.load_fc[d],
306                               "光伏实际": data["pv_act"][d], "光伏_E": fp_ad[d],
307                               "光伏_ARIMA": ar.pv_fc[d]}))
308
309 log.info("对照完成：自适应 {:.1f} 万元 | ARIMA {:.1f} | D {:.1f}",
310         costs.loc[0, "总费用/万元"], costs.loc[1, "总费用/万元"],
311         costs.loc[3, "总费用/万元"])
312 return {"metrics": metrics, "costs": costs,
313        "load_fc": ar.load_fc, "pv_fc": ar.pv_fc}
314
315
316 if __name__ == "__main__":
317     run_comparison()

```

program/src/solve/experiments/q2_bias.py

```

1  """Q2 预测偏差实验：精度最优 vs 决策最优（分位数 / 费用标定 / 场景对冲）。
2
3  【历史专题·旧口径】基于"日循环 + 口径 D"结构（2026-09-12 前），未适配
4  2 日滚动 / 统一执行器口径；`q2.run_hedge` / `q2.run_mc` 的签名已在合流中变更，
5  如需复跑须先适配这两个调用。结论仅作历史引用（见 reports/ 专题文档）。
6
7  E1 三条路线（Q2 口径：0:00 计划 take-or-pay + 5 倍紧急）：
8      基准    无偏预测（口径 D）
9      偏差    光伏折扣（确定性扫描 + 最优 *；理论分位数 _q = F_e^{-1}(0.8)）
10     对冲    两阶段场景 LP（q2.run_hedge）
11  评估：确定性全年费用（2/1-12/31）+ 蒙特卡洛 50 年（共同随机数）。
12
13  运行（program/ 下）：uv run python -m solve.experiments.q2_bias
14  """
15  from __future__ import annotations
16
17  import time
18

```

```

19 import numpy as np
20 import pandas as pd
21
22 import program as pm
23 from solve import q2
24 from solve.common import E0, ROOT, T
25
26
27 def simulate_det(data: dict, delta_kw: float = 0.0) -> dict:
28     """确定性全年模拟：计划用"0:00 预报 - "（光伏折扣），储能跨日连续。"""
29     price = data["price"]
30     E = E0
31     x_all = np.zeros((q2.N_DAY, T))
32     plan_cost = 0.0
33     emerg_cost = 0.0
34     E_feb = None
35     daily = np.zeros(q2.N_DAY)
36     for d in range(q2.N_DAY):
37         pv_raw = q2._hour_to_slots(data["fc0"][d])
38         pv_fc = np.clip(pv_raw - delta_kw, 0.0, None) / 6.0
39         load_kwh = data["load"][d] / 6.0
40         x, _E, _ = q2.plan_day(price, load_kwh, pv_fc, E)
41         ex = q2.exec_day_causal(price, load_kwh, data["pv_act"][d] / 6.0, x, E)
42         x_all[d] = x
43         E = float(ex["E"][-1])
44         if d >= q2.REPORT_START:
45             c_plan = float(price @ x)
46             c_em = float(q2.EMERG_MULT * (price @ ex["e"]))
47             plan_cost += c_plan
48             emerg_cost += c_em
49             daily[d] = c_plan + c_em
50         if d == q2.REPORT_START - 1:
51             E_feb = E
52     return {"plan": plan_cost, "emerg": emerg_cost, "total": plan_cost + emerg_cost,
53           "x": x_all, "E_feb": E_feb, "daily": daily}
54
55
56 def theory_delta(data: dict) -> dict:
57     """理论分位数偏移：e = 预报 - 实际（白天样本），_q = F_e'(0.8)。"""
58     errs = []
59     for d in range(q2.N_DAY):
60         e = q2._hour_to_slots(data["fc0"][d]) - data["pv_act"][d]
61         m = data["pv_act"][d] > 500
62         if m.any():
63             errs.append(e[m])
64     e = np.concatenate(errs)
65     return {
66         "n": int(e.size),
67         "mean_kW": float(e.mean()),
68         "sigma_kW": float(e.std()),
69         "delta_q80_kW": float(np.quantile(e, 0.80)),
70         "delta_normal_kW": float(0.8416 * e.std()),
71     }
72
73
74 def main():
75     pm.init(seed=42, root=str(ROOT))
76     data = q2.load_all()
77     t0 = time.time()
78
79     base = simulate_det(data, 0.0)
80     print(f"[基准 =0] 计划 {base['plan'] / 1e4:.1f} + 紧急 {base['emerg'] / 1e4:.1f} "
81           f"={base['total'] / 1e4:.1f} 万 （自检目标 1231.1 + 155.2 = 1386.3）")
82     cache = {0.0: base}
83
84     deltas = [0, 50, 100, 150, 200, 300, 400, 500]
85     rows = []
86     for dl in deltas:
87         r = cache.get(float(dl))

```

```

88     if r is None:
89         r = simulate_det(data, float(d1))
90         cache[float(d1)] = r
91     rows.append({"delta_kW": d1, "计划费/万元": round(r["plan"] / 1e4, 1),
92                "紧急费/万元": round(r["emerg"] / 1e4, 1),
93                "总费用/万元": round(r["total"] / 1e4, 1)})
94     print(f"={d1:4d} kW: 计划 {r['plan'] / 1e4:7.1f} + 紧急 {r['emerg'] / 1e4:6.1f} "
95           f"={r['total'] / 1e4:7.1f} 万    ({time.time() - t0:.0f}s)")
96 df = pd.DataFrame(rows)
97 df.to_csv(ROOT / "code" / "outputs" / "q2_bias_delta_scan.csv", index=False, encoding="utf-8-sig")
98
99 best = int(df["总费用/万元"].idxmin())
100 d_star = float(df.loc[best, "delta_kW"])
101 print(f"\n确定性最优偏差 *={d_star:.0f} kW (费用 {df.loc[best, '总费用/万元']} 万)")
102
103 th = theory_delta(data)
104 print(f"理论分位数: e = 预报-实际 (n={th['n']}), 均值 {th['mean_kW']:+.1f} kW、"
105       f" {th['sigma_kW']:.1f} kW + _q80 = {th['delta_q80_kW']:.0f} kW (正态近似 {th['delta_normal_kW']:.0f} kW)")
106
107 # 把理论 也加入缓存
108 d_q = float(round(th["delta_q80_kW"] / 50) * 50) # 归到 50 的倍数
109 if d_q not in cache:
110     cache[d_q] = simulate_det(data, d_q)
111 print(f"理论 取整为 {d_q:.0f} kW: 费用 {cache[d_q]['total'] / 1e4:.1f} 万")
112
113 # ---- 对冲策略 ----
114 t1 = time.time()
115 xh = q2.run_hedge(data, n_scen=20, seed=7)
116 plan_h = float(sum(data["price"] @ xh[d] for d in range(q2.REPORT_START, q2.N_DAY)))
117 print(f"对冲计划完成 ({time.time() - t1:.0f}s), 计划费 {plan_h / 1e4:.1f} 万")
118
119 # ---- 蒙特卡洛 (共同随机数) ----
120 n_years = 50
121 strategies = {
122     "基准 (无偏)": (base["x"], base["E_feb"], base["plan"]),
123     f"费用偏差 *={d_star:.0f}kW": (cache[d_star]["x"], cache[d_star]["E_feb"], cache[d_star]["plan"]),
124     f"理论分位数 = {d_q:.0f}kW": (cache[d_q]["x"], cache[d_q]["E_feb"], cache[d_q]["plan"]),
125     "场景对冲": (xh, base["E_feb"], plan_h),
126 }
127 rows_mc = []
128 for name, (x, e_feb, plan) in strategies.items():
129     t2 = time.time()
130     mc, _ = q2.run_mc(data, x, e_feb, n_years=n_years, seed=42)
131     total = plan + mc
132     p95 = float(np.percentile(total, 95))
133     rows_mc.append({
134         "策略": name, "计划费/万元": round(plan / 1e4, 1),
135         "紧急费均值/万元": round(float(mc.mean()) / 1e4, 1),
136         "总费用均值/万元": round(float(total.mean()) / 1e4, 1),
137         "总费用std/万元": round(float(total.std()) / 1e4, 2),
138         "P95/万元": round(p95 / 1e4, 1),
139         "CVaR95/万元": round(float(total[total >= p95].mean()) / 1e4, 1),
140     })
141     print(f" {name:<18} 均值 {total.mean() / 1e4:7.1f} std {total.std() / 1e4:5.2f} "
142           f"P95 {p95 / 1e4:7.1f} ({time.time() - t2:.0f}s)")
143
144 df_mc = pd.DataFrame(rows_mc)
145 df_mc.to_csv(ROOT / "code" / "outputs" / "q2_bias_mc.csv", index=False, encoding="utf-8-sig")
146 print("\n==== Q2 预测目标对照 (确定性 + MC 50 年) =====")
147 print(df.to_string(index=False))
148 print()
149 print(df_mc.to_string(index=False))
150 print(f"\n总用时 {time.time() - t0:.0f}s")
151
152
153 if __name__ == "__main__":
154     main()

```

program/src/solve/experiments/q2_bias_roll.py

```

1  # -*- coding: utf-8 -*-
2  """Q2 偏差策略的滚动标定验证（无前视）：每天用过去 W 天选最优。
3
4  运行（program/ 下）：uv run python -m solve.experiments.q2_bias_roll
5  """
6  from __future__ import annotations
7
8  import numpy as np
9  import pandas as pd
10
11  import program as pm
12  from solve import q2
13  from solve.experiments import q2_bias as qb
14  from solve.common import ROOT
15
16  DELTAS = [0, 50, 100, 150, 200, 250, 300, 350, 400, 500]
17
18
19  def main():
20      pm.init(seed=42, root=str(ROOT))
21      data = q2.load_all()
22      rep = np.arange(q2.REPORT_START, q2.N_DAY)
23      tab = np.zeros((q2.N_DAY, len(DELTAS)))
24      for j, dl in enumerate(DELTAS):
25          r = qb.simulate_det(data, float(dl))
26          tab[:, j] = r["daily"]
27
28      base = tab[rep, 0].sum()
29      i300 = DELTAS.index(300)
30      fixed300 = tab[rep, i300].sum()
31      oracle = tab[rep].min(axis=1).sum()
32
33      rows = [{"策略": "基准 =0", "总费用/万元": round(base / 1e4, 1)}]
34      print(f"基准 =0          {base / 1e4:7.1f} 万")
35      print(f"固定 =300          {fixed300 / 1e4:7.1f} 万（确定性事后最优附近的代表值）")
36      print(f"离线最优（参考）    {oracle / 1e4:7.1f} 万\n")
37      rows.append({"策略": "固定 =300", "总费用/万元": round(fixed300 / 1e4, 1)})
38
39      for W in (14, 30, 60):
40          tot = 0.0
41          picks = []
42          for d in rep:
43              i0 = max(0, d - W)
44              sc = tab[i0:d].mean(axis=0)
45              j = int(np.argmin(sc))
46              tot += tab[d, j]
47              picks.append(DELTAS[j])
48          picks = np.array(picks)
49          print(f"滚动 W={W:2d}          {tot / 1e4:7.1f} 万 平均 ={picks.mean():5.0f} kW "
50                f"（选择分布：{dict(zip(*np.unique(picks, return_counts=True)))}")
51          rows.append({"策略": f"滚动 W={W}", "总费用/万元": round(tot / 1e4, 1)})
52
53      rows.append({"策略": "离线最优（参考）", "总费用/万元": round(oracle / 1e4, 1)})
54      pd.DataFrame(rows).to_csv(ROOT / "code" / "outputs" / "q2_bias_rolling.csv",
55                                index=False, encoding="utf-8-sig")
56      print("\n已保存 code/outputs/q2_bias_rolling.csv")
57
58
59  if __name__ == "__main__":
60      main()

```

program/src/solve/experiments/q2_regsrc.py

```

1  """Q2 口径 E 候选改进：回归型参考量（用户思路）——探针 + 全流程测试。
2
3  原口径：L^ = w1 * L(d-7) + w2 * L(d-14) + w3 * 典型日（可信度权重由费用标定）。
4  新思路：把前两个“来源”换成两个回归预测参考量：
5  参考量1（周期型）：由 y(d-28), y(d-21), y(d-14), y(d-7) 做线性回归，预测 d；
6  参考量2（连续型）：由 y(d-4), y(d-3), y(d-2), y(d-1) 做线性回归，预测 d；

```

```

7  之后仍按三源（参考量1、参考量2、典型日）可信度权重组合，光伏同样处理。
8
9  两种"线性回归"实现：
10  T1 趋势外推：4 点等间隔拟合直线并外推一步（闭式权重）；
11  T2 滚动多元回归：4 个滞后为特征，最近 60 天同槽 OLS，逐日重估（无前视）。
12
13  模式：
14  uv run python -m solve.experiments.q2_regrsrc          # 探针：精度 + 固定权重费用
15  uv run python -m solve.experiments.q2_regrsrc --full   # 全流程：甲（严格）/乙（光伏适配）各建表 + 扫描
16  """
17  from __future__ import annotations
18
19  import hashlib
20  import os
21  import sys
22  import time
23
24  import numpy as np
25  import pandas as pd
26
27  import program as pm
28  from solve import q2
29  from solve.common import E0, ROOT
30  from solve.models.adaptive import AdaptiveWeightModel
31
32  REP = list(range(q2.REPORT_START, q2.N_DAY))
33  LAG_W = (28, 21, 14, 7)  # 周期型滞后（相隔 7 天）
34  LAG_C = (4, 3, 2, 1)    # 连续型滞后
35  LAG_P2 = (8, 6, 4, 2)   # 光伏备选：相隔 2 天
36
37
38  # -----
39  # 预测器
40  # -----
41  def extrap_coef(n: int) -> np.ndarray:
42      """n 个等间隔点（坐标 -n..-1）拟合直线并外推至 0 的线性权重。"""
43      xs = -np.arange(n, 0, -1, dtype=float)
44      A = np.stack([np.ones(n), xs], axis=1)
45      return A @ np.linalg.inv(A.T @ A)[: , 0]
46
47
48  def t1_predict(series: np.ndarray, lags, d: int):
49      pts = [(x, series[d - lag]) for x, lag in zip(-np.arange(len(lags), 0, -1), lags)
50             if d - lag >= 0]
51      if len(pts) < 2:
52          return None
53      c = extrap_coef(len(pts))
54      return np.tensordot(c, np.stack([y for _, y in pts]), axes=(0, 0))
55
56
57  def t2_predict(series: np.ndarray, lags, d: int, Wr: int = 60):
58      """同槽滚动 OLS：特征 = 4 个滞后值，样本 = 最近 Wr 天（k d-1）。"""
59      ks = np.arange(max(0, d - Wr), d)
60      ks = ks[ks >= max(lags)]
61      n = len(ks)
62      if n < 8:
63          return None
64      Y = series[ks]  # (n,144)
65      X = np.stack([series[ks - lag] for lag in lags], axis=1)  # (n,4,144)
66      Xd = np.concatenate([np.ones((n, 1, q2.T)), X], axis=1)  # (n,5,144)
67      XtX = np.einsum("nij,nkj->ikj", Xd, Xd) + 1e-8 * np.eye(5)[: , :, None]
68      XtY = np.einsum("nij,nj->ij", Xd, Y)
69      beta = np.linalg.solve(XtX.transpose(2, 0, 1), XtY.T[... , None])[... , 0]
70      x0 = np.stack([series[d - lag] for lag in lags], axis=1)  # (144,4)
71      return beta[: , 0] + np.einsum("sj,sj->s", beta[: , 1:], x0)
72
73
74  def build_sources(L: np.ndarray, P: np.ndarray, d_from: int = 14, t2_wr: int = 60) -> dict:
75      """为标表所需的所有天构建 10 个参考量矩阵（早期样本不足退化为趋势外推）。"""

```

```

76     keys = ("L_T1w", "L_T1c", "L_T2w", "L_T2c",
77            "P_T1w", "P_T1c", "P_T2w", "P_T2c", "P_T1p2", "P_T2p2")
78     src = {k: np.full((q2.N_DAY, q2.T), np.nan) for k in keys}
79     for d in range(d_from, q2.N_DAY):
80         for key, series, lags in (("L_T1w", L, LAG_W), ("L_T1c", L, LAG_C),
81                                ("P_T1w", P, LAG_W), ("P_T1c", P, LAG_C),
82                                ("P_T1p2", P, LAG_P2)):
83             v = t1_predict(series, lags, d)
84             if v is not None:
85                 src[key][d] = np.clip(v, 0.0, None)
86         for key, series, lags in (("L_T2w", L, LAG_W), ("L_T2c", L, LAG_C),
87                                ("P_T2w", P, LAG_W), ("P_T2c", P, LAG_C),
88                                ("P_T2p2", P, LAG_P2)):
89             v = t2_predict(series, lags, d, Wr=t2_wr)
90             fallback = src[key.replace("T2", "T1")][d]
91             if v is not None:
92                 src[key][d] = np.clip(v, 0.0, None)
93             elif not np.isnan(fallback).any():
94                 src[key][d] = fallback
95     return src
96
97
98 def metrics(y, yhat) -> tuple[float, float]:
99     e = yhat - y
100     return float(np.abs(e).mean()), float(np.sqrt((e ** 2).mean()))
101
102
103 def shift(arr: np.ndarray, k: int) -> np.ndarray:
104     out = np.zeros_like(arr)
105     out[k:] = arr[:-k]
106     return out
107
108
109 # -----
110 # 回归源模型（口径 E 变体：两个回归参考量 + 典型日）
111 # -----
112 class RegSourceModel(AdaptiveWeightModel):
113     """把 forecast 换成"两个回归参考量 + 典型日"的口径 E 变体（标定框架不变）。"""
114
115     CACHE_TAG = "q2e_regsrc" # 与官方 q2e 缓存隔离（q3_proto 只认 tag=q2e 的表）
116
117     def __init__(self, sources, name: str = "regsrc", **kwargs):
118         super().__init__(**kwargs)
119         self.S1L, self.S2L, self.S1P, self.S2P = sources
120         self.NAME = name
121
122     def forecast_kw(self, w, u, d: int):
123         l = w[0] * self.S1L[d] + w[1] * self.S2L[d] + w[2] * self.L_typ
124         p = u[0] * self.S1P[d] + u[1] * self.S2P[d] + u[2] * self.P_typ
125         return l, p
126
127     def forecast(self, w, u, d: int):
128         l, p = self.forecast_kw(w, u, d)
129         return np.clip(l, 0.0, None) / 6.0, np.clip(p, 0.0, None) / 6.0
130
131     def _cache_key(self) -> str:
132         h = hashlib.md5()
133         for arr in (self.S1L, self.S2L, self.S1P, self.S2P, self.L_typ, self.P_typ):
134             h.update(np.ascontiguousarray(arr, dtype=float).tobytes())
135         h.update(np.asarray([self.grid_step, self.steps, self.lr, self.eps_grad], float).tobytes())
136         h.update(q2.CAUSAL_POLICY_VERSION.encode("utf-8"))
137         h.update(self.NAME.encode("utf-8"))
138         return h.hexdigest()[:12]
139
140     def _one_day(self, d: int):
141         """只算网格行（本变体不用 GD 精化，省 1/4 建表时间）。"""
142         n = len(self.grid)
143         row = np.empty((n, n))
144         for i, w in enumerate(self.grid):

```

```

145         for j, u in enumerate(self.grid):
146             row[i, j] = self.day_cost(d, w, u)
147         c = float(row.min())
148         return d, row, np.zeros(6), c, c
149
150
151 # -----
152 # 并行评估 (含裕度)
153 # -----
154 _W = None
155
156
157 def _w_init(payload):
158     global _W
159     _W = payload
160
161
162 def _w_task(t):
163     ci, si, mi, d = t
164     p = _W
165     w, u = p["seqs"][si][d - q2.REPORT_START]
166     l = w[0] * p["S1L"][d] + w[1] * p["S2L"][d] + w[2] * p["L_typ"]
167     pp = u[0] * p["S1P"][d] + u[1] * p["S2P"][d] + u[2] * p["P_typ"]
168     spec = p["margins"][mi]
169     if spec is not None:
170         kind = spec["kind"]
171         if kind == "pv_global":
172             pp = pp - spec["m"]
173         elif kind == "load_kappa":
174             l = l * spec["k"]
175         elif kind == "combo":
176             l = l * spec["k"]
177             pp = pp - spec["m"]
178     l = np.clip(l, 0.0, None) / 6.0
179     pp = np.clip(pp, 0.0, None) / 6.0
180     x, _E, _ = q2.plan_day(p["price"], l, pp, E0, eps=1e-3)
181     ex = q2.exec_day_causal(p["price"], p["L"][d] / 6.0, p["P"][d] / 6.0, x, E0)
182     return ci, d - q2.REPORT_START, float(p["price"] @ x), float(q2.EMERG_MULT * (p["price"] @ ex["e"]))
183
184
185 def eval_configs(model: RegSourceModel, configs, workers: int | None = None) -> pd.DataFrame:
186     """configs: list of (名称, 权重序列, 裕度 spec); 返回年度费用表。"""
187     seqs, sid, margins, mid = [], {}, [], {}
188     for name, seq, spec in configs:
189         if id(seq) not in sid:
190             sid[id(seq)] = len(seqs)
191             seqs.append(seq)
192         key = -1 if spec is None else id(spec)
193         if key not in mid:
194             mid[key] = len(margins)
195             margins.append(spec)
196     tasks = [(ci, sid[id(seq)], mid[-1 if spec is None else id(spec)], d)
197              for ci, (_, seq, spec) in enumerate(configs) for d in REP]
198     payload = {"seqs": seqs, "margins": margins,
199               "S1L": model.S1L, "S2L": model.S2L, "S1P": model.S1P, "S2P": model.S2P,
200               "L_typ": model.L_typ, "P_typ": model.P_typ,
201               "price": model.price, "L": model.L, "P": model.P}
202     n = len(configs)
203     plan = np.zeros(n)
204     emerg = np.zeros(n)
205     nw = workers or min(8, os.cpu_count() or 1)
206     t0 = time.time()
207     if nw > 1:
208         from multiprocessing import Pool
209
210         with Pool(nw, initializer=_w_init, initargs=(payload,)) as pool:
211             for ci, di, c_plan, c_em in pool.imap_unordered(_w_task, tasks, chunksize=64):
212                 plan[ci] += c_plan
213                 emerg[ci] += c_em

```

```

214     else:
215         _w_init(payload)
216         for t in tasks:
217             ci, di, c_plan, c_em = _w_task(t)
218             plan[ci] += c_plan
219             emerg[ci] += c_em
220     df = pd.DataFrame({
221         "配置": [c[0] for c in configs],
222         "计划购电费/万元": np.round(plan / 1e4, 1),
223         "紧急购电费/万元": np.round(emerg / 1e4, 1),
224     })
225     df["总费用/万元"] = (df["计划购电费/万元"] + df["紧急购电费/万元"]).round(1)
226     print(f"  批量评估 {n} 个配置完成 ({time.time() - t0:.0f}s)")
227     return df
228
229
230 # -----
231 # 探针
232 # -----
233 def run_probe(model: AdaptiveWeightModel):
234     L, P = model.L, model.P
235     t0 = time.time()
236     src = build_sources(L, P, d_from=q2.REPORT_START)
237     print(f"预测器构造完成 ({time.time() - t0:.0f}s)")
238
239     model.build_table()
240     ws = model.weights(W=7, use_gd=False)
241     E_L = np.full((q2.N_DAY, q2.T), np.nan)
242     E_P = np.full((q2.N_DAY, q2.T), np.nan)
243     for i, d in enumerate(REP):
244         w, u = ws[i]
245         l_kwh, p_kwh = model.forecast(w, u, d)
246         E_L[d], E_P[d] = l_kwh * 6.0, p_kwh * 6.0
247
248     rows = []
249     for name, series, typ in (("负荷", L, model.L_typ), ("光伏", P, model.P_typ)):
250         y = np.stack([series[d] for d in REP])
251         base = {"原始 d-7": series[np.array(REP) - 7],
252             "原始 d-14": series[np.array(REP) - 14],
253             "原始 d-1": series[np.array(REP) - 1],
254             "原始 d-2": series[np.array(REP) - 2],
255             "典型日": np.stack([typ for _ in REP])}
256         preds = dict(base)
257         if name == "负荷":
258             preds.update({k: np.stack([src[k][d] for d in REP]) for k in
259                 ("L_T1w", "L_T1c", "L_T2w", "L_T2c")})
260             preds["口径E(W=7)"] = np.stack([E_L[d] for d in REP])
261         else:
262             preds.update({k: np.stack([src[k][d] for d in REP]) for k in
263                 ("P_T1w", "P_T1c", "P_T2w", "P_T2c", "P_T1p2", "P_T2p2")})
264             preds["口径E(W=7)"] = np.stack([E_P[d] for d in REP])
265         for k, v in preds.items():
266             mae, rmse = metrics(y, v)
267             rows.append({"变量": name, "预测器": k, "MAE/kW": round(mae, 1),
268                 "RMSE/kW": round(rmse, 1)})
269     acc = pd.DataFrame(rows)
270     pm.save_outputs(acc, "q2_regsrc_probe")
271     print(acc.to_string(index=False))
272
273 def cost_of(load_src, pv_src, w, u):
274     plan = emerg = 0.0
275     for d in REP:
276         l = np.clip(w[0] * load_src[0][d] + w[1] * load_src[1][d]
277             + w[2] * load_src[2][d], 0.0, None) / 6.0
278         p = np.clip(u[0] * pv_src[0][d] + u[1] * pv_src[1][d]
279             + u[2] * pv_src[2][d], 0.0, None) / 6.0
280         x, _E, _ = q2.plan_day(model.price, l, p, E0, eps=1e-3)
281         ex = q2.exec_day_causal(model.price, L[d] / 6.0, P[d] / 6.0, x, E0)
282         plan += float(model.price @ x)

```

```

283     emerg += float(q2.EMERG_MULT * (model.price @ ex["e"]))
284     return plan, emerg
285
286 typL = np.stack([model.L_typ] * q2.N_DAY)
287 typP = np.stack([model.P_typ] * q2.N_DAY)
288 L7, L14 = shift(L, 7), shift(L, 14)
289 P1, P2 = shift(P, 1), shift(P, 2)
290 cases = [
291     ("老口径 等权(L_{d-7},L_{d-14},典型)", (L7, L14, typL), (P1, P2, typP),
292      (1 / 3, 1 / 3, 1 / 3)),
293     ("新T1 等权(周期,连续,典型)", (src["L_T1w"], src["L_T1c"], typL),
294      (src["P_T1w"], src["P_T1c"], typP), (1 / 3, 1 / 3, 1 / 3)),
295     ("新T2 等权(周期,连续,典型)", (src["L_T2w"], src["L_T2c"], typL),
296      (src["P_T2w"], src["P_T2c"], typP), (1 / 3, 1 / 3, 1 / 3)),
297     ("新T2 (0.4,0.4,0.2)", (src["L_T2w"], src["L_T2c"], typL),
298      (src["P_T2w"], src["P_T2c"], typP), (0.4, 0.4, 0.2)),
299 ]
300 rows = []
301 for name, load_src, pv_src, w in cases:
302     t1 = time.time()
303     plan, emerg = cost_of(load_src, pv_src, w, w)
304     tot = (plan + emerg) / 1e4
305     rows.append({"配置": name, "计划费/万元": round(plan / 1e4, 1),
306                "紧急费/万元": round(emerg / 1e4, 1), "总费用/万元": round(tot, 1)})
307     print(f" {name}: {tot:.1f} 万 ({time.time() - t1:.0f}s) ")
308 cost = pd.DataFrame(rows)
309 pm.save_outputs(cost, "q2_regsrc_cost_probe")
310 return acc, cost
311
312
313 # -----
314 # 全流程 (甲/乙)
315 # -----
316 def run_variant(name: str, sources, workers: int | None = None):
317     from solve.q2_tune import seqs_ewma
318
319     print(f"\n===== 变体 {name}: 建表 =====")
320     model = RegSourceModel(sources, name=name).load()
321     model.build_table()
322     t0 = time.time()
323
324     cfgs = []
325     for W in (1, 3, 5, 7, 10, 14, 21, 30, 45, 60, 90, 120):
326         cfgs.append((f"{name} W={W}", model.weights(W=W, use_gd=False), None))
327     for hl in (1, 2, 3, 5, 7, 10, 14, 21, 30):
328         cfgs.append((f"{name} EWMA h={hl}", seqs_ewma(model, hl), None))
329     df1 = eval_configs(model, cfgs, workers)
330     df1.to_csv(pm.outputs_dir() / f"q2_regsrc_{name}_weights.csv",
331               index=False, encoding="utf-8-sig")
332     print(df1.sort_values("总费用/万元").head(5).to_string(index=False))
333
334     best = df1.sort_values("总费用/万元").iloc[0]
335     best_name = best["配置"]
336     best_seq = next(seq for nm, seq, _ in cfgs if nm == best_name)
337     print(f"变体 {name} 权重最优: {best_name} = {best['总费用/万元']} 万元")
338
339     cfgs2 = []
340     for mv in (50, 100, 150, 200, 250, 300):
341         cfgs2.append((f"{best_name} m={mv}", best_seq, {"kind": "pv_global", "m": float(mv)}))
342     for k in (1.005, 1.01, 1.015, 1.02, 1.025, 1.03, 1.035, 1.04, 1.045, 1.05):
343         cfgs2.append((f"{best_name} k={k}", best_seq, {"kind": "load_kappa", "k": k}))
344     for k in (1.01, 1.015, 1.02, 1.025, 1.03):
345         for mv in (25.0, 50.0, 75.0, 100.0):
346             cfgs2.append((f"{best_name} k={k}+m={mv:.0f}", best_seq,
347                           {"kind": "combo", "k": k, "m": mv}))
348     df2 = eval_configs(model, cfgs2, workers)
349     df2.to_csv(pm.outputs_dir() / f"q2_regsrc_{name}_margin.csv",
350               index=False, encoding="utf-8-sig")
351     best2 = df2.sort_values("总费用/万元").iloc[0]

```

```

352     print(f"变体 {name} 含裕度最优: {best2['配置']} = {best2['总费用/万元']} 万元"
353           f" ({time.time() - t0:.0f}s) ")
354     return df1, df2, best2
355
356
357 def run_full(workers: int | None = None, only: list[str] | None = None,
358             tag: str | None = None):
359     pm.init(seed=42, root=str(ROOT))
360     log = pm.get_logger("q2-regsrc")
361     t0 = time.time()
362     base = AdaptiveWeightModel().load()
363     src60 = build_sources(base.L, base.P, 14, t2_wr=60)
364     src120 = build_sources(base.L, base.P, 14, t2_wr=120)
365     for s in (src60, src120):
366         s["L7"], s["L14"] = shift(base.L, 7), shift(base.L, 14)
367     variants = {
368         "甲": (src60, {"L1": "L_T2w", "L2": "L_T2c", "P1": "P_T2w", "P2": "P_T2c"}),
369         "乙": (src60, {"L1": "L_T2w", "L2": "L_T2c", "P1": "P_T2c", "P2": "P_T2p2"}),
370         "丙": (src60, {"L1": "L7", "L2": "L14", "P1": "P_T2c", "P2": "P_T2p2"}),
371         "丁": (src60, {"L1": "L7", "L2": "L14", "P1": "P_T2c", "P2": "P_T2w"}),
372         "丙Wr120": (src120, {"L1": "L7", "L2": "L14", "P1": "P_T2c", "P2": "P_T2p2"}),
373     }
374     if only:
375         variants = {k: v for k, v in variants.items() if k in only}
376     summary = []
377     for name, (s, sel) in variants.items():
378         sources = (s[sel["L1"]], s[sel["L2"]], s[sel["P1"]], s[sel["P2"]])
379         _, _, best2 = run_variant(name, sources, workers)
380         summary.append({"变体": name, "最优配置": best2["配置"],
381                        "计划费/万元": best2["计划购电费/万元"],
382                        "紧急费/万元": best2["紧急购电费/万元"],
383                        "总费用/万元": best2["总费用/万元"],
384                        "相对官方1467.4/%": round(100 * (best2["总费用/万元"] / 1467.4 - 1), 2),
385                        "相对旧最优1406.6/%": round(100 * (best2["总费用/万元"] / 1406.6 - 1), 2)})
386     out = pd.DataFrame(summary)
387     suffix = f"_{tag}" if tag else ""
388     pm.save_outputs(out, f"q2-regsrc_full_summary{suffix}")
389     print("\n==== 汇总 =====")
390     print(out.to_string(index=False))
391     log.info("全流程完成 ({:.0f}s) ", time.time() - t0)
392     return out
393
394
395 if __name__ == "__main__":
396     if "--wr" in sys.argv:
397         run_full(only=["丙", "丙Wr120"], tag="wr")
398     elif "--full" in sys.argv:
399         run_full()
400     else:
401         pm.init(seed=42, root=str(ROOT))
402         _m = AdaptiveWeightModel().load()
403         run_probe(_m)

```

program/src/solve/experiments/q2e_smooth.py

```

1  """Q2 口径 E 权重平滑回溯：新旧权重混合（ ）对年度费用的影响。
2
3  平滑：权重_D = 权重_new_D + (1- ) * 权重_{D-1}（凸组合，负荷 w 与光伏 u 可分别控制）
4  对比基准（因果口径）：W=7 网格（官方 result2, 1467.4 万）、W=1+梯度（1515.4 万）。
5
6  运行（program/ 下）：uv run python -m solve.experiments.q2e_smooth
7  """
8  from __future__ import annotations
9
10 import time
11
12 import numpy as np
13 import pandas as pd
14

```

```

15 import program as pm
16 from solve import q2
17 from solve.common import ROOT
18 from solve.models.adaptive import AdaptiveWeightModel
19 from solve.models.weights import softmax
20
21
22 def smoothed_from_table(model, W, beta, smooth_load=True, smooth_pv=True):
23     """平滑"网格选择"权重序列（无前视）。"""
24     grid = model.grid
25     out = []
26     prev_w = prev_u = None
27     for d in range(q2.REPORT_START, q2.N_DAY):
28         lo = max(model.START, d - W)
29         i, j = np.unravel_index(np.argmin(model.C[lo:d].mean(axis=0)),
30                                (len(grid), len(grid)))
31         w_new, u_new = grid[i].copy(), grid[j].copy()
32         if prev_w is None:
33             prev_w, prev_u = w_new, u_new
34         else:
35             if smooth_load:
36                 w_new = beta * w_new + (1 - beta) * prev_w
37             if smooth_pv:
38                 u_new = beta * u_new + (1 - beta) * prev_u
39             prev_w, prev_u = w_new, u_new
40         out.append((w_new.copy(), u_new.copy()))
41     return out
42
43
44 def smoothed_from_th(model, beta, smooth_load=True, smooth_pv=True):
45     """平滑 W=1 梯度精化权重（TH）序列（无前视）。"""
46     out = []
47     prev_w = prev_u = None
48     for d in range(q2.REPORT_START, q2.N_DAY):
49         theta = model.TH[d - 1]
50         w_new, u_new = softmax(theta[:3]), softmax(theta[3:])
51         if prev_w is None:
52             prev_w, prev_u = w_new, u_new
53         else:
54             if smooth_load:
55                 w_new = beta * w_new + (1 - beta) * prev_w
56             if smooth_pv:
57                 u_new = beta * u_new + (1 - beta) * prev_u
58             prev_w, prev_u = w_new, u_new
59         out.append((w_new.copy(), u_new.copy()))
60     return out
61
62
63 def cost_of(model, weights):
64     df = model.simulate(weights)
65     plan = float(df["计划购电费/元"].sum())
66     em = float(df["紧急购电费/元"].sum())
67     return plan, em
68
69
70 def main():
71     pm.init(seed=42, root=str(ROOT))
72     model = AdaptiveWeightModel().load().build_table()
73     rows = []
74     t0 = time.time()
75
76     plan17, em17 = cost_of(model, model.weights(W=7, use_gd=False))
77     plan11, em11 = cost_of(model, model.weights(W=1, use_gd=True))
78     base7 = plan17 + em17
79     print(f"基准 W=7 网格      : {base7 / 1e4:.1f} 万 (计划 {plan17 / 1e4:.1f} + 紧急 {em17 / 1e4:.1f}) ")
80     print(f"基准 W=1 网格+梯度 : {(plan11 + em11) / 1e4:.1f} 万 (计划 {plan11 / 1e4:.1f} + 紧急 {em11 / 1e4:.1f}) ")
81     rows += [{"配置": "基准 W=7", "总费用/万元": round(base7 / 1e4, 1)},
82             {"配置": "基准 W=1+梯度", "总费用/万元": round((plan11 + em11) / 1e4, 1)}]
83

```

```

84     print("\n网格权重平滑 (W=7 基准): ")
85     for beta in (0.1, 0.2, 0.3, 0.5):
86         for only_pv in (False, True):
87             ws = smoothed_from_table(model, 7, beta, smooth_load=not only_pv)
88             plan, em = cost_of(model, ws)
89             tot = plan + em
90             tag = f"W=7 平滑 = {beta} ({'仅光伏' if only_pv else '负荷+光伏'}) "
91             print(f" {tag:<28} {tot / 1e4:7.1f} 万"
92                   f" (计划 {plan / 1e4:.1f} + 紧急 {em / 1e4:.1f}, vs W=7 {100 * (tot / base7 - 1):+.2f}%) ")
93             rows.append({"配置": tag, "总费用/万元": round(tot / 1e4, 1)})
94
95     print("\n梯度权重平滑 (W=1+梯度 基准): ")
96     for beta in (0.1, 0.3):
97         ws = smoothed_from_th(model, beta)
98         plan, em = cost_of(model, ws)
99         tot = plan + em
100        tag = f"W=1+梯度 平滑 = {beta}"
101        print(f" {tag:<28} {tot / 1e4:7.1f} 万"
102              f" (计划 {plan / 1e4:.1f} + 紧急 {em / 1e4:.1f}) ")
103        rows.append({"配置": tag, "总费用/万元": round(tot / 1e4, 1)})
104
105    pd.DataFrame(rows).to_csv(ROOT / "code" / "outputs" / "q2e_smooth_compare.csv",
106                              index=False, encoding="utf-8-sig")
107    print(f"\n已保存 code/outputs/q2e_smooth_compare.csv (用时 {time.time() - t0:.0f}s) ")
108
109
110 if __name__ == "__main__":
111     main()

```

program/src/solve/experiments/q2e_structure.py

```

1  """Q2 口径 E: 误差分布结构分析与参数参考值推导 (先验版)。
2
3  对官方口径 (历史自适应加权 W=7 网格选择) 在报送期 2025-02-01 ~ 12-31 的
4  预测误差做分布结构分析, 把模型参数与数据分布挂钩:
5
6  1. 净需求偏差裕度 m_ref (对应"决策最优预测"的报童临界比 0.8):
7      m_ref = F_{e}^{-1}(0.8), e = 实际净需求 - 预测净需求;
8      给出全局 / 逐时段 / 逐小时 / 逐月 / 比例式 (e 相对预报光伏) 五种口径;
9  2. 标定窗口 W 与 EWMA 半衰期 h_ref: 由日度偏差序列的 AR(1) 记忆推得;
10 3. 风险参考: 官方计划的日费用尾部分布 (P95/CVaR95), 为风险厌恶系数 扫描定标。
11
12 输出: code/outputs/q2e_structure_{slots,hours,months}.csv、q2e_structure_summary.json
13 图: figures/Q2E_误差结构与参考值.pdf
14 运行 (在 program/ 下): uv run python -m solve.experiments.q2e_structure
15 """
16 from __future__ import annotations
17
18 import json
19 import time
20
21 import numpy as np
22 import pandas as pd
23 from scipy import stats
24
25 import program as pm
26 from solve import q2
27 from solve.common import ROOT
28 from solve.io.report import record
29 from solve.models.adaptive import AdaptiveWeightModel
30
31
32 def acf(x: np.ndarray, max_lag: int) -> np.ndarray:
33     """样本自相关函数 (0..max_lag)。"""
34     x = np.asarray(x, float)
35     x = x - x.mean()
36     out = [1.0]
37     for k in range(1, max_lag + 1):
38         out.append(float(np.corrcoef(x[:-k], x[k:])[0, 1]))

```

```

39     return np.array(out)
40
41
42 def ari_half_life(x: np.ndarray) -> tuple[float, float]:
43     """AR(1) 系数与半衰期 (天); 系数 0 时半衰期记 NaN。"""
44     x = np.asarray(x, float)
45     rho = float(np.corrcoef(x[:-1], x[1:])[0, 1])
46     hl = float(np.log(0.5) / np.log(rho)) if 0.0 < rho < 1.0 else float("nan")
47     return rho, hl
48
49
50 def main() -> dict:
51     pm.init(seed=42, root=str(ROOT))
52     log = pm.get_logger("q2e-structure")
53     t0 = time.time()
54     model = AdaptiveWeightModel().load().build_table()
55     months = model.data["months"]
56
57     rep = list(range(q2.REPORT_START, q2.N_DAY))
58     ws = model.weights(W=7, use_gd=False) # 官方 W=7 网格序列
59
60     n = len(rep)
61     e_net = np.zeros((n, q2.T)) # 实际净需求 - 预测净需求 (kW)
62     e_pv = np.zeros((n, q2.T)) # 预报光伏 - 实际光伏 (kW)
63     e_load = np.zeros((n, q2.T)) # 预报负荷 - 实际负荷 (kW)
64     p_hat = np.zeros((n, q2.T))
65     for i, d in enumerate(rep):
66         w, u = ws[i]
67         l, p = model.forecast_kw(w, u, d)
68         p_hat[i] = np.clip(p, 0.0, None)
69         e_load[i] = l - model.L[d]
70         e_pv[i] = p - model.P[d]
71         e_net[i] = (model.L[d] - model.P[d]) - (l - p)
72
73     # ---- 分布形状 (全样本 / 白天样本) ----
74     day_mask = p_hat > 500.0 # 光照样本 (按预报出力定义)
75     x_all = e_net.ravel()
76     x_day = e_net[day_mask]
77     shape = {
78         "样本数 (全时段)": int(x_all.size),
79         "样本数 (白天)": int(x_day.size),
80         "均值/kW (全时段)": float(x_all.mean()),
81         "标准差/kW (全时段)": float(x_all.std()),
82         "偏度 (全时段)": float(stats.skew(x_all)),
83         "峰度 (全时段)": float(stats.kurtosis(x_all)),
84         "均值/kW (白天)": float(x_day.mean()),
85         "标准差/kW (白天)": float(x_day.std()),
86         "偏度 (白天)": float(stats.skew(x_day)),
87     }
88     qq = {f"q{int(q * 100)}": float(np.quantile(x_day, q))
89           for q in (0.5, 0.6, 0.7, 0.75, 0.8, 0.85, 0.9, 0.95)}
90
91     # ---- 裕度参考值 ----
92     m_global_all = float(np.quantile(x_all, 0.80))
93     m_global_day = float(np.quantile(x_day, 0.80))
94     m_slot = np.quantile(e_net, 0.80, axis=0) # 144
95     m_hour = np.array([np.quantile(e_net[:, 6 * h:6 * (h + 1)], 0.80)
96                        for h in range(24)])
97     m_month = np.array([
98         np.quantile(e_net[months[rep] == mm], 0.80) if (months[rep] == mm).any() else np.nan
99         for mm in range(1, 13)
100     ])
101     # 比例式: 白天 e_net / 预报光伏 的 0.8 分位
102     ratio = (e_net[day_mask] / p_hat[day_mask])
103     rho_ref = float(np.quantile(ratio, 0.80))
104     # 条件结构: 按预报出力分桶的 q80 (检查裕度是否随出力增大)
105     buckets = [(500, 2000), (2000, 3500), (3500, 5000), (5000, 8000)]
106     bucket_rows = []
107     for lo, hi in buckets:

```

```

108     mask = (p_hat > lo) & (p_hat <= hi)
109     if mask.any():
110         bucket_rows.append({
111             "预报光伏区间/kW": f"({lo},{hi})",
112             "样本数": int(mask.sum()),
113             "q80/kW": float(np.quantile(e_net[mask], 0.80)),
114             "均值/kW": float(e_net[mask].mean()),
115         })
116
117 # ---- 时间记忆: 日度偏差的 AR(1) 与 ACF ----
118 b_net = e_net.mean(axis=1) # 日度净需求偏差
119 with np.errstate(invalid="ignore"):
120     day_bias = np.array([
121         e_net[i][day_mask[i]].mean() if day_mask[i].any() else np.nan
122         for i in range(n)
123     ])
124 good = ~np.isnan(day_bias)
125 rho_net, hl_net = ari_half_life(b_net)
126 rho_day, hl_day = ari_half_life(day_bias[good])
127 u1 = np.array([w[0] for w, _ in ws]) # 光伏 d-1 权重序列
128 rho_u1, hl_u1 = ari_half_life(u1)
129 acf_net = acf(b_net, 30)
130 acf_u1 = acf(u1, 30)
131
132 # ---- 风险: 官方计划日费用尾部分布 ----
133 df_official = model.simulate(ws)
134 daily_total = (df_official["计划购电费/元"] + df_official["紧急购电费/元"]).to_numpy()
135 p95 = float(np.percentile(daily_total, 95))
136 risk = {
137     "日费用均值/元": float(daily_total.mean()),
138     "日费用标准差/元": float(daily_total.std()),
139     "日费用P95/元": p95,
140     "日费用CVaR95/元": float(daily_total[daily_total >= p95].mean()),
141     "尾部溢价(P95-均值)/均值": float((p95 - daily_total.mean()) / daily_total.mean()),
142     "日费用偏度": float(stats.skew(daily_total)),
143     "有紧急购电天数占比": float((df_official["紧急购电量/kWh"] > 1e-3).mean()),
144 }
145
146 # ---- 导出 ----
147 pd.DataFrame({
148     "槽序号": np.arange(1, q2.T + 1),
149     "e_net_均值/kW": e_net.mean(axis=0),
150     "e_net_标准差/kW": e_net.std(axis=0),
151     "e_net_q80/kW": m_slot,
152 }).to_csv(pm.outputs_dir() / "q2e_structure_slots.csv", index=False, encoding="utf-8-sig")
153 pd.DataFrame({
154     "小时": np.arange(1, 25),
155     "e_net_均值/kW": [e_net[:, 6 * h:6 * (h + 1)].mean() for h in range(24)],
156     "e_net_标准差/kW": [e_net[:, 6 * h:6 * (h + 1)].std() for h in range(24)],
157     "e_net_q80/kW": m_hour,
158 }).to_csv(pm.outputs_dir() / "q2e_structure_hours.csv", index=False, encoding="utf-8-sig")
159 pd.DataFrame({
160     "月": np.arange(1, 13),
161     "e_net_均值/kW": [
162         e_net[months[rep] == mm].mean() if (months[rep] == mm).any() else np.nan
163         for mm in range(1, 13)],
164     "e_net_q80/kW": m_month,
165 }).to_csv(pm.outputs_dir() / "q2e_structure_months.csv", index=False, encoding="utf-8-sig")
166 pd.DataFrame({
167     "滞后/天": np.arange(31),
168     "日度净需求偏差ACF": acf_net,
169     "光伏d-1权重ACF": acf_u1,
170 }).to_csv(pm.outputs_dir() / "q2e_structure_acf.csv", index=False, encoding="utf-8-sig")
171
172 summary = {
173     **shape,
174     **qq,
175     "m80_全时段/kW": m_global_all,
176     "m80_白天/kW": m_global_day,

```

```

177     "m80_逐时段范围/kW": [float(m_slot.min()), float(m_slot.max())],
178     "m80_逐小时范围/kW": [float(m_hour.min()), float(m_hour.max())],
179     "m80_逐月范围/kW": [float(np.nanmin(m_month)), float(np.nanmax(m_month))],
180     "rho_ref_比例式": rho_ref,
181     "日度偏差AR1系数": rho_net,
182     "日度偏差半衰期/天": hl_net,
183     "白天偏差AR1系数": rho_day,
184     "白天偏差半衰期/天": hl_day,
185     "光伏d-1权重AR1系数": rho_u1,
186     "光伏权重半衰期/天": hl_u1,
187     **risk,
188 }
189 (pm.outputs_dir() / "q2e_structure_summary.json").write_text(
190     json.dumps(summary, ensure_ascii=False, indent=2), encoding="utf-8")
191
192 # ---- 图：四联 ----
193 import matplotlib.pyplot as plt
194
195 fig, axes = plt.subplots(2, 2, figsize=(7.2, 5.2))
196 ax = axes[0, 0]
197 ax.hist(x_day, bins=60, color="#4C72B0", alpha=0.85)
198 ax.axvline(m_global_day, color="#C44E52", ls="--",
199           label=f"白天 q80 = {m_global_day:.0f} kW")
200 ax.set_xlabel("净需求偏差 e / kW")
201 ax.set_ylabel("频数")
202 ax.set_title("(a) 偏差分布 (白天)", fontsize=9)
203 ax.legend(fontsize=8)
204
205 ax = axes[0, 1]
206 tt = np.arange(q2.T) / 6.0
207 ax.plot(tt, m_slot, color="#4C72B0", lw=1.0, label="逐时段 q80")
208 ax.step(np.arange(1, 25), m_hour, where="mid", color="#C44E52",
209        lw=1.4, label="逐小时 q80")
210 ax.axhline(m_global_all, color="#55A868", ls=":", lw=1.2,
211          label=f"全局 q80 = {m_global_all:.0f} kW")
212 ax.set_xlabel("时刻 / h")
213 ax.set_ylabel("裕度参考 / kW")
214 ax.set_title("(b) 分位数裕度曲线", fontsize=9)
215 ax.legend(fontsize=8)
216
217 ax = axes[1, 0]
218 lags = np.arange(31)
219 ax.bar(lags, acf_net, color="#4C72B0", alpha=0.85, label="日度净需求偏差")
220 ax.plot(lags, acf_u1, color="#C44E52", marker="o", ms=2.5, lw=1.0,
221        label="光伏 d-1 权重")
222 if np.isfinite(hl_net):
223     ax.axvline(hl_net, color="#55A868", ls="--", lw=1.2,
224               label=f"半衰期 {hl_net:.0f} 天")
225 ax.set_xlabel("滞后 / 天")
226 ax.set_ylabel("自相关")
227 ax.set_title("(c) 时间记忆", fontsize=9)
228 ax.legend(fontsize=8)
229
230 ax = axes[1, 1]
231 hours = np.arange(24)
232 hmean = np.array([e_net[:, 6 * h:6 * (h + 1)].mean() for h in range(24)])
233 hstd = np.array([e_net[:, 6 * h:6 * (h + 1)].std() for h in range(24)])
234 ax.plot(hours + 0.5, hmean, color="#4C72B0", marker="o", ms=2.5, label="均值")
235 ax.fill_between(hours + 0.5, hmean - hstd, hmean + hstd,
236                color="#4C72B0", alpha=0.2, label="±1 ")
237 ax.axhline(0.0, color="k", lw=0.6)
238 ax.set_xlabel("时刻 / h")
239 ax.set_ylabel("净需求偏差 / kW")
240 ax.set_title("(d) 日内偏差结构", fontsize=9)
241 ax.legend(fontsize=8)
242
243 fig.tight_layout()
244 pm.save_fig(fig, "Q2E_误差结构与参考值", data=pd.DataFrame({
245     "槽序号": np.arange(1, q2.T + 1),

```

```

246         "m80_逐时段_kW": m_slot,
247         "m80_逐小时_kW": np.repeat(m_hour, 6),
248     )))
249
250     # ---- 报告 ----
251     record(
252         "问题二 口径E 误差分布结构与参数参考值",
253         {
254             "样本数（全时段/白天）": f"{shape['样本数（全时段）']}/{shape['样本数（白天）']}",
255             "偏差均值/kW（白天）": round(shape["均值/kW（白天）"], 1),
256             "偏差标准差/kW（白天）": round(shape["标准差/kW（白天）"], 1),
257             "偏差偏度（白天）": round(shape["偏度（白天）"], 3),
258             "q0.8 裕度/kW（全时段）": round(m_global_all, 0),
259             "q0.8 裕度/kW（白天）": round(m_global_day, 0),
260             "逐时段 q80 范围/kW": (
261                 f"{summary['m80_逐时段范围/kW'][0]:.0f} ~ "
262                 f"{summary['m80_逐时段范围/kW'][1]:.0f}"),
263             "逐小时 q80 范围/kW": (
264                 f"{summary['m80_逐小时范围/kW'][0]:.0f} ~ "
265                 f"{summary['m80_逐小时范围/kW'][1]:.0f}"),
266             "逐月 q80 范围/kW": (
267                 f"{summary['m80_逐月范围/kW'][0]:.0f} ~ "
268                 f"{summary['m80_逐月范围/kW'][1]:.0f}"),
269             "比例式参考 rho_ref": round(rho_ref, 4),
270             "日度偏差 AR(1)": round(rho_net, 3),
271             "日度偏差半衰期/天": round(hl_net, 1),
272             "白天偏差半衰期/天": round(hl_day, 1),
273             "光伏 d-1 权重半衰期/天": round(hl_u1, 1),
274             "日费用均值/元": round(risk["日费用均值/元"], 1),
275             "日费用 P95/元": round(p95, 1),
276             "日费用 CVaR95/元": round(risk["日费用CVaR95/元"], 1),
277             "尾部溢价（P95-均值）/均值": round(risk["尾部溢价(P95-均值)/均值"], 3),
278             "有紧急购电天数占比": round(risk["有紧急购电天数占比"], 3),
279         },
280         note=(
281             "对官方口径 E (W=7 网格权重) 在 2025-02-01-12-31 的预测误差做分布结构分析: "
282             "偏差 e = 实际净需求 - 预测净需求。报童临界比 1-1/5=0.8 对应的最优裕度为 e 的 "
283             "0.8 分位数; 分时段/分小时/分月/比例式参考见 code/outputs/q2e_structure_*.csv; "
284             "时间记忆用于 W 与 EWMA 半衰期参考; 日费用尾部用于风险厌恶系数 扫描。"
285             "图 figures/Q2E-误差结构与参考值.pdf。"
286         ),
287     )
288     log.info("结构分析完成 ({:.0f}s): m80_全={:.0f} m80_白天={:.0f} _net={:.3f} "
289             "半衰期={:.1f}天", time.time() - t0, m_global_all, m_global_day,
290             rho_net, hl_net)
291     return summary
292
293
294 if __name__ == "__main__":
295     main()

```

program/src/solve/experiments/q3_proto_abl.py

```

1  """Q3 原型第三批: 调整点消融、信息/纠偏价值分解、结算口径对照、蒙特卡洛尾部。
2
3  主口径: =1 (纯官方预报), 评估"调整机制本身"的价值, 与题面口径一致。
4
5  实验:
6      调整点消融: 8 种调整时刻组合 (含无调整 = Q2), 量化 6/12/18 各点的边际价值
7      价值分解: V_B 无调整、V_C 有调整但不用新预报、V_A 现实 → 信息价值 vs 纠偏价值
8      结算口径: final (计划 vs 最终值一次结算) vs sequential (逐次结算)
9      蒙特卡洛: 残余误差整日块重采样, 比较"有调整 vs 无调整"的费用分布与尾部风险
10
11  运行 (program/ 下):
12      uv run python -m solve.experiments.q3_proto_abl          # 全部 (约 6 分钟)
13      uv run python -m solve.experiments.q3_proto_abl --skip-mc # 只做
14      uv run python -m solve.experiments.q3_proto_abl --years 30
15  """
16  from __future__ import annotations

```

```

17
18 import argparse
19 import time
20
21 import numpy as np
22 import pandas as pd
23
24 from solve import q2
25 from solve import q3_proto as qp
26 from solve.common import EO, ROOT, T
27 from solve.core.residual import latest_forecast
28
29 DAY_START, N_DAYS = 31, 334
30 DAYS = list(range(DAY_START, DAY_START + N_DAYS))
31 LAM = 1.0
32 EMERG_MULT = qp.EMERG_MULT
33
34
35 def total_of(data, lam=LAM, **kw) -> float:
36     return float(sum(qp.simulate_day(data, D, lam, **kw)["total"] for D in DAYS))
37
38
39 # -----
40 # 调整点消融
41 # -----
42 def run_ablation(data) -> dict:
43     combos = [(), (6,), (12,), (18,), (6, 12), (6, 18), (12, 18), (6, 12, 18)]
44     names = {(): "无调整 (Q2)", (6,): "仅 6:00", (12,): "仅 12:00", (18,): "仅 18:00",
45              (6, 12): "6+12", (6, 18): "6+18", (12, 18): "12+18", (6, 12, 18): "全部 (Q3-A)"}
46     res = {}
47     t0 = time.time()
48     for c in combos:
49         res[c] = total_of(data, adj_hours=c)
50         print(f" {names[c]:<12} {res[c] / 1e4:9.1f} 万 ({time.time() - t0:.0f}s)")
51
52     base = res[()]
53     rows = [{"调整点": names[c], "总费用/万元": round(res[c] / 1e4, 1),
54             "vs 无调整": f"{100 * (res[c] / base - 1):+.2f}%" for c in combos]
55     df = pd.DataFrame(rows)
56     print("\n 调整点消融 (=1, 334 天): ")
57     print(df.to_string(index=False))
58     print("\n逐点边际节省 (万元): ")
59     print(f" 单独开 6:00 : {(res[()] - res[(6,)]) / 1e4:6.1f}")
60     print(f" 单独开 12:00 : {(res[()] - res[(12,)]) / 1e4:6.1f}")
61     print(f" 单独开 18:00 : {(res[()] - res[(18,)]) / 1e4:6.1f}")
62     print(f" 6:00 之后再开 12:00 : {(res[(6,)] - res[(6, 12)]) / 1e4:6.1f}")
63     print(f" 6+12 之后再开 18:00 : {(res[(6, 12)] - res[(6, 12, 18)]) / 1e4:6.1f}")
64     df.to_csv(ROOT / "code" / "outputs" / "q3_proto_ablation.csv", index=False, encoding="utf-8-sig")
65     return res
66
67
68 # -----
69 # 信息价值 vs 纠偏价值
70 # -----
71 def run_decomposition(data, res_abl):
72     V_B = res_abl[()] # 无调整
73     V_A = res_abl[(6, 12, 18)] # 现实
74     V_C = total_of(data, adj_hours=(6, 12, 18), use_new_fc=False) # 有调整但无新预报
75     print("\n 调整机制价值分解 (万元): ")
76     print(f" V_B 无调整 : {V_B / 1e4:8.1f}")
77     print(f" V_C 有调整、无新预报 : {V_C / 1e4:8.1f}")
78     print(f" V_A 有调整、新预报 (现实) : {V_A / 1e4:8.1f}")
79     print(f" 总价值 V_B-V_A : {(V_B - V_A) / 1e4:8.1f} ({100 * (V_B - V_A) / V_B:.2f}%)")
80     print(f" 信息价值 V_C-V_A : {(V_C - V_A) / 1e4:8.1f}")
81     print(f" 纠偏价值 V_B-V_C : {(V_B - V_C) / 1e4:8.1f}")
82     return {"V_B": V_B, "V_C": V_C, "V_A": V_A}
83
84
85 # -----

```

```

86 # 结算口径
87 # -----
88 def run_settle(data):
89     print("\n 结算口径对照: ")
90     rows = []
91     for lam in (1.0, 0.5):
92         for settle in ("final", "sequential"):
93             tot = total_of(data, lam=lam, settle=settle)
94             rows.append({" ": lam, "口径": settle, "总费用/万元": round(tot / 1e4, 1)})
95             print(f"   {lam} {settle:<11} {tot / 1e4:9.1f} 万")
96     df = pd.DataFrame(rows)
97     df.to_csv(ROOT / "code" / "outputs" / "q3_proto_settle.csv", index=False, encoding="utf-8-sig")
98     return df
99
100
101 # -----
102 # 蒙特卡洛: 尾部风险
103 # -----
104 def _stats(x):
105     p95 = float(np.percentile(x, 95))
106     return {"mean": float(x.mean()), "std": float(x.std()),
107           "P95": p95, "CVar95": float(x[x >= p95].mean())}
108
109
110 def run_mc(data, years=50, seed=42):
111     price = data["price"]
112     months = np.asarray(data["months"])
113
114     # 策略 A (=1) 的固定决策
115     x_plan_all, x_final_all, fix_adj, fix_no = {}, {}, {}, {}
116     t0 = time.time()
117     for D in DAYS:
118         r = qp.simulate_day(data, D, LAM)
119         x_plan_all[D], x_final_all[D] = r["x_plan"], r["x_final"]
120         fix_adj[D] = float(price @ r["x_final"]) + float(
121             (0.5 * price * np.abs(r["x_final"] - r["x_plan"])) .sum())
122         fix_no[D] = float(price @ r["x_plan"])
123     print(f" 固定决策预计算完成 ({time.time() - t0:.0f}s) ")
124
125     # 残余误差矩阵 (kW): 最新预报 - 实际; 0:00 预报 - 实际
126     R_latest = np.zeros((365, T))
127     R_q2 = np.zeros((365, T))
128     for D in range(365):
129         R_latest[D] = latest_forecast(data, D) - data["pv_act"][D]
130         R_q2[D] = q2._hour_to_slots(data["fc0"][D]) - data["pv_act"][D]
131
132     rng = np.random.default_rng(seed)
133     cost_adj = np.zeros(years)
134     cost_no = np.zeros(years)
135     emerg_adj = np.zeros(years)
136     emerg_no = np.zeros(years)
137     t0 = time.time()
138     for rep in range(years):
139         for D in DAYS:
140             pool = np.where(months == months[D])[0]
141             idx = int(rng.choice(pool))
142             load_kwh = data["load"][D] / 6.0
143
144             pv_s = np.clip(latest_forecast(data, D) + R_latest[idx], 0.0, None) / 6.0
145             ex = q2.exec_day(price, load_kwh, pv_s, x_final_all[D], E0)
146             ec = EMERG_MULT * float(price @ ex["e"])
147             emerg_adj[rep] += ec
148             cost_adj[rep] += fix_adj[D] + ec
149
150             pv_s0 = np.clip(q2._hour_to_slots(data["fc0"][D]) + R_q2[idx], 0.0, None) / 6.0
151             ex0 = q2.exec_day(price, load_kwh, pv_s0, x_plan_all[D], E0)
152             ec0 = EMERG_MULT * float(price @ ex0["e"])
153             emerg_no[rep] += ec0
154             cost_no[rep] += fix_no[D] + ec0

```

```

155         if (rep + 1) % 10 == 0:
156             print(f" MC {rep + 1}/{years}, {time.time() - t0:.0f}s")
157
158     sa, sn = _stats(cost_adj), _stats(cost_no)
159     print("\n 蒙特卡洛 (%d 年, 共同随机数): " % years)
160     print(f" {'指标':<22}{'无调整 (Q2) '}>14}{'有调整 (Q3-A) '}>16}")
161     for k, lab in (("mean", "总费用均值/万元"), ("std", "总费用标准差/万元"),
162                  ("P95", "总费用 P95/万元"), ("CVar95", "总费用 CVar95/万元")):
163         print(f" {'lab':<22}{sn[k] / 1e4:>14.1f}{sa[k] / 1e4:>16.1f}")
164     print(f" {'紧急费均值/万元':<22}{emerg_no.mean() / 1e4:>14.1f}{emerg_adj.mean() / 1e4:>16.1f}")
165     print(f" {'紧急费 P95/万元':<22}{np.percentile(emerg_no, 95) / 1e4:>14.1f}"
166           f"{np.percentile(emerg_adj, 95) / 1e4:>16.1f}")
167     print(f" 总费用均值下降 {100 * (1 - sa['mean'] / sn['mean']):+.2f}%, "
168           f"std 下降 {100 * (1 - sa['std'] / sn['std']):+.1f}%, "
169           f"P95 下降 {100 * (1 - sa['P95'] / sn['P95']):+.1f}%")
170
171     out = pd.DataFrame({
172         "无调整_总费用_元": cost_no, "有调整_总费用_元": cost_adj,
173         "无调整_紧急费_元": emerg_no, "有调整_紧急费_元": emerg_adj,
174     })
175     out.to_csv(ROOT / "code" / "outputs" / "q3_proto_mc.csv", index=False, encoding="utf-8-sig")
176     return out
177
178
179 def main():
180     ap = argparse.ArgumentParser()
181     ap.add_argument("--skip-mc", action="store_true")
182     ap.add_argument("--skip-abl", action="store_true")
183     ap.add_argument("--years", type=int, default=50)
184     ap.add_argument("--seed", type=int, default=42)
185     args = ap.parse_args()
186
187     qp.pm.init(seed=args.seed, root=str(ROOT))
188     data = qp.load_extended()
189
190     res_abl = None
191     if not args.skip_abl:
192         print(" 调整点消融 (8 种组合) .....")
193         res_abl = run_ablation(data)
194         print("\n 价值分解.....")
195         run_decomposition(data, res_abl)
196         print("\n 结算口径.....")
197         run_settle(data)
198
199     if not args.skip_mc:
200         print("\n 蒙特卡洛尾部.....")
201         run_mc(data, years=args.years, seed=args.seed)
202
203
204 if __name__ == "__main__":
205     main()

```

program/src/solve/experiments/q3_proto_adjmix.py

```

1  """Q3 原型第五批：调整层组合预报修正（消除“信息退化”）。
2
3  发现：0:00 计划用组合预测（较准），调整层却用纯官方预报（较不准）→ 调整会把更准的
4  计划改坏（实时模式下 =0.5 的“仅 6:00”反而比无调整贵）。本实验让调整层也用同一组合权重。
5
6  运行（program/ 下）：uv run python -m solve.experiments.q3_proto_adjmix
7  """
8  from __future__ import annotations
9
10 import time
11
12 import pandas as pd
13
14 from solve import q3_proto as qp
15 from solve.common import ROOT

```

```

16
17 DAYS = list(range(31, 365))
18
19
20 def total(data, lam, **kw):
21     return float(sum(qp.simulate_day_rt(data, D, lam, **kw)["total"] for D in DAYS))
22
23
24 def main():
25     qp.pm.init(seed=42, root=str(ROOT))
26     data = qp.load_extended()
27     jobs = []
28     for lam, combos in [(0.5, [(("无调整", ()), ("全开", (6, 12, 18))), ("12+18", (12, 18))), ("仅6", (6,))]),
29                         (0.3, [(("全开", (6, 12, 18)))]):
30         for name, c in combos:
31             jobs.append((lam, name, c))
32
33     rows = []
34     for lam, name, c in jobs:
35         for adj_lam, tag in ((None, "官方"), (lam, "组合")):
36             if not c and tag == "组合":
37                 continue
38             t0 = time.time()
39             tot = total(data, lam, adj_hours=c, adj_lam=adj_lam)
40             rows.append({" ": lam, "调整点": name, "调整层": tag,
41                         "总费用/万元": round(tot / 1e4, 1)})
42             print(f"={lam} {name:<6} 调整层={tag} {tot / 1e4:8.1f} 万 ({time.time() - t0:.0f}s)")
43     pd.DataFrame(rows).to_csv(ROOT / "code" / "outputs" / "q3_proto_adjmix.csv",
44                             index=False, encoding="utf-8-sig")
45     print("已保存 code/outputs/q3_proto_adjmix.csv")
46
47
48 if __name__ == "__main__":
49     main()

```

program/src/solve/experiments/q3_proto_final.py

```

1 """Q3 原型第六批：保守裕度扫描 + 调整层场景对冲。
2
3 运行 (program/ 下) : uv run python -m solve.experiments.q3_proto_final
4 """
5 from __future__ import annotations
6
7 import time
8
9 import pandas as pd
10
11 from solve import q3_proto as qp
12 from solve.common import ROOT
13
14 DAYS = list(range(31, 365))
15
16
17 def total(data, fn, lam, **kw):
18     return float(sum(fn(data, D, lam, **kw)["total"] for D in DAYS))
19
20
21 def main():
22     qp.pm.init(seed=42, root=str(ROOT))
23     data = qp.load_extended()
24     rows = []
25
26     print(" 保守裕度扫描 (=0.5, 全开, 调整层组合) : ")
27     for m in (0.0, 0.005, 0.01, 0.02, 0.03):
28         t0 = time.time()
29         tot = total(data, qp.simulate_day_rt, 0.5, adj_lam=0.5, margin=m)
30         rows.append({"实验": "margin", "参数": f"{m:.3f}", "总费用/万元": round(tot / 1e4, 1)})
31         print(f"  margin={m:.3f} {tot / 1e4:8.1f} 万 ({time.time() - t0:.0f}s)")
32

```

```

33     print("\n 调整层场景对冲（全开）：")
34     for lam in (0.5, 1.0):
35         arg = {"adj_lam": (lam if lam < 1 else None)}
36         t0 = time.time()
37         base = total(data, qp.simulate_day_rt, lam, **arg)
38         t1 = time.time()
39         hedge = total(data, qp.simulate_day_rt_hedge, lam, n_scen=10, seed=7, **arg)
40         rows += [{"实验": "hedge", "参数": f"lam={lam} base", "总费用/万元": round(base / 1e4, 1)},
41                 {"实验": "hedge", "参数": f"lam={lam} hedge", "总费用/万元": round(hedge / 1e4, 1)}]
42         print(f"    =lam=: 基准 {base / 1e4:8.1f} 万 ({t1 - t0:.0f}s) + 对冲 {hedge / 1e4:8.1f} 万 "
43               f" ({time.time() - t1:.0f}s) 差 {(hedge - base) / 1e4:+.1f} 万")
44
45     pd.DataFrame(rows).to_csv(ROOT / "code" / "outputs" / "q3_proto_final.csv",
46                               index=False, encoding="utf-8-sig")
47     print("已保存 code/outputs/q3_proto_final.csv")
48
49
50 if __name__ == "__main__":
51     main()

```

program/src/solve/experiments/q3_proto_lh.py

```

1  """Q3 原型第七批：对冲模式下组合权重扫描 + 调整点确认。
2
3  运行（program/ 下）：uv run python -m solve.experiments.q3_proto_lh
4  """
5  from __future__ import annotations
6
7  import time
8
9  import pandas as pd
10
11  from solve import q3_proto as qp
12  from solve.common import ROOT
13
14  DAYS = list(range(31, 365))
15
16
17  def run(data, lam, adj_lam, adj_hours=(6, 12, 18)):
18      outs = [qp.simulate_day_rt_hedge(data, D, lam, adj_hours=adj_hours,
19                                       adj_lam=adj_lam, n_scen=10, seed=7) for D in DAYS]
20      return {
21          "total": sum(o["total"] for o in outs),
22          "plan": sum(o["plan_cost"] for o in outs),
23          "adj": sum(o["adjust_net"] for o in outs),
24          "emerg": sum(o["emerg"] for o in outs),
25      }
26
27
28  def main():
29      qp.pm.init(seed=42, root=str(ROOT))
30      data = qp.load_extended()
31      rows = []
32
33      print("对冲模式 * 组合权重扫描（全开调整，6/12 对冲）：")
34      for lam in (0.3, 0.5, 0.7, 1.0):
35          t0 = time.time()
36          r = run(data, lam, None if lam == 1.0 else lam)
37          rows.append({"配置": f"={lam} 全开对冲", "总费用/万元": round(r["total"] / 1e4, 1),
38                      "计划/万元": round(r["plan"] / 1e4, 1),
39                      "调整/万元": round(r["adj"] / 1e4, 1),
40                      "紧急/万元": round(r["emerg"] / 1e4, 1)})
41          print(f"    =lam:<4) 总 {r['total'] / 1e4:8.1f} 万 "
42                f" (计划 {r['plan'] / 1e4:.1f} + 调整 {r['adj'] / 1e4:+.1f} + 紧急 {r['emerg'] / 1e4:.1f}) "
43                f" ({time.time() - t0:.0f}s)")
44
45      print("\n对冲模式 * 调整点确认（=1）：")
46      for hs, name in [(12, 18), "12+18"), ((6, 12, 18), "全开")]:
47          t0 = time.time()

```

```

48     r = run(data, 1.0, None, adj_hours=hs)
49     rows.append({"配置": f"=1 {name} 对冲", "总费用/万元": round(r["total"] / 1e4, 1),
50                 "计划/万元": round(r["plan"] / 1e4, 1),
51                 "调整/万元": round(r["adj"] / 1e4, 1),
52                 "紧急/万元": round(r["emerg"] / 1e4, 1)})
53     print(f" {name:<6} 总 {r['total'] / 1e4:8.1f} 万 "
54           f" (计划 {r['plan'] / 1e4:.1f} + 调整 {r['adj'] / 1e4:+.1f} + 紧急 {r['emerg'] / 1e4:.1f}) "
55           f" {time.time() - t0:.0f}s")
56
57     pd.DataFrame(rows).to_csv(ROOT / "code" / "outputs" / "q3_proto_lambda_hedge.csv",
58                               index=False, encoding="utf-8-sig")
59     print("已保存 code/outputs/q3_proto_lambda_hedge.csv")
60
61
62 if __name__ == "__main__":
63     main()

```

program/src/solve/experiments/q3_proto_rt.py

```

1  """Q3 原型第四批：实时执行模式对比与调整点稳健性。
2
3  实时执行 = 每段先用实际功率执行到下一调整时刻（得到真实储电量），调整 LP 以该
4  储电量为初值（利用已实现信息），段执行终端跟踪最近一次规划轨迹。
5
6  运行（program/ 下）：uv run python -m solve.experiments.q3_proto_rt
7  """
8  from __future__ import annotations
9
10 import time
11
12 import pandas as pd
13
14 from solve import q3_proto as qp
15 from solve.common import ROOT
16
17 DAY_START, N_DAYS = 31, 334
18 DAYS = list(range(DAY_START, DAY_START + N_DAYS))
19
20
21 def run(data, fn, lam, **kw):
22     outs = [fn(data, D, lam, **kw) for D in DAYS]
23     return {
24         "total": sum(o["total"] for o in outs),
25         "plan": sum(o["plan_cost"] for o in outs),
26         "adj": sum(o["adjust_net"] for o in outs),
27         "emerg": sum(o["emerg"] for o in outs),
28     }
29
30
31 def main():
32     qp.pm.init(seed=42, root=str(ROOT))
33     data = qp.load_extended()
34
35     print("模式对比 (=1, 三点调整, 334 天): ")
36     for name, fn in (("事后执行 (全天一次执行 LP)", qp.simulate_day),
37                     ("实时执行 (实际重放+分段)", qp.simulate_day_rt)):
38         t0 = time.time()
39         r = run(data, fn, 1.0)
40         print(f" {name} 总 {r['total'] / 1e4:8.1f} 万 "
41               f" (计划 {r['plan'] / 1e4:.1f} + 调整 {r['adj'] / 1e4:+.1f} "
42               f"+ 紧急 {r['emerg'] / 1e4:.1f}) {time.time() - t0:.0f}s")
43
44     print("\n实时执行模式 • 调整点消融: ")
45     combos = [(), (6,), (12,), (18,), (6, 12), (6, 18), (12, 18), (6, 12, 18)]
46     rows = []
47     for lam in (1.0, 0.5):
48         for c in combos:
49             t0 = time.time()
50             r = run(data, qp.simulate_day_rt, lam, adj_hours=c)

```

```

51         rows.append({" ": lam, "调整点": str(c),
52                     "总费用/万元": round(r["total"] / 1e4, 1),
53                     "紧急费/万元": round(r["emerg"] / 1e4, 1)})
54         print(f"={lam} {str(c):<12} 总 {r['total'] / 1e4:8.1f} 万 "
55               f"紧急 {r['emerg'] / 1e4:5.1f} 万 ({time.time() - t0:.0f}s)")
56     df = pd.DataFrame(rows)
57     df.to_csv(ROOT / "code" / "outputs" / "q3_proto_rt.csv", index=False, encoding="utf-8-sig")
58     print("\n已保存 code/outputs/q3_proto_rt.csv")
59
60
61 if __name__ == "__main__":
62     main()

```

program/src/solve/experiments/q3_proto_seg.py

```

1  """Q3 原型第二批：单 滚动策略改进 (W/EWMA/范围约束) + 分时段 网格。
2
3  第一部分（秒级，复用 q3_proto 的全网格费用表）：
4      C 基线 (W=7) 之外，评估 W {3,14,30,60}、指数加权窗口、 范围约束。
5  第二部分（约 8-10 分钟）：分时段 （[0,12h] 与 [12h,24h] 各一个权重）7×7 组合网格，
6      滚动窗口选择（无前视）与离线最优（参考）。
7
8  用法（program/ 下）：
9      uv run python -m solve.experiments.q3_proto_seg          # 全部
10     uv run python -m solve.experiments.q3_proto_seg --skip-seg # 仅第一部分
11     uv run python -m solve.experiments.q3_proto_seg --skip-single
12 """
13 from __future__ import annotations
14
15 import argparse
16 import time
17
18 import numpy as np
19 import pandas as pd
20
21 from solve import q2
22 from solve import q3_proto as qp
23 from solve.common import E0, ROOT, T
24
25 SEG_GRID = np.round(np.array([0.0, 0.2, 0.4, 0.5, 0.6, 0.8, 1.0]), 2)
26 DAY_START, N_DAYS = 31, 334      # 与 q3_proto 全年表一致
27 I_START = 7                      # 评估起点（给滚动窗口预热；所有策略统一）
28 W_SEG = 7
29
30
31 # -----
32 # 单 表：滚动规则对比
33 # -----
34 def load_single_table():
35     path = ROOT / "code" / "outputs" / "q3_proto_lambda_table.csv"
36     df = pd.read_csv(path, index_col=0)
37     grid = np.array([float(c.split("=")[1]) for c in df.columns])
38     return df.to_numpy(float), grid, df.index.to_numpy()
39
40
41 def _pick(rows: np.ndarray, grid: np.ndarray, alpha=None, lo=0.0, hi=1.0) -> int:
42     if len(rows) == 0:
43         return int(np.argmin(np.abs(grid - 0.5)))
44     if alpha is None:
45         score = rows.mean(axis=0)
46     else:
47         w = alpha ** np.arange(len(rows) - 1, -1, -1, dtype=float)
48         score = (rows * w[:, None]).sum(axis=0) / w.sum()
49     valid = (grid >= lo) & (grid <= hi)
50     return int(np.argmin(np.where(valid, score, np.inf)))
51
52
53 def analyze_single(table, grid, dates):
54     n = table.shape[0]

```

```

55     idx_a = len(grid) - 1
56     rules = {}
57
58     for W in (3, 7, 14, 30, 60):
59         cost = np.empty(n)
60         for i in range(n):
61             i0 = max(0, i - W)
62             li = _pick(table[i0:i], grid)
63             cost[i] = table[i, li]
64             rules[f"动态 W={W}"] = cost
65
66     for W, a in ((14, 0.6), (30, 0.8), (14, 0.3)):
67         cost = np.empty(n)
68         for i in range(n):
69             i0 = max(0, i - W)
70             li = _pick(table[i0:i], grid, alpha=a)
71             cost[i] = table[i, li]
72             rules[f"EWMA W={W} a={a}"] = cost
73
74     for W, lo, hi in ((7, 0.3, 0.7), (30, 0.2, 0.8), (14, 0.3, 0.7)):
75         cost = np.empty(n)
76         for i in range(n):
77             i0 = max(0, i - W)
78             li = _pick(table[i0:i], grid, lo=lo, hi=hi)
79             cost[i] = table[i, li]
80             rules[f"限幅[{lo},{hi}] W={W}"] = cost
81
82     # 固定与参考
83     i_half = int(np.argmax(np.abs(grid - 0.5)))
84     rules["固定 =0.5"] = table[:, i_half].copy()
85     rules["A 纯官方"] = table[:, idx_a].copy()
86     # 离线最优 (仅参考, 信息前视)
87     rules["离线最优(参考)"] = table.min(axis=1)
88
89     ev = slice(I_START, None)
90     A = table[ev, idx_a].sum()
91     rows = []
92     for name, cost in rules.items():
93         tot = cost[ev].sum()
94         rows.append({
95             "规则": name,
96             "总费用/万元": round(tot / 1e4, 1),
97             "vs A": f"{100 * (tot / A - 1):+.2f}%",
98         })
99     out = pd.DataFrame(rows)
100     print("\n===== 单 表: 滚动规则对比 (%d-%d 天, n=%d) =====" % (I_START + 1, n, n - I_START))
101     print(out.to_string(index=False))
102     return out, rules
103
104
105 # -----
106 # 分时段 网格
107 # -----
108 def _day_cost(data, D, lam_vec):
109     """分时段组合的单日 Q3 费用 (标量四元组)。"""
110     price = data["price"]
111     load_kwh = data["load"][D] / 6.0
112     f0 = q2._hour_to_slots(data["fc0"][D])
113     ph = qp.hist_forecast(data, D)
114     pv_plan = (lam_vec * f0 + (1.0 - lam_vec) * ph) / 6.0
115
116     x_plan, E_plan, _ = q2.plan_day(price, load_kwh, pv_plan, E0, eps=qp.EPS_TH)
117     x_final = x_plan.copy()
118     for t0, fc, pub in ((36, data["fc6"][D], 6), (72, data["fc12"][D], 12), (108, data["fc18"][D], 18)):
119         pv_new = np.clip(qp.fc_slots(fc, pub), 0.0, None) / 6.0
120         x_adj, _E_adj = qp.adjust_day(price, load_kwh, pv_new, x_plan, float(E_plan[t0 - 1]), t0)
121         x_final[t0:] = x_adj
122
123     ex = q2.exec_day(price, load_kwh, data["pv_act"][D] / 6.0, x_final, E0)

```

```

124     base = float((price * np.minimum(x_plan, x_final)).sum())
125     dev_lo = float((0.5 * price * np.maximum(x_plan - x_final, 0.0)).sum())
126     dev_hi = float((1.5 * price * np.maximum(x_final - x_plan, 0.0)).sum())
127     emerg = float((qp.EMERG_MULT * price * ex["e"]).sum())
128     plan_cost = float(price @ x_plan)
129     total = base + dev_lo + dev_hi + emerg
130     return total, plan_cost, base + dev_lo + dev_hi - plan_cost, emerg
131
132
133 def run_seg_table(force=False):
134     out = ROOT / "code" / "outputs" / "q3_proto_seg_table.npz"
135     if out.exists() and not force:
136         z = np.load(out)
137         print(f"分时段表命中缓存: {out.name}")
138         return z["table"], z["dates"]
139
140     data = qp.load_extended()
141     ng = len(SEG_GRID)
142     table = np.zeros((N_DAYS, ng, ng))
143     dates = [data["dates"][d] for d in range(DAY_START, DAY_START + N_DAYS)]
144     ts = time.time()
145     for i, D in enumerate(range(DAY_START, DAY_START + N_DAYS)):
146         cost_grid = np.zeros((ng, ng))
147         for a_i, lam_a in enumerate(SEG_GRID):
148             for p_i, lam_p in enumerate(SEG_GRID):
149                 lam_vec = np.concatenate([np.full(72, lam_a), np.full(72, lam_p)])
150                 cost_grid[a_i, p_i] = _day_cost(data, D, lam_vec)[0]
151         table[i] = cost_grid
152         if (i + 1) % 20 == 0 or i == N_DAYS - 1:
153             print(f" 分时段 {i + 1}/{N_DAYS} 天, 用时 {time.time() - ts:.0f}s")
154     np.savez_compressed(out, table=table, dates=np.array(dates))
155     print(f"分时段表已保存: {out}")
156     return table, np.array(dates)
157
158
159 def analyze_seg(table, dates, single_table, grid):
160     n = table.shape[0]
161     ng = len(SEG_GRID)
162     cost_seg = np.empty(n)
163     cost_seg_off = np.empty(n)
164     picks = []
165     for i in range(n):
166         i0 = max(0, i - W_SEG)
167         score = table[i0:i].mean(axis=0) if i > i0 else table[:,i].mean(axis=0)
168         a_i, p_i = np.unravel_index(np.argmin(score), (ng, ng))
169         cost_seg[i] = table[i, a_i, p_i]
170         picks.append((SEG_GRID[a_i], SEG_GRID[p_i]))
171         a_i2, p_i2 = np.unravel_index(np.argmin(table[i]), (ng, ng))
172         cost_seg_off[i] = table[i, a_i2, p_i2]
173     picks = np.array(picks)
174
175     i_half = int(np.argmin(np.abs(grid - 0.5)))
176     idx_a = len(grid) - 1
177     i_half_seg = int(np.argmin(np.abs(SEG_GRID - 0.5)))
178     ev = slice(I_START, None)
179
180     def report(name, cost):
181         tot = cost[ev].sum()
182         a_tot = single_table[ev, idx_a].sum()
183         print(f" {name:<26}{tot / 1e4:>10.1f} {100 * (tot / a_tot - 1):>7.2f}%")
184
185     print("\n==== 分时段 对比 (n=%d) ===== % (n - I_START))")
186     print(f" {'策略':<26}{'总费用/万元':>10} {'vs A':>7}")
187     report("A 纯官方", single_table[:, idx_a])
188     report("固定 =0.5 (单)", single_table[:, i_half])
189     report("固定 0.5/0.5 (分时段)", table[:, i_half_seg, i_half_seg])
190     # 单 动态 W=7 (复算)
191     c7 = np.empty(n)
192     for i in range(n):

```

```

193     i0 = max(0, i - 7)
194     li = _pick(single_table[i0:i], grid)
195     c7[i] = single_table[i, li]
196     report("动态单 W=7", c7)
197     report("动态分时段 W=7", cost_seg)
198     report("分时段离线最优(参考)", cost_seg_off)
199
200     print("\n 分时段滚动规则扫描 (W × 限幅): ")
201     for W in (14, 30, 60):
202         for lo, hi in ((0.0, 1.0), (0.2, 0.8), (0.3, 0.7)):
203             cost = np.empty(n)
204             for i in range(n):
205                 i0 = max(0, i - W)
206                 score = table[i0:i].mean(axis=0) if i > i0 else table[:i].mean(axis=0)
207                 valid = (SEG_GRID >= lo - 1e-9) & (SEG_GRID <= hi + 1e-9)
208                 score = np.where(valid, score, np.inf)
209                 a_i, p_i = np.unravel_index(np.argmin(score), (ng, ng))
210                 cost[i] = table[i, a_i, p_i]
211                 report(f"分时段 W={W} 限[{lo},{hi}]", cost)
212
213     # 单 最优规则 (同评估区间, 用于公平对比)
214     c30 = np.empty(n)
215     for i in range(n):
216         i0 = max(0, i - 30)
217         li = _pick(single_table[i0:i], grid, lo=0.2, hi=0.8)
218         c30[i] = single_table[i, li]
219     report("单 W=30 限[0.2,0.8]", c30)
220     print(f" 分时段滚动选择: _am 均值 {picks[I_START:, 0].mean():.2f}, "
221           f" _pm 均值 {picks[I_START:, 1].mean():.2f}")
222
223     df = pd.DataFrame({
224         "日期": dates,
225         "动态分时段费用": cost_seg,
226         "动态单 费用": c7,
227         "_am": picks[:, 0],
228         "_pm": picks[:, 1],
229     })
230     df.to_csv(ROOT / "code" / "outputs" / "q3_proto_seg_picks.csv", index=False, encoding="utf-8-sig")
231     return cost_seg
232
233
234 def main():
235     ap = argparse.ArgumentParser()
236     ap.add_argument("--skip-seg", action="store_true")
237     ap.add_argument("--skip-single", action="store_true")
238     ap.add_argument("--force-seg", action="store_true")
239     args = ap.parse_args()
240
241     single, grid, dates = load_single_table()
242     if not args.skip_single:
243         analyze_single(single, grid, dates)
244
245     if not args.skip_seg:
246         table, dates2 = run_seg_table(force=args.force_seg)
247         analyze_seg(table, dates2, single, grid)
248
249
250 if __name__ == "__main__":
251     main()

```

program/src/solve/experiments/q3_proto_smooth.py

```

1 """Q3 原型第八批: 历史预测权重的参数平滑 (新旧参数混合) + 动态 。
2
3 - 参数平滑:  $u_D = \alpha \cdot u_{\text{new}} + (1-\alpha) \cdot u_{\{D-1\}}$  ( $\alpha=1$  为不平滑)
4 - 评估基座: 实时执行 + 全开调整 + 调整层组合
5
6 运行 (program/ 下): uv run python -m solve.experiments.q3_proto_smooth
7 """

```

```

8 from __future__ import annotations
9
10 import time
11
12 import numpy as np
13 import pandas as pd
14
15 from solve import q3_proto as qp
16 from solve.common import ROOT
17
18 DAYS = list(range(31, 365))
19 LAM_SUB = np.array([0.2, 0.4, 0.5, 0.6, 0.7, 0.8, 1.0])
20
21
22 def daily_costs(data, fn, lam, **kw):
23     return np.array([fn(data, D, lam, **kw)["total"] for D in DAYS])
24
25
26 def main():
27     qp.pm.init(seed=42, root=str(ROOT))
28     base = qp.load_extended()
29     rows = []
30
31     print(" 参数平滑 beta 扫描 (实时模式, 固定 =0.5, 调整层组合): ")
32     tot_by_beta = {}
33     for beta in (0.1, 0.3, 0.5, 0.7, 1.0):
34         d2 = dict(base)
35         d2["U_SMOOTH"] = qp.make_smooth_u(base, beta)
36         t0 = time.time()
37         tot = daily_costs(d2, qp.simulate_day_rt, 0.5, adj_lam=0.5).sum()
38         tot_by_beta[beta] = tot
39         rows.append({"实验": "beta", "参数": beta, "总费用/万元": round(tot / 1e4, 1)})
40         print(f"  beta={beta:.1f}  {tot / 1e4:8.1f} 万  ({time.time() - t0:.0f}s)")
41     beta_star = min(tot_by_beta, key=tot_by_beta.get)
42     print(f"  -> 最优 beta*={beta_star}")
43
44     print("\n  网格 + 动态 (实时, 无对冲): ")
45     tabs = {}
46     for beta in (1.0, beta_star):
47         d2 = dict(base)
48         d2["U_SMOOTH"] = qp.make_smooth_u(base, beta)
49         t0 = time.time()
50         tab = np.zeros((len(DAYS), len(LAM_SUB)))
51         for i, D in enumerate(DAYS):
52             for j, lam in enumerate(LAM_SUB):
53                 tab[i, j] = qp.simulate_day_rt(d2, D, float(lam), adj_lam=float(lam))["total"]
54         tabs[beta] = tab
55         print(f"  beta={beta}  网格完成 ({time.time() - t0:.0f}s) ")
56
57     def dyn(tab, W=30, lo=0.2, hi=0.8):
58         cost = np.zeros(len(DAYS))
59         for i in range(len(DAYS)):
60             i0 = max(0, i - W)
61             score = tab[i0:i].mean(axis=0) if i > i0 else tab[:i].mean(axis=0)
62             valid = (LAM_SUB >= lo - 1e-9) & (LAM_SUB <= hi + 1e-9)
63             li = int(np.argmin(np.where(valid, score, np.inf)))
64             cost[i] = tab[i, li]
65         return cost
66
67     for beta in (1.0, beta_star):
68         tab = tabs[beta]
69         ev = tab[7:]
70         d_cost = dyn(tab)[7:]
71         print(f"\n  beta={beta}:")
72         for j, lam in enumerate(LAM_SUB):
73             print(f"    固定 ={lam:.1f}: {ev.sum(axis=0)[j] / 1e4:8.1f} 万 ")
74         print(f"  动态 W=30 限[0.2,0.8]: {d_cost.sum() / 1e4:8.1f} 万 ")
75         print(f"  离线最优 (参考): {ev.min(axis=1).sum() / 1e4:8.1f} 万 ")
76     rows.append({"实验": "lambda", "参数": f"beta={beta} 动态",

```

```

77         "总费用/万元": round(d_cost.sum() / 1e4, 1)})
78
79     print("\n 对冲模式对比 (=0.7): ")
80     for beta in (1.0, beta_star):
81         d2 = dict(base)
82         d2["U_SMOOTH"] = qp.make_smooth_u(base, beta)
83         t0 = time.time()
84         tot = sum(qp.simulate_day_rt_hedge(d2, D, 0.7, adj_lam=0.7, n_scen=10, seed=7)["total"]
85                 for D in DAYS)
86         rows.append({"实验": "hedge", "参数": f"beta={beta}", "总费用/万元": round(tot / 1e4, 1)})
87         print(f"  beta={beta}: {tot / 1e4:8.1f} 万  ({time.time() - t0:.0f}s)")
88
89     pd.DataFrame(rows).to_csv(ROOT / "code" / "outputs" / "q3_proto_smooth.csv",
90                             index=False, encoding="utf-8-sig")
91     print("已保存 code/outputs/q3_proto_smooth.csv")
92
93
94 if __name__ == "__main__":
95     main()

```

program/src/solve/experiments/q4_price_dyn.py

```

1  """Q4 电价 v 参数的动态修正 (新旧混合平滑) 验证。
2
3  流程:
4  1. 复用 q4_price_fit 的逐日费用表 (daily_costs) 做滚动费用标定 → v*_D 序列 (无前视)
5  2. 动态修正: v_D =  *v*_D + (1- ) * v_{D-1} ( {0.1,0.3,0.5,0.7})
6  3. 用平滑后的连续 v 序列重新模拟全年 (跨日连续储能), 对比 334 天费用
7
8  复用的公共实现: models/price.forecast_price (三源预测)、models/price.rolling_v_star (v* 选择)。
9  运行 (program/ 下): uv run python -m solve.experiments.q4_price_dyn
10 """
11 from __future__ import annotations
12
13 import time
14
15 import numpy as np
16 import pandas as pd
17
18 import program as pm
19 from solve import q2
20 from solve.common import DATA_C, EO, ROOT
21 from solve.data.attachments import load_extended
22 from solve.models.adaptive import hist_forecast_asof, hist_load_forecast_asof
23 from solve.models.price import forecast_price, forecast_price_next_asof, rolling_v_star
24
25 DAY_START = q2.REPORT_START
26 N_DAY = q2.N_DAY
27
28
29 def run_year_vseq(p4, p_typ, data, v_seq):
30     """按给定 v 序列模拟全年 (v_seq: {day: v}; 缺省天用纯典型日), 返回 334 天费用。
31
32     统一口径: 48 小时滚动计划 (末端自由) + 逐槽因果执行; 结算用附件 4 真实价。
33     """
34     load = data["load"]; pv_act = data["pv_act"]; fc0 = data["fc0"]
35     E = EO
36     tot = 0.0
37     for D in range(N_DAY):
38         v = v_seq.get(D, (0.0, 0.0, 1.0))
39         p_hat = forecast_price(D, v, p4, p_typ) if D >= 7 else p_typ
40         p_hat1 = forecast_price_next_asof(D, v, p4, p_typ)
41         xh, Eh, _ = q2.plan_horizon(
42             np.concatenate([p_hat, p_hat1]),
43             np.concatenate([hist_load_forecast_asof(data, D, D),
44                             hist_load_forecast_asof(data, D + 1, D)] / 6.0,
45             np.concatenate([q2._hour_to_slots(fc0[D]),
46                             hist_forecast_asof(data, D + 1, D)] / 6.0,
47             E, None, eps=1e-3,

```

```

48     )
49     x, e_day_end = xh[:q2.T], float(Eh[q2.T - 1])
50     ex = q2.exec_day_causal(
51         p4[D], load[D] / 6.0, pv_act[D] / 6.0, x, E, e_end=e_day_end
52     )
53     E = float(ex["E"][-1])
54     if D >= DAY_START:
55         tot += float(p4[D] @ x + q2.EMERG_MULT * (p4[D] @ ex["e"]))
56     return tot
57
58
59 def main():
60     pm.init(seed=42, root=str(ROOT))
61     data = load_extended()
62     p_typ = data["price"]
63     df4 = pm.read_table(DATA_C / "附件4.xlsx")
64     p4 = df4.iloc[:, 1:145].to_numpy(float)
65
66     # 滚动费用标定 → v*_D (共享实现, 无前视)
67     v_star_all = rolling_v_star()
68     v_star = {D: tuple(v_star_all[D]) for D in range(DAY_START, N_DAY)}
69     print(f"v* 逐日选择: v1 均值 {np.mean([v[0] for v in v_star.values()]):.2f}、"
70           f"v2 {np.mean([v[1] for v in v_star.values()]):.2f}、"
71           f"v3 {np.mean([v[2] for v in v_star.values()]):.2f}")
72
73     t0 = time.time()
74     base = run_year_vseq(p4, p_typ, data, v_star)
75     print(f"滚动费用标定 (不平滑): {base / 1e4:.1f} 万元 ({time.time() - t0:.0f}s)")
76
77     rows = [{"配置": "滚动 * 不平滑", "总费用/万元": round(base / 1e4, 1)}]
78     for beta in (0.1, 0.3, 0.5, 0.7):
79         v_sm = {}
80         prev = None
81         for D in range(DAY_START, N_DAY):
82             v_new = np.array(v_star[D])
83             prev = v_new if prev is None else beta * v_new + (1 - beta) * prev
84             v_sm[D] = tuple(prev)
85         tot = run_year_vseq(p4, p_typ, data, v_sm)
86         tag = f"动态修正 = {beta}"
87         print(f"{tag}: {tot / 1e4:.1f} 万元 (vs 不平滑 {100 * (tot / base - 1):+.2f}%) "
88               f"({time.time() - t0:.0f}s)")
89         rows.append({"配置": tag, "总费用/万元": round(tot / 1e4, 1)})
90
91     pd.DataFrame(rows).to_csv(ROOT / "code" / "outputs" / "q4_price_dyn.csv",
92                               index=False, encoding="utf-8-sig")
93     print("已保存 code/outputs/q4_price_dyn.csv")
94
95
96 if __name__ == "__main__":
97     main()

```

program/src/solve/experiments/q4_price_eda.py

```

1  """Q4 附件 4 波动电价结构分析 (EDA): 日内/周内/年内结构、自相关、预测源可解释性。
2
3  运行 (program/ 下): uv run python -m solve.experiments.q4_price_eda
4  """
5  from __future__ import annotations
6
7  import json
8
9  import numpy as np
10 import pandas as pd
11
12 import program as pm
13 from solve.common import DATA_C, ROOT
14
15 T = 144
16

```

```

17
18 def _acf_curve(x: np.ndarray, max_lag: int, step: int = 1) -> tuple[np.ndarray, np.ndarray]:
19     x = x - x.mean()
20     n = len(x)
21     denom = float((x * x).sum())
22     lags = np.arange(1, max_lag + 1, step)
23     vals = np.array([float((x[:n - k] * x[k:]).sum() / denom) for k in lags])
24     return lags, vals
25
26
27 def main():
28     pm.init(seed=42, root=str(ROOT))
29     df = pm.read_table(DATA_C / "附件4.xlsx")
30     price = df.iloc[:, 1:145].to_numpy(float)
31     dates = pd.to_datetime(df.iloc[:, 0])
32
33     a1 = pm.read_table(DATA_C / "附件1.xlsx")
34     p1 = a1["电价"].to_numpy(float)
35
36     out = {}
37     out["基本统计"] = {
38         "均值": float(price.mean()), "标准差": float(price.std()),
39         "最小": float(price.min()), "P25": float(np.percentile(price, 25)),
40         "中位": float(np.median(price)), "P75": float(np.percentile(price, 75)),
41         "P95": float(np.percentile(price, 95)), "P99": float(np.percentile(price, 99)),
42         "最大": float(price.max()),
43         "偏度": float(pd.Series(price.ravel()).skew()),
44         "超额峰度": float(pd.Series(price.ravel()).kurt()),
45     }
46     print("=== 基本统计 ===")
47     for k, v in out["基本统计"].items():
48         print(f"    {k}: {v:.4f}")
49
50     # 附件 1 = 逐槽全年均值?
51     diff = np.abs(price.mean(axis=0) - p1)
52     print(f"\n附件 1 电价 vs 附件 4 逐槽均值: 最大差 {diff.max():.2e} (应为 0)")
53     out["附件1_vs_均值_最大差"] = float(diff.max())
54
55     # ---- 结构分解 ----
56     m_slot = price.mean(axis=0) # 日内形状
57     day_mean = price.mean(axis=1) # 每天均值
58     resid_intra = price - m_slot # 扣除日内形状
59     R2_intra = 1 - resid_intra.var() / price.var()
60     weekday = np.array([d.weekday() for d in dates])
61     month = np.array([d.month for d in dates])
62     # 周内/月内对"天均值"的解释
63     wd_eff = np.array([day_mean[weekday == w].mean() for w in weekday])
64     mo_eff = np.array([day_mean[month == m].mean() for m in month])
65     R2_wd = 1 - (day_mean - wd_eff).var() / day_mean.var()
66     R2_mo = 1 - (day_mean - mo_eff).var() / day_mean.var()
67     print(f"\n=== 方差分解 ===")
68     print(f"    日内形状 R² (144 槽均值解释总方差): {R2_intra:.3f}")
69     print(f"    周内效应对天均值的 R²: {R2_wd:.3f}")
70     print(f"    月内效应对天均值的 R²: {R2_mo:.3f}")
71     out["方差分解"] = {"日内形状R2": float(R2_intra), "周内R2天均值": float(R2_wd),
72                        "月内R2天均值": float(R2_mo)}
73
74     print("\n    周内均值 (周一~周日):")
75     print("        ", np.round([day_mean[weekday == w].mean() for w in range(7)], 4))
76     print("    月均值:")
77     print("        ", np.round([day_mean[month == m].mean() for m in range(1, 13)], 4))
78
79     # ---- 自相关 ----
80     lags, acf = _acf_curve(price.ravel(), 1008, 1) # 到 7 天 (逐槽)
81     print(f"\n=== 自相关 (槽级) ===")
82     for k in (1, 6, 36, 144, 288, 432, 720, 1008):
83         idx = np.where(lags == k)[0]
84         if len(idx):
85             print(f"    lag {k:4d} 槽 ({k * 10 / 60:5.1f} h) : {acf[idx[0]]:.3f}")

```

```

86 dm_lags, dm_acf = _acf_curve(day_mean, 14)
87 print(" 天均值 ACF (lag 1..7 天):", np.round(dm_acf[:7], 3))
88 out["ACF"] = {"slot_lags": lags.tolist(), "slot_acf": acf.tolist(),
89              "daymean_lags": dm_lags.tolist(), "daymean_acf": dm_acf.tolist()}
90
91 # ---- 预测源可解释性 (D 与源的正确对齐) ----
92 P = price
93 print(f"\n=== 预测源相关性 (D 与源, 逐槽) ===")
94 r_all = {}
95 for lag, name in ((1, "D-1 同时刻"), (2, "D-2 同时刻"), (7, "D-7 同时刻")):
96     a, b = P[lag:], P[:-lag]
97     rr = float(np.corrcoef(a.ravel(), b.ravel())[0, 1])
98     r_all[name] = rr
99     print(f" {name:<12}: r = {rr:.3f}")
100 rr = float(np.corrcoef(P.ravel(), np.tile(p1, (365, 1)).ravel())[0, 1])
101 r_all["典型日(附件1)"] = rr
102 print(f" {'典型日(附件1)':<12}: r = {rr:.3f}")
103 out["源相关"] = r_all
104
105 # 三源联合回归 (逐槽最小二乘, 全样本)
106 m = 365 - 7
107 X = np.stack([P[6:365 - 1], P[5:365 - 2], P[3:365 - 7]], axis=2) # (n,144,3)
108 Y = P[7:]
109 Xf = X.reshape(-1, 3)
110 Yf = Y.ravel()
111 Xd = np.column_stack([Xf, np.ones(len(Xf))])
112 coef, *_ = np.linalg.lstsq(Xd, Yf, rcond=None)
113 pred = Xd @ coef
114 R2 = 1 - ((Yf - pred) ** 2).sum() / ((Yf - Yf.mean()) ** 2).sum()
115 print(f"\n三源 (D-1/D-2/D-7) 联合回归 R² = {R2:.3f}, 系数 {np.round(coef[:3], 3)}")
116 out["三源回归"] = {"R2": float(R2), "coef": coef[:3].tolist()}
117
118 # ---- 极端价格 ----
119 thr = np.percentile(price, 99)
120 mask = price >= thr
121 slot_share = mask.sum(axis=0) / mask.sum()
122 top_slots = np.argsort(slot_share)[: -1][:10]
123 hot_hours = sorted(set(int(s * 10 // 60) for s in top_slots))
124 print(f"\n=== 极端价格 (P99={thr:.3f}) ===")
125 print(f" 样本数 {int(mask.sum())} (占 {100 * mask.mean():.2f}%), 最集中时段小时: {hot_hours}")
126 out["极端价格"] = {"P99": float(thr), "n": int(mask.sum()), "热点小时": hot_hours}
127
128 # ---- 与负荷/光伏的相关性 ----
129 sheets = pm.read_sheets(DATA_C / "附件2.xlsx")
130 load = sheets["小区负载"].iloc[:, 1:145].to_numpy(float)
131 pv = sheets["光伏发电实际功率"].iloc[:, 1:145].to_numpy(float)
132 print(f"\n=== 与负荷/光伏的同时刻相关 ===")
133 for name, arr in (("负荷", load), ("光伏", pv)):
134     rr = float(np.corrcoef(price.ravel(), arr.ravel())[0, 1])
135     print(f" {name}: r = {rr:.3f}")
136     out.setdefault("相关_负荷光伏", {})[name] = rr
137
138 # ---- 电价峰谷的日内时段特征 (用于口径说明) ----
139 lo = np.argsort(m_slot)[:12]
140 hi = np.argsort(m_slot)[: -1][:12]
141 print(f"\n日内均值最低时段 (槽索引):", lo.tolist())
142 print(f"日内均值最高时段 (槽索引):", sorted(hi.tolist()))
143 out["日内极值槽"] = {"最低": sorted(lo.tolist()), "最高": sorted(hi.tolist())}
144
145 with open(ROOT / "code" / "outputs" / "q4_price_eda.json", "w", encoding="utf-8") as f:
146     json.dump(out, f, ensure_ascii=False, indent=2)
147
148 # ---- 图: ACF + 结构分解 ----
149 import matplotlib.pyplot as plt
150 fig, ax = plt.subplots(figsize=(7, 4))
151 ax.plot(lags / 6.0, acf, lw=0.9)
152 for k, lab in ((144 / 6, "1 天"), (720 / 6, "5 天"), (1008 / 6, "7 天")):
153     ax.axvline(k, color="gray", ls="--", lw=0.7)
154     ax.text(k, ax.get_ylim()[1] * 0.9, lab, fontsize=8, ha="center")

```

```

155     ax.set_xlabel("滞后 / 小时"); ax.set_ylabel("ACF")
156     pm.save_fig(fig, "Q4_电价自相关", data=pd.DataFrame({"滞后_小时": lags / 6.0, "ACF": acf}))
157
158     fig2, axes = plt.subplots(1, 3, figsize=(10, 3.2))
159     t_h = (np.arange(T) + 0.5) / 6.0
160     axes[0].plot(t_h, m_slot, lw=1.2); axes[0].set_xlabel("时刻 / h"); axes[0].set_ylabel("电价 / (元 • kWh$^{-1}$)")
161     axes[0].set_title("日内均值曲线", fontsize=9)
162     axes[1].bar(np.arange(7), [day_mean[weekday == w].mean() for w in range(7)],
163                color="#4C72B0"); axes[1].set_xticks(np.arange(7))
164     axes[1].set_xticklabels(["一", "二", "三", "四", "五", "六", "日"])
165     axes[1].set_title("周内均值 (天均值)", fontsize=9)
166     axes[2].bar(np.arange(1, 13), [day_mean[month == m].mean() for m in range(1, 13)],
167                color="#55A868")
168     axes[2].set_title("月均值 (天均值)", fontsize=9)
169     fig2.tight_layout()
170     pm.save_fig(fig2, "Q4_电价结构分解",
171                data=pd.DataFrame({"时段": np.arange(T), "日内均值": m_slot}))
172     print("\n已保存 code/outputs/q4_price_eda.json 与两张图")
173
174
175 if __name__ == "__main__":
176     main()

```

legacy

program/src/solve/legacy/2.py

```

1  from data_reader import df21, df22, df1
2  from lp_model import LPModel
3  import pandas as pd
4  import numpy as np
5
6
7  if __name__ == "__main__":
8      data_L, data_P = df21.to_numpy(), df22.to_numpy()
9      prices, L, solar = (
10         np.tile(df1.to_numpy()[:, 1], 344),
11         data_L[:, 1:].flatten(),
12         data_P[:, 1:].flatten(),
13     )
14
15     model = LPModel()
16
17     x, fun = model(prices, L, solar, 6000, 6000)
18     print(fun)
19     df1 = pd.read_excel("./data/C/附件5/result1.xlsx")
20
21     # df1["购电量"] = x[:144]
22     # df1.to_excel("./res/result1.xlsx")

```

program/src/solve/legacy/__init__.py

```

1  """早期脚本（原样保留，勿改）。
2
3  这些脚本是 C 题最初的手写探索（2.py / solve1.py 等），依赖同目录下的
4  ``data_reader`` / ``lp_model`` 以绝对模块名导入，运行方式：
5
6      cd program
7      uv run python src/solve/legacy/2.py
8
9  （脚本目录会被加入 sys.path，因此 data_reader / lp_model 可直接导入；
10  路径 ``./data/C/...`` 相对 cwd，必须在 program/ 下运行。）
11  """

```

program/src/solve/legacy/arima.py

```

1 import numpy as np
2 import pandas as pd
3 import numpy as np
4 import matplotlib.pyplot as plt
5
6 from statsmodels.tsa.arima.model import ARIMA
7 from statsmodels.tsa.stattools import adfuller
8
9 from sklearn.metrics import mean_squared_error, mean_absolute_error
10
11 from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
12
13
14 class Arima:
15     def __init__(self, series):
16         self.model = ARIMA

```

program/src/solve/legacy/data_reader.py

```

1 import matplotlib.pyplot as plt
2 import numpy as np
3 import pandas as pd
4 from sklearn.metrics import mean_absolute_error, mean_squared_error
5 from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
6 from statsmodels.tsa.arima.model import ARIMA
7 from statsmodels.tsa.stattools import adfuller
8
9 df1 = pd.read_excel("./data/C/附件1.xlsx") # 典型天
10 df22 = pd.read_excel("./data/C/附件2.xlsx", sheet_name="光伏发电实际功率")
11 df21 = pd.read_excel("./data/C/附件2.xlsx", sheet_name="小区负载")
12 df3 = pd.read_excel("./data/C/附件3.xlsx") # 天气预测
13 df4 = pd.read_excel("./data/C/附件4.xlsx") # 价格实际波动
14
15 dst = "./figures/"
16
17
18 def steady():
19     data = df22.to_numpy()[1:, 1:]
20
21     data = data.flatten()[1:343].reshape(-1, 7).sum(axis=0)
22
23     # print(adfuller(data))
24
25     # data = data[1:] - data[:-1]
26
27     adf = adfuller(data)
28
29     print(adf)
30
31     # fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))
32
33     # plot_acf(data, lags=40, ax=ax1) # 图上看截尾/拖尾
34     # plot_pacf(data, lags=40, method="yw", ax=ax2)
35
36     plt.plot(data)
37
38     plt.show()
39
40
41 def see():
42     data = df21.to_numpy()[1:, 1:]
43
44     data = data[1:343].reshape(-1, 7)
45     for row in data:
46         plt.scatter(np.arange(len(row)), row, s=4)
47
48     plt.show()
49
50

```

```

51 if __name__ == "__main__":
52     # steady()
53     see()

```

program/src/solve/legacy/lp_model.py

```

1  import numpy as np
2  from data_reader import df1
3  from scipy.optimize import linprog
4
5
6  class LPModel:
7      def __init__(self):
8          self.E_up = 10800.0
9          self.E_down = 1200.0
10
11          self.P = 5000.0
12          self.eta = 0.9
13
14      def __call__(self, prices, L, solar, E0, Ee=None):
15          "传入 价格 小区载荷 , 光伏发电 , 起始值和终点值"
16          T = len(prices)
17
18          def _idx(m, n):
19              return m * T + n
20
21          C = np.hstack([prices, np.zeros(T * 3)])
22
23          A_uq = np.zeros((T, T * 4))
24
25          for t in range(T):
26              A_uq[t, _idx(2, t)], A_uq[t, _idx(1, t)], A_uq[t, _idx(0, t)] = (
27                  1.0,
28                  -1.0,
29                  -1.0,
30              )
31
32          B_uq = solar - L
33
34          A_eq = np.zeros((T + 1, T * 4))
35          for t in range(T):
36              A_eq[t, _idx(3, t)] = 1.0
37              if t:
38                  A_eq[t, _idx(3, t - 1)] = -1.0
39
40              A_eq[t, _idx(1, t)] = 1 / self.eta
41              A_eq[t, _idx(2, t)] = -self.eta
42
43          A_eq[T, _idx(3, T - 1)] = 1.0 if Ee is not None else 0.0
44          B_eq = np.zeros(T + 1)
45          B_eq[0], B_eq[T] = E0, Ee if Ee is not None else 0.0
46
47          bounds = (
48              [(0, None) for _ in range(T)]
49              + [(0, self.P / 6) for _ in range(T)]
50              + [(0, self.P / 6) for _ in range(T)]
51              + [(self.E_down, self.E_up) for _ in range(T)]
52          )
53
54          res = linprog(C, A_ub=A_uq, b_ub=B_uq, A_eq=A_eq, b_eq=B_eq, bounds=bounds)
55
56          return res.x, res.fun
57
58
59 if __name__ == "__main__":
60     data = df1.to_numpy()
61     prices, L, solar = data[:, 1], data[:, 2] / 6, data[:, 3] / 6
62
63     model = LPModel()

```

```

64
65     x, fun = model(prices, L, solar, 6000, 6000)
66
67     print(fun)

```

program/src/solve/legacy/solve1.py

```

1  from data_reader import df1
2  from lp_model import LPModel
3  import pandas as pd
4
5
6  if __name__ == "__main__":
7      data = df1.to_numpy()
8      prices, L, solar = data[:, 1], data[:, 2] / 6, data[:, 3] / 6
9
10     model = LPModel()
11
12     x, fun = model(prices, L, solar, 6000, 6000)
13     print(fun)
14     df1 = pd.read_excel("./data/C/附件5/result1.xlsx")
15
16     df1["购电量"] = x[:144]
17     df1.to_excel("./res/result1.xlsx")

```

program/src/solve/legacy/solve2.py

```

1  from sklearn.linear_model import LinearRegression

```

工具箱

program/src/program/__init__.py

```

1  """program —— 数学建模便利工具包（CUMCM / 华为杯 / MCM 通用）。
2
3  十一个模块覆盖数学建模全流程：
4
5  =====
6  模块                               用途
7  =====
8  :mod:`program.config`               全局配置：中文字体、论文风格、输出目录、随机种子
9  :mod:`program.utils`                随机种子、计时、日志、结果缓存、运行环境、格式化
10 :mod:`program.dataio`                数据读写（csv/xlsx/json/pkl）、缺失处理、概览
11 :mod:`program.plotting`              论文级绘图：PDF 矢量输出 + 常用图型
12 :mod:`program.stats`                 统计检验、相关、回归、置信区间（中文结论）
13 :mod:`program.fitting`               曲线拟合、多项式、插值、平滑
14 :mod:`program.metrics`               回归 / 分类 / 聚类评估指标
15 :mod:`program.optimize`              LP / MILP / NLP / 全局优化 / 背包 / 指派 / TSP
16 :mod:`program.ml`                    机器学习快捷流程（训练 + 交叉验证 + 评估）
17 :mod:`program.ode`                   常微分方程求解与参数反演
18 :mod:`program.report`                 RESULTS_REPORT 记录、LaTeX 表格、结果文件输出
19  =====
20
21  标准脚本骨架::
22
23      import program as pm
24
25      pm.init()                        # 中文字体 + 风格 + 随机种子
26      df = pm.read_table("data/附件1.xlsx") # 数据
27      ...                               # 建模
28      pm.record_result("问题一结果", {"最优值": 42}) # 结果写进 reports/RESULTS_REPORT.md
29      pm.line(x, y, xlabel="t", ylabel="y", save="q1_curve") # 图存 figures/q1_curve.pdf
30
31  完整说明见 ``program/AGENTS.md``。
32  """
33

```

```

34 from __future__ import annotations
35
36 from . import ( # noqa: F401 (统一导入, 便于 pm.stats.xxx 访问)
37     config,
38     dataio,
39     fitting,
40     metrics,
41     ml,
42     ode,
43     optimize,
44     plotting,
45     report,
46     stats,
47     utils,
48 )
49
50 # 高频函数提升到顶层, 便于 pm.read_table / pm.save_fig 直接使用
51 from .config import ( # noqa: F401
52     find_cjk_font,
53     figures_dir,
54     init,
55     outputs_dir,
56     project_root,
57     reports_dir,
58     set_project_root,
59 )
60 from .dataio import read_sheets, read_table, save_table, summarize # noqa: F401
61 from .plotting import ( # noqa: F401
62     bar,
63     bar_group,
64     box,
65     corr_heatmap,
66     dual_axis,
67     errorbar,
68     heatmap,
69     hist,
70     line,
71     pie,
72     radar,
73     roc,
74     save_fig,
75     scatter,
76     scatter_fit,
77     surface3d,
78 )
79 from .report import record, record_environment, record_result, save_outputs, to_latex # noqa: F401
80 from .utils import DataCache, Timer, fmt, get_logger, markdown_table, set_seed # noqa: F401
81
82 __version__ = "0.2.0"
83
84 __all__ = [
85     # 子模块
86     "config", "dataio", "fitting", "metrics", "ml", "ode", "optimize",
87     "plotting", "report", "stats", "utils",
88     # 配置与工具
89     "init", "set_seed", "get_logger", "Timer", "DataCache",
90     "project_root", "set_project_root", "figures_dir", "reports_dir", "outputs_dir",
91     "find_cjk_font", "fmt", "markdown_table",
92     # 数据
93     "read_table", "read_sheets", "save_table", "summarize",
94     # 绘图
95     "save_fig", "line", "scatter", "scatter_fit", "bar", "bar_group", "hist",
96     "box", "heatmap", "corr_heatmap", "radar", "dual_axis", "pie", "errorbar",
97     "surface3d", "roc",
98     # 报告
99     "record", "record_result", "record_environment", "save_outputs", "to_latex",
100 ]
101
102

```

```

103 def main() -> None:
104     """CLI 入口 (`uv run program`)：打印包信息与模块清单。"""
105     print(f"program —— 数学建模便利工具包 v{__version__}")
106     print()
107     print("模块: ")
108     for name in (
109         "config", "utils", "dataio", "plotting", "stats", "fitting",
110         "metrics", "optimize", "ml", "ode", "report",
111     ):
112         mod = __import__(f"program.{name}", fromlist=[name])
113         doc = (mod.__doc__ or "").strip().splitlines()[0]
114         print(f"  program.{name:<10} {doc}")
115     print()
116     print("快速开始: ")
117     print("  import program as pm")
118     print("  pm.init()")
119     print("  df = pm.read_table('data.xlsx')")
120     print("  pm.record_result('问题-结果', {'目标值': 1.23})")
121     print("  pm.line(x, y, save='fig1')")
122     print()
123     print("详细文档: program/AGENTS.md")

```

program/src/program/config.py

```

1  """全局配置与初始化（中文字体、论文风格、随机种子、输出目录）。
2
3  用法::
4
5      import program as pm
6
7      pm.init()                # 一键初始化：中文字体 + 论文风格 + 随机种子
8      pm.init(seed=0, root=r"D:\case") # 自定义随机种子与项目根目录
9
10  输出目录约定（全部从 :func:`project_root` 派生，默认为当前工作目录，
11  可用环境变量 ``MATHMODEL_ROOT`` 或 :func:`set_project_root` 覆盖）：
12
13  - ``figures``          论文图表（:func:`program.plotting.save_fig` 默认输出）
14  - ``reports``         ``RESULTS_REPORT.md`` 等报告（:func:`program.report.record_result`）
15  - ``code/outputs``    中间数据、图表数据、log（:func:`program.report.save_outputs`）
16  """
17
18  from __future__ import annotations
19
20  import os
21  from pathlib import Path
22
23  # -----
24  # 论文配色: seaborn "deep" 风格 10 色循环，对色盲友好、印刷清晰
25  # -----
26  PALETTE: tuple[str, ...] = (
27      "#4C72B0", "#DD8452", "#55A868", "#C44E52", "#8172B3",
28      "#937860", "#DA8BC3", "#8C8C8C", "#CCB974", "#64B5CD",
29  )
30
31  # 中文字体候选（按平台常见度排序）
32  CJK_FONTS: tuple[str, ...] = (
33      "Microsoft YaHei", "SimHei", "SimSun", "KaiTi", "FangSong", # Windows
34      "PingFang SC", "Hiragino Sans GB", "Songti SC", "Heiti SC", # macOS
35      "Noto Sans CJK SC", "Noto Serif CJK SC", "Source Han Sans SC",
36      "WenQuanYi Zen Hei", "WenQuanYi Micro Hei", # Linux
37  )
38
39  _ROOT: Path | None = None
40
41
42  # -----
43  # 项目根目录与输出目录
44  # -----
45  def set_project_root(path: str | Path) -> Path:

```

```

46     """显式指定项目根目录（覆盖环境变量 ``MATHMODEL_ROOT`` 与当前工作目录）。"""
47     global _ROOT
48     _ROOT = Path(path).expanduser().resolve()
49     return _ROOT
50
51
52 def project_root() -> Path:
53     """项目根目录：显式设置 > 环境变量 ``MATHMODEL_ROOT`` > 当前工作目录。"""
54     if _ROOT is not None:
55         return _ROOT
56     env = os.environ.get("MATHMODEL_ROOT")
57     if env:
58         return Path(env).expanduser().resolve()
59     return Path.cwd()
60
61
62 def figures_dir() -> Path:
63     """论文图表目录 ``<root>/figures``。"""
64     return project_root() / "figures"
65
66
67 def reports_dir() -> Path:
68     """报告目录 ``<root>/reports``。"""
69     return project_root() / "reports"
70
71
72 def outputs_dir() -> Path:
73     """中间数据目录 ``<root>/code/outputs``。"""
74     return project_root() / "code" / "outputs"
75
76
77 def figure_data_dir() -> Path:
78     """图表对应的数据目录 ``<root>/code/outputs/figure_data``。"""
79     return outputs_dir() / "figure_data"
80
81
82 # -----
83 # 中文字体探测
84 # -----
85 def find_cjk_font() -> str | None:
86     """返回本机第一个可用的中文字体名，找不到返回 ``None``。"""
87     from matplotlib import font_manager
88
89     available = {f.name for f in font_manager.fontManager.ttflist}
90     for name in CJK_FONTS:
91         if name in available:
92             return name
93     return None
94
95
96 # -----
97 # 一键初始化
98 # -----
99 def init(
100     seed: int | None = 42,
101     root: str | Path | None = None,
102     chinese: bool = True,
103     backend: str | None = "Agg",
104     figsize: tuple[float, float] = (7.0, 4.3),
105     verbose: bool = True,
106 ) -> dict:
107     """初始化绘图环境与随机种子，脚本开头调用一次即可。
108
109     Parameters
110     -----
111     seed : int | None
112         全局随机种子（random / numpy / torch / PYTHONHASHSEED），``None`` 表示不设置。
113     root : str | Path | None
114         项目根目录，传给 :func:`set_project_root`。

```

```

115     chinese : bool
116         是否自动探测并启用中文字体。
117     backend : str | None
118         matplotlib 后端, 默认 ``"Agg"`` (无窗口、稳定保存文件);
119         需要交互式窗口时传 ``None`` 或 ``"TkAgg"``。
120     figsize : (float, float)
121         默认图幅尺寸 (英寸), 论文单栏常用 7x4.3。
122     verbose : bool
123         是否打印初始化摘要。
124
125     Returns
126     -----
127     dict
128         实际生效的配置摘要 (根目录、字体、后端、种子)。
129     """
130     if root is not None:
131         set_project_root(root)
132
133     if seed is not None:
134         from .utils import set_seed
135
136         set_seed(seed)
137
138     # 后端必须在 pyplot 首次导入前设置
139     import matplotlib
140
141     if backend:
142         try:
143             matplotlib.use(backend, force=True)
144         except Exception: # 后端不可用时静默回退
145             pass
146
147     import matplotlib.pyplot as plt
148     from cycler import cycler
149
150     font_name = find_cjk_font() if chinese else None
151
152     rc: dict = {
153         # 图幅与分辨率
154         "figure.figsize": figsize,
155         "figure.dpi": 110,
156         "savefig.dpi": 300,
157         "savefig.bbox": "tight",
158         # 字号
159         "font.size": 10.5,
160         "axes.titlesize": 11.5,
161         "axes.labelsize": 10.5,
162         "xtick.labelsize": 9.5,
163         "ytick.labelsize": 9.5,
164         "legend.fontsize": 9.5,
165         # 网格与边框
166         "axes.grid": True,
167         "grid.alpha": 0.35,
168         "grid.linestyle": "--",
169         "grid.linewidth": 0.6,
170         "axes.spines.top": False,
171         "axes.spines.right": False,
172         # 线条与配色
173         "lines.linewidth": 1.6,
174         "lines.markersize": 5,
175         "legend.frameon": False,
176         "axes.prop_cycle": cycler(color=PALETTE),
177         # 负号与数学字体
178         "axes.unicode_minus": False,
179         "mathtext.fontset": "dejavusans",
180     }
181     if font_name:
182         rc["font.family"] = "sans-serif"
183         rc["font.sans-serif"] = [font_name, "DejaVu Sans", "Arial"]

```

```

184     plt.rcParams.update(rc)
185
186     summary = {
187         "root": str(project_root()),
188         "font": font_name or "（未找到中文字体，中文可能显示为方框）",
189         "backend": matplotlib.get_backend(),
190         "seed": seed,
191     }
192     if verbose:
193         from loguru import logger
194
195         logger.info(
196             "program 初始化完成 | root={root} | 中文字体={font} | 后端={backend} | seed={seed}",
197             **summary,
198         )
199     return summary

```

program/src/program/dataio.py

```

1  """数据读取、清洗与概览。
2
3  覆盖数学建模常见数据格式（csv / tsv / txt / xlsx / json / pkl），
4  自动处理中文编码（UTF-8 / GBK），并提供缺失值、数据概览等清洗工具。
5
6  用法::
7
8      import program as pm
9
10     df = pm.read_table("data/附件1.xlsx", sheet="Sheet1")
11     print(pm.dataio.summarize(df))          # 数据概览（可直接贴进报告）
12     df = pm.dataio.fill_missing(df)         # 数值列均值 / 类别列众数填充
13     X_train, X_test, y_train, y_test = pm.dataio.train_test_split_df(
14         df, target="销量", test_size=0.2, seed=42
15     )
16 """
17
18 from __future__ import annotations
19
20 import datetime
21 import json
22 from pathlib import Path
23 from typing import Any, Sequence
24
25 import numpy as np
26 import pandas as pd
27
28 from .utils import markdown_table
29
30
31 # -----
32 # 读取
33 # -----
34 def _read_csv_smart(
35     path: Path,
36     sep: str | None,
37     encoding: str | None,
38     **kwargs: Any,
39 ) -> pd.DataFrame:
40     if sep is None:
41         kwargs.setdefault("engine", "python")
42     encodings = [encoding] if encoding else ["utf-8-sig", "gb18030"]
43     last_err: Exception | None = None
44     for enc in encodings:
45         try:
46             return pd.read_csv(path, sep=sep, encoding=enc, **kwargs)
47         except UnicodeDecodeError as err: # 中文附件常见 GBK 编码
48             last_err = err
49     raise last_err if last_err else ValueError(f"无法读取 {path}")
50

```

```

51
52 def read_table(
53     path: str | Path,
54     *,
55     sheet: str | int | None = 0,
56     sep: str | None = None,
57     encoding: str | None = None,
58     **kwargs: Any,
59 ) -> pd.DataFrame:
60     """读取数据表, 按扩展名自动分派, 返回 DataFrame。
61
62     - ``.csv``: 默认逗号分隔, 优先 UTF-8, 失败自动尝试 GB18030 (GBK 超集)
63     - ``.tsv``: 制表符分隔
64     - ``.txt``: 自动嗅探分隔符
65     - ``.xlsx / .xls / .xlsm``: Excel, ``sheet`` 指定工作表 (默认第一个)
66     - ``.json``: 尝试 ``pd.read_json``, 失败则按记录列表解析
67     - ``.pkl / .pickle``: pickle
68
69     其余 ``**kwargs`` 透传给底层 ``pandas`` 读取函数。
70     """
71     p = Path(path)
72     if not p.exists():
73         raise FileNotFoundError(f"数据文件不存在: {p.resolve()}")
74     suf = p.suffix.lower()
75
76     if suf == ".csv":
77         return _read_csv_smart(p, "," if sep is None else sep, encoding, **kwargs)
78     if suf == ".tsv":
79         return _read_csv_smart(p, "\t" if sep is None else sep, encoding, **kwargs)
80     if suf == ".txt":
81         return _read_csv_smart(p, sep, encoding, **kwargs)
82     if suf in {".xlsx", ".xls", ".xlsm"}:
83         return pd.read_excel(p, sheet_name=sheet if sheet is not None else 0, **kwargs)
84     if suf == ".json":
85         try:
86             return pd.read_json(p, **kwargs)
87         except ValueError:
88             data = load_json(p)
89             if isinstance(data, dict):
90                 data = list(data.values())[0] if data else []
91             return pd.DataFrame(data)
92     if suf in {".pkl", ".pickle"}:
93         return pd.read_pickle(p)
94     raise ValueError(f"暂不支持的文件类型: {suf} (支持 csv/tsv/txt/xlsx/json/pkl)")
95
96
97 def read_sheets(path: str | Path, **kwargs: Any) -> dict[str, pd.DataFrame]:
98     """读取 Excel 全部工作表, 返回 ``{表名: DataFrame}``。"""
99     return pd.read_excel(path, sheet_name=None, **kwargs)
100
101
102 # -----
103 # 写出
104 # -----
105 def save_table(df: pd.DataFrame, path: str | Path, *, index: bool = False) -> Path:
106     """把 DataFrame 写出到 csv / xlsx / json / pkl (按扩展名分派)。
107
108     写 csv 时使用 ``utf-8-sig`` 编码, Excel 打开不乱码。
109     """
110     p = Path(path)
111     p.parent.mkdir(parents=True, exist_ok=True)
112     suf = p.suffix.lower()
113     if suf == ".csv":
114         df.to_csv(p, index=index, encoding="utf-8-sig")
115     elif suf in {".xlsx", ".xls"}:
116         df.to_excel(p, index=index)
117     elif suf == ".json":
118         save_json(df.to_dict(orient="records"), p)
119     elif suf in {".pkl", ".pickle"}:

```

```

120         df.to_pickle(p)
121     else:
122         raise ValueError(f"暂不支持的写出格式: {suf}")
123     return p
124
125
126 def _json_default(o: Any) -> Any:
127     if isinstance(o, (np.integer,)):
128         return int(o)
129     if isinstance(o, (np.floating,)):
130         return None if np.isnan(o) else float(o)
131     if isinstance(o, np.ndarray):
132         return o.tolist()
133     if isinstance(o, np.bool_):
134         return bool(o)
135     if isinstance(o, Path):
136         return str(o)
137     if isinstance(o, (datetime.date, datetime.datetime, pd.Timestamp)):
138         return o.isoformat()
139     raise TypeError(f"JSON 无法序列化类型: {type(o)!r}")
140
141
142 def save_json(obj: Any, path: str | Path, indent: int = 2) -> Path:
143     """保存 JSON (自动处理 numpy 标量/数组、路径、日期)。"""
144     p = Path(path)
145     p.parent.mkdir(parents=True, exist_ok=True)
146     p.write_text(
147         json.dumps(obj, ensure_ascii=False, indent=indent, default=_json_default),
148         encoding="utf-8",
149     )
150     return p
151
152
153 def load_json(path: str | Path) -> Any:
154     """读取 JSON 文件。"""
155     return json.loads(Path(path).read_text(encoding="utf-8"))
156
157
158 # -----
159 # 清洗
160 # -----
161 def clean_columns(df: pd.DataFrame, *, lower: bool = False) -> pd.DataFrame:
162     """规整列名: 去除首尾空格、把「Unnamed」空列名替换为 ``col{i}``。"""
163     df = df.copy()
164     new_cols: list[str] = []
165     for i, c in enumerate(df.columns):
166         name = str(c).strip()
167         if not name or name.lower().startswith("unnamed"):
168             name = f"col{i}"
169         if lower:
170             name = name.lower()
171         new_cols.append(name)
172     df.columns = new_cols
173     return df
174
175
176 def missing_report(df: pd.DataFrame) -> pd.DataFrame:
177     """每列缺失情况统计表 (缺失数 / 缺失占比 / 非空数)。"""
178     n = len(df)
179     missing = df.isna().sum()
180     return pd.DataFrame(
181         {
182             "列名": [str(c) for c in df.columns],
183             "类型": [str(t) for t in df.dtypes],
184             "非空数": (n - missing).to_numpy(),
185             "缺失数": missing.to_numpy(),
186             "缺失占比": (missing / max(n, 1)).to_numpy(),
187         }
188     )

```

```

189
190
191 def fill_missing(
192     df: pd.DataFrame,
193     strategy: str = "auto",
194     cols: Sequence[str] | None = None,
195 ) -> pd.DataFrame:
196     """缺失值填充, 返回新 DataFrame (不修改原表)。
197
198     Parameters
199     -----
200     strategy : str
201         - ``"auto"``: 数值列均值、类别列众数 (推荐)
202         - ``"mean" / "median" / "mode" / "zero" / "ffill" / "bfill"``
203         - ``"drop"``: 删除含缺失的行
204     cols : 序列 | None
205         只处理指定列, 默认全部列。
206     """
207     if strategy == "drop":
208         return df.dropna(subset=list(cols) if cols else None).copy()
209
210     out = df.copy()
211     target_cols = list(cols) if cols else list(out.columns)
212     for c in target_cols:
213         s = out[c]
214         if not s.isna().any():
215             continue
216         if strategy == "auto":
217             st = "mean" if pd.api.types.is_numeric_dtype(s) else "mode"
218         else:
219             st = strategy
220         if st == "mean":
221             out[c] = s.fillna(s.mean())
222         elif st == "median":
223             out[c] = s.fillna(s.median())
224         elif st == "mode":
225             m = s.mode()
226             out[c] = s.fillna(m.iloc[0] if len(m) else "未知")
227         elif st == "zero":
228             out[c] = s.fillna(0)
229         elif st == "ffill":
230             out[c] = s.ffill()
231         elif st == "bfill":
232             out[c] = s.bfill()
233         else:
234             raise ValueError(f"未知填充策略: {strategy!r}")
235     return out
236
237
238 # -----
239 # 概览与转换
240 # -----
241 def column_report(df: pd.DataFrame) -> pd.DataFrame:
242     """每列概况: 类型 / 非空 / 缺失 / 唯一值数 / 最小值 / 最大值 / 均值。"""
243     rows = []
244     for c in df.columns:
245         s = df[c]
246         row: dict[str, Any] = {
247             "列名": str(c),
248             "类型": str(s.dtype),
249             "非空数": int(s.notna().sum()),
250             "缺失数": int(s.isna().sum()),
251             "唯一值": int(s.nunique(dropna=True)),
252         }
253         if pd.api.types.is_numeric_dtype(s):
254             row.update(
255                 最小值=s.min(),
256                 最大值=s.max(),
257                 均值=s.mean(),

```

```

258         )
259         else:
260             row.update(最小值=None, 最大值=None, 均值=None)
261             rows.append(row)
262         return pd.DataFrame(rows)
263
264
265 def summarize(df: pd.DataFrame, n: int = 5, title: str = "数据概览") -> str:
266     """生成 DataFrame 的 Markdown 概览 (形状 + 列信息 + 前 n 行)。
267
268     直接把返回值交给 ``report.record`` 即可写入 RESULTS_REPORT。
269     """
270     parts = [
271         f"***{title}***: {df.shape[0]} 行 × {df.shape[1]} 列, "
272         f"重复行 {int(df.duplicated().sum())}, 缺失单元格 {int(df.isna().sum().sum())}"
273     ]
274     parts.append(markdown_table(column_report(df)))
275     parts.append(f"***前 {n} 行**\n")
276     parts.append(markdown_table(df.head(n)))
277     return "\n\n".join(parts)
278
279
280 def split_xy(
281     df: pd.DataFrame,
282     target: str,
283     drop: Sequence[str] | None = None,
284 ) -> tuple[pd.DataFrame, pd.Series]:
285     """按目标列拆分特征矩阵 X 与标签 y (y 从 X 中移除, ``drop`` 可再排除列)。"""
286     if target not in df.columns:
287         raise KeyError(f"目标列不存在: {target}; 现有列: {list(df.columns)}")
288     y = df[target]
289     drop_cols = [target] + list(drop or [])
290     X = df.drop(columns=drop_cols)
291     return X, y
292
293
294 def to_numpy(df: pd.DataFrame, *, numeric_only: bool = True) -> np.ndarray:
295     """DataFrame → float ndarray (默认只取数值列)。"""
296     d = df.select_dtypes(include="number") if numeric_only else df
297     return np.asarray(d, dtype=float)
298
299
300 def train_test_split_df(
301     df: pd.DataFrame,
302     target: str | None = None,
303     *,
304     test_size: float = 0.2,
305     seed: int = 42,
306     stratify: bool = False,
307 ) -> tuple:
308     """DataFrame 版的训练/测试划分 (内部固定随机种子)。
309
310     - ``target=None``: 返回 ``(train_df, test_df)``
311     - ``target="列名"``: 返回 ``(X_train, X_test, y_train, y_test)`` , 保持 DataFrame/Series
312     """
313     from sklearn.model_selection import train_test_split
314
315     if target is None:
316         return train_test_split(df, test_size=test_size, random_state=seed)
317     X, y = split_xy(df, target)
318     return train_test_split(
319         X, y, test_size=test_size, random_state=seed,
320         stratify=y if stratify else None,
321     )

```

program/src/program/fitting.py

```

1 """曲线拟合、插值与平滑。
2

```

```

3  - :func:`fit_curve`: 任意函数最小二乘拟合 (curve_fit 包装, 带 R2/标准误)
4  - :func:`polyfit` / :func:`poly_scan`: 多项式拟合与自动定阶
5  - :func:`interp` / :func:`spline_smooth`: 插值与平滑样条
6  - :func:`smooth`: Savitzky-Golay / 滑动平均 / 指数平滑
7
8  用法::
9
10     import numpy as np
11     from program import fitting
12
13     def growth(x, a, b, c):
14         return a / (1 + np.exp(-b * (x - c)))    # Logistic
15
16     r = fitting.fit_curve(growth, t, y, p0=[10, 0.5, 5], names=["K", "r", "t0"])
17     print(r.summary())
18     y_hat = r.predict(t_new)
19 """
20
21 from __future__ import annotations
22
23 from dataclasses import dataclass
24 from typing import Any, Callable, Sequence
25
26 import numpy as np
27
28 from .utils import fmt
29
30
31 # -----
32 # 拟合结果容器
33 # -----
34 @dataclass
35 class FitResult:
36     """拟合结果: ``predict`` 求预测值, ``summary`` 打印摘要。"""
37
38     func: Callable[..., np.ndarray]
39     params: np.ndarray
40     stderr: np.ndarray | None
41     r2: float
42     rmse: float
43     n: int
44     names: list[str]
45     kind: str = ""
46     x: Any = None
47     y: Any = None
48
49     def predict(self, x: Sequence[float]) -> np.ndarray:
50         """用拟合参数计算预测值。"""
51         return self.func(np.asarray(x, dtype=float), *self.params)
52
53     @property
54     def residuals(self) -> np.ndarray:
55         if self.x is None or self.y is None:
56             raise ValueError("未保存原始数据, 无法计算残差")
57         return np.asarray(self.y, dtype=float) - self.predict(self.x)
58
59     def param_dict(self) -> dict[str, float]:
60         return {name: float(v) for name, v in zip(self.names, self.params)}
61
62     def summary(self) -> str:
63         lines = [
64             f"拟合结果{' (' + self.kind + ') ' if self.kind else ''}: "
65             f"R2 = {fmt(self.r2, 5)}, RMSE = {fmt(self.rmse, 5)}, n = {self.n}"
66         ]
67         for i, (name, v) in enumerate(zip(self.names, self.params)):
68             if self.stderr is not None and i < len(self.stderr) and np.isfinite(self.stderr[i]):
69                 lines.append(f"    {name} = {fmt(v, 6)} ± {fmt(self.stderr[i], 3)}")
70             else:
71                 lines.append(f"    {name} = {fmt(v, 6)}")

```

```

72         return "\n".join(lines)
73
74     __str__ = summary
75
76
77 def _metrics(y: np.ndarray, y_hat: np.ndarray) -> tuple[float, float]:
78     ss_res = float(np.sum((y - y_hat) ** 2))
79     ss_tot = float(np.sum((y - y.mean()) ** 2))
80     r2 = 1 - ss_res / ss_tot if ss_tot > 0 else float("nan")
81     return r2, float(np.sqrt(np.mean((y - y_hat) ** 2)))
82
83
84 # -----
85 # 通用曲线拟合
86 # -----
87 def fit_curve(
88     func: Callable[..., Any],
89     x: Sequence[float],
90     y: Sequence[float],
91     p0: Sequence[float] | None = None,
92     *,
93     bounds: tuple = (-np.inf, np.inf),
94     names: Sequence[str] | None = None,
95     sigma: Sequence[float] | None = None,
96     maxfev: int = 20000,
97     kind: str = "",
98 ) -> FitResult:
99     """非线性最小二乘拟合 ``y = func(x, *params)``。
100
101     Parameters
102     -----
103     func : callable
104         ``f(x, a, b, ...) -> y``，第一个参数是自变量。
105     x, y : 序列
106         观测数据。
107     p0 : 序列 | None
108         参数初值（建议给出，避免陷入局部最优）。
109     bounds : tuple
110         ``(下界, 上界)``，元素为标量或与参数等长的序列。
111     names : 序列 | None
112         参数名（用于摘要），默认 ``p0, p1, ...``。
113
114     Returns
115     -----
116     FitResult
117         ``r.predict(x_new)`` / ``r.summary()`` / ``r.param_dict()``。
118     """
119     from scipy.optimize import curve_fit
120
121     xa = np.asarray(x, dtype=float).ravel()
122     ya = np.asarray(y, dtype=float).ravel()
123     popt, pcov = curve_fit(
124         func, xa, ya, p0=p0 if p0 is None else np.asarray(p0, dtype=float),
125         sigma=sigma, bounds=bounds, maxfev=maxfev,
126     )
127     diag = np.clip(np.diag(pcov), 0, None) if pcov is not None else np.full_like(popt, np.nan)
128     perr = np.sqrt(diag)
129     if names is None:
130         names = [f"p{i}" for i in range(len(popt))]
131     r2, rmse = _metrics(ya, func(xa, *popt))
132     return FitResult(
133         func=func, params=np.asarray(popt), stderr=np.asarray(perr),
134         r2=r2, rmse=rmse, n=len(xa), names=list(names),
135         kind=kind or getattr(func, "__name__", ""), x=xa, y=ya,
136     )
137
138
139 # -----
140 # 多项式

```

```

141 # -----
142 def _poly_func(x: np.ndarray, *coef: float) -> np.ndarray:
143     return np.polyval(coef, x)
144
145
146 def polyfit(
147     x: Sequence[float],
148     y: Sequence[float],
149     degree: int = 1,
150 ) -> FitResult:
151     """多项式拟合 (含系数标准误, ``params`` 按降幂排列)。"""
152     xa = np.asarray(x, dtype=float).ravel()
153     ya = np.asarray(y, dtype=float).ravel()
154     coef, cov = np.polyfit(xa, ya, degree, cov=True)
155     perr = np.sqrt(np.clip(np.diag(cov), 0, None))
156     r2, rmse = _metrics(ya, np.polyval(coef, xa))
157     names = [f"a{i}" for i in range(degree + 1)]
158     return FitResult(
159         func=_poly_func, params=np.asarray(coef), stderr=np.asarray(perr),
160         r2=r2, rmse=rmse, n=len(xa), names=names, kind=f"多项式(deg={degree})",
161         x=xa, y=ya,
162     )
163
164
165 def poly_str(coef: Sequence[float], var: str = "x", nd: int = 4) -> str:
166     """多项式系数 (降幂) + 可读表达式, 如 ``0.5x^{2} - 1.2x + 3``。"""
167     coef = np.asarray(coef, dtype=float)
168     deg = len(coef) - 1
169     terms: list[tuple[str, str]] = []
170     for i, c in enumerate(coef):
171         power = deg - i
172         if abs(c) < 1e-12:
173             continue
174         sign = "-" if c < 0 else "+"
175         mag = fmt(abs(c), nd)
176         if power == 0:
177             body = mag
178         elif power == 1:
179             body = f"{mag}{var}"
180         else:
181             body = f"{mag}{var}^{{{power}}}"
182         terms.append((sign, body))
183     if not terms:
184         return "0"
185     s = terms[0][1] if terms[0][0] == "+" else "-" + terms[0][1]
186     for sign, body in terms[1:]:
187         s += f" {sign} {body}"
188     return s
189
190
191 def poly_scan(
192     x: Sequence[float],
193     y: Sequence[float],
194     max_degree: int = 5,
195 ) -> "pd.DataFrame":
196     """扫描 1-max_degree 阶多项式, 返回对比表 (用于自动定阶)。
197
198     列: ``degree, r2, adj_r2, rmse, aic`` —— R² 提升有限时选更简单的阶数。
199     """
200     import pandas as pd
201
202     xa = np.asarray(x, dtype=float).ravel()
203     ya = np.asarray(y, dtype=float).ravel()
204     n = len(xa)
205     rows = []
206     for deg in range(1, max_degree + 1):
207         coef = np.polyfit(xa, ya, deg)
208         yhat = np.polyval(coef, xa)
209         r2, rmse = _metrics(ya, yhat)

```

```

210     sse = float(np.sum((ya - yhat) ** 2))
211     k = deg + 1
212     adj = 1 - (1 - r2) * (n - 1) / max(n - k - 1, 1)
213     aic = n * np.log(max(sse / n, 1e-300)) + 2 * k
214     rows.append({"degree": deg, "r2": r2, "adj_r2": adj, "rmse": rmse, "aic": aic})
215     return pd.DataFrame(rows)
216
217
218 # -----
219 # 插值与平滑
220 # -----
221 def interp(
222     x: Sequence[float],
223     y: Sequence[float],
224     x_new: Sequence[float],
225     kind: str = "cubic",
226 ) -> np.ndarray:
227     """一维插值: ``kind`` {"linear", "cubic", "pchip", "nearest"}``.
228
229     曲线光滑推荐 ``"cubic"``; 防震荡 (数据陡变) 推荐 ``"pchip"``。
230     """
231     xa = np.asarray(x, dtype=float).ravel()
232     ya = np.asarray(y, dtype=float).ravel()
233     xb = np.asarray(x_new, dtype=float).ravel()
234     if kind == "linear":
235         return np.interp(xb, xa, ya)
236     if kind == "cubic":
237         from scipy.interpolate import CubicSpline
238
239         return CubicSpline(xa, ya)(xb)
240     if kind == "pchip":
241         from scipy.interpolate import PchipInterpolator
242
243         return PchipInterpolator(xa, ya)(xb)
244     if kind == "nearest":
245         idx = np.abs(xb[:, None] - xa[None, :]).argmin(axis=1)
246         return ya[idx]
247     raise ValueError(f"不支持的插值类型: {kind!r}")
248
249
250 def spline_smooth(
251     x: Sequence[float],
252     y: Sequence[float],
253     s: float | None = None,
254     k: int = 3,
255 ) -> Callable[[np.ndarray], np.ndarray]:
256     """平滑样条 (UnivariateSpline), 返回可调用对象。
257
258     ``s`` 越大越光滑 (``None`` 自动按残差平方和目标选择)。
259     """
260     from scipy.interpolate import UnivariateSpline
261
262     return UnivariateSpline(np.asarray(x, float).ravel(), np.asarray(y, float).ravel(), s=s, k=k)
263
264
265 def smooth(
266     y: Sequence[float],
267     window: int = 7,
268     *,
269     method: str = "savgol",
270     polyorder: int = 2,
271 ) -> np.ndarray:
272     """一维数据平滑。
273
274     - ``"savgol"``: Savitzky-Golay 滤波 (默认, 保峰形, 实验数据首选)
275     - ``"moving"``: 居中滑动平均
276     - ``"ewma"``: 指数加权平均
277     """
278     import pandas as pd

```

```

279 ya = np.asarray(y, dtype=float).ravel()
280 window = int(window)
281 if method == "savgol":
282     from scipy.signal import savgol_filter
283
284     w = window if window % 2 == 1 else window + 1
285     w = min(w, len(ya) if len(ya) % 2 == 1 else len(ya) - 1)
286     return savgol_filter(ya, w, min(polyorder, max(w - 2, 1) if w > 2 else 1))
287 if method == "moving":
288     return pd.Series(ya).rolling(window, center=True, min_periods=1).mean().to_numpy()
289 if method == "ewma":
290     return pd.Series(ya).ewm(span=window, adjust=False).mean().to_numpy()
291 raise ValueError(f"未知平滑方法: {method!r}")
292

```

program/src/program/metrics.py

```

1 """模型评估指标（回归 / 分类 / 聚类）。
2
3 用法::
4
5     import program as pm
6
7     m = pm.metrics.regression_metrics(y_true, y_pred, n_features=5)
8     print(pm.metrics.str_metrics(m))      # 多行文本，可直接贴报告
9     pm.metrics.confusion_table(y_true, y_pred)
10 """
11
12 from __future__ import annotations
13
14 from typing import Any, Sequence
15
16 import numpy as np
17 import pandas as pd
18
19 from .utils import fmt
20
21
22 # -----
23 # 回归
24 # -----
25 def regression_metrics(
26     y_true: Sequence[float],
27     y_pred: Sequence[float],
28     n_features: int | None = None,
29 ) -> dict[str, float]:
30     """回归指标: MAE / MSE / RMSE / MAPE / SMAPE / R2 / NSE (+ 调整 R2)。
31
32     MAPE/SMAPE 自动忽略 ``y_true = 0`` 且分母同时为 0 的点。
33     """
34     y = np.asarray(y_true, dtype=float).ravel()
35     yhat = np.asarray(y_pred, dtype=float).ravel()
36     if len(y) != len(yhat):
37         raise ValueError(f"长度不一致: y_true {len(y)}, y_pred {len(yhat)}")
38     n = len(y)
39     err = y - yhat
40     mae = float(np.mean(np.abs(err)))
41     mse = float(np.mean(err**2))
42     rmse = float(np.sqrt(mse))
43     denom = np.where(np.abs(y) > 1e-12, np.abs(y), np.nan)
44     mape = float(np.nanmean(np.abs(err) / denom) * 100)
45     smape_den = np.abs(y) + np.abs(yhat)
46     smape = float(np.nanmean(np.where(smape_den > 1e-12, 2 * np.abs(err) / smape_den, np.nan)) * 100)
47     ss_res = float(np.sum(err**2))
48     ss_tot = float(np.sum((y - y.mean()) ** 2))
49     r2 = 1 - ss_res / ss_tot if ss_tot > 0 else float("nan")
50     nse = 1 - ss_res / ss_tot if ss_tot > 0 else float("nan")
51     out = {
52         "MAE": mae,

```

```

53     "MSE": mse,
54     "RMSE": rmse,
55     "MAPE(%)": mape,
56     "SMAPE(%)": smape,
57     "R2": r2,
58     "NSE": nse,
59     "MaxErr": float(np.max(np.abs(err))),
60     "n": float(n),
61 }
62 if n_features is not None and n > n_features + 1:
63     out["AdjR2"] = 1 - (1 - r2) * (n - 1) / (n - n_features - 1)
64 return out
65
66
67 # -----
68 # 分类
69 # -----
70 def classification_metrics(
71     y_true: Sequence,
72     y_pred: Sequence,
73     y_proba: Sequence[float] | None = None,
74     labels: Sequence | None = None,
75 ) -> dict[str, float]:
76     """分类指标: accuracy / precision / recall / F1 (macro、weighted) 与 AUC。
77
78     ``y_proba`` 为每个类别的预测概率（形状 ``(n, n_classes)``），二分类时也可
79     直接传正类概率（1D）。
80
81     """
82     from sklearn.metrics import (
83         accuracy_score,
84         f1_score,
85         precision_score,
86         recall_score,
87         roc_auc_score,
88     )
89     y = np.asarray(y_true).ravel()
90     yp = np.asarray(y_pred).ravel()
91     out: dict[str, float] = {
92         "accuracy": float(accuracy_score(y, yp)),
93         "precision_macro": float(precision_score(y, yp, average="macro", zero_division=0, labels=labels)),
94         "recall_macro": float(recall_score(y, yp, average="macro", zero_division=0, labels=labels)),
95         "f1_macro": float(f1_score(y, yp, average="macro", zero_division=0, labels=labels)),
96         "f1_weighted": float(f1_score(y, yp, average="weighted", zero_division=0, labels=labels)),
97         "n": float(len(y)),
98     }
99     if y_proba is not None:
100         proba = np.asarray(y_proba, dtype=float)
101         n_classes = len(np.unique(y))
102         try:
103             if n_classes > 2:
104                 out["auc_ovr"] = float(
105                     roc_auc_score(y, proba, multi_class="ovr", labels=labels)
106                 )
107             else:
108                 p1 = proba[:, 1] if proba.ndim == 2 else proba.ravel()
109                 out["auc"] = float(roc_auc_score(y, p1))
110         except Exception:
111             pass
112     return out
113
114
115 def confusion_table(
116     y_true: Sequence,
117     y_pred: Sequence,
118     labels: Sequence | None = None,
119 ) -> pd.DataFrame:
120     """混淆矩阵（行 = 真实类别，列 = 预测类别）。"""
121     return pd.crosstab(

```

```

122         pd.Series(np.asarray(y_true).ravel(), name="真实"),
123         pd.Series(np.asarray(y_pred).ravel(), name="预测"),
124         dropna=False,
125     )
126
127
128 # -----
129 # 聚类
130 # -----
131 def cluster_metrics(X, labels: Sequence[int]) -> dict[str, float]:
132     """聚类内部指标: 轮廓系数、Calinski-Harabasz、Davies-Bouldin。
133
134     轮廓系数越大越好; CH 越大越好; DB 越小越好。
135     """
136     from sklearn.metrics import (
137         calinski_harabasz_score,
138         davies_bouldin_score,
139         silhouette_score,
140     )
141
142     Xa = np.asarray(X, dtype=float)
143     lab = np.asarray(labels).ravel()
144     out: dict[str, float] = {}
145     if len(set(lab.tolist())) > 1:
146         out["silhouette"] = float(silhouette_score(Xa, lab))
147         out["calinski_harabasz"] = float(calinski_harabasz_score(Xa, lab))
148         out["davies_bouldin"] = float(davies_bouldin_score(Xa, lab))
149     return out
150
151
152 def str_metrics(metrics: dict[str, Any], nd: int = 4, indent: str = " ") -> str:
153     """把指标 dict 转成多行文本 (``MAE = 1.234``), 便于写入报告。"""
154     return "\n".join(f"{indent}{k} = {fmt(v, nd)}" for k, v in metrics.items())

```

program/src/program/ml.py

```

1  """机器学习快捷流程: 训练 / 交叉验证 / 评估一体化。
2
3  常用模型可用字符串直接指定, 内部自动划分数据、标准化(按需)、
4  交叉验证并给出测试集指标, 每个环节都固定随机种子。
5
6  - 回归: ``"linear" "ridge" "lasso" "rf" "gbr" "svr" "knn" "mlp"``
7  - 分类: ``"logreg" "rf" "dt" "svc" "knn" "mlp"``
8  - 无监督: :func:`kmeans` / :func:`elbow` / :func:`pca`
9
10  用法::
11
12      import program as pm
13
14      r = pm.ml.fit_regressor("rf", X, y, test_size=0.2, cv=5, seed=42)
15      print(r.summary())
16      print(pm.metrics.str_metrics(r.test_metrics))
17
18      c = pm.ml.fit_classifier("logreg", X, y)
19      pm.plotting.roc(y_test, proba, save="q2_roc")
20  """
21
22  from __future__ import annotations
23
24  from dataclasses import dataclass, field
25  from typing import Any, Sequence
26
27  import numpy as np
28  import pandas as pd
29
30  from .metrics import classification_metrics, cluster_metrics, regression_metrics, str_metrics
31  from .utils import fmt, set_seed
32
33

```

```

34 # -----
35 # 模型工厂
36 # -----
37 def _regressor_table(seed: int) -> dict[str, Any]:
38     from sklearn.ensemble import GradientBoostingRegressor, RandomForestRegressor
39     from sklearn.linear_model import Lasso, LinearRegression, Ridge
40     from sklearn.neighbors import KNeighborsRegressor
41     from sklearn.neural_network import MLPRegressor
42     from sklearn.svm import SVR
43
44     return {
45         "linear": LinearRegression(),
46         "ridge": Ridge(alpha=1.0),
47         "lasso": Lasso(alpha=0.01, max_iter=10000),
48         "rf": RandomForestRegressor(n_estimators=300, random_state=seed, n_jobs=-1),
49         "gbr": GradientBoostingRegressor(random_state=seed),
50         "svr": SVR(),
51         "knn": KNeighborsRegressor(),
52         "mlp": MLPRegressor(hidden_layer_sizes=(64, 32), max_iter=3000, random_state=seed),
53     }
54
55
56 def _classifier_table(seed: int) -> dict[str, Any]:
57     from sklearn.ensemble import RandomForestClassifier
58     from sklearn.linear_model import LogisticRegression
59     from sklearn.neighbors import KNeighborsClassifier
60     from sklearn.neural_network import MLPClassifier
61     from sklearn.svm import SVC
62     from sklearn.tree import DecisionTreeClassifier
63
64     return {
65         "logreg": LogisticRegression(max_iter=2000, random_state=seed),
66         "rf": RandomForestClassifier(n_estimators=300, random_state=seed, n_jobs=-1),
67         "dt": DecisionTreeClassifier(random_state=seed),
68         "svc": SVC(probability=True),
69         "knn": KNeighborsClassifier(),
70         "mlp": MLPClassifier(hidden_layer_sizes=(64, 32), max_iter=3000, random_state=seed),
71     }
72
73
74 _SCALE_NEEDED = {
75     "SVR", "KNeighborsRegressor", "MLPRegressor",
76     "LogisticRegression", "SVC", "KNeighborsClassifier", "MLPClassifier",
77 }
78
79
80 def make_model(model: Any, task: str = "regression", seed: int = 42):
81     """字符串 → sklearn 估计器；传入估计器实例则原样返回。"""
82     if not isinstance(model, str):
83         return model
84     key = model.lower().replace("-", "_").replace(" ", "_")
85     table = _regressor_table(seed) if task == "regression" else _classifier_table(seed)
86     if key not in table:
87         raise ValueError(f"未知模型 {model!r}; 可选: {sorted(table)}")
88     return table[key]
89
90
91 def _needs_scaling(estimator: Any) -> bool:
92     return type(estimator).__name__ in _SCALE_NEEDED
93
94
95 def _prepared(model: Any, task: str, seed: int, scale: str | bool):
96     est = make_model(model, task, seed)
97     if scale == "auto":
98         scale = _needs_scaling(est)
99     if scale:
100         from sklearn.pipeline import make_pipeline
101         from sklearn.preprocessing import StandardScaler

```

```

103     est = make_pipeline(StandardScaler(), est)
104     return est
105
106
107 def _feature_names(X: Any) -> list[str] | None:
108     if isinstance(X, pd.DataFrame):
109         return [str(c) for c in X.columns]
110     return None
111
112
113 # -----
114 # 训练结果容器
115 # -----
116 @dataclass
117 class TrainResult:
118     """ 监督学习训练结果。 """
119
120     model: Any
121     task: str
122     cv_scores: np.ndarray
123     cv_scoring: str
124     test_metrics: dict[str, float]
125     y_test: Any
126     y_pred: np.ndarray
127     y_proba: np.ndarray | None = None
128     X_test: Any = None
129     feature_names: list[str] | None = None
130     extra: dict[str, Any] = field(default_factory=dict)
131
132     def summary(self) -> str:
133         lines = [f"【{self.task}】交叉验证 ({len(self.cv_scores)} 折, {self.cv_scoring}): "
134                 f"{fmt(self.cv_scores.mean(), 5)} ± {fmt(self.cv_scores.std(), 3)}"]
135         lines.append("测试集指标: ")
136         lines.append(str_metrics(self.test_metrics))
137         return "\n".join(lines)
138
139     __str__ = summary
140
141
142 # -----
143 # 监督学习快捷流程
144 # -----
145 def fit_regressor(
146     model: Any = "rf",
147     X: Any = None,
148     y: Any = None,
149     *,
150     test_size: float = 0.2,
151     cv: int = 5,
152     seed: int = 42,
153     scale: str | bool = "auto",
154     scoring: str | None = None,
155 ) -> TrainResult:
156     """ 训练回归模型: 划分 → 拟合 → 交叉验证 → 测试集评估。 """
157
158     Parameters
159     -----
160     model : str | 估计器
161         模型名 (见模块文档) 或自定义 sklearn 估计器。
162     X, y : 数据
163         特征与目标 (支持 DataFrame / ndarray)。
164     cv : int
165         交叉验证折数 (在训练集上做)。
166     scale : ``"auto"`` | bool
167         是否标准化 (``"auto"`` 对 SVR/KNN/MLP 等自动开启)。
168
169     Returns
170     -----
171     TrainResult

```

```

172     ``r.model`` 训练好的模型; ``r.y_test`` / ``r.y_pred`` 供画图;
173     ``r.test_metrics`` 指标 dict。
174     """
175     from sklearn.model_selection import cross_val_score, train_test_split
176
177     set_seed(seed)
178     est = _prepared(model, "regression", seed, scale)
179     Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=test_size, random_state=seed)
180     est.fit(Xtr, ytr)
181     cv_scores = cross_val_score(est, Xtr, ytr, cv=cv, scoring=scoring or "r2")
182     y_pred = est.predict(Xte)
183     n_features = np.asarray(X).shape[1] if np.ndim(X) == 2 else None
184     metrics = regression_metrics(yte, y_pred, n_features=n_features)
185     return TrainResult(
186         model=est, task="回归", cv_scores=np.asarray(cv_scores),
187         cv_scoring=scoring or "r2", test_metrics=metrics,
188         y_test=yte, y_pred=np.asarray(y_pred),
189         X_test=Xte, feature_names=_feature_names(X),
190     )
191
192
193 def fit_classifier(
194     model: Any = "rf",
195     X: Any = None,
196     y: Any = None,
197     *,
198     test_size: float = 0.2,
199     cv: int = 5,
200     seed: int = 42,
201     scale: str | bool = "auto",
202     scoring: str | None = None,
203     stratify: bool = True,
204 ) -> TrainResult:
205     """训练分类模型: 划分(默认分层) → 拟合 → 交叉验证 → 测试集评估。"""
206     from sklearn.model_selection import cross_val_score, train_test_split
207
208     set_seed(seed)
209     est = _prepared(model, "classification", seed, scale)
210     ya = np.asarray(y).ravel()
211     Xtr, Xte, ytr, yte = train_test_split(
212         X, y, test_size=test_size, random_state=seed,
213         stratify=ya if stratify else None,
214     )
215     est.fit(Xtr, ytr)
216     cv_scores = cross_val_score(est, Xtr, ytr, cv=cv, scoring=scoring or "accuracy")
217     y_pred = est.predict(Xte)
218     y_proba = est.predict_proba(Xte) if hasattr(est, "predict_proba") else None
219     metrics = classification_metrics(yte, y_pred, y_proba)
220     return TrainResult(
221         model=est, task="分类", cv_scores=np.asarray(cv_scores),
222         cv_scoring=scoring or "accuracy", test_metrics=metrics,
223         y_test=yte, y_pred=np.asarray(y_pred), y_proba=y_proba,
224         X_test=Xte, feature_names=_feature_names(X),
225     )
226
227
228 def cross_validate(
229     model: Any,
230     X: Any,
231     y: Any,
232     *,
233     cv: int = 5,
234     task: str | None = None,
235     seed: int = 42,
236     scale: str | bool = "auto",
237     scoring: str | None = None,
238 ) -> dict[str, Any]:
239     """K 折交叉验证 (返回均值/标准差/每折得分)。"""
240     from sklearn.base import is_classifier

```

```

241     from sklearn.model_selection import cross_val_score
242
243     set_seed(seed)
244     if task is None:
245         probe = make_model(model, "regression", seed) if isinstance(model, str) else model
246         task = "classification" if is_classifier(probe) else "regression"
247     est = _prepared(model, task, seed, scale)
248     scores = np.asarray(cross_val_score(est, X, y, cv=cv, scoring=scoring))
249     return {
250         "task": task,
251         "scoring": scoring or ("accuracy" if task == "classification" else "r2"),
252         "scores": scores,
253         "mean": float(scores.mean()),
254         "std": float(scores.std()),
255     }
256
257
258 def compare_models(
259     models: Sequence[Any],
260     X: Any,
261     y: Any,
262     *,
263     task: str = "regression",
264     test_size: float = 0.2,
265     cv: int = 5,
266     seed: int = 42,
267 ) -> pd.DataFrame:
268     """批量训练并对比多个模型，返回对比表（模型 / CV 均值 / CV 标准差 / 测试指标）。
269
270     用法::
271
272         table = pm.ml.compare_models(
273             ["linear", "ridge", "rf", "gbr"], X, y, task="regression"
274         )
275     """
276     rows = []
277     key = "R2" if task == "regression" else "accuracy"
278     for m in models:
279         name = m if isinstance(m, str) else type(m).__name__
280         res = (
281             fit_regressor(m, X, y, test_size=test_size, cv=cv, seed=seed)
282             if task == "regression"
283             else fit_classifier(m, X, y, test_size=test_size, cv=cv, seed=seed)
284         )
285         row = {
286             "模型": name,
287             f"CV_{res.cv_scoring}_均值": res.cv_scores.mean(),
288             f"CV_{res.cv_scoring}_标准差": res.cv_scores.std(),
289             f"测试集_{key}": res.test_metrics.get(key, float("nan")),
290             "测试集_RMSE": res.test_metrics.get("RMSE", float("nan")),
291         }
292         rows.append(row)
293     return pd.DataFrame(rows).sort_values(f"测试集_{key}", ascending=False)
294
295
296 # -----
297 # 无监督
298 # -----
299 def standardize(X: Any) -> tuple[Any, Any]:
300     """标准化 (Z-score)，返回 ``(标准化数据, scaler)``。
301
302     DataFrame 输入返回 DataFrame（保持列名与索引），否则返回 ndarray。
303     """
304     from sklearn.preprocessing import StandardScaler
305
306     sc = StandardScaler()
307     if isinstance(X, pd.DataFrame):
308         arr = sc.fit_transform(X)
309         return pd.DataFrame(arr, columns=X.columns, index=X.index), sc

```

```

310     return sc.fit_transform(np.asarray(X, dtype=float)), sc
311
312
313 @dataclass
314 class ClusterResult:
315     """聚类结果。"""
316
317     labels: np.ndarray
318     centers: np.ndarray
319     metrics: dict[str, float]
320     model: Any
321     X_scaled: Any = None
322     extra: dict[str, Any] = field(default_factory=dict)
323
324     def summary(self) -> str:
325         import collections
326
327         counts = collections.Counter(self.labels.tolist())
328         lines = [f"聚类完成: k = {len(counts)}, 各簇样本数 = {dict(sorted(counts.items()))}"]
329         lines.append(str_metrics(self.metrics))
330         return "\n".join(lines)
331
332     __str__ = summary
333
334
335 def kmeans(
336     X: Any,
337     k: int = 3,
338     *,
339     seed: int = 42,
340     scale: bool = True,
341     n_init: int = 10,
342 ) -> ClusterResult:
343     """K-means 聚类（返回标签、原始尺度质心与内部指标）。"""
344     from sklearn.cluster import KMeans
345
346     set_seed(seed)
347     Xa = X
348     sc = None
349     if scale:
350         Xa, sc = standardize(X)
351     model = KMeans(n_clusters=k, n_init=n_init, random_state=seed)
352     labels = model.fit_predict(Xa)
353     centers = (
354         sc.inverse_transform(model.cluster_centers_) if sc is not None
355         else model.cluster_centers_
356     )
357     metrics = cluster_metrics(np.asarray(Xa, dtype=float), labels)
358     return ClusterResult(
359         labels=np.asarray(labels), centers=np.asarray(centers),
360         metrics=metrics, model=model, X_scaled=Xa,
361         extra={"k": k, "inertia": float(model.inertia_)},
362     )
363
364
365 def elbow(
366     X: Any,
367     k_range: Sequence[int] = range(2, 11),
368     *,
369     seed: int = 42,
370     scale: bool = True,
371 ) -> pd.DataFrame:
372     """肘部法则扫描：每个 k 的 inertia 与轮廓系数（用于确定最优簇数）。"""
373     from sklearn.cluster import KMeans
374
375     set_seed(seed)
376     Xa = X
377     if scale:
378         Xa, _ = standardize(X)

```

```

379     Xa = np.asarray(Xa, dtype=float)
380     rows = []
381     for k in k_range:
382         model = KMeans(n_clusters=k, n_init=10, random_state=seed)
383         labels = model.fit_predict(Xa)
384         row = {"k": k, "inertia": float(model.inertia_)}
385         row["silhouette"] = cluster_metrics(Xa, labels).get("silhouette", float("nan"))
386         rows.append(row)
387     return pd.DataFrame(rows)
388
389
390 @dataclass
391 class PCAResult:
392     """主成分分析结果。"""
393
394     scores: np.ndarray
395     loadings: pd.DataFrame | None
396     explained_ratio: np.ndarray
397     model: Any
398     feature_names: list[str] | None = None
399
400     @property
401     def cumulative(self) -> np.ndarray:
402         return np.cumsum(self.explained_ratio)
403
404     def summary(self) -> str:
405         parts = [
406             f"主成分方差解释率: {' '.join(f'PC{i + 1}={r:.4f}' for i, r in enumerate(self.explained_ratio))}",
407             f"累计解释率: {self.cumulative[-1]:.4f}",
408         ]
409         return "\n".join(parts)
410
411     __str__ = summary
412
413
414 def pca(X: Any, n_components: int = 2, *, scale: bool = True) -> PCAResult:
415     """主成分分析（返回得分矩阵、载荷矩阵、解释方差比）。"""
416     from sklearn.decomposition import PCA
417
418     Xa = X
419     if scale:
420         Xa, _ = standardize(X)
421     Xa = np.asarray(Xa, dtype=float)
422     model = PCA(n_components=n_components)
423     scores = model.fit_transform(Xa)
424     names = _feature_names(X)
425     loadings = None
426     if names is not None:
427         loadings = pd.DataFrame(
428             model.components_.T,
429             index=names,
430             columns=[f"PC{i + 1}" for i in range(model.n_components_)],
431         )
432     return PCAResult(
433         scores=scores, loadings=loadings,
434         explained_ratio=model.explained_variance_ratio_,
435         model=model, feature_names=names,
436     )

```

program/src/program/ode.py

```

1 """常微分方程：初值问题求解与参数反演（传染病、动力学、种群模型）。
2
3 ``func`` 统一使用 ``solve_ivp`` 的接口 ``f(t, y, *params)``（注意不是
4 ``odeint`` 的 ``func(y, t)``），如 SIR 模型::
5
6     def sir(t, y, beta, gamma):
7         S, I, R = y
8         return [-beta * S * I, beta * S * I - gamma * I, gamma * I]

```

```

9
10     sol = pm.ode.solve_ivp_model(sir, [0.99, 0.01, 0], (0, 60), params=(0.3, 0.1), t_eval=t)
11
12 用法（参数反演）::
13
14     fit = pm.ode.fit_ode_params(sir, [0.99, 0.01, 0], t_data, y_data, p0=[0.2, 0.1],
15                                bounds=([0.001, 0.001], [2, 2]), param_names=[" ", " "])
16     print(fit.summary())
17     y_sim = fit.simulate(t_grid)
18 """
19
20 from __future__ import annotations
21
22 from collections.abc import Callable, Sequence
23 from dataclasses import dataclass
24 from typing import Any
25
26 import numpy as np
27
28 from .utils import fmt
29
30
31 def solve_ivp_model(
32     func: Callable[..., Any],
33     y0: Sequence[float],
34     t_span: tuple[float, float],
35     *,
36     params: Sequence[float] = (),
37     t_eval: Sequence[float] | None = None,
38     method: str = "RK45",
39     rtol: float = 1e-6,
40     atol: float = 1e-9,
41     **kwargs: Any,
42 ):
43     """求解常微分方程组初值问题，返回 ``scipy.integrate.OdeResult``。
44
45     ``func(t, y, *params)``: 默认提高精度 (rtol=1e-6, atol=1e-9)。
46     结果用 ``sol.t`` (时间) 与 ``sol.y`` (形状 ``(变量数, 时间点数)``) 访问。
47     """
48     from scipy.integrate import solve_ivp
49
50     y0a = np.atleast_1d(np.asarray(y0, dtype=float))
51     f = lambda t, y: func(t, y, *params)
52     return solve_ivp(
53         f, t_span, y0a, t_eval=t_eval, method=method, rtol=rtol, atol=atol, **kwargs
54     )
55
56
57 @dataclass
58 class ODEFitResult:
59     """ODE 参数反演结果。"""
60
61     params: np.ndarray
62     param_names: list[str]
63     cost: float
64     rmse: float
65     r2: float
66     success: bool
67     func: Callable[..., Any]
68     y0: np.ndarray
69     t_span: tuple[float, float]
70     t_data: np.ndarray
71     y_data: np.ndarray
72     method: str = "RK45"
73
74     def simulate(self, t_eval: Sequence[float] | None = None) -> np.ndarray:
75         """用拟合参数模拟，返回 ``(时间点数, 变量数)`` 数组。"""
76         t = (
77             np.asarray(t_eval, dtype=float).ravel()

```

```

78         if t_eval is not None
79         else self.t_data
80     )
81     sol = solve_ivp_model(
82         self.func,
83         self.y0,
84         self.t_span,
85         params=self.params,
86         t_eval=t,
87         method=self.method,
88     )
89     return sol.y.T
90
91 predict = simulate
92
93 def summary(self) -> str:
94     lines = [
95         f"ODE 参数反演{'成功' if self.success else '失败'}: "
96         f"RMSE = {fmt(self.rmse, 5)}, R² = {fmt(self.r2, 5)}"
97     ]
98     for name, v in zip(self.param_names, self.params):
99         lines.append(f" {name} = {fmt(v, 6)}")
100    return "\n".join(lines)
101
102 __str__ = summary
103
104
105 def fit_ode_params(
106     func: Callable[..., Any],
107     y0: Sequence[float],
108     t_data: Sequence[float],
109     y_data: Sequence[float],
110     p0: Sequence[float],
111     *,
112     bounds: tuple = (-np.inf, np.inf),
113     param_names: Sequence[str] | None = None,
114     method: str = "RK45",
115     rtol: float = 1e-6,
116     atol: float = 1e-9,
117     verbose: bool = False,
118 ) -> ODEFitResult:
119     """用最小二乘反演 ODE 参数（`scipy.optimize.least_squares`）。
120
121     Parameters
122     -----
123     func : callable
124         ``f(t, y, *params)``。
125     y0 : 序列
126         初值（已知且固定；如需一起拟合请自行扩展参数向量）。
127     t_data, y_data : 序列
128         观测时间点（需单调递增）与观测值；``y_data`` 形状 ``(n_t,)``
129         或 ``(n_t, n_vars)``。
130     p0 : 序列
131         参数初值。
132     bounds : tuple
133         ``(下界, 上界)``；强烈建议给出，避免参数跑到无物理意义区域。
134
135     """
136     from scipy.optimize import least_squares
137
138     t = np.asarray(t_data, dtype=float).ravel()
139     Yobs = np.asarray(y_data, dtype=float)
140     if Yobs.ndim == 1:
141         Yobs = Yobs[:, None]
142     y0a = np.atleast_1d(np.asarray(y0, dtype=float))
143     t_span = (float(t.min()), float(t.max()))
144
145     def residual(p: np.ndarray) -> np.ndarray:
146         try:
147             sol = solve_ivp_model(

```

```

147         func,
148         y0a,
149         t_span,
150         params=p,
151         t_eval=t,
152         method=method,
153         rtol=rtol,
154         atol=atol,
155     )
156     if not sol.success or sol.y.shape[1] != len(t):
157         return np.full(Yobs.size, 1e6)
158     return (sol.y.T - Yobs).ravel()
159 except Exception:
160     return np.full(Yobs.size, 1e6)
161
162 res = least_squares(
163     residual,
164     np.asarray(p0, dtype=float),
165     bounds=bounds,
166     verbose=2 if verbose else 0,
167 )
168 resid = res.fun
169 rmse = float(np.sqrt(np.mean(resid**2)))
170 ss_tot = float(np.sum((Yobs - Yobs.mean()) ** 2))
171 r2 = 1 - float(np.sum(resid**2)) / ss_tot if ss_tot > 0 else float("nan")
172 if param_names is None:
173     param_names = [f"p{i}" for i in range(len(res.x))]
174 return ODEFitResult(
175     params=np.asarray(res.x),
176     param_names=list(param_names),
177     cost=float(res.cost),
178     rmse=rmse,
179     r2=r2,
180     success=bool(res.success),
181     func=func,
182     y0=y0a,
183     t_span=t_span,
184     t_data=t,
185     y_data=Yobs,
186     method=method,
187 )

```

program/src/program/optimize.py

```

1  """优化求解：线性/整数规划、非线性规划、全局优化与经典离散问题。
2
3  - :func:`solve_lp`: 线性规划 / 混合整数线性规划 (HiGHS, 支持最大化)
4  - :func:`solve_nlp`: 非线性规划 (scipy.minimize 包装)
5  - :func:`global_minimize`: 全局优化 (差分进化)
6  - :func:`knapsack` / :func:`assignment` / :func:`tsp_nearest_neighbor` + :func:`tsp_2opt`
7
8  优化类赛题注意：先保证「可行解」，再优化目标值；结果用
9  :func:`program.report.record_result` 记录目标值与约束校验。
10
11  用法::
12
13      import program as pm
14
15      r = pm.optimize.solve_lp(
16          c=[3, 5], A_ub=[[1, 0], [0, 2], [3, 2]], b_ub=[4, 12, 18],
17          maximize=True,
18      )
19      print(r.summary())
20  """
21
22  from __future__ import annotations
23
24  from dataclasses import dataclass, field
25  from typing import Any, Callable, Sequence

```

```

26
27 import numpy as np
28
29 from .utils import fmt
30
31
32 # -----
33 # 结果容器
34 # -----
35 @dataclass
36 class OptResult:
37     """优化结果 (``str(r)`` 直接看结论)。"""
38
39     x: np.ndarray
40     fun: float
41     success: bool
42     message: str
43     solver: str = ""
44     extra: Dict[str, Any] = field(default_factory=dict)
45
46     def summary(self) -> str:
47         xs = np.asarray(self.x).ravel()
48         x_str = ", ".join(f"{v:.6g}" for v in xs[:10]) + (" ..." if len(xs) > 10 else "")
49         status = "成功" if self.success else "失败"
50         msg = (
51             f"【{self.solver or '优化'}】{status}: 目标值 = {fmt(self.fun, 8)}"
52             f"\n 最优解 x = [{x_str}]"
53         )
54         if not self.success and self.message:
55             msg += f"\n 信息: {self.message}"
56         return msg
57
58     __str__ = summary
59
60
61 # -----
62 # 线性 / 整数规划
63 # -----
64 def solve_lp(
65     c: Sequence[float],
66     A_ub: Any = None,
67     b_ub: Any = None,
68     A_eq: Any = None,
69     b_eq: Any = None,
70     bounds: Sequence[tuple[float | None, float | None]] | None = None,
71     *,
72     maximize: bool = False,
73     integer: bool | Sequence[int] | None = None,
74     options: dict | None = None,
75 ) -> OptResult:
76     """线性规划 (``integer=None``) 或混合整数线性规划 (HiGHS) 。
77
78     Parameters
79     -----
80     c : 序列
81         目标函数系数。
82     A_ub, b_ub, A_eq, b_eq : 数组 | None
83         不等式约束 ``A_ub x ≤ b_ub`` 与等式约束 ``A_eq x = b_eq``。
84     bounds : 序列[[下界, 上界]] | None
85         变量边界, ``None`` 表示  $\pm\infty$ ; 整体默认为 ``(0, +∞)`` (非负)。
86     maximize : bool
87         是否最大化 (默认最小化)。
88     integer : bool | 序列 | None
89         ``None``/``False`` 连续; ``True`` 全部整数; 序列则逐变量指定
90         (1 = 整数, 0 = 连续, 可用于 0-1 变量配合 bounds=(0,1))。
91
92     Returns
93     -----
94     OptResult

```

```

95     """
96     from scipy.optimize import Bounds, LinearConstraint, linprog, milp
97
98     c_arr = np.asarray(c, dtype=float).ravel()
99     n = len(c_arr)
100     sign = -1.0 if maximize else 1.0
101
102     if integer is None or integer is False:
103         res = linprog(
104             sign * c_arr, A_ub=A_ub, b_ub=b_ub, A_eq=A_eq, b_eq=b_eq,
105             bounds=bounds, method="highs", options=options or {},
106         )
107         x = res.x if res.x is not None else np.full(n, np.nan)
108         return OptResult(
109             x=np.asarray(x), fun=float(sign * res.fun) if res.fun is not None else float("nan"),
110             success=bool(res.success), message=str(res.message),
111             solver="HiGHS 线性规划", extra={"maximize": maximize},
112         )
113
114     # 整数规划
115     integrality = (
116         np.ones(n, dtype=int) if integer is True
117         else np.asarray(integer, dtype=int)
118     )
119     if bounds is None:
120         bl = Bounds(np.zeros(n), np.full(n, np.inf))
121     else:
122         lb = np.array([b[0] if b[0] is not None else -np.inf for b in bounds], dtype=float)
123         ub = np.array([b[1] if b[1] is not None else np.inf for b in bounds], dtype=float)
124         bl = Bounds(lb, ub)
125     cons = []
126     if A_ub is not None:
127         cons.append(
128             LinearConstraint(np.asarray(A_ub, dtype=float), -np.inf, np.asarray(b_ub, dtype=float))
129         )
130     if A_eq is not None:
131         b_eq_arr = np.asarray(b_eq, dtype=float)
132         cons.append(LinearConstraint(np.asarray(A_eq, dtype=float), b_eq_arr, b_eq_arr))
133     res = milp(
134         sign * c_arr, constraints=cons if cons else (),
135         integrality=integrality, bounds=bl, options=options or {},
136     )
137     x = res.x if res.x is not None else np.full(n, np.nan)
138     return OptResult(
139         x=np.asarray(x), fun=float(sign * res.fun) if res.fun is not None else float("nan"),
140         success=bool(res.success), message=str(res.message),
141         solver="HiGHS 整数规划", extra={"maximize": maximize, "n_integer": int(integrality.sum())},
142     )
143
144
145 # -----
146 # 非线性与全局优化
147 # -----
148 def solve_nlp(
149     func: Callable[[np.ndarray], float],
150     x0: Sequence[float],
151     *,
152     bounds: Sequence[tuple[float | None, float | None]] | None = None,
153     constraints: Any = None,
154     maximize: bool = False,
155     method: str | None = None,
156     options: dict | None = None,
157 ) -> OptResult:
158     """非线性规划（`scipy.optimize.minimize` 包装，支持最大化与约束）。
159
160     Parameters
161     -----
162     func : callable
163         ``f(x) -> float``。

```

```

164 x0 : 序列
165     初值（建议多试几个初值以判断局部/全局）。
166 bounds : 序列[[下界, 上界]] | None
167     变量边界。
168 constraints : dict | 序列 | None
169     scipy 约束格式, 如
170     ``{"type": "ineq", "fun": lambda x: x[0] + x[1] - 1}``.
171 maximize : bool
172     是否最大化。
173 """
174 from scipy.optimize import minimize
175
176 sign = -1.0 if maximize else 1.0
177
178 def f(x: np.ndarray) -> float:
179     return sign * float(func(x))
180
181 res = minimize(
182     f, np.asarray(x0, dtype=float), method=method,
183     bounds=bounds, constraints=constraints, options=options or {},
184 )
185 return OptResult(
186     x=np.asarray(res.x), fun=float(sign * res.fun), success=bool(res.success),
187     message=str(res.message), solver=f"minimize/{method or '默认方法'}",
188     extra={"maximize": maximize, "iterations": getattr(res, "nit", None)},
189 )
190
191 def global_minimize(
192     func: Callable[[np.ndarray], float],
193     bounds: Sequence[tuple[float, float]],
194     *,
195     maximize: bool = False,
196     seed: int = 42,
197     **de_kwargs: Any,
198 ) -> OptResult:
199     """全局优化（差分进化, 适合多峰/非凸目标, 结果可复现）。
200
201     ``de_kwargs`` 透传 ``scipy.optimize.differential_evolution``,
202     如 ``maxiter=200, popsize=30, tol=1e-8``。
203     """
204     from scipy.optimize import differential_evolution
205
206     sign = -1.0 if maximize else 1.0
207     res = differential_evolution(
208         lambda x: sign * float(func(x)), bounds, seed=seed, polish=True, **de_kwargs
209     )
210     return OptResult(
211         x=np.asarray(res.x), fun=float(sign * res.fun), success=bool(res.success),
212         message=str(res.message), solver="differential_evolution",
213         extra={"maximize": maximize, "nfev": int(res.nfev)},
214     )
215
216
217
218 # -----
219 # 经典离散问题
220 # -----
221 @dataclass
222 class KnapsackResult:
223     """0-1 背包结果。"""
224
225     best_value: float
226     items: list[int]
227     total_weight: float
228
229     def summary(self) -> str:
230         return (
231             f"【0-1 背包】最优价值 = {fmt(self.best_value)}, "
232             f"总重量 = {fmt(self.total_weight)}, 选中物品 (0 起) = {self.items}"

```

```

233     )
234
235     __str__ = summary
236
237
238 def knapsack(
239     values: Sequence[float],
240     weights: Sequence[float],
241     capacity: float,
242 ) -> KnapsackResult:
243     """0-1 背包（动态规划， $O(n \cdot C)$ ）。
244
245     要求 ``weights`` 与 ``capacity`` 为整数（非整数请先统一缩放为整数，如乘 1000）。
246     """
247     v = np.asarray(values, dtype=float).ravel()
248     w = np.asarray(weights).ravel()
249     if not np.allclose(w, np.round(w)) or not float(capacity).is_integer():
250         raise ValueError("weights 与 capacity 必须为整数（可先统一放大为整数）")
251     w = w.astype(int)
252     W = int(capacity)
253     n = len(v)
254     dp = np.zeros(W + 1)
255     keep = np.zeros((n, W + 1), dtype=bool)
256     for i in range(n):
257         for c in range(W, w[i] - 1, -1):
258             cand = dp[c - w[i]] + v[i]
259             if cand > dp[c]:
260                 dp[c] = cand
261                 keep[i, c] = True
262     c = W
263     items: list[int] = []
264     for i in range(n - 1, -1, -1):
265         if keep[i, c]:
266             items.append(i)
267             c -= w[i]
268     items.reverse()
269     return KnapsackResult(
270         best_value=float(dp[W]), items=items,
271         total_weight=float(w[items].sum()) if items else 0.0,
272     )
273
274
275 @dataclass
276 class Assignment:
277     """指派问题结果。"""
278
279     rows: np.ndarray
280     cols: np.ndarray
281     total: float
282
283     def pairs(self) -> list[tuple[int, int]]:
284         return [(int(r), int(c)) for r, c in zip(self.rows, self.cols)]
285
286     def summary(self) -> str:
287         return f"【指派问题】最小总成本 = {fmt(self.total)}, 匹配 = {self.pairs()}"
288
289     __str__ = summary
290
291
292 def assignment(cost: Any, *, maximize: bool = False) -> Assignment:
293     """指派问题（匈牙利算法）。
294
295     ``cost[i, j]`` 表示第 i 个工人做第 j 项任务的成本（最大化时传收益矩阵）。
296     """
297     from scipy.optimize import linear_sum_assignment
298
299     c = np.asarray(cost, dtype=float)
300     rows, cols = linear_sum_assignment(c, maximize=maximize)
301     return Assignment(rows=rows, cols=cols, total=float(c[rows, cols].sum()))

```

```

302
303
304 def distance_matrix(points: Any, metric: str = "euclidean") -> np.ndarray:
305     """点坐标 → 距离矩阵 (``scipy.spatial.distance.cdist``)。"""
306     from scipy.spatial.distance import cdist
307
308     return cdist(np.asarray(points, dtype=float), np.asarray(points, dtype=float), metric=metric)
309
310
311 def _tour_length(dist: np.ndarray, tour: Sequence[int]) -> float:
312     t = np.asarray(tour, dtype=int)
313     return float(dist[t, np.roll(t, -1)].sum())
314
315
316 def tsp_nearest_neighbor(dist: np.ndarray, start: int = 0) -> list[int]:
317     """TSP 最近邻构造启发式，返回访问顺序（不含回到起点）。"""
318     n = dist.shape[0]
319     unvisited = set(range(n))
320     unvisited.discard(start)
321     tour = [start]
322     cur = start
323     while unvisited:
324         nxt = min(unvisited, key=lambda j: dist[cur, j])
325         tour.append(nxt)
326         unvisited.discard(nxt)
327         cur = nxt
328     return tour
329
330
331 def tsp_2opt(
332     dist: np.ndarray,
333     tour: Sequence[int] | None = None,
334     *,
335     max_iter: int = 5000,
336 ) -> tuple[list[int], float]:
337     """TSP 2-opt 局部搜索改进，返回 ``(访问顺序, 总路程)``。
338
339     ``tour=None`` 时先用最近邻构造再改进。确定性算法，可复现。
340
341     用法::
342
343         d = pm.optimize.distance_matrix(points)
344         tour, length = pm.optimize.tsp_2opt(d)
345     """
346     dist = np.asarray(dist, dtype=float)
347     t = list(tour) if tour is not None else tsp_nearest_neighbor(dist)
348     n = len(t)
349     improved = True
350     it = 0
351     while improved and it < max_iter:
352         improved = False
353         for i in range(n - 1):
354             for j in range(i + 2, n):
355                 if i == 0 and j == n - 1:
356                     continue
357                 a, b = t[i], t[i + 1]
358                 c, d = t[j], t[(j + 1) % n]
359                 delta = dist[a, c] + dist[b, d] - dist[a, b] - dist[c, d]
360                 if delta < -1e-10:
361                     t[i + 1:j + 1] = t[i + 1:j + 1][::-1]
362                     improved = True
363                     it += 1
364                     if it >= max_iter:
365                         break
366                 if it >= max_iter:
367                     break
368     return t, _tour_length(dist, t)

```

program/src/program/plotting.py

```

1  """论文级绘图工具：中文字体、PDF 矢量输出、常用图型一套搞定。
2
3  约定
4  ----
5  - 所有函数返回 ``(fig, ax)``（或自定义说明），并可选 ``save="图名"`` 参数：
6    传入即按 ``figures/图名.pdf`` 保存，同时把作图数据存到
7    ``code/outputs/figure_data/图名.csv``（满足 RESULTS_REPORT 数据可追溯要求）。
8  - 不在图内写大标题：标题交给论文 caption。
9  - 默认输出矢量 PDF（300 dpi），也支持 ``save_fig(..., formats=("pdf","png"))``。
10
11  用法::
12
13      import program as pm
14
15      pm.init() # 必须：设置中文字体与论文风格
16      pm.line(x, [y1, y2], labels=["预测值", "真实值"], xlabel="时间/s", ylabel="温度/°C", save="q1_temp")
17  """
18
19  from __future__ import annotations
20
21  from pathlib import Path
22  from typing import Any, Mapping, Sequence
23
24  import numpy as np
25
26  import matplotlib.pyplot as plt
27
28  from .config import figure_data_dir, figures_dir
29
30  _FONT_READY = False
31
32
33  def _bootstrap_font() -> None:
34      """未调用 ``config.init()`` 时的兜底中文字体设置（只做一次）。"""
35      global _FONT_READY
36      if _FONT_READY:
37          return
38      _FONT_READY = True
39      from .config import find_cjk_font
40
41      name = find_cjk_font()
42      if name:
43          current = list(plt.rcParams.get("font.sans-serif", []))
44          if name not in current:
45              plt.rcParams["font.sans-serif"] = [name] + [c for c in current if c != "DejaVu Sans"]
46              plt.rcParams["axes.unicode_minus"] = False
47
48
49  # -----
50  # 基础设施
51  # -----
52  def save_fig(
53      fig,
54      name: str | Path,
55      *,
56      outdir: str | Path | None = None,
57      formats: tuple[str, ...] = ("pdf",),
58      dpi: int | None = None,
59      close: bool = False,
60      data: Any = None,
61      transparent: bool = False,
62      tight: bool = True,
63  ) -> list[Path]:
64      """保存图形为论文可用的矢量文件，并可选保存作图数据。
65
66      Parameters
67      -----
68      fig : Figure
69          matplotlib 图对象。

```

```

70 name : str | Path
71     文件名（可带子目录或扩展名）。带扩展名时 ``formats`` 被忽略。
72 outdir : 路径 | None
73     输出目录，默认 ``figures``。
74 formats : tuple[str, ...]
75     输出格式，默认只输出 ``pdf``；预览可加 ``"png"``。
76 data : DataFrame | dict | ndarray | None
77     作图数据，保存到 ``code/outputs/figure_data/<name>.csv``（或 json）。
78
79 Returns
80 -----
81 list[Path]
82     实际写出的文件路径列表。
83 """
84 _bootstrap_font()
85 out = Path(outdir) if outdir is not None else figures_dir()
86 name_p = Path(str(name))
87 stem = name_p.stem if name_p.suffix else name_p.name
88 base = out / name_p.parent / stem
89 fmts = [name_p.suffix.lstrip(".").lower()] if name_p.suffix else list(formats)
90
91 if tight:
92     try:
93         fig.tight_layout()
94     except Exception:
95         pass
96
97 paths: list[Path] = []
98 for f in fmts:
99     p = base.with_suffix("." + f)
100    p.parent.mkdir(parents=True, exist_ok=True)
101    fig.savefig(p, dpi=dpi, transparent=transparent)
102    paths.append(p)
103
104 if data is not None:
105     import pandas as pd
106
107     ddir = figure_data_dir()
108     rel = base.relative_to(out)
109     ddir = ddir / rel.parent
110     ddir.mkdir(parents=True, exist_ok=True)
111     try:
112         if isinstance(data, pd.Series):
113             data = data.to_frame()
114         if isinstance(data, pd.DataFrame):
115             data.to_csv(ddir / f"{stem}.csv", index=False, encoding="utf-8-sig")
116         elif isinstance(data, dict):
117             try:
118                 pd.DataFrame(data).to_csv(
119                     ddir / f"{stem}.csv", index=False, encoding="utf-8-sig"
120                 )
121             except ValueError: # 不等长序列
122                 from .dataio import save_json
123
124                 save_json(data, ddir / f"{stem}.json")
125         else:
126             arr = np.asarray(data)
127             if arr.ndim == 2:
128                 pd.DataFrame(arr).to_csv(
129                     ddir / f"{stem}.csv", index=False, encoding="utf-8-sig"
130                 )
131             else:
132                 pd.DataFrame({stem: arr}).to_csv(
133                     ddir / f"{stem}.csv", index=False, encoding="utf-8-sig"
134                 )
135     except Exception as err: # 数据记录失败不阻断绘图
136         print(f"[save_fig] 作图数据保存失败: {err}")
137
138 if close:

```

```

139     plt.close(fig)
140     return paths
141
142
143 def _get_ax(ax, figsize=None):
144     _bootstrap_font()
145     if ax is not None:
146         return ax.figure, ax
147     fig, ax = plt.subplots(figsize=figsize)
148     return fig, ax
149
150
151 def _maybe_save(save, fig, data=None):
152     if save is None or save is False:
153         return
154     save_fig(fig, save, data=data, close=False)
155
156
157 def _as_series_list(ys: Any) -> list[np.ndarray]:
158     """把多种输入的 y 统一成「一组曲线」: list[ndarray]。"""
159     if isinstance(ys, np.ndarray) and ys.ndim == 2:
160         return [ys[:, j] for j in range(ys.shape[1])]
161     if isinstance(ys, (list, tuple)) and len(ys) > 0 and hasattr(ys[0], "__len__"):
162         return [np.asarray(y) for y in ys]
163     return [np.asarray(ys)]
164
165
166 # -----
167 # 常用图型
168 # -----
169 def line(
170     x: Sequence,
171     ys: Any,
172     *,
173     labels: Sequence[str] | None = None,
174     xlabel: str = "",
175     ylabel: str = "",
176     marker: str | None = None,
177     linestyle: str = "-",
178     save: str | Path | None = None,
179     figsize: tuple[float, float] | None = None,
180     ax=None,
181     legend: bool = True,
182     **plot_kw: Any,
183 ):
184     """折线图（支持多条曲线）。
185
186     ``ys`` 可以是 1D 序列、2D 数组（每列一条曲线）或一组序列；
187     ``labels`` 给每条曲线命名。返回 ``(fig, ax)``。
188     """
189     fig, ax = _get_ax(ax, figsize)
190     series = _as_series_list(ys)
191     for i, y in enumerate(series):
192         lab = labels[i] if labels and i < len(labels) else None
193         ax.plot(x, y, marker=marker, linestyle=linestyle, label=lab, **plot_kw)
194     ax.set_xlabel(xlabel)
195     ax.set_ylabel(ylabel)
196     has_label = labels is not None or any(
197         not ln.get_label().startswith("_") for ln in ax.get_lines()
198     )
199     if legend and has_label:
200         ax.legend()
201     data: dict[str, Any] = {"x": np.asarray(x).ravel()}
202     multi = len(series) > 1
203     for i, y in enumerate(series):
204         data[f"y{i + 1}"] if multi else "y" = np.asarray(y).ravel()
205     _maybe_save(save, fig, data=data)
206     return fig, ax
207

```

```

208
209 def scatter(
210     x: Sequence,
211     y: Sequence,
212     *,
213     xlabel: str = "",
214     ylabel: str = "",
215     label: str | None = None,
216     s: float | None = None,
217     save: str | Path | None = None,
218     figsize: tuple[float, float] | None = None,
219     ax=None,
220     **kw: Any,
221 ):
222     """散点图。返回 ``(fig, ax)``。"""
223     fig, ax = _get_ax(ax, figsize)
224     ax.scatter(x, y, label=label, s=s, **kw)
225     ax.set_xlabel(xlabel)
226     ax.set_ylabel(ylabel)
227     if label:
228         ax.legend()
229     _maybe_save(save, fig, data={"x": x, "y": y})
230     return fig, ax
231
232
233 def scatter_fit(
234     x: Sequence,
235     y: Sequence,
236     *,
237     degree: int = 1,
238     xlabel: str = "",
239     ylabel: str = "",
240     show_eq: bool = True,
241     color: str | None = None,
242     save: str | Path | None = None,
243     figsize: tuple[float, float] | None = None,
244     ax=None,
245     **kw: Any,
246 ):
247     """散点 + 多项式拟合线（含  $R^2$  / 表达式标注）。
248
249     返回 ``(fig, ax, info)``，``info = {"coef", "r2", "rmse"}``。
250     """
251     from .fitting import poly_str
252
253     fig, ax = _get_ax(ax, figsize)
254     xa = np.asarray(x, dtype=float)
255     ya = np.asarray(y, dtype=float)
256     ax.scatter(xa, ya, **kw)
257     coef = np.polyfit(xa, ya, degree)
258     xx = np.linspace(xa.min(), xa.max(), 200)
259     yy = np.polyval(coef, xx)
260     yhat = np.polyval(coef, xa)
261     ss_res = float(np.sum((ya - yhat) ** 2))
262     ss_tot = float(np.sum((ya - ya.mean()) ** 2))
263     r2 = 1 - ss_res / ss_tot if ss_tot > 0 else float("nan")
264     rmse = float(np.sqrt(np.mean((ya - yhat) ** 2)))
265     line_label = f"拟合曲线  $R^2 = {r2:.4f}"
266     ax.plot(xx, yy, color=color, label=line_label)
267     if show_eq and degree <= 3:
268         ax.text(
269             0.03, 0.97, f"$y = {poly_str(coef)}$\\n$R^2 = {r2:.4f}$",
270             transform=ax.transAxes, va="top", ha="left",
271             bbox=dict(boxstyle="round", fc="white", ec="0.6", alpha=0.85),
272         )
273     ax.set_xlabel(xlabel)
274     ax.set_ylabel(ylabel)
275     ax.legend()
276     _maybe_save(save, fig, data={"x": xa, "y": ya})$ 
```

```

277     return fig, ax, {"coef": coef, "r2": r2, "rmse": rmse}
278
279
280 def bar(
281     labels: Sequence[str],
282     values: Sequence[float],
283     *,
284     errors: Sequence[float] | None = None,
285     xlabel: str = "",
286     ylabel: str = "",
287     orient: str = "v",
288     color: str | None = None,
289     value_labels: bool = False,
290     rot: float = 0,
291     save: str | Path | None = None,
292     figsize: tuple[float, float] | None = None,
293     ax=None,
294     **kw: Any,
295 ):
296     """柱状图（`orient="h"` 为条形图）。返回 `(fig, ax)`。"""
297     fig, ax = _get_ax(ax, figsize)
298     pos = np.arange(len(labels))
299     if orient == "h":
300         ax.barh(pos, values, xerr=errors, color=color, capsize=3, **kw)
301         ax.set_yticks(pos, labels)
302         ax.set_xlabel(xlabel or ylabel)
303         if value_labels:
304             for i, v in enumerate(values):
305                 ax.text(v, i, f"{v:.3g}", va="center")
306     else:
307         ax.bar(pos, values, yerr=errors, color=color, capsize=3, **kw)
308         ax.set_xticks(pos, labels)
309         if rot:
310             plt.setp(ax.get_xticklabels(), rotation=rot, ha="right" if rot > 0 else "center")
311         ax.set_xlabel(xlabel)
312         ax.set_ylabel(ylabel)
313         if value_labels:
314             for i, v in enumerate(values):
315                 ax.text(i, v, f"{v:.3g}", ha="center", va="bottom")
316     _maybe_save(save, fig, data={"类别": list(labels), "数值": list(values)})
317     return fig, ax
318
319
320 def bar_group(
321     labels: Sequence[str],
322     series: Mapping[str, Sequence[float]],
323     *,
324     xlabel: str = "",
325     ylabel: str = "",
326     rot: float = 0,
327     save: str | Path | None = None,
328     figsize: tuple[float, float] | None = None,
329     ax=None,
330     **kw: Any,
331 ):
332     """分组柱状图。`series = {"方法A": [...], "方法B": [...]}`。返回 `(fig, ax)`。"""
333     fig, ax = _get_ax(ax, figsize)
334     x = np.arange(len(labels))
335     n = len(series)
336     width = 0.8 / max(n, 1)
337     for i, (name, vals) in enumerate(series.items()):
338         offset = (i - (n - 1) / 2) * width
339         ax.bar(x + offset, vals, width=width * 0.92, label=name, **kw)
340     ax.set_xticks(x, labels)
341     if rot:
342         plt.setp(ax.get_xticklabels(), rotation=rot, ha="right" if rot > 0 else "center")
343     ax.set_xlabel(xlabel)
344     ax.set_ylabel(ylabel)
345     ax.legend()

```

```

346     _maybe_save(save, fig, data={"类别": list(labels), **{k: list(v) for k, v in series.items()}})
347     return fig, ax
348
349
350 def hist(
351     x: Sequence[float],
352     *,
353     bins: int = 30,
354     density: bool = True,
355     kde: bool = True,
356     xlabel: str = "",
357     ylabel: str = "",
358     label: str | None = None,
359     save: str | Path | None = None,
360     figsize: tuple[float, float] | None = None,
361     ax=None,
362     **kw: Any,
363 ):
364     """直方图 + 可选 KDE 密度曲线（默认密度归一）。返回 ``(fig, ax)``。"""
365     fig, ax = _get_ax(ax, figsize)
366     xa = np.asarray(x, dtype=float)
367     ylab = ylabel or ("概率密度" if density else "频数")
368     ax.hist(xa, bins=bins, density=density, alpha=0.75, label=label, **kw)
369     if kde and len(xa) >= 5:
370         from scipy.stats import gaussian_kde
371
372         kde_obj = gaussian_kde(xa)
373         grid = np.linspace(xa.min(), xa.max(), 200)
374         ax.plot(grid, kde_obj(grid), color="crimson", lw=1.8, label="KDE 密度")
375     ax.set_xlabel(xlabel)
376     ax.set_ylabel(ylabel)
377     if label or (kde and len(xa) >= 5):
378         ax.legend()
379     _maybe_save(save, fig, data={"值": xa})
380     return fig, ax
381
382
383 def box(
384     data: Mapping[str, Sequence[float]] | np.ndarray,
385     *,
386     xlabel: str = "",
387     ylabel: str = "",
388     save: str | Path | None = None,
389     figsize: tuple[float, float] | None = None,
390     ax=None,
391     **kw: Any,
392 ):
393     """箱线图。`data` 为 `{组名: 数据}` 或 2D 数组（每列一组）。返回 ``(fig, ax)``。"""
394     fig, ax = _get_ax(ax, figsize)
395     if isinstance(data, Mapping):
396         groups = {str(k): np.asarray(v, dtype=float) for k, v in data.items()}
397     else:
398         arr = np.asarray(data, dtype=float)
399         arr = arr[:, None] if arr.ndim == 1 else arr
400         groups = {f"组{j + 1}": arr[:, j] for j in range(arr.shape[1])}
401     ax.boxplot(
402         list(groups.values()),
403         tick_labels=list(groups.keys()),
404         patch_artist=True,
405         **kw,
406     )
407     ax.set_xlabel(xlabel)
408     ax.set_ylabel(ylabel)
409     _maybe_save(save, fig, data=groups)
410     return fig, ax
411
412
413 def heatmap(
414     matrix: Any,

```

```

415     *,
416     xticklabels: Sequence[str] | None = None,
417     yticklabels: Sequence[str] | None = None,
418     annot: bool = False,
419     fmt: str = ".2f",
420     cmap: str = "RdBu_r",
421     vmin: float | None = None,
422     vmax: float | None = None,
423     cbar_label: str = "",
424     xlabel: str = "",
425     ylabel: str = "",
426     rot: float = 45,
427     save: str | Path | None = None,
428     figsize: tuple[float, float] | None = None,
429     ax=None,
430 ):
431     """热力图。`matrix` 支持 DataFrame (自动取行列名) 或 ndarray。返回 ``(fig, ax)``。"""
432     import pandas as pd
433
434     fig, ax = _get_ax(ax, figsize)
435     if isinstance(matrix, pd.DataFrame):
436         xticklabels = xticklabels or [str(c) for c in matrix.columns]
437         yticklabels = yticklabels or [str(i) for i in matrix.index]
438         arr = matrix.to_numpy(dtype=float)
439     else:
440         arr = np.asarray(matrix, dtype=float)
441     im = ax.imshow(arr, cmap=cmap, aspect="auto", vmin=vmin, vmax=vmax)
442     n_r, n_c = arr.shape
443     ax.set_xticks(range(n_c), xticklabels if xticklabels is not None else range(n_c))
444     ax.set_yticks(range(n_r), yticklabels if yticklabels is not None else range(n_r))
445     if rot:
446         plt.setp(ax.get_xticklabels(), rotation=rot, ha="right")
447     if annot and n_r * n_c <= 200:
448         finite = arr[np.isfinite(arr)]
449         vmax_abs = max(float(np.abs(finite).max()), 1e-12) if finite.size else 1.0
450         for i in range(n_r):
451             for j in range(n_c):
452                 val = arr[i, j]
453                 txt = format(val, fmt) if np.isfinite(val) else "—"
454                 shade = "white" if np.isfinite(val) and abs(val) > 0.6 * vmax_abs else "black"
455                 ax.text(j, i, txt, ha="center", va="center", fontsize=8, color=shade)
456     ax.set_xlabel(xlabel)
457     ax.set_ylabel(ylabel)
458     cb = fig.colorbar(im, ax=ax, shrink=0.85)
459     if cbar_label:
460         cb.set_label(cbar_label)
461     ax.grid(False)
462     _maybe_save(
463         save, fig,
464         data=pd.DataFrame(arr, index=yticklabels, columns=xticklabels),
465     )
466     return fig, ax
467
468
469 def corr_heatmap(
470     df,
471     *,
472     method: str = "pearson",
473     annot: bool = True,
474     save: str | Path | None = None,
475     figsize: tuple[float, float] | None = None,
476     ax=None,
477 ):
478     """相关系数热力图 (-1~1 固定色阶, 适合论文)。返回 ``(fig, ax)``。"""
479     from .stats import corr_matrix
480
481     corr = corr_matrix(df, method=method)
482     fig, ax = heatmap(
483         corr, annot=annot, fmt=".2f", cmap="RdBu_r",

```

```

484         vmin=-1, vmax=1, cbar_label="相关系数",
485         save=save, figsize=figsize, ax=ax,
486     )
487     return fig, ax
488
489
490 def radar(
491     labels: Sequence[str],
492     series: Mapping[str, Sequence[float]],
493     *,
494     normalize: bool = False,
495     fill: bool = True,
496     ylim: tuple[float, float] | None = None,
497     save: str | Path | None = None,
498     figsize: tuple[float, float] | None = None,
499 ):
500     """雷达图（综合评价/多方案对比）。`series = {"方案A": [...], ...}`。
501
502     ``normalize=True`` 时按指标（列）做 min-max 归一化到 [0, 1]。
503     返回 ``(fig, ax)``。
504     """
505     _bootstrap_font()
506     fig = plt.figure(figsize=figsize)
507     ax = fig.add_subplot(projection="polar")
508     n = len(labels)
509     angles = np.linspace(0, 2 * np.pi, n, endpoint=False).tolist()
510     angles_c = angles + angles[:1]
511
512     mat = np.array([list(v) for v in series.values()], dtype=float)
513     if normalize:
514         lo = mat.min(axis=0)
515         hi = mat.max(axis=0)
516         rng = np.where(hi - lo == 0, 1, hi - lo)
517         mat = (mat - lo) / rng
518
519     for (name, _), vals in zip(series.items(), mat):
520         vals_c = np.concatenate([vals, vals[:1]])
521         ax.plot(angles_c, vals_c, label=str(name))
522         if fill:
523             ax.fill(angles_c, vals_c, alpha=0.10)
524     ax.set_xticks(angles, [str(l) for l in labels])
525     if ylim:
526         ax.set_ylim(*ylim)
527     ax.legend(loc="upper right", bbox_to_anchor=(1.25, 1.1))
528     _maybe_save(save, fig, data={"指标": list(labels), **{k: list(v) for k, v in series.items()}})
529     return fig, ax
530
531
532 def dual_axis(
533     x: Sequence,
534     y1: Sequence,
535     y2: Sequence,
536     *,
537     labels: tuple[str, str] = ("左轴", "右轴"),
538     xlabel: str = "",
539     ylabel1: str = "",
540     ylabel2: str = "",
541     colors: tuple[str, str] = ("4C72B0", "C44E52"),
542     save: str | Path | None = None,
543     figsize: tuple[float, float] | None = None,
544     ax=None,
545 ):
546     """双纵轴折线图（量纲不同的两条曲线）。返回 ``(fig, ax, ax2)``。"""
547     fig, ax = _get_ax(ax, figsize)
548     ax2 = ax.twinx()
549     ax.plot(x, y1, color=colors[0], label=labels[0])
550     ax2.plot(x, y2, color=colors[1], label=labels[1])
551     ax.set_xlabel(xlabel)
552     ax.set_ylabel(ylabel1 or labels[0], color=colors[0])

```

```

553     ax2.set_ylabel(ylabel2 or labels[1], color=colors[1])
554     ax.tick_params(axis="y", colors=colors[0])
555     ax2.tick_params(axis="y", colors=colors[1])
556     lines = ax.get_lines() + ax2.get_lines()
557     ax.legend(lines, [l.get_label() for l in lines], loc="best")
558     ax.grid(True, alpha=0.3)
559     _maybe_save(save, fig, data={"x": list(x), labels[0]: list(y1), labels[1]: list(y2)})
560     return fig, ax, ax2
561
562
563 def pie(
564     labels: Sequence[str],
565     values: Sequence[float],
566     *,
567     startangle: float = 90,
568     autopct: str | None = "%1.1f%%",
569     explode: Sequence[float] | None = None,
570     save: str | Path | None = None,
571     figsize: tuple[float, float] | None = None,
572     ax=None,
573 ):
574     """饼图（占比结构）。返回 ``(fig, ax)``。"""
575     fig, ax = _get_ax(ax, figsize)
576     ax.pie(values, labels=labels, startangle=startangle, autopct=autopct, explode=explode)
577     ax.axis("equal")
578     _maybe_save(save, fig, data={"类别": list(labels), "数值": list(values)})
579     return fig, ax
580
581
582 def errorbar(
583     x: Sequence,
584     y: Sequence,
585     yerr: Sequence | None = None,
586     *,
587     xlabel: str = "",
588     ylabel: str = "",
589     label: str | None = None,
590     fmt: str = "o-",
591     capsize: float = 3,
592     save: str | Path | None = None,
593     figsize: tuple[float, float] | None = None,
594     ax=None,
595     **kw: Any,
596 ):
597     """带误差棒的折线/散点图。返回 ``(fig, ax)``。"""
598     fig, ax = _get_ax(ax, figsize)
599     ax.errorbar(x, y, yerr=yerr, fmt=fmt, capsize=capsize, label=label, **kw)
600     ax.set_xlabel(xlabel)
601     ax.set_ylabel(ylabel)
602     if label:
603         ax.legend()
604     data = {"x": list(x), "y": list(y)}
605     if yerr is not None:
606         data["yerr"] = list(np.asarray(yerr).ravel())
607     _maybe_save(save, fig, data=data)
608     return fig, ax
609
610
611 def surface3d(
612     X: Any,
613     Y: Any,
614     Z: Any,
615     *,
616     xlabel: str = "",
617     ylabel: str = "",
618     zlabel: str = "",
619     cmap: str = "viridis",
620     elev: float = 30,
621     azim: float = -60,

```

```

622     cbar: bool = True,
623     save: str | Path | None = None,
624     figsize: tuple[float, float] = (7.5, 5.5),
625 ):
626     """三维曲面图（参数敏感性、二维优化景观、调参曲面）。返回 ``(fig, ax)``。"""
627     _bootstrap_font()
628     fig = plt.figure(figsize=figsize)
629     ax = fig.add_subplot(111, projection="3d")
630     surf = ax.plot_surface(X, Y, Z, cmap=cmap, linewidth=0, antialiased=True, alpha=0.95)
631     ax.set_xlabel(xlabel)
632     ax.set_ylabel(ylabel)
633     ax.set_zlabel(zlabel)
634     ax.view_init(elev=elev, azim=azim)
635     if cbar:
636         fig.colorbar(surf, ax=ax, shrink=0.6, pad=0.1, label=zlabel)
637     _maybe_save(
638         save, fig,
639         data={"X": np.asarray(X).ravel(), "Y": np.asarray(Y).ravel(), "Z": np.asarray(Z).ravel()},
640     )
641     return fig, ax
642
643
644 def roc(
645     y_true: Sequence,
646     y_score: Sequence[float],
647     *,
648     xlabel: str = "假阳性率 FPR",
649     ylabel: str = "真阳性率 TPR",
650     save: str | Path | None = None,
651     figsize: tuple[float, float] | None = None,
652     ax=None,
653 ):
654     """二分类 ROC 曲线（含 AUC 标注）。返回 ``(fig, ax, auc_value)``。"""
655     from sklearn.metrics import auc as sk_auc
656     from sklearn.metrics import roc_curve
657
658     fig, ax = _get_ax(ax, figsize)
659     fpr, tpr, _ = roc_curve(y_true, y_score)
660     auc_val = float(sk_auc(fpr, tpr))
661     ax.plot(fpr, tpr, lw=1.8, label=f"ROC (AUC = {auc_val:.4f}) ")
662     ax.plot([0, 1], [0, 1], "k--", lw=0.9, alpha=0.6)
663     ax.set_xlabel(xlabel)
664     ax.set_ylabel(ylabel)
665     ax.legend(loc="lower right")
666     _maybe_save(save, fig, data={"FPR": fpr, "TPR": tpr})
667     return fig, ax, auc_val

```

program/src/program/report.py

```

1  """结果记录与论文素材输出: RESULTS_REPORT 追加、LaTeX 表格、结果文件。
2
3  典型用法（每个子问题算完后）::
4
5      import program as pm
6
7      pm.report.record_result(
8          "问题一结果",
9          {"最优成本": 1234.5, "总利润": 6789.0},
10         note="求解器: HiGHS, 约束全部满足。",
11     )
12     pm.report.record_result("问题一灵敏度", sensitivity_df)
13     pm.report.save_outputs(sensitivity_df, "q1_sensitivity")
14     print(pm.report.to_latex(result_table, caption="问题一结果", label="tab:q1"))
15 """
16
17 from __future__ import annotations
18
19 from pathlib import Path
20 from typing import Any, Sequence

```

```

21
22 import numpy as np
23
24 from .config import reports_dir
25 from .utils import fmt, markdown_table
26
27 REPORT_NAME = "RESULTS_REPORT.md"
28
29
30 # -----
31 # 报告写入
32 # -----
33 def report_path(path: str | Path | None = None) -> Path:
34     """RESULTS_REPORT 路径 (默认 ``<root>/reports/RESULTS_REPORT.md``) 。”"""
35     return Path(path) if path is not None else reports_dir() / REPORT_NAME
36
37
38 def ensure_report(title: str = "# 计算结果", path: str | Path | None = None) -> Path:
39     """确保报告文件存在 (不存在时写入标题), 返回路径。"""
40     p = report_path(path)
41     if not p.exists():
42         p.parent.mkdir(parents=True, exist_ok=True)
43         p.write_text(title.rstrip() + "\n\n", encoding="utf-8")
44     return p
45
46
47 def record(text: str, path: str | Path | None = None) -> Path:
48     """向 RESULTS_REPORT 追加一段 Markdown (自动建目录/文件) 。”"""
49     p = ensure_report(path=path)
50     with p.open("a", encoding="utf-8") as f:
51         f.write(text.rstrip() + "\n\n")
52     return p
53
54
55 def record_result(
56     title: str,
57     data: Any,
58     *,
59     note: str | None = None,
60     path: str | Path | None = None,
61     level: int = 3,
62 ) -> Path:
63     """把一段结果以 ``### 标题 + 表格`` 的形式追加进 RESULTS_REPORT。
64
65     Parameters
66     -----
67     title : str
68         小节标题, 如 ``"问题一结果"``。
69     data : dict | DataFrame | list | str
70         - ``dict[str, 标量]`` → 两列「指标 / 数值」表格;
71         - ``dict[str, 序列]`` 或 DataFrame → 普通表格;
72         - ``str`` → 原样写入正文。
73     note : str | None
74         表格前的补充说明 (方法、约束校验、结论等)。
75     level : int
76         Markdown 标题级别, 默认 3 (``###``)。
77
78     """
79     import pandas as pd
80
81     parts = [f"{'#' * level} {title}"]
82     if note:
83         parts.append(note)
84     if isinstance(data, str):
85         parts.append(data)
86     elif isinstance(data, pd.DataFrame):
87         parts.append(markdown_table(data))
88     elif isinstance(data, dict):
89         scalars = all(np.isscalar(v) or v is None or isinstance(v, str) for v in data.values())
90         if scalars:

```

```

90         rows = [{"指标": k, "数值": fmt(v)} for k, v in data.items()]
91         parts.append(markdown_table(rows))
92     else:
93         parts.append(markdown_table(data))
94     else:
95         parts.append(markdown_table(data))
96     return record("\n\n".join(parts), path=path)
97
98
99 def record_environment(path: str | Path | None = None) -> Path:
100     """把运行环境 (Python 与库版本) 写入报告。"""
101     from .utils import env_markdown
102
103     return record("## 运行环境\n\n" + env_markdown(), path=path)
104
105
106 # -----
107 # 论文素材
108 # -----
109 def to_latex(
110     df,
111     *,
112     caption: str | None = None,
113     label: str | None = None,
114     float_format: str = "%.4f",
115     index: bool = False,
116     escape: bool = True,
117     column_format: str | None = None,
118 ) -> str:
119     """DataFrame → LaTeX 表格字符串 (booktabs 风格, 可直接贴进论文)。
120
121     注意: ``float_format`` 为 printf 风格字符串; 含下划线的列名请传 ``escape=False``
122     或改用 ``\\textbackslash`` 转义。
123     """
124     kwargs: dict = dict(
125         index=index,
126         escape=escape,
127         float_format=float_format,
128         caption=caption,
129         label=label,
130     )
131     if column_format:
132         kwargs["column_format"] = column_format
133     try:
134         return df.to_latex(**kwargs)
135     except TypeError:
136         # 兼容新旧 pandas 对 float_format 类型的差异
137         import pandas as pd
138
139         kwargs["float_format"] = lambda v: f"{float(v):.4f}"
140         latex = df.to_latex(**kwargs)
141         _ = pd # 保持引用避免误删导入
142         return latex
143
144
145 def save_outputs(
146     data: Any,
147     name: str,
148     *,
149     fmt: str | None = None,
150     outdir: str | Path | None = None,
151 ) -> Path:
152     """把结果数据保存到 ``code/outputs/<name>.<ext>`` 并返回文件路径。
153
154     - DataFrame → ``.csv`` (``utf-8-sig``, Excel 直接打开不乱码)
155     - dict / list → ``.json``
156     - ndarray → ``.csv`` (1D 为单列, 2D 带列名 col0、col1...)
157     - str → ``.md``
158     - 其他 → ``.pkl``

```

```

159 """
160 from .config import outputs_dir
161
162 outdir = Path(outdir) if outdir is not None else outputs_dir()
163 outdir.mkdir(parents=True, exist_ok=True)
164
165 import pandas as pd
166
167 if fmt is None:
168     if isinstance(data, pd.DataFrame):
169         fmt = "csv"
170     elif isinstance(data, (dict, list, tuple)):
171         fmt = "json"
172     elif isinstance(data, np.ndarray):
173         fmt = "csv"
174     elif isinstance(data, str):
175         fmt = "md"
176     else:
177         fmt = "pkl"
178 fmt = fmt.lower().lstrip(".")
179
180 p = outdir / f"{name}.{fmt}"
181 if fmt == "csv":
182     if isinstance(data, np.ndarray):
183         if data.ndim == 1:
184             data = pd.DataFrame({name: data})
185         else:
186             data = pd.DataFrame(data, columns=[f"col{i}" for i in range(data.shape[1])])
187     if isinstance(data, pd.Series):
188         data = data.to_frame()
189     data.to_csv(p, index=False, encoding="utf-8-sig")
190 elif fmt == "json":
191     from .dataio import save_json
192
193     save_json(data, p)
194 elif fmt in {"md", "txt"}:
195     p.write_text(str(data), encoding="utf-8")
196 else:
197     import pickle
198
199     with p.open("wb") as f:
200         pickle.dump(data, f, protocol=pickle.HIGHEST_PROTOCOL)
201 return p

```

program/src/program/stats.py

```

1 """统计检验与回归分析（统一的 TestResult 输出，含中文结论）。
2
3 覆盖数学建模高频统计需求：正态性、t 检验、方差分析、卡方、相关性、
4 一元/多元线性回归、置信区间与 bootstrap。
5
6 用法::
7
8     import program as pm
9
10     r = pm.stats.compare_groups(group_a, group_b, group_c) # 自动选 ANOVA / Kruskal
11     print(r) # 直接可读的中文结论
12     pm.stats.ols(df, y="销量", xs=["价格", "广告费"]).summary() # 多元回归摘要
13 """
14
15 from __future__ import annotations
16
17 from dataclasses import dataclass, field
18 from typing import Any, Sequence
19
20 import numpy as np
21 import pandas as pd
22 from scipy import stats as sps
23

```

```

24 from .utils import fmt
25
26
27 # -----
28 # 统一检验结果
29 # -----
30 @dataclass
31 class TestResult:
32     """假设检验结果 (`str(r)` 直接给出中文结论)。"""
33
34     name: str
35     statistic: float
36     pvalue: float
37     h0: str = ""
38     alpha: float = 0.05
39     extra: dict[str, Any] = field(default_factory=dict)
40
41     @property
42     def significant(self) -> bool:
43         """p < alpha 时为 True (拒绝 H0)。"""
44         return bool(self.pvalue < self.alpha)
45
46     def __str__(self) -> str:
47         sign = "<" if self.significant else " "
48         verdict = "拒绝 H0" if self.significant else "不拒绝 H0"
49         msg = (
50             f"【{self.name}】统计量 = {fmt(self.statistic)}, "
51             f"p = {fmt(self.pvalue)} {sign} = {self.alpha} -> {verdict}"
52         )
53         if self.h0:
54             msg += f" (H0: {self.h0}) "
55         if self.extra:
56             extras = ", ".join(f"{k} = {fmt(v)}" for k, v in self.extra.items())
57             msg += f"\n附: {extras}"
58         return msg
59
60     def to_dict(self) -> dict[str, Any]:
61         d = {"检验": self.name, "统计量": self.statistic, "p值": self.pvalue,
62             "结论": "拒绝 H0" if self.significant else "不拒绝 H0"}
63         d.update(self.extra)
64         return d
65
66
67 def _clean(x: Sequence[float]) -> np.ndarray:
68     a = np.asarray(x, dtype=float).ravel()
69     return a[np.isfinite(a)]
70
71
72 def _cohens_d(a: np.ndarray, b: np.ndarray) -> float:
73     n1, n2 = len(a), len(b)
74     s2 = ((n1 - 1) * a.var(ddof=1) + (n2 - 1) * b.var(ddof=1)) / max(n1 + n2 - 2, 1)
75     return float((a.mean() - b.mean()) / np.sqrt(s2)) if s2 > 0 else float("nan")
76
77
78 # -----
79 # 分布与均值检验
80 # -----
81 def normality(x: Sequence[float], alpha: float = 0.05) -> TestResult:
82     """正态性检验: n = 5000 用 Shapiro-Wilk, 否则用 D'Agostino K²。"""
83     a = _clean(x)
84     n = len(a)
85     if n < 8:
86         raise ValueError(f"正态性检验至少需要 8 个有效样本, 当前 {n}")
87     if n <= 5000:
88         stat, p = sps.shapiro(a)
89         method = "Shapiro-Wilk"
90     else:
91         stat, p = sps.normaltest(a)
92         method = "D'Agostino K²"

```

```

93     return TestResult(
94         f"正态性检验 ({method})", float(stat), float(p),
95         h0="数据服从正态分布", alpha=alpha, extra={"n": n},
96     )
97
98
99 def t_test_one(x: Sequence[float], mu: float = 0, alpha: float = 0.05) -> TestResult:
100     """单样本 t 检验 (样本均值是否等于 ``mu``)。"""
101     a = _clean(x)
102     stat, p = sps.ttest_1samp(a, mu)
103     return TestResult(
104         "单样本 t 检验", float(stat), float(p),
105         h0=f"总体均值等于 {mu}", alpha=alpha,
106         extra={"样本均值": a.mean(), "样本量": len(a)},
107     )
108
109
110 def t_test_ind(
111     a: Sequence[float],
112     b: Sequence[float],
113     *,
114     equal_var: bool = False,
115     alpha: float = 0.05,
116 ) -> TestResult:
117     """独立双样本 t 检验: 默认 Welch (不假设方差齐性)。"""
118     x, y = _clean(a), _clean(b)
119     stat, p = sps.ttest_ind(x, y, equal_var=equal_var)
120     return TestResult(
121         "独立双样本 t 检验 (Welch)" if not equal_var else "独立双样本 t 检验",
122         float(stat), float(p),
123         h0="两组均值相等", alpha=alpha,
124         extra={"均值A": x.mean(), "均值B": y.mean(),
125               "nA": len(x), "nB": len(y), "Cohen's d": _cohens_d(x, y)},
126     )
127
128
129 def t_test_paired(a: Sequence[float], b: Sequence[float], alpha: float = 0.05) -> TestResult:
130     """配对样本 t 检验。"""
131     x, y = _clean(a), _clean(b)
132     if len(x) != len(y):
133         raise ValueError("配对检验要求两组长度相同")
134     stat, p = sps.ttest_rel(x, y)
135     return TestResult(
136         "配对样本 t 检验", float(stat), float(p),
137         h0="差值的均值为 0", alpha=alpha,
138         extra={"均值差": (x - y).mean(), "n": len(x)},
139     )
140
141
142 def f_test_var(a: Sequence[float], b: Sequence[float], alpha: float = 0.05) -> TestResult:
143     """方差齐性 F 检验 (两样本)。"""
144     x, y = _clean(a), _clean(b)
145     stat = x.var(ddof=1) / y.var(ddof=1)
146     df1, df2 = len(x) - 1, len(y) - 1
147     p = 2 * min(sps.f.cdf(stat, df1, df2), 1 - sps.f.cdf(stat, df1, df2))
148     return TestResult(
149         "方差齐性 F 检验", float(stat), float(p),
150         h0="两组方差相等", alpha=alpha,
151         extra={"方差A": x.var(ddof=1), "方差B": y.var(ddof=1)},
152     )
153
154
155 # -----
156 # 多样本比较
157 # -----
158 def anova(*groups: Sequence[float], alpha: float = 0.05) -> TestResult:
159     """单因素方差分析 (ANOVA, H0: 各组均值相等)。"""
160     gs = [_clean(g) for g in groups]
161     if len(gs) < 2:

```

```

162         raise ValueError("ANOVA 至少需要两组")
163     stat, p = sps.f_oneway(*gs)
164     return TestResult(
165         "单因素方差分析 ANOVA", float(stat), float(p),
166         h0="各组均值相等", alpha=alpha,
167         extra={f"组{i + 1}均值": g.mean() for i, g in enumerate(gs)},
168     )
169
170
171 def kruskal(*groups: Sequence[float], alpha: float = 0.05) -> TestResult:
172     """Kruskal-Wallis 非参数检验（不满足正态/方差齐性时使用）。"""
173     gs = [_clean(g) for g in groups]
174     stat, p = sps.kruskal(*gs)
175     return TestResult(
176         "Kruskal-Wallis 非参数检验", float(stat), float(p),
177         h0="各组分布相同", alpha=alpha,
178     )
179
180
181 def compare_groups(*groups: Sequence[float], alpha: float = 0.05) -> TestResult:
182     """多组均值差异比较（自动选择方法）。
183
184     先检验每组正态性（Shapiro）与方差齐性（Levene）：
185     全部通过 → ANOVA；否则 → Kruskal-Wallis 非参数检验。
186     """
187     gs = [_clean(g) for g in groups]
188     if len(gs) < 2:
189         raise ValueError("至少需要两组数据")
190     normal_ok = all(len(g) >= 8 and sps.shapiro(g).pvalue > alpha for g in gs)
191     var_ok = sps.levene(*gs).pvalue > alpha
192     if normal_ok and var_ok:
193         return anova(*gs, alpha=alpha)
194     r = kruskal(*gs, alpha=alpha)
195     r.name += "（自动选择：正态性/方差齐性未通过）"
196     return r
197
198
199 def chi2_test(table: Any, *, correction: bool = False, alpha: float = 0.05) -> TestResult:
200     """列联表卡方独立性检验。
201
202     Parameters
203     -----
204     table : array-like | DataFrame
205         列联表（行 × 列的观测频数）。
206     """
207     arr = np.asarray(table, dtype=float)
208     chi2, p, dof, expected = sps.chi2_contingency(arr, correction=correction)
209     n = arr.sum()
210     r, c = arr.shape
211     cramers_v = float(np.sqrt(chi2 / (n * min(r - 1, c - 1)))) if min(r - 1, c - 1) > 0 else float("nan")
212     return TestResult(
213         "卡方独立性检验", float(chi2), float(p),
214         h0="行变量与列变量独立", alpha=alpha,
215         extra={"自由度": dof, "Cramér's V": cramers_v,
216             "最小期望频数": float(expected.min())},
217     )
218
219
220 # -----
221 # 相关性与回归
222 # -----
223 def corr_test(
224     x: Sequence[float],
225     y: Sequence[float],
226     *,
227     method: str = "pearson",
228     alpha: float = 0.05,
229 ) -> TestResult:
230     """两变量相关性检验（pearson / spearman / kendall）。"""

```

```

231     xa, ya = _clean(x), _clean(y)
232     n = min(len(xa), len(ya))
233     if method == "pearson":
234         stat, p = sps.pearsonr(xa[:n], ya[:n])
235     elif method == "spearman":
236         stat, p = sps.spearmanr(xa[:n], ya[:n])
237     elif method == "kendall":
238         stat, p = sps.kendalltau(xa[:n], ya[:n])
239     else:
240         raise ValueError(f"未知相关方法: {method!r}")
241     return TestResult(
242         f"{method} 相关检验", float(stat), float(p),
243         h0="两变量不相关", alpha=alpha, extra={"n": n},
244     )
245
246
247 def corr_matrix(df: pd.DataFrame, *, method: str = "pearson") -> pd.DataFrame:
248     """数值列相关系数矩阵 (pearson / spearman / kendall)。"""
249     num = df.select_dtypes(include="number")
250     return num.corr(method=method)
251
252
253 @dataclass
254 class LinReg:
255     """一元线性回归结果: ``y = slope * x + intercept``。"""
256
257     slope: float
258     intercept: float
259     r: float
260     r2: float
261     pvalue: float
262     stderr: float
263     n: int
264
265     def predict(self, x: Sequence[float]) -> np.ndarray:
266         return self.slope * np.asarray(x, dtype=float) + self.intercept
267
268     def __str__(self) -> str:
269         return (
270             f"y = {fmt(self.slope, 5)} x + {fmt(self.intercept, 5)}"
271             f" (R² = {fmt(self.r2, 5)}, p = {fmt(self.pvalue)}, n = {self.n}) "
272         )
273
274
275 def linregress_summary(x: Sequence[float], y: Sequence[float]) -> LinReg:
276     """一元线性回归摘要 (含 R²、斜率标准误、p 值)。"""
277     xa, ya = _clean(x), _clean(y)
278     n = min(len(xa), len(ya))
279     res = sps.linregress(xa[:n], ya[:n])
280     return LinReg(
281         slope=float(res.slope), intercept=float(res.intercept),
282         r=float(res.rvalue), r2=float(res.rvalue) ** 2,
283         pvalue=float(res.pvalue), stderr=float(res.stderr), n=n,
284     )
285
286
287 def ols(df: pd.DataFrame, y: str, xs: Sequence[str], *, add_constant: bool = True):
288     """多元线性回归 (statsmodels OLS), 返回结果对象。
289
290     打印 ``.summary()`` 查看完整统计信息; 用 :func:`ols_text` 直接拿字符串。
291     """
292     import statsmodels.api as sm
293
294     X = df[list(xs)].astype(float)
295     if add_constant:
296         X = sm.add_constant(X)
297     model = sm.OLS(df[y].astype(float), X).fit()
298     return model
299

```

```

300
301 def ols_text(df: pd.DataFrame, y: str, xs: Sequence[str], *, add_constant: bool = True) -> str:
302     """多元线性回归摘要（纯文本，可直接贴进报告/论文附录）。"""
303     return str(ols(df, y, xs, add_constant=add_constant).summary())
304
305
306 # -----
307 # 区间估计
308 # -----
309 def ci_mean(x: Sequence[float], alpha: float = 0.05) -> tuple[float, float]:
310     """总体均值的 t 置信区间，返回 ``(下界, 上界)``。"""
311     a = _clean(x)
312     n = len(a)
313     se = a.std(ddof=1) / np.sqrt(n)
314     h = se * sps.t.ppf(1 - alpha / 2, n - 1)
315     return float(a.mean() - h), float(a.mean() + h)
316
317
318 @dataclass
319 class BootstrapCI:
320     """Bootstrap 置信区间结果。"""
321
322     stat: float
323     lo: float
324     hi: float
325     se: float
326     alpha: float
327     n_boot: int
328
329     def __str__(self) -> str:
330         lv = f"{1 - self.alpha:.0%}"
331         return (
332             f"Bootstrap {lv}% CI = [{fmt(self.lo)}, {fmt(self.hi)}], "
333             f"点估计 = {fmt(self.stat)}, 标准误 = {fmt(self.se)} (B = {self.n_boot}) "
334         )
335
336
337 def bootstrap_ci(
338     x: Sequence[float],
339     stat_fn=np.mean,
340     *,
341     n_boot: int = 10000,
342     alpha: float = 0.05,
343     seed: int = 42,
344 ) -> BootstrapCI:
345     """Bootstrap 置信区间（对任意统计量 ``stat_fn``）。
346
347     Parameters
348     -----
349     x : 序列
350         样本数据。
351     stat_fn : callable
352         统计量函数，默认均值。
353     """
354     a = _clean(x)
355     rng = np.random.default_rng(seed)
356     stats_boot = np.array(
357         [stat_fn(rng.choice(a, size=len(a), replace=True)) for _ in range(n_boot)]
358     )
359     lo, hi = np.percentile(stats_boot, [100 * alpha / 2, 100 * (1 - alpha / 2)])
360     return BootstrapCI(
361         stat=float(stat_fn(a)), lo=float(lo), hi=float(hi),
362         se=float(stats_boot.std(ddof=1)), alpha=alpha, n_boot=n_boot,
363     )

```

program/src/program/utils.py

```

1 """通用工具：随机种子、计时、日志、缓存、运行环境、格式化。
2

```

```

3  本模块是纯工具层，不依赖绘图与建模库，可在任何脚本中安全导入。
4  """
5
6  from __future__ import annotations
7
8  import os
9  import pickle
10 import platform
11 import random
12 import sys
13 import time
14 from contextlib import ContextDecorator
15 from importlib import metadata
16 from pathlib import Path
17 from typing import Any, Callable, Iterable, Sequence
18
19 import numpy as np
20
21 # -----
22 # 随机种子
23 # -----
24 def set_seed(seed: int = 42) -> int:
25     """一次性固定 random / numpy / torch (若已安装) 的随机种子。
26
27     所有涉及随机过程的脚本（采样、优化、神经网络、bootstrap）必须先调用本函数，
28     保证结果可复现。
29     """
30     os.environ["PYTHONHASHSEED"] = str(seed)
31     random.seed(seed)
32     np.random.seed(seed)
33     if "torch" in sys.modules: # 不主动导入 torch (加载慢)，已导入才设置
34         import torch
35
36         torch.manual_seed(seed)
37         if torch.cuda.is_available():
38             torch.cuda.manual_seed_all(seed)
39     return seed
40
41
42 # -----
43 # 计时
44 # -----
45 class Timer(ContextDecorator):
46     """计时器：上下文管理器兼装饰器。
47
48     用法::
49
50         with Timer("求解"):
51             heavy_work()
52
53         @Timer("训练")
54         def train(): ...
55     """
56
57     def __init__(self, label: str = "", logger: Any = None):
58         self.label = label
59         self.logger = logger
60         self.elapsed: float = 0.0
61
62     def _log(self, msg: str) -> None:
63         if self.logger is not None:
64             self.logger.info(msg)
65         else:
66             print(msg)
67
68     def __enter__(self) -> "Timer":
69         self._start = time.perf_counter()
70         return self
71

```

```

72     def __exit__(self, *exc: Any) -> bool:
73         self.elapsed = time.perf_counter() - self._start
74         self._log(f"{self.label or '耗时'}: {self.elapsed:.3f} s")
75         return False
76
77
78 def timeit(func: Callable) -> Callable:
79     """装饰器：运行函数并打印耗时，原样返回函数结果。"""
80
81     def wrapper(*args: Any, **kwargs: Any) -> Any:
82         t0 = time.perf_counter()
83         result = func(*args, **kwargs)
84         print(f"{func.__name__} 耗时: {time.perf_counter() - t0:.3f} s")
85         return result
86
87     wrapper.__name__ = getattr(func, "__name__", "wrapper")
88     return wrapper
89
90
91 # -----
92 # 日志
93 # -----
94 def get_logger(name: str = "mathmodel", log_file: str | Path | None = None):
95     """返回 loguru 日志器（带模块名绑定）。
96
97     Parameters
98     -----
99     name : str
100         模块/脚本名，出现在每条日志的 ``module`` 字段里。
101     log_file : str | Path | None
102         可选日志文件路径；传入后同时写入文件。
103
104     用法::
105
106         log = get_logger("q1")
107         log.info("开始求解")
108     """
109     from loguru import logger
110
111     log = logger.bind(module=name)
112     if log_file is not None:
113         p = Path(log_file)
114         p.parent.mkdir(parents=True, exist_ok=True)
115         # loguru 的 add 返回 sink id; 重复调用同一文件会重复写，用属性去重
116         added = getattr(get_logger, "_files", set())
117         if str(p.resolve()) not in added:
118             logger.add(p, encoding="utf-8", level="DEBUG", enqueue=False)
119             added.add(str(p.resolve()))
120             get_logger._files = added
121     return log
122
123
124 # -----
125 # 结果缓存
126 # -----
127 class DataCache:
128     """某次计算的中间结果缓存（pickle 序列化）。
129
130     默认目录 ``code/outputs/cache/<name>/``。用途：避免重复训练/重复求解，
131     同时把关键中间结果沉淀为文件（RESULTS_REPORT 要求记录数据来源）。
132
133     用法::
134
135         cache = DataCache("q1")
136         params = cache.get_or_compute("params", heavy_fit, x, y)
137     """
138
139     def __init__(self, name: str, root: str | Path | None = None):
140         from .config import outputs_dir

```

```

141
142     base = Path(root) if root is not None else outputs_dir() / "cache"
143     self.dir = base / name
144     self.dir.mkdir(parents=True, exist_ok=True)
145
146     def path(self, key: str) -> Path:
147         """缓存文件路径 (``<key>.pkl``)。"""
148         return self.dir / f"{key}.pkl"
149
150     def exists(self, key: str) -> bool:
151         return self.path(key).exists()
152
153     def save(self, key: str, obj: Any) -> Path:
154         """写入缓存, 返回文件路径。"""
155         p = self.path(key)
156         with p.open("wb") as f:
157             pickle.dump(obj, f, protocol=pickle.HIGHEST_PROTOCOL)
158         return p
159
160     def load(self, key: str) -> Any:
161         """读取缓存; 不存在则抛 ``FileNotFoundError``。"""
162         p = self.path(key)
163         if not p.exists():
164             raise FileNotFoundError(f"缓存不存在: {p}")
165         with p.open("rb") as f:
166             return pickle.load(f)
167
168     def get_or_compute(self, key: str, func: Callable, *args: Any, **kwargs: Any) -> Any:
169         """存在则读缓存, 否则调用 ``func(*args, **kwargs)`` 并缓存结果。"""
170         if self.exists(key):
171             return self.load(key)
172         result = func(*args, **kwargs)
173         self.save(key, result)
174         return result
175
176     def clear(self) -> int:
177         """清空缓存目录, 返回删除的文件数。"""
178         n = 0
179         for p in self.dir.glob("*.pkl"):
180             p.unlink()
181             n += 1
182         return n
183
184
185 # -----
186 # 运行环境
187 # -----
188 def env_info() -> dict[str, str]:
189     """返回运行环境关键信息 (平台、Python 与主要库版本)。"""
190     info = {
191         "platform": platform.platform(),
192         "python": sys.version.split()[0],
193     }
194     for pkg in (
195         "numpy", "scipy", "pandas", "matplotlib", "scikit-learn",
196         "statsmodels", "sympy", "torch", "networkx", "pulp", "openpyxl", "loguru",
197     ):
198         try:
199             info[pkg] = metadata.version(pkg)
200         except metadata.PackageNotFoundError:
201             pass
202     return info
203
204
205 def env_markdown() -> str:
206     """运行环境的 Markdown 片段 (直接可贴进 RESULTS_REPORT)。"""
207     info = env_info()
208     lines = ["| 项目 | 版本 |", "| --- | --- |"]
209     labels = {"platform": "操作系统", "python": "Python"}

```

```

210     for k, v in info.items():
211         lines.append(f"| {labels.get(k, k)} | {v} |")
212     return "\n".join(lines)
213
214
215 # -----
216 # 格式化
217 # -----
218 def fmt(x: Any, nd: int = 4) -> str:
219     """把数值格式化成论文/报告友好的字符串。
220
221     - ``None`` / NaN -> ``--`` / ``NaN``
222     - 绝对值过大或过小 -> 科学计数法 (3 位有效数字)
223     - 其余 -> 至多 ``nd`` 位有效数字
224     """
225     if x is None:
226         return "--"
227     if isinstance(x, str):
228         return x
229     try:
230         f = float(x)
231     except (TypeError, ValueError):
232         return str(x)
233     if np.isnan(f):
234         return "NaN"
235     if np.isinf(f):
236         return "∞" if f > 0 else "-∞"
237     if f == 0:
238         return "0"
239     a = abs(f)
240     if a >= 1e5 or a < 1e-3:
241         return f"{f:.3e}"
242     return f"{f:.{nd}g}"
243
244
245 def markdown_table(
246     data: Any,
247     headers: Sequence[str] | None = None,
248     nd: int = 4,
249     index: bool = False,
250 ) -> str:
251     """把表格数据转成 Markdown 表格字符串。
252
253     Parameters
254     -----
255     data : DataFrame | list[dict] | list[list] | dict[str, list]
256         表格数据。
257     headers : list[str] | None
258         自定义表头; ``list[list]`` / ``DataFrame`` 默认取第一行或列名。
259     nd : int
260         数字保留的有效位数。
261     index : bool
262         是否输出 DataFrame 索引列。
263     """
264     import pandas as pd
265
266     if isinstance(data, pd.DataFrame):
267         df = data.reset_index() if index else data
268         cols = [str(c) for c in df.columns]
269         rows = df.itertuples(index=False, name=None)
270         body = [[fmt(v, nd) for v in row] for row in rows]
271     elif isinstance(data, dict):
272         cols = list(headers) if headers else [str(k) for k in data]
273         body = [[fmt(v, nd) for v in data[c]] for c in data]
274     elif isinstance(data, (list, tuple)):
275         seq = list(data)
276         if not seq:
277             return "(空表)"
278         if isinstance(seq[0], dict):

```

```

279         cols = list(headers) if headers else [str(k) for k in seq[0]]
280         body = [[fmt(item.get(c, ""), nd) for c in cols] for item in seq]
281     else:
282         cols = list(headers) if headers else [str(i) for i in range(len(seq[0]))]
283         body = [[fmt(v, nd) for v in row] for row in seq]
284     else:
285         raise TypeError(f"markdown_table 不支持的类型: {type(data)!r}")
286
287     out = ["| " + " | ".join(cols) + " |",
288           "| " + " | ".join("---" for _ in cols) + " |"]
289     out += ["| " + " | ".join(str(v) for v in row) + " |" for row in body]
290     return "\n".join(out)
291
292
293 def ensure_dir(path: str | Path) -> Path:
294     """创建目录（存在则跳过），返回 ``Path``。"""
295     p = Path(path)
296     p.mkdir(parents=True, exist_ok=True)
297     return p
298
299
300 def flatten(items: Iterable) -> list:
301     """展平一层嵌套序列。"""
302     out: list = []
303     for it in items:
304         out.extend(it)
305     return out

```

测试

program/tests/c_results_audit.py

```

1  """C 题五份官方结果文件的结构审计（独立于写出脚本，提交前固定检查）。
2
3  校验内容: sheet 名与顺序、日期连续性、行数、数值非负/有限、全天购电量与
4  源电价逐槽费用重算、储能端点/跨日连续性与 SOC 界、时间标签。
5
6  注意: 官方模板（附件 5）把 24:00 储电量放在 4:00-8:00 行，本脚本按模板校验。
7
8  运行（program/ 下）: uv run python tests/c_results_audit.py
9  """
10 from __future__ import annotations
11
12 from datetime import date, datetime, timedelta
13 from pathlib import Path
14
15 import numpy as np
16 import openpyxl
17
18 ROOT = Path(__file__).resolve().parents[2]
19 RESULTS = ROOT / "results"
20 DATA_C = ROOT / "program" / "data" / "C"
21
22 EXPECTED = {
23     "result1.xlsx": (["计划购电量", "充放电量"], 1, 6),
24     "result2.xlsx": (["计划购电量", "充放电量", "紧急购电量"], 334, 2004),
25     "result3.xlsx": (["计划购电量", "调整购电量", "充放电量", "紧急购电量"], 334, 2004),
26     "result4-2.xlsx": (["计划购电量", "充放电量", "紧急购电量"], 334, 2004),
27     "result4-3.xlsx": (["计划购电量", "调整购电量", "充放电量", "紧急购电量"], 334, 2004),
28 }
29
30
31 def _as_date(value) -> date:
32     if isinstance(value, datetime):
33         return value.date()
34     if isinstance(value, date):
35         return value

```

```

36     return datetime.fromisoformat(str(value)).date()
37
38
39 def _prices() -> tuple[np.ndarray, dict[date, np.ndarray]]:
40     wb = openpyxl.load_workbook(DATA_C / "附件1.xlsx", read_only=True, data_only=True)
41     static = np.asarray([r[1] for r in list(wb.active.values)[1:145]], dtype=float)
42     wb.close()
43     wb = openpyxl.load_workbook(DATA_C / "附件4.xlsx", read_only=True, data_only=True)
44     dynamic = {
45         _as_date(r[0]): np.asarray(r[1:145], dtype=float)
46         for r in list(wb.active.values)[1:] if r[0] is not None
47     }
48     wb.close()
49     return static, dynamic
50
51
52 def audit_workbook(name: str, sheets: list[str], n_days: int, n_charge: int) -> None:
53     path = RESULTS / name
54     assert path.exists(), f"缺少 {path}"
55     wb = openpyxl.load_workbook(path, read_only=True, data_only=True)
56     assert wb.sheetnames == sheets, (name, wb.sheetnames)
57
58     # 计划/调整购电量: 日行 × 144 槽 + 全天电量/费用汇总列
59     if name == "result1.xlsx":
60         rows = [r for r in wb["计划购电量"].iter_rows(min_row=2, values_only=True)
61                  if r[0] is not None]
62         assert len(rows) == 144, (name, "计划购电量", len(rows))
63         values = np.asarray([r[1] for r in rows], dtype=float)
64         assert np.isfinite(values).all() and (values >= -1e-7).all()
65     static_price, dynamic_price = _prices()
66     purchase_rows: dict[str, list[tuple]] = {}
67     for sheet in ("计划购电量", "调整购电量"):
68         if sheet not in sheets or name == "result1.xlsx":
69             continue
70         rows = [r for r in wb[sheet].iter_rows(min_row=2, values_only=True) if r[0] is not None]
71         purchase_rows[sheet] = rows
72         assert len(rows) == n_days, (name, sheet, len(rows))
73         dates = [_as_date(r[0]) for r in rows]
74         expected = [date(2025, 2, 1) + timedelta(days=i) for i in range(n_days)]
75         assert dates == expected, (name, sheet, "日期不连续或范围错误", dates[:1], dates[-1:])
76         for i, r in enumerate(rows):
77             slots = np.asarray(r[1:145], dtype=float)
78             assert np.isfinite(slots).all() and (slots >= -1e-7).all()
79             assert abs(float(r[145]) - float(slots.sum())) < 1e-4, (name, sheet, "勾稽")
80             assert np.isfinite(float(r[146])) and float(r[146]) >= -1e-7
81             price = dynamic_price[dates[i]] if name.startswith("result4-") else static_price
82             if sheet == "调整购电量":
83                 plan = np.asarray(purchase_rows["计划购电量"][i][1:145], dtype=float)
84                 expected_fee = float(price @ slots + (0.5 * price * np.abs(slots - plan)).sum())
85             else:
86                 expected_fee = float(price @ slots)
87             assert abs(float(r[146]) - expected_fee) < 1e-4, (
88                 name, sheet, "费用列不能由源电价重算", float(r[146]), expected_fee
89             )
90
91     # 充放电量: 0:00/24:00 行(官方模板位于第 1/2 块) 标签与 SOC
92     charge = [r for r in wb["充放电量"].iter_rows(min_row=2, values_only=True)
93               if r[1] is not None]
94     assert len(charge) == n_charge, (name, "充放电量", len(charge))
95     cc, dc, ec = (1, 2, 4) if name == "result1.xlsx" else (2, 3, 5)
96     e0 = e24 = None
97     first_block = charge[0]
98     second_block = charge[1] if len(charge) > 1 else None
99     if name == "result1.xlsx":
100         assert first_block[3] == "0:00" and first_block[ec] is not None
101         assert second_block[3] == "24:00" and second_block[ec] is not None
102         e0, e24 = float(first_block[ec]), float(second_block[ec])
103     for r in charge:
104         assert float(r[cc]) >= -1e-7 and float(r[dc]) >= -1e-7
105         if r[ec] is not None:

```

```

105         assert 1200.0 - 1e-6 <= float(r[ec]) <= 10800.0 + 1e-6, (name, r[ec])
106     if name == "result1.xlsx":
107         assert abs(e0 - e24) < 1e-6, (name, "0:00/24:00 储电量不相等", e0, e24)
108     else:
109         plan_dates = [_as_date(r[0]) for r in purchase_rows["计划购电量"]]
110         e0s, e24s, charge_dates = [], [], []
111         for i in range(n_days):
112             block = charge[6 * i:6 * (i + 1)]
113             assert block[0][4] == "00:00" and block[1][4] == "24:00", (
114                 name, i, "0:00/24:00 标签错误")
115             charge_dates.append(_as_date(block[0][0]))
116             e0s.append(float(block[0][ec]))
117             e24s.append(float(block[1][ec]))
118         assert charge_dates == plan_dates, (name, "充放表日期与计划表不一致")
119         assert max(abs(e24s[i] - e0s[i + 1]) for i in range(n_days - 1)) < 1e-6, (
120             name, "跨日 SOC 不连续")
121
122     # 紧急购电量: 电量非负
123     if "紧急购电量" in sheets:
124         for r in wb["紧急购电量"].iter_rows(min_row=2, values_only=True):
125             if r[2] is not None:
126                 assert float(r[2]) >= -1e-7, (name, r[:3])
127
128     # 时间标签: 与附件 5 模板一致 (附件与模板标签相差一个 10 分钟刻度,
129     # 一律按行序号对齐; 重写文件时不得改动模板标签)
130     tpl = openpyxl.load_workbook(DATA_C / "附件5" / name, read_only=True, data_only=True)
131     t_rows = list(tpl["计划购电量"].iter_rows(values_only=True))
132     hdr = next(wb["计划购电量"].iter_rows(min_row=1, max_row=1, values_only=True))
133
134     def _norm(v):
135         return None if v is None else str(v)
136
137     assert [_norm(v) for v in hdr] == [_norm(v) for v in t_rows[0]], (
138         name, "计划购电量表头/时段标签与附件 5 模板不一致")
139     if name == "result1.xlsx":
140         labels = [_norm(r[0]) for r in wb["计划购电量"].iter_rows(min_row=2, values_only=True)
141                  if r[0] is not None]
142         assert labels == [_norm(r[0]) for r in t_rows[1:]], (
143             name, "逐行时段标签与附件 5 模板不一致")
144         label_col = 0 if name == "result1.xlsx" else 1 # 充放电电表的“时间段”列
145         blocks = [str(r[label_col]) for r in wb["充放电电量"].iter_rows(min_row=2, values_only=True)
146                  if r[label_col] is not None]
147         expected_blocks = ["0:00-4:00", "4:00-8:00", "8:00-12:00",
148                            "12:00-16:00", "16:00-20:00", "20:00-24:00"]
149         assert blocks and all(b == expected_blocks[i % 6] for i, b in enumerate(blocks)), (
150             name, "充放电电量块标签错误", blocks[:6])
151     tpl.close()
152     wb.close()
153
154
155 def main() -> None:
156     for name, (sheets, n_days, n_charge) in EXPECTED.items():
157         audit_workbook(name, sheets, n_days, n_charge)
158         print(f"ok - {name}")
159     print("C result audit: PASS")
160
161
162 if __name__ == "__main__":
163     main()

```

program/tests/docs_examples_test.py

```

1 """验证 docs/cookbook.md 中的核心示例代码可运行 (预测 / 优化 / 评价三套骨架)。
2
3 运行::
4
5     uv run python tests/docs_examples_test.py
6 """
7

```

```

8 from __future__ import annotations
9
10 import sys
11 import tempfile
12 from pathlib import Path
13
14 sys.stdout.reconfigure(encoding="utf-8")
15
16 import numpy as np
17 import pandas as pd
18
19 import program as pm
20
21 tmp = Path(tempfile.mkdtemp(prefix="docs_examples_"))
22 pm.init(root=tmp, seed=42, verbose=False)
23 rng = np.random.default_rng(0)
24
25 # ----- 预测类骨架
26 n = 150
27 df = pd.DataFrame(
28     {
29         "价格": rng.uniform(5, 20, n),
30         "广告费": rng.uniform(0, 10, n),
31         "促销力度": rng.uniform(0, 1, n),
32     }
33 )
34 df["销量"] = 80 - 2 * df["价格"] + 3 * df["广告费"] + 10 * df["促销力度"] + rng.normal(0, 3, n)
35 pm.record("## 数据理解\n\n" + pm.summarize(df))
36
37 X, y = pm.dataio.split_xy(df, target="销量")
38 table = pm.ml.compare_models(["linear", "ridge", "rf", "gbr"], X, y, task="regression", cv=5, seed=42)
39 pm.record_result("模型对比", table)
40 best = str(table.iloc[0]["模型"])
41 res = pm.ml.fit_regressor(best, X, y, test_size=0.2, cv=5, seed=42)
42 pm.record_result("最终模型指标", res.test_metrics, note=f"模型: {best}")
43 pm.scatter(res.y_test, res.y_pred, xlabel="真实值", ylabel="预测值", save="q2_pred_vs_true")
44 resid = np.asarray(res.y_test) - res.y_pred
45 pm.hist(resid, xlabel="残差", save="q2_residual")
46 pm.line(np.arange(len(resid)), [np.asarray(res.y_test), res.y_pred],
47         labels=["真实值", "预测值"], xlabel="样本序号", ylabel="销量", save="q2_series")
48 assert res.test_metrics["R2"] > 0.8, res.test_metrics
49 print("OK 预测类骨架")
50
51 # ----- 优化类骨架
52 c = [3, 5]
53 A_ub = np.array([[1, 0], [0, 2], [3, 2]])
54 b_ub = np.array([4, 12, 18])
55 r = pm.optimize.solve_lp(c=c, A_ub=A_ub, b_ub=b_ub, maximize=True)
56 assert r.success, r.message
57 violation = float(np.max(A_ub @ r.x - b_ub))
58 pm.record_result(
59     "问题一结果",
60     {f"x{i + 1}": v for i, v in enumerate(r.x)} | {"目标值": r.fun},
61     note=f"最大约束违反量 = {violation:.2e} ( 0 即全部满足)",
62 )
63 rows = []
64 for delta in (-0.10, -0.05, 0.0, 0.05, 0.10):
65     r2 = pm.optimize.solve_lp(c=c, A_ub=A_ub, b_ub=b_ub * (1 + delta), maximize=True)
66     rows.append({"扰动": f"{delta:+.0%}", "最优目标值": r2.fun})
67 sens = pd.DataFrame(rows)
68 pm.record_result("灵敏度分析", sens)
69 pm.line(sens["扰动"], sens["最优目标值"], marker="o",
70         xlabel="约束参数扰动", ylabel="最优目标值", save="q1_sensitivity")
71 pm.bar(["原方案", "优化方案"], [28.0, r.fun], value_labels=True,
72        ylabel="总收益/万元", save="q1_compare")
73 assert abs(r.fun - 36) < 1e-8
74 print("OK 优化类骨架")
75
76 # ----- 评价类骨架

```

```

77 names = ["方案A", "方案B", "方案C", "方案D"]
78 indicators = ["成本", "效率", "稳定性", "环保性"]
79 raw = rng.uniform(1, 10, (4, 4))
80 directions = np.array([-1, 1, 1, 1])
81
82 Z = (raw - raw.min(0)) / (raw.max(0) - raw.min(0))
83 Z[:, directions < 0] = 1 - Z[:, directions < 0]
84 P = (Z + 1e-12) / (Z + 1e-12).sum(0)
85 E = -(P * np.log(P)).sum(0) / np.log(len(Z))
86 W = (1 - E) / (1 - E).sum()
87 score = Z @ W
88 rank = pd.Series(score, index=names).rank(ascending=False).astype(int)
89 table = pd.DataFrame({"方案": names, "综合得分": score, "排名": rank})
90 pm.record_result("综合评价结果", table)
91 pm.record_result("权重", pd.DataFrame({"指标": indicators, "权重": W}))
92 pm.bar(names, score, value_labels=True, ylabel="综合得分", save="q3_score")
93 pm.radar(indicators, {nm: Z[i].tolist() for i, nm in enumerate(names)}, save="q3_radar")
94 assert abs(W.sum() - 1) < 1e-9
95 print("OK 评价类骨架")
96
97 # ----- 分类类骨架
98 dfc = pd.DataFrame(rng.normal(size=(160, 3)), columns=["a", "b", "c"])
99 dfc["标签"] = (dfc["a"] + dfc["b"] > 0).astype(int)
100 Xc, yc = pm.dataio.split_xy(dfc, target="标签")
101 table_c = pm.ml.compare_models(["logreg", "rf", "knn"], Xc, yc, task="classification", cv=5, seed=42)
102 res_c = pm.ml.fit_classifier("rf", Xc, yc, cv=5, seed=42)
103 pm.record_result("测试集指标", res_c.test_metrics)
104 pm.record_result("混淆矩阵", pm.metrics.confusion_table(res_c.y_test, res_c.y_pred))
105 pm.roc(res_c.y_test, res_c.y_proba[:, 1], save="q2_roc")
106 assert res_c.test_metrics["accuracy"] > 0.9
107 print("OK 分类类骨架")
108
109 # ----- 产物检查
110 report = (tmp / "reports" / "RESULTS_REPORT.md").read_text(encoding="utf-8")
111 figs = sorted(p.name for p in (tmp / "figures").glob("*.pdf"))
112 fdata = sorted(p.name for p in (tmp / "code" / "outputs" / "figure_data").glob("*.csv"))
113 assert "模型对比" in report and "综合评价结果" in report
114 assert len(figs) == 8, figs
115 assert len(fdata) == 8, fdata
116 print(f"OK 报告与图表产物 ({len(figs)} 图 / {len(fdata)} 数据)")
117 print("\n 文档示例代码全部可运行")

```

program/tests/model_consistency_test.py

```

1 """模型一致性守卫测试（唯一参数源 + 统一构造）。
2
3 校验：
4 1. ``consistency.py`` 的参数值即规范值；
5 2. ``q2_tune.seqs_ewma`` 与 ``consistency.ewma_weights_from_table`` 完全一致（EWMA 唯一源）；
6 3. ``q3_proto`` 的历史预测使用 EWMA 权重（与手工公式一致）；
7 4. 统一参数被各引擎以默认值引用（/m/情景数）。
8
9 运行（program/ 下）：uv run python tests/model_consistency_test.py
10 """
11 from __future__ import annotations
12
13 import inspect
14
15 import numpy as np
16
17 import program as pm
18 from solve import consistency as cs
19 from solve import q2_tune, q3, q3_proto, q4
20 from solve.q2_adaptive import AdaptiveWeightModel
21 from solve.common import ROOT
22
23
24 def main() -> None:
25     pm.init(seed=42, root=str(ROOT))

```

```

26
27 # 1) 参数值 = 规范值
28 assert cs.EWMA_HL == 5.0, cs.EWMA_HL
29 assert cs.KAPPA == 1.02, cs.KAPPA
30 assert cs.MARGIN == 25.0, cs.MARGIN
31 assert cs.N_SCEN == 40, cs.N_SCEN
32 assert cs.Q3_LAM == 0.7 and cs.Q3_ADJ_LAM == 0.7
33 assert cs.Q3_USE_HEDGE is False
34 assert cs.START_DAY == 31 and cs.PLAN_HORIZON == 288
35
36 # 2) EWMA 序列唯一源
37 model = AdaptiveWeightModel().load().build_table()
38 seq = q2_tune.seqs_ewma(model, cs.EWMA_HL)
39 table = cs.ewma_weights_from_table(model.C, model.grid, hl=cs.EWMA_HL)
40 days = list(range(q2_tune.REPORT_START, q2_tune.q2.N_DAY))
41 assert len(seq) == len(days)
42 for i, d in enumerate(days):
43     assert np.allclose(seq[i][0], table[d][0]) and np.allclose(seq[i][1], table[d][1])
44
45 # 3) q3_proto 历史预测 = EWMA 权重手工公式 (负荷/光伏两通道)
46 data = q3_proto.load_extended()
47 assert "EWMA_WU" in data
48 for D in (60, 200):
49     w, u = data["EWMA_WU"][D]
50     pv = q3_proto.hist_forecast(data, D)
51     manual_pv = np.clip(u[0] * data["pv_act"][D - 1] + u[1] * data["pv_act"][D - 2]
52                     + u[2] * data["pv_typ"], 0.0, None)
53     assert np.allclose(pv, manual_pv), D
54     load = q3_proto.hist_load_forecast(data, D)
55     manual_l = np.clip(w[0] * data["load"][D - 7] + w[1] * data["load"][D - 14]
56                     + w[2] * data["load_typ"], 0.0, None)
57     assert np.allclose(load, manual_l), D
58
59 # 4) 默认参数来自唯一源
60 sig = inspect.signature(q3_proto.plan_two_day)
61 assert sig.parameters["kappa"].default == cs.KAPPA
62 assert sig.parameters["margin"].default == cs.MARGIN
63 sig2 = inspect.signature(q3_proto.simulate_day_rt_hedge)
64 assert sig2.parameters["n_scen"].default == cs.N_SCEN
65 assert sig2.parameters["kappa"].default == cs.KAPPA
66 assert sig2.parameters["margin"].default == cs.MARGIN
67 assert sig2.parameters["exec_policy"].default == cs.EXEC_POLICY
68 assert cs.EXEC_POLICY == "free"
69 assert q3.LAM == cs.Q3_LAM and q3.ADJ_LAM == cs.Q3_ADJ_LAM
70 assert q3.USE_HEDGE == cs.Q3_USE_HEDGE
71 assert inspect.signature(q4.simulate_day_q3).parameters["hedge"].default == cs.Q3_USE_HEDGE
72 assert inspect.signature(q4.run_year_q3).parameters["hedge"].default == cs.Q3_USE_HEDGE
73
74 print("model consistency test: PASS")
75
76
77 if __name__ == "__main__":
78     main()

```

program/tests/smoke_test.py

```

1 """program 工具包冒烟测试: 覆盖全部 11 个模块, 验证核心功能可运行且结果正确。
2
3 运行::
4
5     uv run python tests/smoke_test.py
6 """
7
8 from __future__ import annotations
9
10 import sys
11 import tempfile
12 from pathlib import Path
13

```

```

14 sys.stdout.reconfigure(encoding="utf-8")
15
16 import numpy as np
17 import pandas as pd
18
19 import program as pm
20
21 PASSED: list[str] = []
22
23
24 def check(name: str, cond: bool, detail: str = "") -> None:
25     if not cond:
26         raise AssertionError(f" {name} {detail}")
27     PASSED.append(name)
28     print(f" ok - {name}")
29
30
31 def section(title: str) -> None:
32     print(f"\n== {title} ==")
33
34
35 def main() -> None:
36     tmp = Path(tempfile.mkdtemp(prefix="program_smoke_"))
37     pm.config.set_project_root(tmp)
38     pm.init(seed=42, verbose=False)
39     rng = np.random.default_rng(42)
40
41     # ----- utils
42     section("utils / config")
43     from program.utils import DataCache, Timer, env_markdown, fmt, markdown_table
44
45     check("fmt", fmt(1234.5678) == "1235" and "e" in fmt(1.23e-6))
46     check("markdown_table", "| a |" in markdown_table({"a": [1, 2]}, nd=3))
47     with Timer("noop"):
48         pass
49     cache = DataCache("smoke", root=tmp / "cache")
50     cache.save("k", {"v": 1})
51     check("DataCache", cache.get_or_compute("k", lambda: 2)["v"] == 1)
52     check("env_markdown", "numpy" in env_markdown())
53
54     # ----- dataio
55     section("dataio")
56     df = pd.DataFrame(
57         {
58             "x": [1, 2, 3, 4, 5],
59             "y": [2.0, np.nan, 6.0, 8.0, 10.0],
60             "g": ["a", "b", "a", "b", "a"],
61         }
62     )
63     csv_p = pm.save_table(df, tmp / "data" / "t.csv")
64     check("csv roundtrip", len(pm.read_table(csv_p)) == 5)
65     xlsx_p = pm.save_table(df, tmp / "data" / "t.xlsx")
66     check("xlsx roundtrip", pm.read_table(xlsx_p)["y"].isna().sum() == 1)
67     check("read_sheets", "Sheet1" in pm.read_sheets(xlsx_p))
68     filled = pm.dataio.fill_missing(df)
69     check("fill_missing", filled["y"].isna().sum() == 0)
70     check("clean_columns", list(pm.dataio.clean_columns(df).columns) == ["x", "y", "g"])
71     check("missing_report", pm.dataio.missing_report(df)["缺失数"].sum() == 1)
72     s = pm.summarize(df)
73     check("summarize", "数据概览" in s and "前 5 行" in s)
74     X, y = pm.dataio.split_xy(filled, target="y")
75     check("split_xy", X.shape == (5, 2) and len(y) == 5)
76     Xtr, Xte, ytr, yte = pm.dataio.train_test_split_df(filled, target="y", seed=1)
77     check("train_test_split_df(target)", len(Xtr) + len(Xte) == 5 and len(ytr) + len(yte) == 5)
78     tr, te = pm.dataio.train_test_split_df(filled, seed=1)
79     check("train_test_split_df(默认)", len(tr) + len(te) == 5)
80     js_p = pm.dataio.save_json({"a": np.arange(3)}, tmp / "data" / "t.json")
81     check("json roundtrip", pm.dataio.load_json(js_p)["a"] == [0, 1, 2])
82

```

```

83 # ----- plotting
84 section("plotting")
85 xs = np.linspace(0, 2 * np.pi, 100)
86 pm.line(xs, [np.sin(xs), np.cos(xs)], labels=["sin", "cos"],
87         xlabel="x", ylabel="y", save="p_line")
88 check("line", (tmp / "figures" / "p_line.pdf").exists())
89 pm.scatter(xs, np.sin(xs), xlabel="x", ylabel="sin", save="p_scatter")
90 _, _, info = pm.scatter_fit(xs, 2 * xs + 1 + rng.normal(0, 0.3, 100),
91                             xlabel="x", ylabel="y", save="p_fit")
92 check("scatter_fit", abs(info["coef"][0] - 2) < 0.3)
93 pm.bar(["A", "B", "C"], [1, 2, 3], ylabel="值", save="p_bar")
94 pm.bar_group(["A", "B"], {"m1": [1, 2], "m2": [2, 1]}, save="p_bar_group")
95 pm.hist(rng.normal(size=200), xlabel="x", save="p_hist")
96 pm.box({"A": rng.normal(size=50), "B": rng.normal(1, 1, 50)}, save="p_box")
97 pm.heatmap(df[["x", "y"]].fillna(0).to_numpy(), annot=True, save="p_heat")
98 pm.corr_heatmap(filled[["x", "y"]], save="p_corr")
99 pm.radar(["m1", "m2", "m3"], {"A": [1, 2, 3], "B": [3, 1, 2]}, save="p_radar")
100 pm.dual_axis(xs, np.sin(xs), 100 * np.cos(xs), save="p_dual")
101 pm.pie(["A", "B"], [3, 1], save="p_pie")
102 pm.errorbar(xs[:10], np.sin(xs[:10]), yerr=0.1, save="p_err")
103 XX, YY = np.meshgrid(np.linspace(-2, 2, 30), np.linspace(-2, 2, 30))
104 pm.surface3d(XX, YY, XX**2 + YY**2, xlabel="x", ylabel="y", zlabel="z", save="p_3d")
105 yb = (rng.normal(size=200) > 0).astype(int)
106 prob = np.clip(yb * 0.7 + rng.normal(0, 0.2, 200) + 0.2, 0, 1)
107 _, _, auc = pm.roc(yb, prob, save="p_roc")
108 check("roc", auc > 0.9)
109 figs = sorted(p.name for p in (tmp / "figures").glob("*.pdf"))
110 check("全部图已输出 PDF", len(figs) == 15, f"实际 {len(figs)}: {figs}")
111 check(
112     "图表数据已记录",
113     all(
114         (tmp / "code" / "outputs" / "figure_data" / f).exists()
115         for f in ("p_line.csv", "p_fit.csv")
116     ),
117 )
118
119 # ----- stats
120 section("stats")
121 a = rng.normal(0, 1, 100)
122 b = rng.normal(0.6, 1, 100)
123 r_t = pm.stats.t_test_ind(a, b)
124 check("t_test_ind", r_t.significant and abs(r_t.statistic) > 2)
125 check("normality", pm.stats.normality(a).pvalue > 0.01)
126 check("t_test_one", not pm.stats.t_test_one(a, mu=0).significant)
127 check("t_test_paired", pm.stats.t_test_paired(a, a + rng.normal(0, 0.1, 100)).pvalue is not None)
128 check("anova", pm.stats.anova(*[rng.normal(m, 1, 60) for m in (0, 0, 1)]).pvalue < 0.05)
129 check("kruskal", pm.stats.kruskal(a, b).pvalue < 0.05)
130 check("compare_groups", pm.stats.compare_groups(a, b).pvalue < 0.05)
131 check("chi2", pm.stats.chi2_test([10, 20], [30, 15])).pvalue < 0.05)
132 check("corr_test", abs(pm.stats.corr_test(np.arange(50), np.arange(50)).statistic - 1) < 1e-9)
133 df_c = pd.DataFrame(rng.normal(size=(100, 3)), columns=["a", "b", "c"])
134 check("corr_matrix", pm.stats.corr_matrix(df_c).shape == (3, 3))
135 lr = pm.stats.linregress_summary(np.arange(20), 2 * np.arange(20) + 1)
136 check("linregress", abs(lr.slope - 2) < 1e-9 and lr.r2 > 0.999)
137 df_r = pd.DataFrame({"y": 3 * df_c["a"] - 2 * df_c["b"] + rng.normal(0, 0.1, 100), "x1": df_c["a"], "x2": df_c["b"]})
138 ols_res = pm.stats.ols(df_r, y="y", xs=["x1", "x2"])
139 check("ols", abs(ols_res.params["x1"] - 3) < 0.1)
140 check("ols_text", "R-squared" in pm.stats.ols_text(df_r, y="y", xs=["x1", "x2"]))
141 lo, hi = pm.stats.ci_mean(a)
142 check("ci_mean", lo < a.mean() < hi)
143 bci = pm.stats.bootstrap_ci(a, n_boot=2000, seed=1)
144 check("bootstrap_ci", bci.lo < bci.stat < bci.hi)
145
146 # ----- fitting
147 section("fitting")
148 from program import fitting
149
150 def expo(x, aa, bb):
151     return aa * np.exp(-bb * x)

```

```

152
153 xf = np.linspace(0, 2, 60)
154 yf = 3 * np.exp(-1.5 * xf) + rng.normal(0, 0.01, 60)
155 fr = fitting.fit_curve(expo, xf, yf, p0=[1, 1], names=["a", "b"])
156 check("fit_curve", abs(fr.params[0] - 3) < 0.1 and abs(fr.params[1] - 1.5) < 0.05)
157 check("fit_curve.summary", "R2" in fr.summary() and "+" in fr.summary())
158 pf = fitting.polyfit(xf, 2 * xf**2 - 3 * xf + 1, degree=2)
159 check("polyfit", np.allclose(pf.params, [2, -3, 1], atol=1e-6))
160 check("poly_str", "x2" in fitting.poly_str(pf.params))
161 check("poly_scan", len(fitting.poly_scan(xf, yf, max_degree=3)) == 3)
162 xi = np.linspace(0.1, 1.9, 20)
163 for kind in ("linear", "cubic", "pchip"):
164     check(f"interp/{kind}", fitting.interp(xf, yf, xi, kind=kind).shape == (20,))
165 spl = fitting.spline_smooth(xf, yf, s=len(xf) * 0.001)
166 check("spline_smooth", spl(xi).shape == (20,))
167 for m in ("savgol", "moving", "ewma"):
168     check(f"smooth/{m}", fitting.smooth(yf, 7, method=m).shape == yf.shape)
169
170 # ----- metrics
171 section("metrics")
172 yt = np.array([1.0, 2, 3, 4, 5])
173 m = pm.metrics.regression_metrics(yt, yt + 0.1, n_features=2)
174 check("regression_metrics", abs(m["MAE"] - 0.1) < 1e-9 and "AdjR2" in m)
175 y_cls = np.array([0, 1, 0, 1, 0, 1, 0, 1])
176 p_cls = np.array([0, 1, 0, 1, 0, 0, 0, 1])
177 cm = pm.metrics.classification_metrics(y_cls, p_cls, y_proba=np.clip(p_cls + 0.1, 0, 1))
178 check("classification_metrics", cm["accuracy"] > 0.8 and "auc" in cm)
179 ct = pm.metrics.confusion_table(y_cls, p_cls)
180 check("confusion_table", ct.shape == (2, 2))
181 blobs = np.vstack([rng.normal(m, 0.3, (30, 2)) for m in (0, 3)]).astype(float)
182 lab = np.r_[np.zeros(30, int), np.ones(30, int)]
183 check("cluster_metrics", pm.metrics.cluster_metrics(blobs, lab)["silhouette"] > 0.8)
184 check("str_metrics", "MAE" in pm.metrics.str_metrics(m))
185
186 # ----- optimize
187 section("optimize")
188 r_lp = pm.optimize.solve_lp(
189     [3, 5], A_ub=[[1, 0], [0, 2], [3, 2]], b_ub=[4, 12, 18], maximize=True
190 )
191 check("solve_lp", r_lp.success and abs(r_lp.fun - 36) < 1e-8, str(r_lp))
192 r_ip = pm.optimize.solve_ip([1, 1], A_ub=[[2, 2]], b_ub=[7], maximize=True, integer=True)
193 check("solve_ip(integer)", r_ip.success and abs(r_ip.fun - 3) < 1e-8)
194 r_nlp = pm.optimize.solve_nlp(lambda v: (v[0] - 1) ** 2 + (v[1] - 2) ** 2, x0=[0, 0])
195 check("solve_nlp", abs(r_nlp.fun) < 1e-6 and np.allclose(r_nlp.x, [1, 2], atol=1e-3))
196 r_g = pm.optimize.global_minimize(lambda v: (v[0] - 1.5) ** 2, bounds=[(0, 3)])
197 check("global_minimize", abs(r_g.fun) < 1e-6)
198 r_kp = pm.optimize.knapsack([60, 100, 120], [10, 20, 30], 50)
199 check("knapsack", r_kp.best_value == 220 and r_kp.items == [1, 2])
200 r_as = pm.optimize.assignment([[4, 1, 3], [2, 0, 5], [3, 2, 2]])
201 check("assignment", r_as.total == 5, str(r_as))
202 pts = np.array([[0, 0], [1, 0], [1, 1], [0, 1]], dtype=float)
203 d = pm.optimize.distance_matrix(pts)
204 tour_nn = pm.optimize.tsp_nearest_neighbor(d)
205 tour_2o, length = pm.optimize.tsp_2opt(d, tour_nn)
206 check("tsp", abs(length - 4.0) < 1e-9 and sorted(tour_2o) == [0, 1, 2, 3])
207
208 # ----- ml
209 section("ml")
210 Xm = rng.normal(size=(120, 3))
211 ym = Xm @ np.array([1.0, -2.0, 3.0]) + rng.normal(0, 0.1, 120)
212 r_reg = pm.ml.fit_regressor("ridge", Xm, ym, cv=3, seed=0)
213 check("fit_regressor", r_reg.test_metrics["R2"] > 0.99, r_reg.summary())
214 Xc = np.vstack([rng.normal(0, 1, (60, 2)), rng.normal(2.5, 1, (60, 2))])
215 yc = np.r_[np.zeros(60, int), np.ones(60, int)]
216 r_cls = pm.ml.fit_classifier("logreg", Xc, yc, cv=3, seed=0)
217 check("fit_classifier", r_cls.test_metrics["accuracy"] > 0.9 and r_cls.y_proba is not None)
218 cv_res = pm.ml.cross_validate("rf", Xm, ym, cv=3, seed=0)
219 check("cross_validate", 0 <= cv_res["mean"] <= 1)
220 table = pm.ml.compare_models(["linear", "ridge"], Xm, ym, cv=3, seed=0)

```

```

221 check("compare_models", len(table) == 2 and "测试集_R2" in table.columns)
222 Xs, sc = pm.ml.standardize(pd.DataFrame(Xm, columns=["a", "b", "c"]))
223 check("standardize", abs(float(np.asarray(Xs).mean())) < 1e-9)
224 cl = pm.ml.kmeans(blobs, k=2, seed=0)
225 check("kmeans", cl.metrics["silhouette"] > 0.8 and cl.centers.shape == (2, 2))
226 elbow_df = pm.ml.elbow(blobs, k_range=range(2, 5), seed=0)
227 check("elbow", len(elbow_df) == 3 and elbow_df["inertia"].is_monotonic_decreasing)
228 pc = pm.ml.pca(pd.DataFrame(Xm, columns=["a", "b", "c"]), n_components=2)
229 check("pca", pc.scores.shape == (120, 2) and pc.loadings is not None and pc.cumulative[-1] <= 1)
230
231 # ----- ode
232 section("ode")
233
234 def sir(t, y, beta, gamma):
235     s, i, r = y
236     return [-beta * s * i, beta * s * i - gamma * i, gamma * i]
237
238 t_grid = np.linspace(0, 40, 41)
239 sol = pm.ode.solve_ivp_model(sir, [0.99, 0.01, 0], (0, 40), params=(0.4, 0.2), t_eval=t_grid)
240 check("solve_ivp_model", sol.success and sol.y.shape == (3, 41))
241 y_obs = sol.y.T + rng.normal(0, 0.002, (41, 3))
242 fit_ode = pm.ode.fit_ode_params(
243     sir, [0.99, 0.01, 0], t_grid, y_obs, p0=[0.3, 0.15],
244     bounds=([0.05, 0.05], [1.0, 1.0]), param_names=["beta", "gamma"],
245 )
246 check("fit_ode_params", abs(fit_ode.params[0] - 0.4) < 0.05 and abs(fit_ode.params[1] - 0.2) < 0.03,
247     fit_ode.summary())
248 check("ODEFitResult.simulate", fit_ode.simulate(t_grid).shape == (41, 3))
249
250 # ----- report
251 section("report")
252 pm.record_result("问题一结果", {"目标值": 36.0, "状态": "最优"}, note="smoke test 写入")
253 pm.record_result("模型对比", table)
254 pm.record("正文段落: 验证 record 直接追加。")
255 text = (tmp / "reports" / "RESULTS_REPORT.md").read_text(encoding="utf-8")
256 check("record_result", "问题一结果" in text and "| 指标 | 数值 |" in text)
257 check("record(DataFrame)", "模型对比" in text)
258 tex = pm.to_latex(pd.DataFrame({"指标": ["R2"], "值": [0.9961]}), caption="测试表", label="tab:smoke")
259 check("to_latex", "tabular" in tex)
260 out_p = pm.save_outputs({"最优值": 36.0}, "q1_result")
261 check("save_outputs(json)", out_p.exists() and out_p.suffix == ".json")
262 out_csv = pm.save_outputs(table, "q1_models")
263 check("save_outputs(csv)", out_csv.suffix == ".csv" and out_csv.exists())
264
265 print(f"\n SMOKE TEST OK: {len(PASSED)} 项检查全部通过")
266 print(f" 临时输出目录: {tmp}")
267
268
269 if __name__ == "__main__":
270     main()

```

program/tests/solve_causal_test.py

```

1  """C 题因果执行与无前视残差池的轻量回归测试。
2
3  运行 (program/ 下): uv run python tests/solve_causal_test.py
4  """
5  from __future__ import annotations
6
7  import numpy as np
8
9  from solve import q2
10 from solve import q3_proto as qp
11
12
13 def check_dispatch() -> None:
14     # 完全平衡: 不充放、不紧急购电
15     ex = q2.exec_segment_causal(
16         np.full(6, 10.0), np.zeros(6), np.full(6, 10.0), 6000.0, 6000.0

```

```

17 )
18 assert np.allclose(ex["E"], 6000.0)
19 assert np.allclose(ex["c"], 0.0)
20 assert np.allclose(ex["d"], 0.0)
21 assert np.allclose(ex["e"], 0.0)
22
23 # 先富余 100 kWh, 再缺口 81 kWh; 双向 0.9 效率后恰好回到起点
24 ex = q2.exec_segment_causal(
25     np.array([0.0, 81.0]), np.array([100.0, 0.0]), np.zeros(2), 6000.0, 6000.0
26 )
27 assert np.allclose(ex["c"], [100.0, 0.0], atol=1e-8)
28 assert np.allclose(ex["d"], [0.0, 81.0], atol=1e-8)
29 assert np.allclose(ex["e"], 0.0, atol=1e-8)
30 assert abs(ex["E"][-1] - 6000.0) < 1e-8
31
32 # 约束全回代: 功率平衡、SOC 动态、终端、充放互斥
33 load = np.array([1000.0, 1000.0])
34 pv = np.zeros(2)
35 buy = np.zeros(2)
36 ex = q2.exec_segment_causal(load, pv, buy, 6000.0, 6000.0)
37 balance = pv + buy + ex["d"] + ex["e"] - load - ex["c"] - ex["s"]
38 dynamics = np.diff(np.r_[6000.0, ex["E"]]) - (
39     q2.ETA * ex["c"] - ex["d"] / q2.ETA
40 )
41 assert np.max(np.abs(balance)) < 1e-8
42 assert np.max(np.abs(dynamics)) < 1e-8
43 assert abs(ex["E"][-1] - 6000.0) < 1e-8
44 assert not np.any((ex["c"] > 1e-9) & (ex["d"] > 1e-9))
45
46 # 随机场景回代 (终端限制在 m 步可达域内, 与真实使用一致)
47 rng = np.random.default_rng(0)
48 max_up, max_down = q2.ETA * q2.P_MAX_E, q2.P_MAX_E / q2.ETA
49 for _ in range(100):
50     m = int(rng.integers(1, 40))
51     ld = rng.uniform(0, 4000, m)
52     pv = rng.uniform(0, 4000, m)
53     x = rng.uniform(0, 4000, m)
54     e0 = rng.uniform(q2.E_MIN, q2.E_MAX)
55     band = min(m * max_up, m * max_down) * 0.9
56     e1 = float(np.clip(e0 + rng.uniform(-band, band), q2.E_MIN, q2.E_MAX))
57     ex = q2.exec_segment_causal(ld, pv, x, e0, e1)
58     bal = pv + x + ex["d"] + ex["e"] - ld - ex["c"] - ex["s"]
59     dyn = np.diff(np.r_[e0, ex["E"]]) - (q2.ETA * ex["c"] - ex["d"] / q2.ETA)
60     assert np.max(np.abs(bal)) < 1e-7 and np.max(np.abs(dyn)) < 1e-7
61     assert ex["c"].max() <= q2.P_MAX_E + 1e-8
62     assert ex["d"].max() <= q2.P_MAX_E + 1e-8
63     assert abs(ex["E"][-1] - e1) < 1e-7
64
65
66 def check_causal_pool() -> None:
67     months = np.repeat(np.arange(1, 13), [31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31])
68     data = {"months": months}
69     for D in (31, 45, 59, 120, 364):
70         pool = qp.causal_residual_pool(data, D)
71         assert len(pool) > 0
72         assert int(pool.max()) < D
73         assert int(pool.min()) >= max(14, D - 90)
74
75
76 def check_two_day_terminal() -> None:
77     """48 小时远端守恒, 但首日 24:00 SOC 必须是自由变量而非固定 6000.0."""
78     price = np.r_[np.ones(q2.T), np.full(q2.T, 10.0)]
79     load = np.r_[np.zeros(q2.T), np.full(q2.T, 20.0)]
80     pv = np.zeros(2 * q2.T)
81     _x, E, _ = q2.plan_horizon(price, load, pv, 6000.0, 6000.0, eps=1e-6)
82     assert abs(E[-1] - 6000.0) < 1e-7
83     assert abs(E[q2.T - 1] - 6000.0) > 1.0
84
85

```

```

86 def check_hedge_probability_weights() -> None:
87     """Q2 等概率场景补救成本必须除以场景数。"""
88     captured = {}
89     original = q2.pm.optimize.solve_lp
90
91     class FakeResult:
92         success = True
93         message = ""
94         x = np.zeros(q2.T + 4 * 5 * q2.T)
95
96     def fake_solve(c, **kwargs):
97         captured["c"] = np.asarray(c, dtype=float)
98         return FakeResult()
99
100    q2.pm.optimize.solve_lp = fake_solve
101    try:
102        q2.hedge_day(
103            np.ones(q2.T), np.zeros(q2.T), np.zeros(24),
104            np.zeros((1, 24)), 6000.0, 6000.0,
105            n_scen=4, rng=np.random.default_rng(0),
106        )
107    finally:
108        q2.pm.optimize.solve_lp = original
109
110    c = captured["c"]
111    assert np.allclose(c[:q2.T], 1.0)
112    for s in range(4):
113        base = q2.T + s * 5 * q2.T
114        assert np.allclose(c[base + 4 * q2.T:base + 5 * q2.T], 5.0 / 4.0)
115
116    fc = np.full(24, 100.0)
117    residual = np.full(24, 20.0) # 预报比实际高 20
118    assert np.allclose(q2._scenario_from_residual(fc, residual), 80.0)
119
120
121 def check_publication_residual() -> None:
122     """6:00 残差不得拼入同一历史日 12:00/18:00 的预报。"""
123     n = 20
124     data = {
125         "fc0": np.zeros((n, 24)),
126         "fc6": np.full((n, 24), 6.0),
127         "fc12": np.full((n, 24), 12.0),
128         "fc18": np.full((n, 24), 18.0),
129         "pv_act": np.zeros((n, q2.T)),
130     }
131     expected = qp.fc_slots(data["fc6"][15], 6) / 6.0
132     actual = qp.forecast_residual(data, 15, 6, None)
133     assert np.allclose(actual, expected)
134     # 12:00 以后仍应保持 6:00 发布口径，而不是历史“最新预报”拼接口径。
135     assert not np.allclose(actual[72:108], qp.latest_forecast(data, 15)[72:108] / 6.0)
136
137
138 if __name__ == "__main__":
139     check_dispatch()
140     check_two_day_terminal()
141     check_causal_pool()
142     check_hedge_probability_weights()
143     check_publication_residual()
144     print("solve_causal_test: PASS")

```

program/tests/solve_refactor_test.py

```

1 """C 题重构金标测试：冻结公共函数在重构前（2026-09-13 tidy 分支起点）的数值行为。
2
3 重构期间使用：入口从子包/门面导出后，下面所有数值与等价关系必须逐位复现。
4 运行（program/ 下）：uv run python tests/solve_refactor_test.py
5 """
6 from __future__ import annotations
7

```

```

8 import tempfile
9 from pathlib import Path
10
11 import numpy as np
12 import program as pm
13
14 from solve import q1, q2, q3
15 from solve import q3_proto as qp
16 from solve.q2_adaptive import AdaptiveWeightModel, simplex_grid, softmax
17 from solve.q4 import plan_horizon
18 from solve.experiments.q4_price_dyn import forecast_price as fp_dyn
19 from solve.q4_price_fit import forecast_price as fp_fit
20
21 # -----
22 # 固定问题（所有检查共用；与 2026-09-13 采集脚本一致）
23 # -----
24 _RNG = np.random.default_rng(0)
25 PRICE = 0.4 + 0.3 * np.sin(np.arange(q2.T) / 24 * 2 * np.pi) + 0.05 * _RNG.random(q2.T)
26 LOAD_KWH = (500 + 200 * _RNG.random(q2.T)) / 6.0
27 PV_KWH = np.clip(300 * np.sin(np.arange(q2.T) / 48 * np.pi), 0, None) / 6.0
28 EO = 6000.0
29
30 GOLD_OBJ_Q1 = 1817.5053528737549
31 GOLD_X_SUM = 13801.814739472791
32 GOLD_EXEC = (11657.42226926288, 9442.512038102926, 0.0, 0.0, 6000.0)
33 GOLD_CACHE_KEY = "2e1bd10564a7"
34
35
36 def check_lp_golden() -> None:
37     """q1.build_lp q2.plan_day(eps=0); q4.plan_horizon plan_day（多日版）。"""
38     _, sol = q1.build_lp(PRICE, LOAD_KWH, PV_KWH)
39     assert abs(float(PRICE @ sol["x"]) - GOLD_OBJ_Q1) < 1e-9
40
41     x2, E2, _ = q2.plan_day(PRICE, LOAD_KWH, PV_KWH, EO, eps=0.0)
42     assert abs(float(x2.sum()) - GOLD_X_SUM) < 1e-9
43     assert abs(float(E2[-1]) - EO) < 1e-8
44     assert np.allclose(sol["x"], x2, atol=1e-10)
45     assert np.allclose(sol["E"], E2, atol=1e-10)
46
47     for eps in (0.0, 1e-3):
48         xh, Eh, _ = plan_horizon(PRICE, LOAD_KWH, PV_KWH, EO, EO, eps=eps)
49         xd, Ed, _ = q2.plan_day(PRICE, LOAD_KWH, PV_KWH, EO, eps=eps)
50         assert np.allclose(xh, xd, atol=1e-10)
51         assert np.allclose(Eh, Ed, atol=1e-10)
52
53     # 自由末端: e_terminal=None 时末端不锚定（48 小时远端守恒由 solve_causal_test 覆盖）
54     xf, Ef, _ = plan_horizon(PRICE, LOAD_KWH, PV_KWH, EO, None, eps=0.0)
55     assert len(xf) == q2.T and len(Ef) == q2.T
56
57
58 def check_exec_golden() -> None:
59     """q2.exec_day qp.exec_segment_hindsight(t0=0, e_end=e_start)（事后下界 LP）。"""
60     x, _, _ = q2.plan_day(PRICE, LOAD_KWH, PV_KWH, EO, eps=0.0)
61     ex = q2.exec_day(PRICE, LOAD_KWH, PV_KWH, x, EO)
62     ex_h = qp.exec_segment_hindsight(PRICE, LOAD_KWH, PV_KWH, x, EO, EO, 0)
63     for key in ("c", "d", "s", "E", "e"):
64         assert np.allclose(ex[key], ex_h[key], atol=1e-12), key
65     sums = (
66         float(ex["c"].sum()), float(ex["d"].sum()), float(ex["s"].sum()),
67         float(ex["e"].sum()), float(ex["E"][-1]),
68     )
69     assert np.allclose(sums, GOLD_EXEC, atol=1e-9)
70
71
72 def check_slots_golden() -> None:
73     """槽位映射（相位敏感）: hour_to_slots 与 fc_slots 的发布时刻对齐。"""
74     y = 100.0 + 10.0 * np.arange(1, 25)
75     h2s = q2.hour_to_slots(y)
76     assert abs(float(h2s[0]) - 9.166666666666666) < 1e-12

```

```

77     assert abs(float(h2s[35]) - 159.16666666666666) < 1e-12
78     assert abs(float(h2s[71]) - 219.16666666666666) < 1e-12
79     assert abs(float(h2s[143]) - 339.16666666666667) < 1e-12
80     assert abs(float(h2s.sum()) - 31380.0) < 1e-9
81
82     fc_gold = {
83         6: (110.0, 110.0, 174.16666666666666, 279.16666666666667, 24510.0),
84         12: (110.0, 110.0, 114.16666666666666, 219.16666666666669, 19470.0),
85         18: (110.0, 110.0, 110.0, 159.16666666666669, 16590.0),
86     }
87     for pub, (g0, g40, g80, g143, gsum) in fc_gold.items():
88         f = qp.fc_slots(y, pub)
89         assert abs(float(f[0]) - g0) < 1e-12
90         assert abs(float(f[40]) - g40) < 1e-12
91         assert abs(float(f[80]) - g80) < 1e-12
92         assert abs(float(f[143]) - g143) < 1e-12
93         assert abs(float(f.sum()) - gsum) < 1e-9
94
95
96 def check_weights_golden() -> None:
97     """softmax / simplex_grid / 因果策略版本 / Q2E 缓存哈希输入。"""
98     s = softmax(np.array([1.0, 2.0, 3.0]))
99     assert np.allclose(s, [0.09003057317038046, 0.24472847105479764, 0.6652409557748218])
100     grid = simplex_grid(0.2)
101     assert grid.shape == (21, 3)
102     assert np.allclose(grid[3], [0.0, 0.6, 0.4])
103     assert np.allclose(grid[20], [1.0, 0.0, 0.0])
104     assert abs(float(grid.sum()) - 21.0) < 1e-9
105
106     assert q2.CAUSAL_POLICY_VERSION == "free-no-terminal-v1"
107
108     m = AdaptiveWeightModel(W=1)
109     m.L = np.arange(365 * 144.0).reshape(365, 144) % 50
110     m.P = m.L * 0.5
111     m.price = np.linspace(0.2, 0.9, 144)
112     m.L_typ = m.L[0]
113     m.P_typ = m.P[0]
114     assert m._cache_key() == GOLD_CACHE_KEY
115
116
117 def check_price_golden() -> None:
118     """电价预测唯一实现: q4_price_fit q4_price_dyn, 且数值冻结。"""
119     p4 = 0.3 + 0.4 * np.abs(np.sin(np.arange(365 * 144).reshape(365, 144) / 37.0))
120     p_typ = p4[100] * 0.9
121     v = np.array([0.2, 0.3, 0.5])
122     a = fp_fit(30, v, p4, p_typ)
123     b = fp_dyn(30, v, p4, p_typ)
124     assert np.array_equal(a, b)
125     assert abs(float(a.sum()) - 73.61137670353284) < 1e-9
126     assert abs(float(a[0]) - 0.48820724638685353) < 1e-12
127
128
129 def check_forecast_golden() -> None:
130     """hist_forecast (历史口E 权重) 与 latest_forecast (按段最新预报) 冻结。"""
131     pv_act = np.clip(
132         300 * np.sin(np.arange(365 * 144).reshape(365, 144) / 48.0 * np.pi), 0, None
133     )
134     pv_typ = np.clip(250 * np.sin(np.arange(144) / 48.0 * np.pi), 0, None)
135     th = np.zeros((365, 6))
136     th[:, 3:6] = np.array([0.3, 0.3, 0.4])
137     h = qp.hist_forecast({"pv_act": pv_act, "pv_typ": pv_typ, "TH": th}, 30)
138     assert abs(float(h.sum()) - 14289.678975896264) < 1e-9
139     assert abs(float(h[0]) - 3.124641251856605e-12) < 1e-12
140     assert abs(float(h[143]) - 12.138252243609397) < 1e-12
141
142     data = {
143         "fc0": np.tile(100.0 + np.arange(24.0), (365, 1)),
144         "fc6": np.tile(200.0 + np.arange(24.0), (365, 1)),
145         "fc12": np.tile(300.0 + np.arange(24.0), (365, 1)),

```

```

146         "fc18": np.tile(400.0 + np.arange(24.0), (365, 1)),
147     }
148     lf = qp.latest_forecast(data, 10)
149     assert abs(float(lf[10]) - 100.75) < 1e-12
150     assert abs(float(lf[40]) - 200.0) < 1e-12
151     assert abs(float(lf[80]) - 300.4166666666667) < 1e-12
152     assert abs(float(lf[120]) - 401.0833333333333) < 1e-12
153     assert abs(float(lf.sum()) - 36000.0) < 1e-9
154
155
156 def check_record_replace() -> None:
157     """record() 语义: 同名章节先删除再追加(临时根目录, 不触碰真实报告)。"""
158     tmp = Path(tempfile.mkdtemp(prefix="solve_refactor_"))
159     pm.init(root=str(tmp))
160     q3.record("金标章节", {"a": 1}, note="first-note")
161     q3.record("金标章节", {"a": 2}, note="second-note")
162     text = (pm.reports_dir() / "RESULTS_REPORT.md").read_text(encoding="utf-8")
163     assert text.count("### 金标章节") == 1
164     assert "second-note" in text and "first-note" not in text
165
166
167 def check_facade_api() -> None:
168     """兼容面: q2 / q3_proto / q2_adaptive 必须保留这些公开名。"""
169     from solve import q2_adaptive
170
171     required_q2 = [
172         "plan_day", "plan_horizon", "exec_day", "exec_day_causal", "exec_segment_causal",
173         "run_exec", "load_all", "run_deterministic", "run_mc", "run_hedge", "hedge_day",
174         "write_result2", "verify", "_hour_to_slots", "_events", "_fmt_time",
175         "_scenario_from_residual", "CAUSAL_POLICY_VERSION", "EMERG_MULT",
176         "EPS_THROUGHPUT", "N_DAY", "REPORT_START", "T",
177     ]
178     for name in required_q2:
179         assert hasattr(q2, name), f"q2.{name} 缺失"
180
181     required_qp = [
182         "adjust_day", "adjust_day_hedge", "causal_residual_pool",
183         "exec_segment_hindsight", "exec_segment_causal", "run_exec", "fc_slots",
184         "forecast_at_publish", "forecast_residual", "hist_forecast",
185         "hist_load_forecast", "hist_forecast_asof", "hist_load_forecast_asof",
186         "joint_residual_blocks", "load_extended", "make_smooth_u", "plan_two_day",
187         "simulate_day", "simulate_day_rt", "simulate_day_rt_hedge",
188         "latest_forecast", "perfect_day", "E0",
189         "EPS_TH", "EMERG_MULT", "LAMBDA_GRID", "W_WINDOW", "pm",
190     ]
191     for name in required_qp:
192         assert hasattr(qp, name), f"q3_proto.{name} 缺失"
193
194     required_q2e = [
195         "AdaptiveWeightModel", "softmax", "simplex_grid", "total_of", "record",
196         "_verify_result2", "_init_worker", "N_DAY", "REPORT_START",
197     ]
198     for name in required_q2e:
199         assert hasattr(q2_adaptive, name), f"q2_adaptive.{name} 缺失"
200
201
202 if __name__ == "__main__":
203     check_lp_golden()
204     check_exec_golden()
205     check_slots_golden()
206     check_weights_golden()
207     check_price_golden()
208     check_forecast_golden()
209     check_record_replace()
210     check_facade_api()
211     print("solve_refactor_test: PASS")

```

工程配置

program/pyproject.toml

```
1 [project]
2 name = "program"
3 version = "0.1.0"
4 description = "Add your description here"
5 readme = "README.md"
6 authors = [
7     { name = "sophia7320", email = "2060205344@qq.com" }
8 ]
9 requires-python = ">=3.14"
10 dependencies = [
11     "matplotlib>=3.11.1",
12     "numpy>=2.5.3",
13     "scipy>=1.18.1",
14     "sympy>=1.14.0",
15     "scikit-learn>=1.9.0",
16     "torch>=2.11.0",
17     "torchvision>=0.26.0",
18     "tqdm>=4.70.0",
19     "loguru>=0.7.3",
20     "pandas>=3.0.5",
21     "openpyxl>=3.1.5",
22     "pulp>=3.3.2",
23     "networkx>=3.6.1",
24     "statsmodels>=0.15.0",
25 ]
26
27 [project.scripts]
28 program = "program:main"
29
30 [build-system]
31 requires = ["uv_build>=0.12.3,<0.13.0"]
32 build-backend = "uv_build"
33
34 [[tool.uv.index]]
35 name = "pytorch-cu130"
36 url = "https://download.pytorch.org/whl/cu130"
37 explicit = true
38
39 [tool.uv.sources]
40 torch = [
41     { index = "pytorch-cu130", marker = "sys_platform == 'linux' or sys_platform == 'win32' },
42 ]
43 torchvision = [
44     { index = "pytorch-cu130", marker = "sys_platform == 'linux' or sys_platform == 'win32' },
45 ]
46
47 [dependency-groups]
48 dev = [
49     "mkdocs>=1.6.1",
50     "mkdocs-material>=9.7.7",
51 ]
```

附录 D 支撑材料文件列表

支撑材料压缩包的目录与文件应与表 17 逐项一致，用于支撑正文中的模型、结果与结论。其中，源程序应能够直接运行并复现主要数值结果；数据资料应附来源说明；较大篇幅的中间结果与论文图表放入 results/；人工智能工具使用情况另附 PDF 说明文件。

表 17 支撑材料压缩包文件列表

序号	文件或目录	内容与用途
1	src/	产生正文全部数值、表格与图件的完整可运行源程序（Python/uv 工程）及运行说明
2	data/	题目附件 1–5（含结果模板）及数据来源与口径说明
3	results/	五份正式结果表、参数扫描与中间输出、论文图表 PDF
4	AI 工具使用详情.pdf	AI 工具名称与版本、使用目的、关键交互记录、采纳内容及人工修改情况
5	README.txt	软件环境、依赖版本、随机种子与一键复现步骤